

Passive Sonar Classification

Thursday 7:04 PM

hey chatgpt i was going to build my project "passive sonar based target ship classification it is an binary classification wether target ship is present or not present "

That's a solid project, and it's actually a good fit for both research and placements because it combines **signal processing**, **machine learning/deep learning**, and **real-world defense applications**.

Project Title

Passive Sonar-Based Binary Target Ship Classification Using Deep Learning

Objective:

Develop an AI model that analyzes passive sonar recordings and classifies them into:

- **Class 0:** Target Ship Not Present
- **Class 1:** Target Ship Present

Unlike active sonar, passive sonar only listens to underwater sounds without transmitting any signal, making it ideal for stealth surveillance.

Overall Workflow



Step 1: Dataset

You need two classes.

Positive Class

Audio containing target ships.

Possible datasets:

- DeepShip
- ShipsEar
- Passive sonar recordings from research datasets

Negative Class

Audio without ships.

Examples:

- Ocean ambient noise
- Rain
- Waves
- Wind
- Marine life
- Underwater silence

These can come from ambient ocean noise datasets or by extracting non-ship segments from longer recordings.

Step 2: Audio Preprocessing

Typical preprocessing:

```
Load WAV
↓
Convert to mono
↓
16 kHz sampling rate
↓
Normalize amplitude
↓
Band-pass filtering (optional)
↓
Segment into 3-second clips
```

Each clip becomes one training sample.

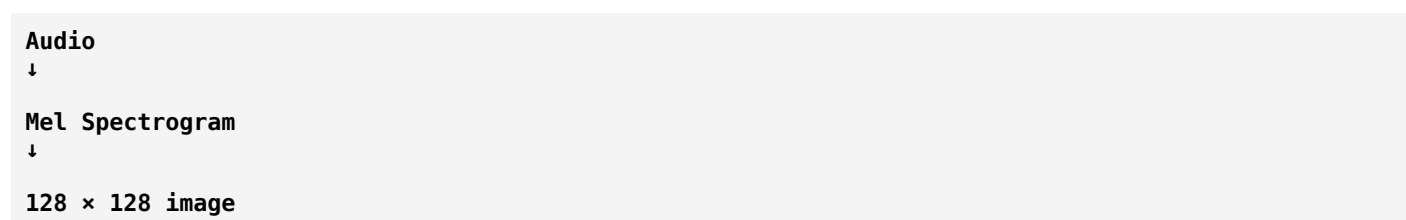
Step 3: Feature Extraction

Instead of feeding raw audio directly, convert it into image-like representations.

Mel Spectrogram

Most commonly used.

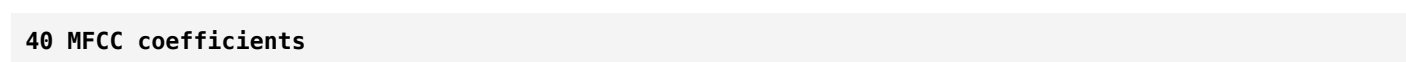
Input:



MFCC

Captures the spectral characteristics of sound.

Usually:



STFT

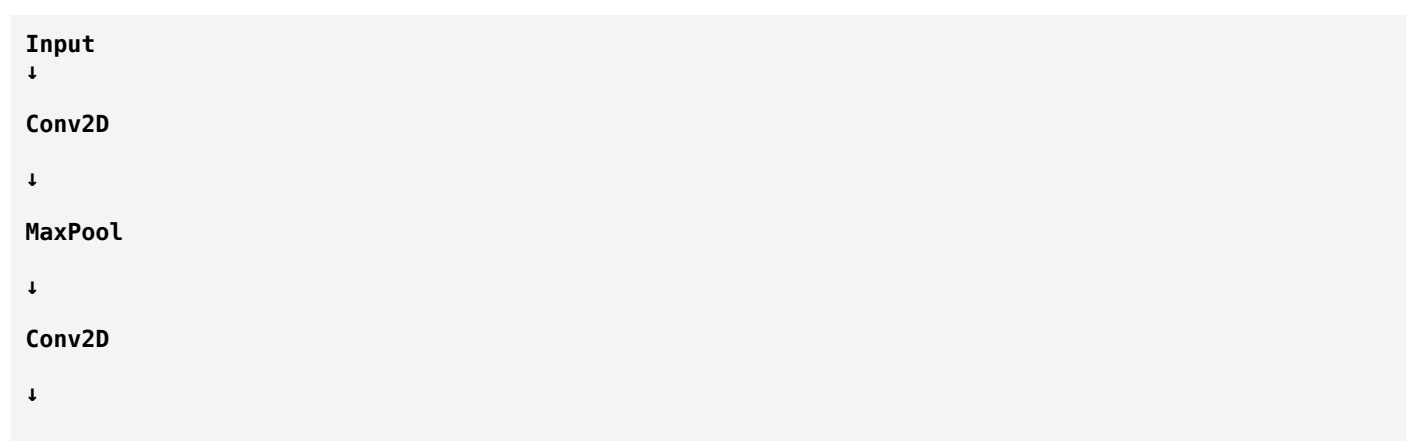
Shows how frequencies change over time.

LOFARGRAM

Very popular in naval sonar applications because it highlights machinery tonal components.

Step 4: Deep Learning Model

Option 1: Simple CNN



Flatten

↓

Dense

↓

Sigmoid

Good for a baseline model.

Option 2: ResNet18

Advantages:

- Faster training
- Good performance
- Suitable for limited datasets

Option 3: EfficientNet-B0

Advantages:

- High accuracy
- Lower parameter count
- Good balance between speed and performance

Option 4: MobileNetV3

Useful if deployment on embedded hardware or edge devices is a goal.

Binary Classification Output

Final layer:

```
Python
Dense(1, activation='sigmoid')
```

Prediction:

0 → No Ship

1 → Ship Present

Loss function:

Binary Cross Entropy

Training

Example settings:

```
Epochs = 30
Batch size = 32
Learning rate = 0.0001
Optimizer = Adam
```

Evaluation Metrics

Report:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC
- Confusion Matrix

Example:

	Predicted	
	Ship	No Ship
Actual Ship	96	4
Actual No	7	93

Suggested Folder Structure

```
PassiveSonar/
├── Dataset/
│   ├── Ship/
│   └── NoShip/
├── preprocessing/
├── feature_extraction/
├── models/
├── training/
├── evaluation/
├── results/
└── app/
```

Technology Stack

- **Programming Language:** Python
 - **Deep Learning:** TensorFlow/Keras or PyTorch
 - **Audio Processing:** Librosa
 - **Numerical Computing:** NumPy
 - **Visualization:** Matplotlib
 - **Machine Learning Utilities:** Scikit-learn
 - **Notebook Environment:** Jupyter Notebook or Google Colab
-

Research Novelty Ideas

To make your project more research-oriented, consider one or more of these enhancements:

- Compare multiple feature representations (Mel Spectrogram vs MFCC vs LOFARGRAM).
 - Benchmark different CNN architectures (CNN, ResNet18, EfficientNet-B0, MobileNetV3).
 - Apply data augmentation techniques such as time shifting, pitch shifting, and additive ocean noise.
 - Use attention mechanisms (e.g., CBAM or SE blocks) to improve feature learning.
 - Evaluate robustness across different signal-to-noise ratio (SNR) levels.
 - Use explainability methods like Grad-CAM to visualize which spectrogram regions influence predictions.
-

Expected Deliverables

1. Curated binary sonar dataset (Ship / No Ship).
2. Preprocessing pipeline for audio normalization and segmentation.
3. Feature extraction scripts for Mel Spectrogram, MFCC, and optionally LOFARGRAM.
4. Trained deep learning model with saved weights.
5. Performance evaluation with standard metrics and confusion matrix.
6. Inference script that accepts a new sonar recording and predicts **"Ship Present"** or **"Ship Not Present"**.
7. Research report documenting methodology, experiments, and results.

This design is appropriate for a final-year engineering project and can be extended into a publishable research prototype by incorporating robust evaluation and comparing multiple deep learning architectures.

ok as of now lets focus on intial stage related to digita audio processing steps

That's the right approach. In a passive sonar project, **Digital Audio Processing (DAP)** is the foundation. If the preprocessing is weak, even the best deep learning model will perform poorly.

Here's a structured roadmap for the **initial Digital Audio Processing stage**.

Passive Sonar Signal Processing Pipeline



Let's understand each step.

Step 1: Audio Acquisition

Input

Usually sonar recordings are stored as

- WAV
- FLAC
- MP3 (less common)
- Hydrophone recordings

Example

```
ship_001.wav
```

A WAV file consists of sampled amplitude values over time.

Example

```
Time (s)
0.00
0.01
```

```
0.02
0.03
...

Amplitude

0.24
0.35
0.48
0.27
...
```

Step 2: Load Audio

Read the waveform into memory.

Output

```
Amplitude array

[0.14,
0.18,
0.22,
0.30,
...
]
```

and

```
Sampling Rate

16000 Hz
```

Step 3: Resampling

Different datasets may have different sampling rates.

Example

Dataset A

```
44100 Hz
```

Dataset B

```
22050 Hz
```

Dataset C

```
16000 Hz
```

A deep learning model expects consistent input, so convert everything to one sampling rate.

Common choices

```
16000 Hz
```

or

22050 Hz

For passive sonar, **16 kHz** is often sufficient unless you're studying very high-frequency signatures.

Step 4: Convert Stereo to Mono

Some recordings have two channels.

Left

Right

Most sonar models use one channel.

Convert

Stereo

↓

Mono

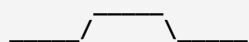
Usually by averaging

Mono = (Left + Right)/2

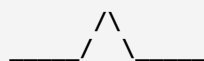
Step 5: Remove DC Offset

Sometimes the waveform is shifted above or below zero.

Bad waveform



Good waveform



Remove the mean value.

signal = signal - mean(signal)

Why?

- Better filtering
- Better FFT
- Better normalization

Step 6: Normalization

Different recordings have different loudness.

Example

Recording A

Amplitude
-0.2 to 0.2

Recording B

Amplitude
-0.9 to 0.9

Normalize both to the same scale.

Common normalization

-1
to
+1

This ensures the model focuses on patterns rather than recording volume.

Step 7: Noise Reduction (Optional)

Ocean recordings contain many background sounds:

- Waves
- Rain
- Wind
- Marine mammals
- Other ships
- Sensor noise

A simple approach is to apply a denoising filter while being careful not to remove useful ship signatures.

Step 8: Band-pass Filtering

Ships produce most of their acoustic energy in certain frequency ranges.

Instead of keeping every frequency,

0 Hz ----- 8000 Hz

retain only the frequencies of interest.

Example

50 Hz

↓

3000 Hz

Benefits

- Removes very low-frequency drift
- Removes high-frequency electronic noise
- Improves signal quality

The exact band depends on your dataset and target ships.

Step 9: Segmentation

A 5-minute recording is too long for a neural network.

Split into fixed-size windows.

Example

5 minute recording

↓

3 second clips

↓

100 clips

Each clip becomes one training sample.

Step 10: Windowing

Before spectral analysis, apply a window function to each segment.

Common choices

- Hamming
- Hann
- Blackman

Example

Raw segment



↓

Hamming Window



Windowing reduces **spectral leakage**, producing cleaner frequency analysis.

Step 11: Feature Extraction (Next Stage)

After preprocessing, convert each segment into representations suitable for machine learning.

Popular options:

- FFT (Fast Fourier Transform)
- STFT (Short-Time Fourier Transform)
- Mel Spectrogram
- MFCC
- LOFARGRAM (widely used in passive sonar)

Complete Initial Pipeline



Concepts you should understand before moving to feature extraction

Make sure you're comfortable with these fundamentals:

1. **Sampling** and sampling rate (Nyquist theorem).
2. **Quantization** and bit depth (e.g., 16-bit PCM).

3. **Digital audio representation** (how a waveform becomes an array of samples).
4. **Time domain vs. frequency domain**.
5. **Fourier Transform** and why it reveals frequency content.
6. **Filtering** (low-pass, high-pass, band-pass).
7. **Windowing** and spectral leakage.
8. **Segmentation** (frame size and overlap).

Once these concepts are clear, you'll be ready to move into **feature extraction**, where you'll learn why **FFT** → **STFT** → **Spectrogram** → **Mel Spectrogram** → **MFCC** → **LOFARGRAM** are used and how each contributes to passive sonar target detection.

ok in first what is the fourier tranform and what its role here

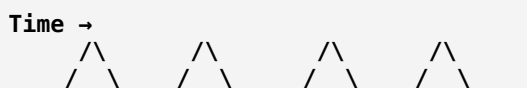
Excellent. Before learning **FFT (Fast Fourier Transform)**, you must first understand **Fourier Transform (FT)**. It is the single most important concept in digital signal processing, especially for sonar.

Let's build the intuition instead of just looking at formulas.

Why Do We Need Fourier Transform?

Imagine your hydrophone records this sound:

Ship Engine Sound



This graph is called the **time-domain signal**.

It tells us:

- How the sound changes over time.
- The amplitude (loudness) at each instant.

But here's the problem:

Can you tell which frequencies are present in this sound just by looking at the waveform?

No.

The waveform does not directly reveal:

- Whether the sound contains a 100 Hz component.
- Whether there's a 500 Hz propeller tone.
- Whether there are harmonics at 1000 Hz.

Think of Music

Imagine someone plays three notes together on a piano:

- C
- E
- G

Your ears hear a **single combined sound**.

But your brain can distinguish the individual notes.

Similarly, a sonar recording is a mixture of many frequencies:

```
Ship Sound
=
Engine Noise
+
Propeller Noise
+
Gearbox Noise
+
Water Noise
+
Wave Noise
+
Marine Life
```

The hydrophone records only the combined signal.

What Does Fourier Transform Do?

Fourier Transform answers this question:

"Which frequencies make up this signal, and how strong is each one?"

It converts:

```
Time Domain
```

into

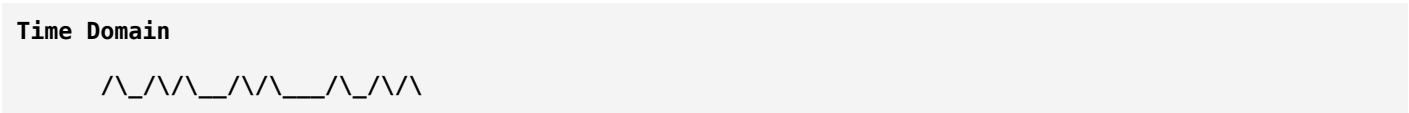
```
Frequency Domain
```

Example

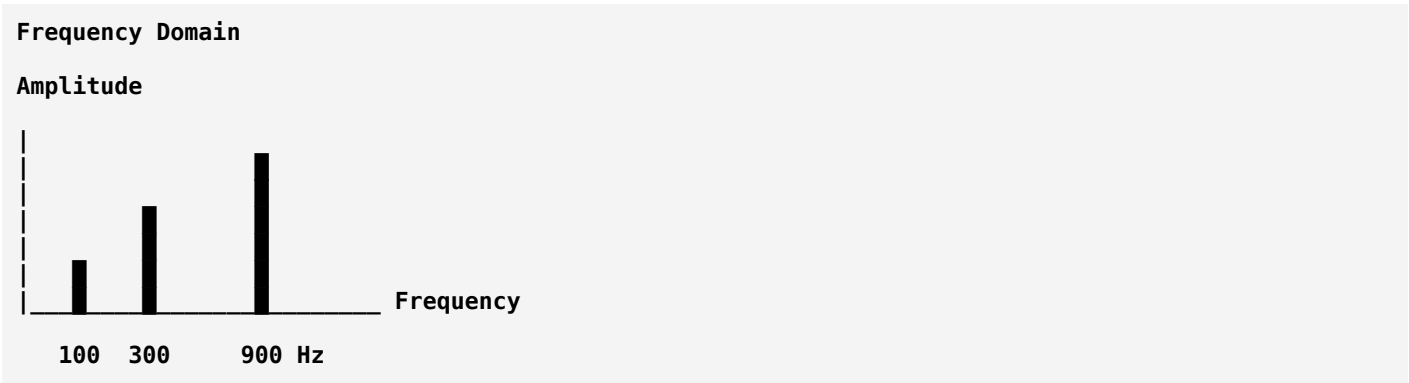
Suppose the recorded waveform is made of:

- 100 Hz engine vibration
- 300 Hz propeller vibration
- 900 Hz machinery vibration

The hydrophone records only one mixed waveform:



After applying the Fourier Transform:



Now you can clearly see the dominant frequencies.

Time Domain vs Frequency Domain

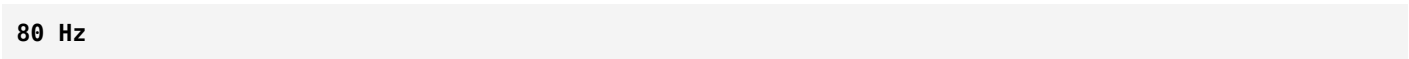
Time Domain	Frequency Domain
Shows amplitude over time	Shows amplitude at each frequency
Easy to hear	Easy to analyze
Complex waveform	Simple frequency peaks
Raw hydrophone output	Output of Fourier Transform

Why Is This Important for Passive Sonar?

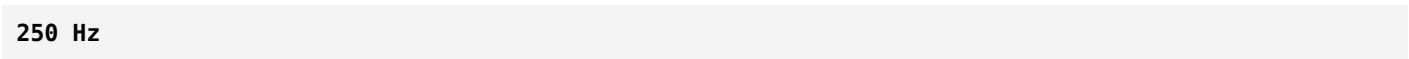
Ships produce characteristic frequencies.

For example:

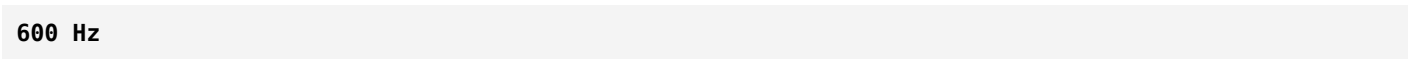
Engine



Propeller



Gearbox



Harmonics

1200 Hz
1800 Hz

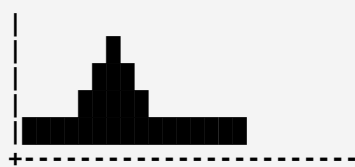
If you only look at the waveform, these are hidden.

Fourier Transform reveals them.

Example: Ship vs Ocean Noise

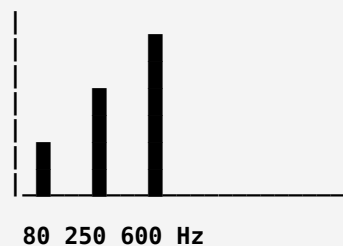
Ocean Noise

Frequency Spectrum



This is spread across many frequencies without strong tonal peaks.

Ship Present



Notice the distinct peaks caused by the ship's machinery.

This difference is what your AI model can learn.

The Mathematical Idea

Any complex signal can be represented as a sum of simple sine and cosine waves.

For example:

Signal

=
100 Hz sine wave
+
300 Hz sine wave
+
900 Hz sine wave

The Fourier Transform decomposes the signal into these individual frequency components.

An Analogy

Imagine a fruit smoothie.

The blender mixes:

- Apples
- Bananas
- Mangoes
- Milk

Once blended, it looks like one drink.

The Fourier Transform is like a magical machine that tells you:

- 2 apples
- 1 banana
- 3 mangoes
- 250 ml milk

Even though you only gave it the smoothie.

Similarly:

Recorded Audio

becomes

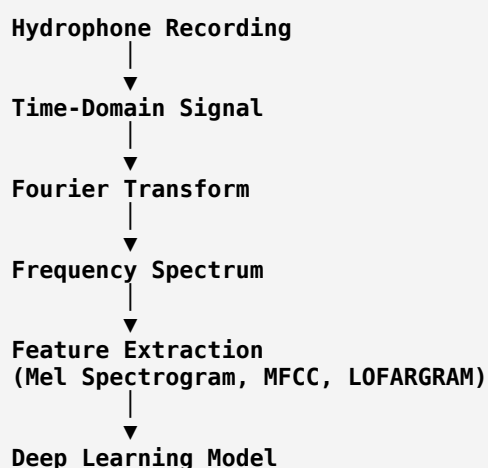
100 Hz → Strong

250 Hz → Medium

600 Hz → Weak

1000 Hz → Strong

Where Fourier Transform Fits in Your Pipeline



Notice that **Mel Spectrograms, MFCCs, and LOFARGRAMs all rely on frequency analysis**, which begins with the Fourier Transform.

Why Not Feed the Raw Waveform to AI?

You could, but for many passive sonar tasks it's less efficient.

A raw waveform mainly tells the model **when** the signal changes.

A frequency representation tells the model **what frequencies** are present, which often correspond directly to physical sources like engines, propellers, and pumps. These frequency patterns are much more informative for distinguishing **"ship present"** from **"ship absent."**

Key Takeaways

- **Time domain** shows how the signal changes over time.
- **Frequency domain** shows which frequencies are present and how strong they are.
- The **Fourier Transform** converts a signal from the time domain to the frequency domain.
- In passive sonar, different ship components generate characteristic frequency signatures.
- These signatures become the basis for advanced features such as **spectrograms, Mel spectrograms, MFCCs, and LOFARGRAMs**.

Next Step

Once you're comfortable with the Fourier Transform, the next concept is **Fast Fourier Transform (FFT)**. FFT is **not a different transform**—it's simply a much faster algorithm for computing the Fourier Transform on digital signals, making it practical for processing long sonar recordings efficiently.

ok tell about fft

Great! Now that you understand **Fourier Transform (FT)**, learning **Fast Fourier Transform (FFT)** becomes much easier.

The most important thing to remember is:

FFT is not a new transform. It is simply a fast algorithm to compute the Fourier Transform on digital data.

Why Was FFT Invented?

Imagine you have a 5-second sonar recording sampled at 16 kHz.

That means:

$$5 \times 16000 = 80,000 \text{ samples}$$

If we compute the Fourier Transform directly (using the Discrete Fourier Transform, DFT), the computer must perform an enormous number of calculations.

The DFT has a time complexity of:

$$O(N^2)$$

where **N** is the number of samples.

For 80,000 samples:

$$80,000 \times 80,000 \\ = 6.4 \text{ billion operations}$$

That's far too slow for practical use.

FFT solves this by reducing the complexity to:

$$O(N \log_2 N)$$

For 80,000 samples, that's roughly:

$$80,000 \times \log_2(80,000) \\ \approx 80,000 \times 16 \\ \approx 1.3 \text{ million operations}$$

Instead of **6.4 billion**, you need only about **1.3 million** operations—a massive speedup.

DFT vs FFT

DFT	FFT
Mathematical transform	Fast algorithm to compute the DFT
Slow	Very fast
$O(N^2)$	$O(N \log N)$
Rarely used directly	Used in almost every DSP application

So when people say:

"Apply FFT"

they actually mean:

"Compute the Fourier Transform using the Fast Fourier Transform algorithm."

How FFT Works (Intuition)

Suppose you have 8 audio samples:

[2, 5, 1, 3, 4, 7, 6, 8]

Instead of analyzing all 8 samples together, FFT uses a **divide-and-conquer** strategy.

It splits the data into:

Original Signal



Even Samples

2 1 4 6

Odd Samples

5 3 7 8

Then it keeps dividing:

Even

2 4

1 6

and

Odd

5 7

3 8

until only very small pieces remain.

It computes the transforms of these small pieces and then combines them efficiently.

This reuse of intermediate calculations is what makes FFT so fast.

If you've studied **Merge Sort**, the idea is similar:

- Divide the problem.
- Solve smaller parts.
- Combine the results.

FFT in Passive Sonar

Suppose your hydrophone records this signal:

Time Domain

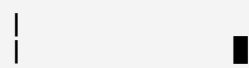
$$\wedge _ \wedge \wedge _ \wedge _ \wedge \wedge _$$

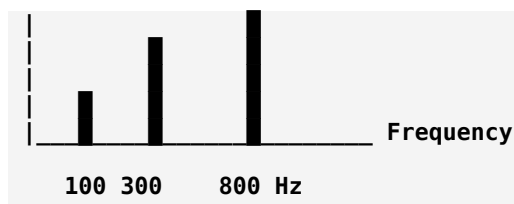
This waveform contains many sounds mixed together.

Applying FFT gives:

Frequency Spectrum

Amplitude





Now you know:

- 100 Hz is present.
- 300 Hz is present.
- 800 Hz is present.

These frequencies often correspond to physical components such as engines, propellers, or pumps.

What Does FFT Output?

Suppose you compute an FFT on a signal.

Instead of another waveform, you get values like:

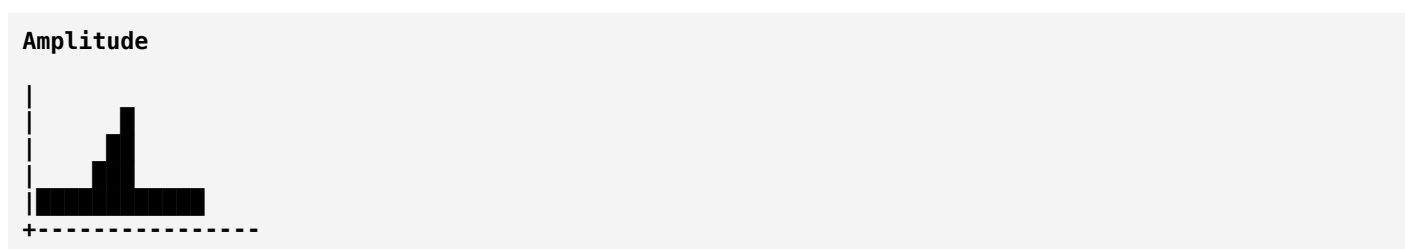
Frequency (Hz)	Magnitude
0	2
50	10
100	80
150	5
200	2
250	65
300	4

The **frequency** tells you *which frequency is present*, and the **magnitude** tells you *how strong that frequency is*.

Plotting these magnitudes gives the frequency spectrum.

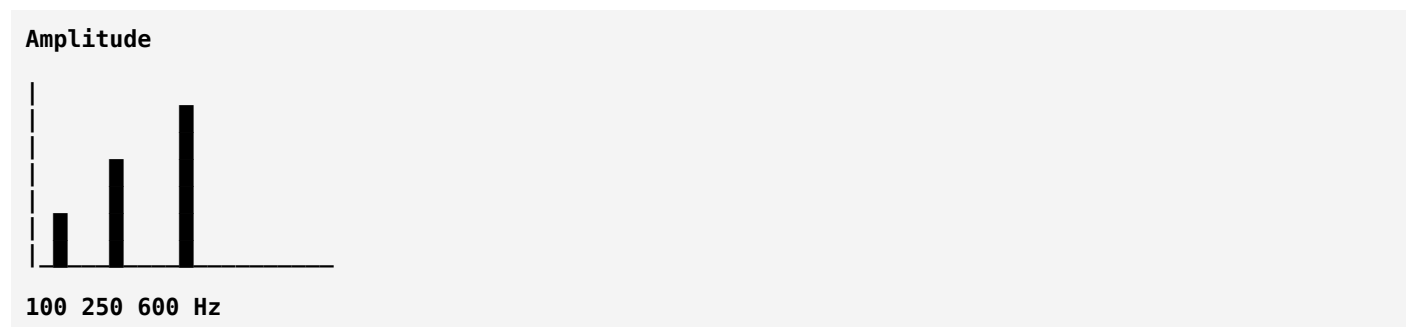
Example: Ship Detection

No Ship



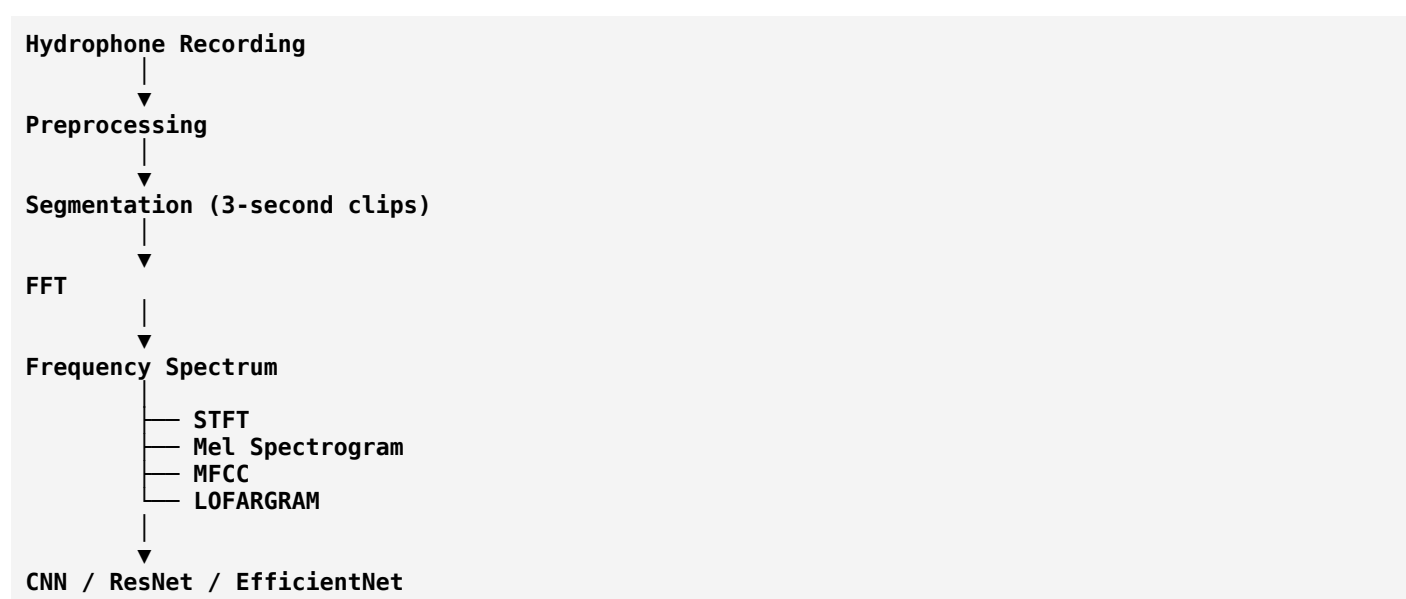
This is mostly random background noise.

Ship Present



Notice the sharp peaks at certain frequencies. These are often called **tonal components**, produced by rotating machinery and engines.

Where FFT Is Used in Your Project



Notice something important:

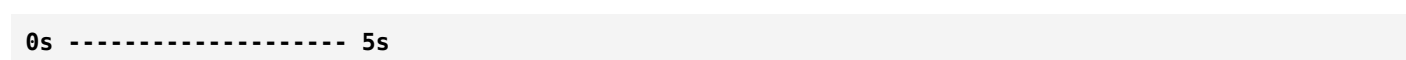
FFT is rarely the final feature used for deep learning.

Instead, FFT is the **building block** for more advanced representations.

Why Isn't FFT Alone Enough?

FFT analyzes the **entire signal at once**.

Imagine a 5-second recording:



If a ship passes only between **2 and 3 seconds**, the FFT will tell you that those frequencies exist, but **not when they occurred**.

This is a limitation because FFT has excellent **frequency resolution** but no **time resolution**.

To solve this, we use the **Short-Time Fourier Transform (STFT)**, which computes FFT over many short overlapping windows, allowing us to see **how the frequency content changes over time**.

Summary

- **Fourier Transform** converts a signal from the time domain to the frequency domain.
- **DFT** is the mathematical method used for digital signals.
- **FFT** is a fast algorithm for computing the DFT.
- FFT reduces computation from **$O(N^2)$** to **$O(N \log N)$** .
- In passive sonar, FFT reveals the frequency signatures produced by ship machinery.
- FFT is the foundation for **STFT, spectrograms, Mel spectrograms, MFCCs, and LOFARGRAMs**.

Next Concept

The next concept is **Short-Time Fourier Transform (STFT)**. It addresses the main limitation of FFT by answering not only "**Which frequencies are present?**" but also "**At what time do those frequencies occur?**" This makes it one of the most important tools in audio and passive sonar analysis.

ok tell me about stft

Excellent. **STFT (Short-Time Fourier Transform)** is one of the most important concepts in audio signal processing. If you understand STFT well, you'll naturally understand **spectrograms, Mel spectrograms, MFCCs, and LOFARGRAMs** because they are all built on top of it.

Let's build the intuition first.

Why Do We Need STFT?

We learned that FFT tells us:

"Which frequencies are present in the signal?"

But it cannot tell us:

"When did those frequencies occur?"

This is a major problem for sonar.

Example

Suppose you have a **6-second** passive sonar recording.

Time

0s-----6s

During these 6 seconds:

- 0–2 s → Only ocean noise
- 2–4 s → Ship passes by
- 4–6 s → Ocean noise again

Timeline:

0 2 4 6
Noise | Ship | Noise

If you apply **one FFT** to the entire 6-second signal:

Entire Signal
↓
FFT

Output:

100 Hz
250 Hz
600 Hz

The FFT tells you these frequencies exist somewhere in the recording.

But it **does not tell you**:

- Did they appear at 1 second?
- At 3 seconds?
- At 5 seconds?

It loses all timing information.

The Solution: STFT

Instead of analyzing the whole signal at once, STFT analyzes **small pieces (frames)** of the signal.

Think of watching a movie.

A movie is not one giant image.

It is made of many frames:

Movie
Frame1
Frame2
Frame3
Frame4
...

Similarly, STFT divides audio into many short frames.

Step 1: Split the Audio

Suppose we have:

6 seconds

Divide it into:

0–1 s

1–2 s

2–3 s

3–4 s

4–5 s

5–6 s

Each frame is analyzed separately.

Step 2: Apply FFT to Each Frame

Frame 1

FFT

↓

Spectrum 1

Frame 2

FFT

↓

Spectrum 2

Frame 3

FFT

↓

Spectrum 3

Continue until the end.

STFT Is Simply

STFT
=
Many FFTs

Each FFT is performed on a short time window.

That's why it's called:

Short-Time Fourier Transform

Visual Representation

Original waveform:

Time →
/\ /\ //\ /\ /_

Sliding window:

```
[#####]
  [#####]
    [#####]
      [#####]
```

Each window:

FFT

The results are stacked together.

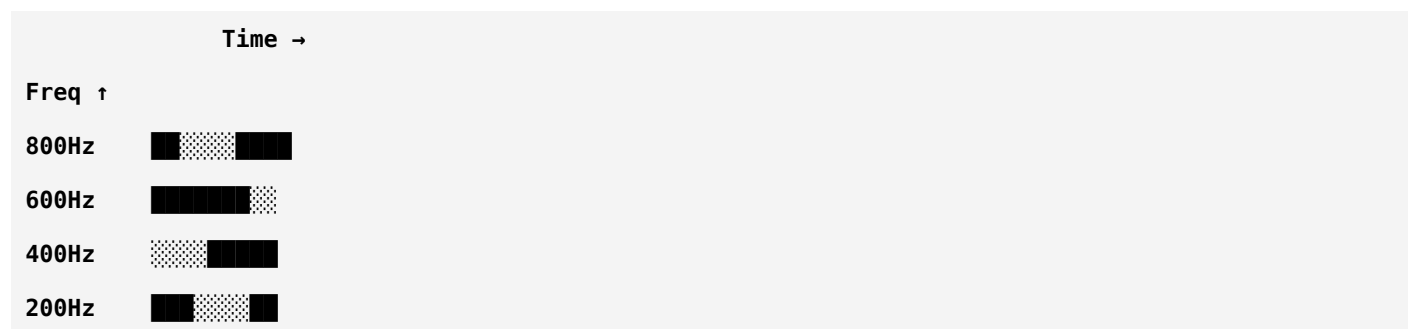
Final STFT Output

Instead of one spectrum,
we obtain **many spectra**.

Time →
Spectrum1
Spectrum2
Spectrum3
Spectrum4
Spectrum5

These are combined into a 2D representation.

STFT Matrix



Rows

↓

Frequency

Columns

↓

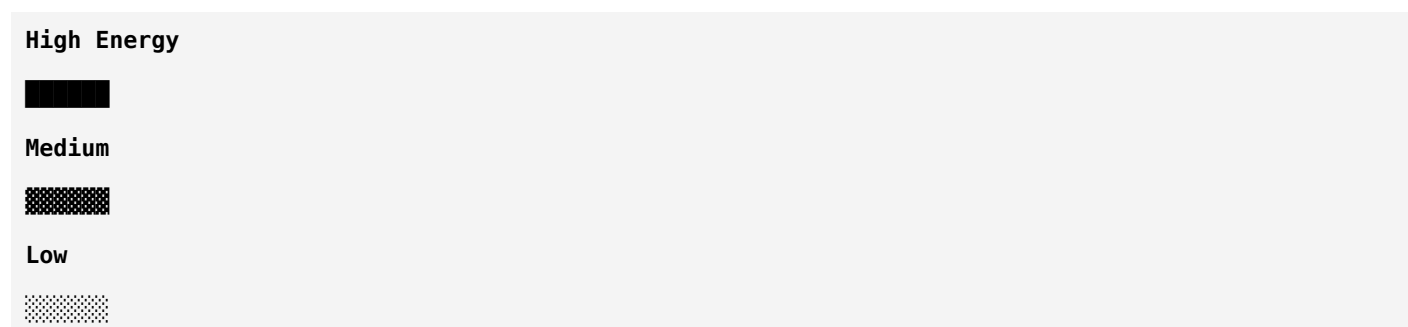
Time

Each cell represents:

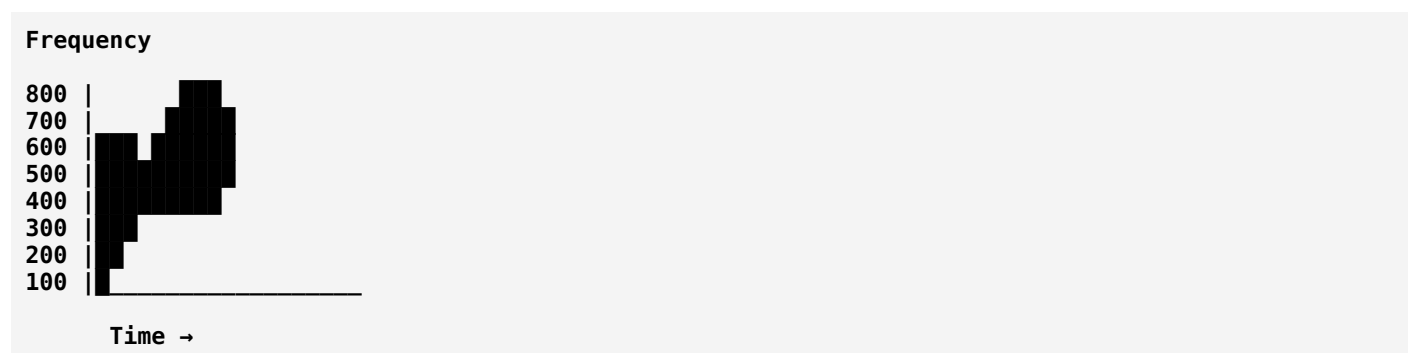
How much energy a particular frequency has at a particular time.

Spectrogram

If we color the STFT matrix,
we obtain a **spectrogram**.



Example:



This image is exactly what CNNs like to process.

Ship Detection Example

Suppose

0–2 sec

Ocean

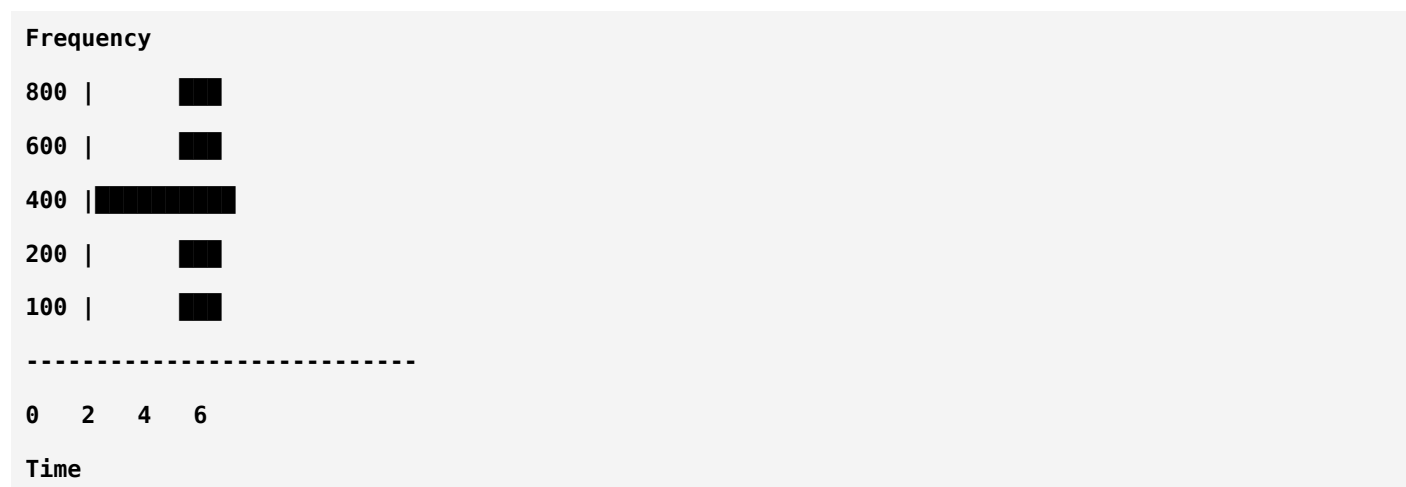
2–4 sec

Ship

4–6 sec

Ocean

The spectrogram might look like



Notice

Between

2–4 sec

new frequency bands appear.

That tells us

A ship is present only during that interval.

FFT cannot tell us this.

STFT can.

Window Size

This is an important concept.

Suppose your window is

50 ms

You get:

- Excellent time resolution
- Poor frequency resolution

Suppose the window is

500 ms

You get:

- Excellent frequency resolution
- Poor time resolution

This is called the **time-frequency trade-off**.

Example

Small window



You know exactly **when** something happened, but frequency estimates are less precise.

Large window



You know the frequencies more accurately, but not the exact timing.

Window Overlap

STFT usually uses overlapping windows.

Example

Window 1

#####

Window 2

#####

Window 3

#####

Common overlap

50%

or

75%

Why?

Because it prevents sudden changes between adjacent windows and produces a smoother, more continuous spectrogram.

STFT in Your Passive Sonar Project

Hydrophone Recording

↓

Preprocessing

↓

Segmentation

↓

Windowing (Hamming)

↓

STFT

↓

Spectrogram

↓

Mel Spectrogram

↓

CNN

Notice something important:

Mel Spectrogram = STFT + Mel Filter Bank

So if you understand STFT, you've already understood the core computation behind Mel spectrograms.

FFT vs STFT

FFT	STFT
One FFT for the entire signal	Many FFTs on short windows
Shows frequencies present	Shows frequencies over time
Output is a 1D frequency spectrum	Output is a 2D time-frequency matrix
No timing information	Preserves timing information
Good for stationary signals	Better for changing signals like speech and sonar

Why STFT Is Important for Ship Classification

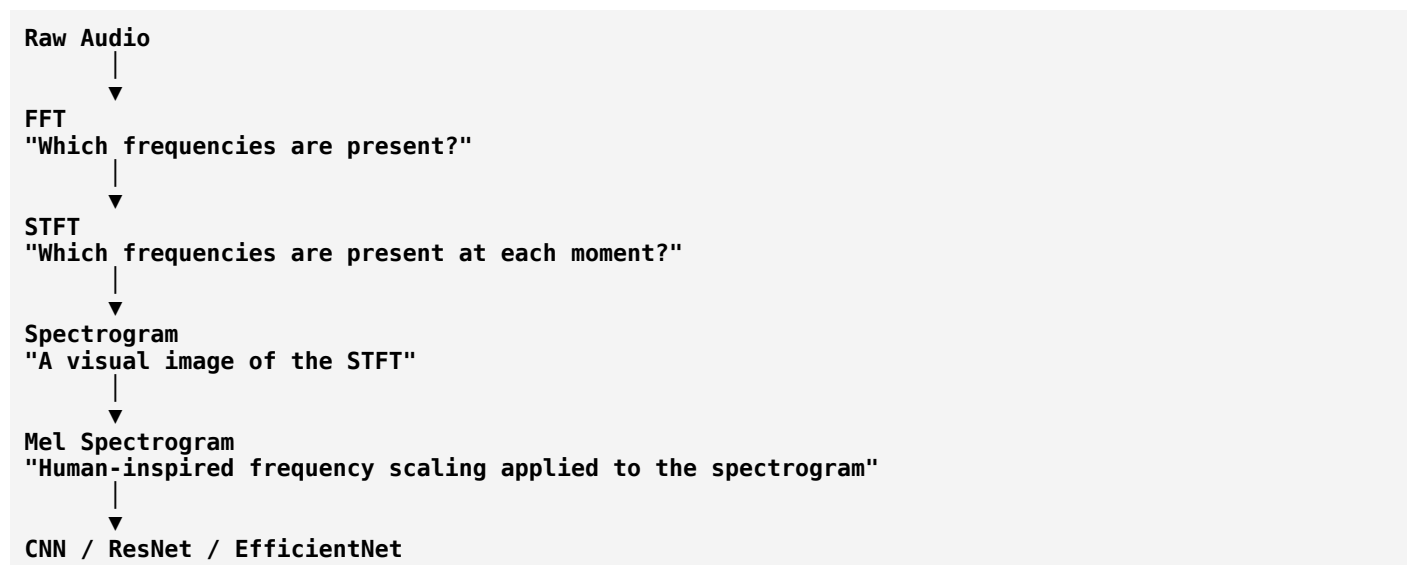
A ship's acoustic signature changes over time because of:

- Propeller speed variations.
- Engine load changes.
- Doppler shifts as the ship moves.
- Background ocean noise fluctuations.

STFT captures **both frequency and timing**, allowing your model to distinguish these dynamic patterns instead of seeing only an average frequency spectrum.

Summary

Think of the progression like this:



For your project, the next natural topic is **Spectrograms**. You'll see that a spectrogram isn't a new transform—it's simply a **visual representation of the STFT**, and understanding that makes the rest of the feature extraction pipeline much easier.

ok tell about spectrograms

Perfect. Now that you understand **FFT** and **STFT**, **Spectrograms** become very easy.

In fact, many people think STFT and Spectrogram are different, but they are closely related.

STFT is the computation.

Spectrogram is the visualization of the STFT.

Let's understand this step by step.

What is a Spectrogram?

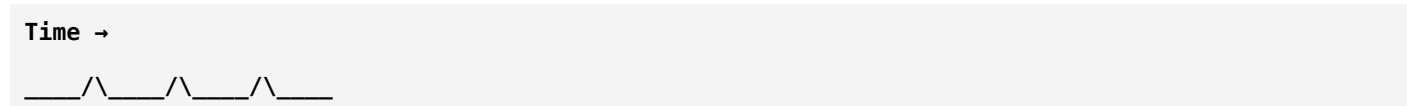
A **spectrogram** is a **2D image** that shows:

- **Time** (X-axis)
- **Frequency** (Y-axis)
- **Energy/Intensity** (Color or Brightness)

Instead of looking at the waveform, we look at **how the frequency content changes over time**.

Why Do We Need a Spectrogram?

Suppose your hydrophone records this signal:



This waveform doesn't tell us:

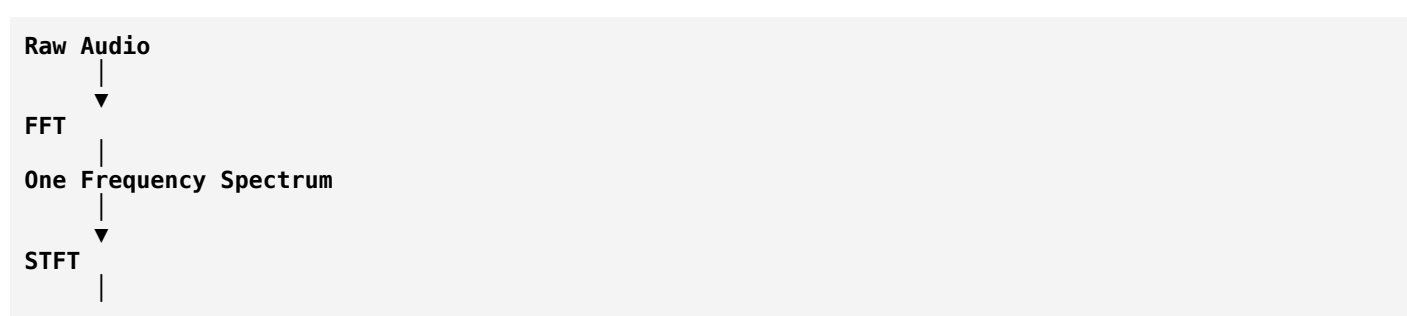
- Which frequencies are present?
- When they appear?
- Which frequencies are strongest?

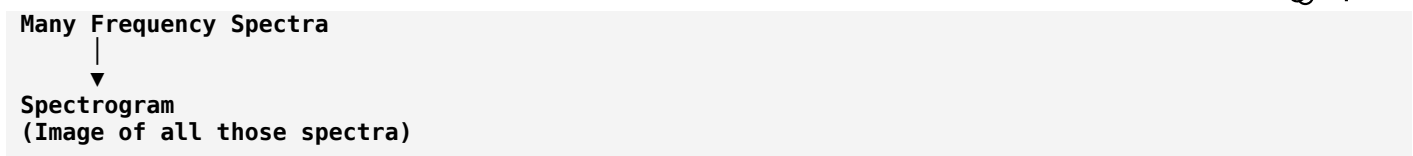
After applying STFT, we get frequency information for every short window.

Instead of reading thousands of numbers, we convert them into an image.

That image is the **spectrogram**.

Relationship Between FFT, STFT, and Spectrogram





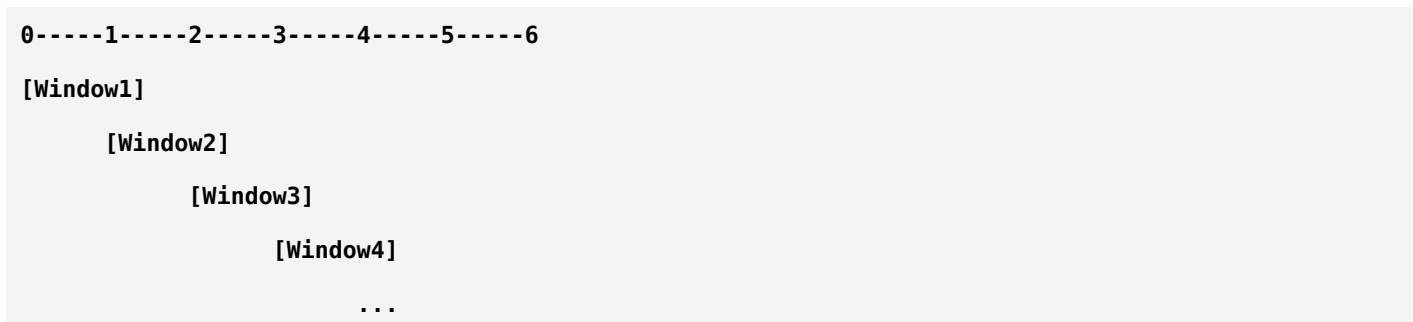
Think of it like this:

- **FFT** → One photograph
- **STFT** → Many photographs over time
- **Spectrogram** → A movie strip made from those photographs

How is a Spectrogram Created?

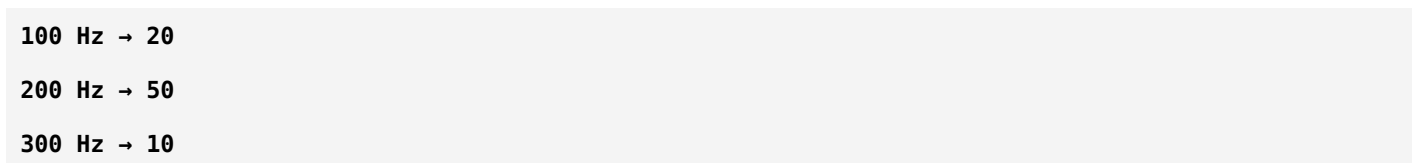
Suppose we have a 6-second audio.

Step 1: Split into windows

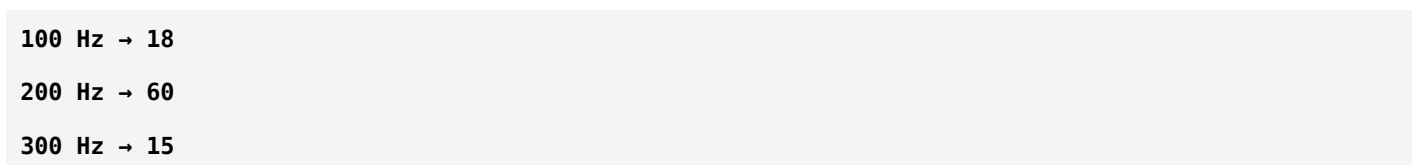


Step 2: FFT on every window

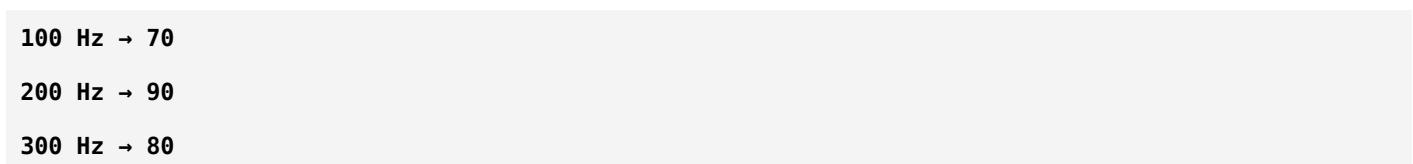
Window 1



Window 2



Window 3



Each window produces one frequency spectrum.

Step 3: Arrange them as columns

	Time →
Freq	
300Hz	10 15 80 ...
200Hz	50 60 90 ...
100Hz	20 18 70 ...

This matrix is the STFT magnitude.

Step 4: Convert numbers into colors

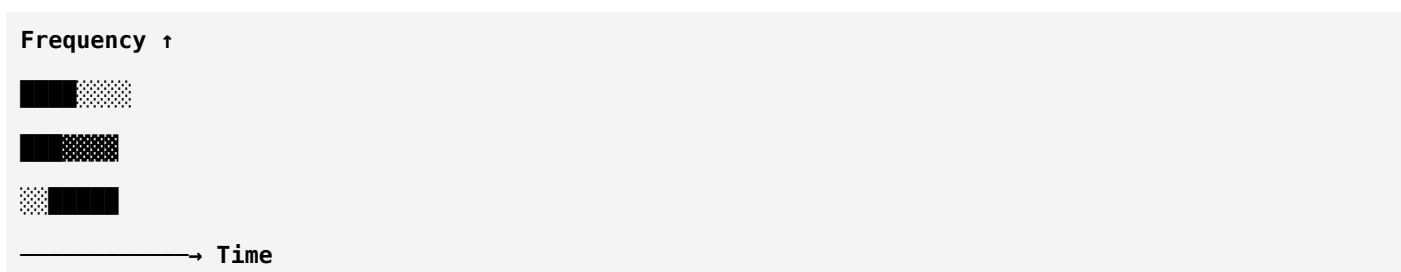
Suppose

0 = Black
50 = Gray
100 = White

Then

10 → Dark
50 → Gray
90 → Bright

The result becomes



This image is called a **spectrogram**.

Understanding the Axes

	Time →
	0s 1s 2s 3s 4s
800Hz	
600Hz	
400Hz	
200Hz	

100Hz

X-axis

Represents

Time

Y-axis

Represents

Frequency

Color

Represents

Signal Energy

or

Magnitude

Bright color

↓

Strong frequency

Dark color

↓

Weak frequency

Real Passive Sonar Example

Suppose

0–2 sec

Ocean Noise

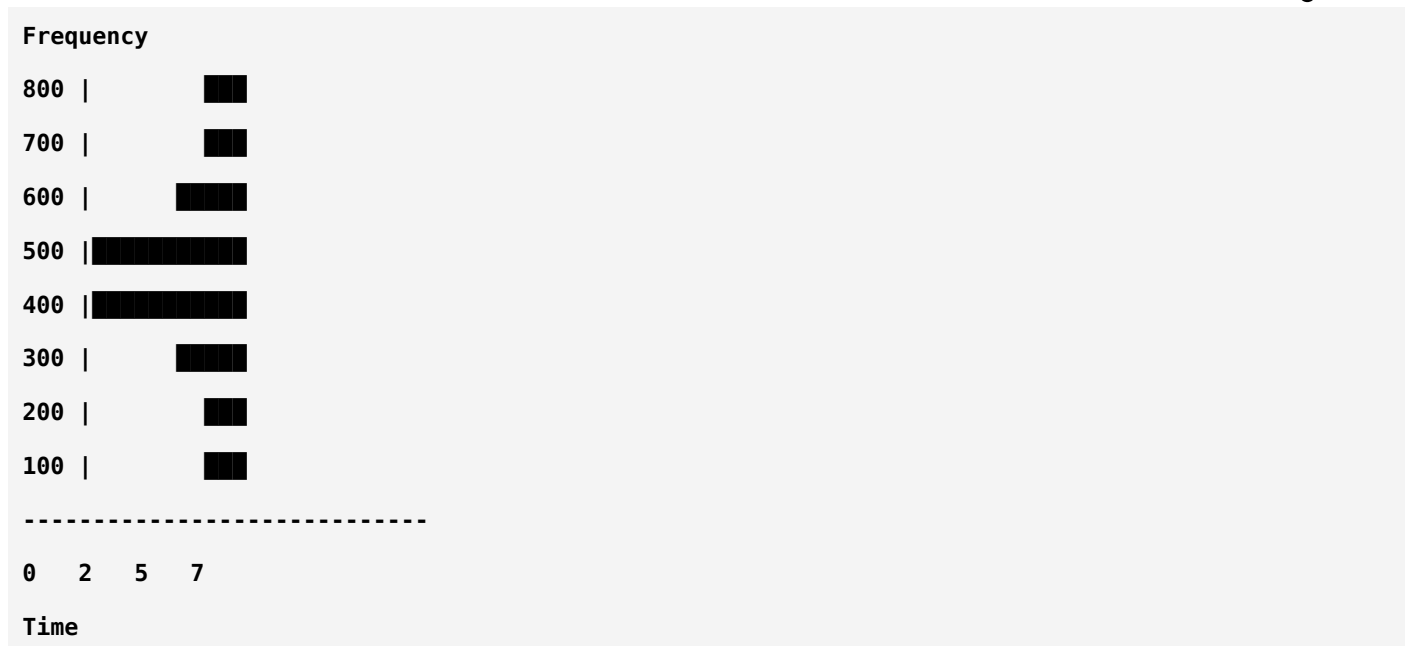
2–5 sec

Ship Passing

5–7 sec

Ocean Noise

The spectrogram might look like



Notice

Only during

2–5 sec

new frequencies appear.

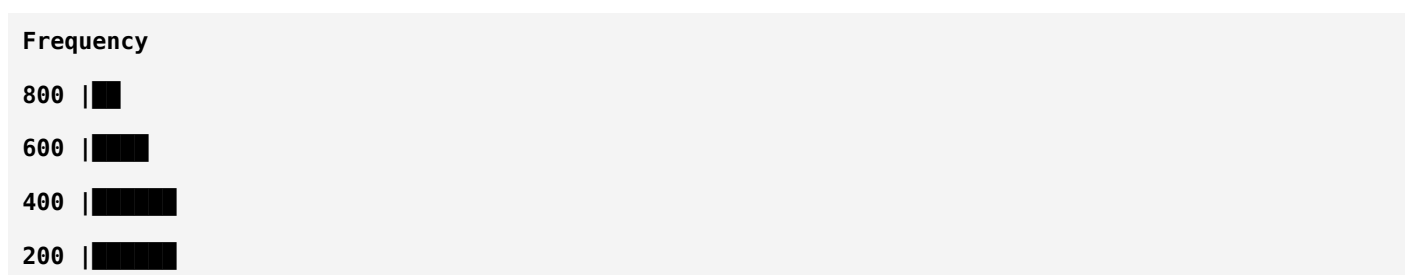
This indicates a ship is present.

Ship Signature

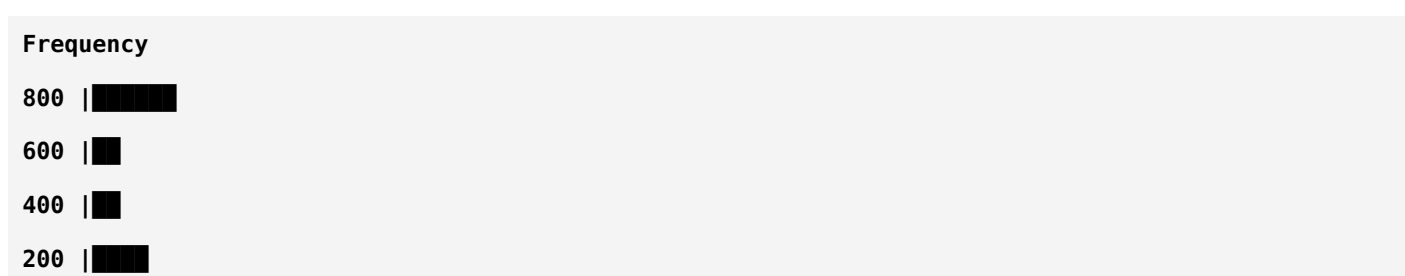
Different ships create different frequency patterns.

For example

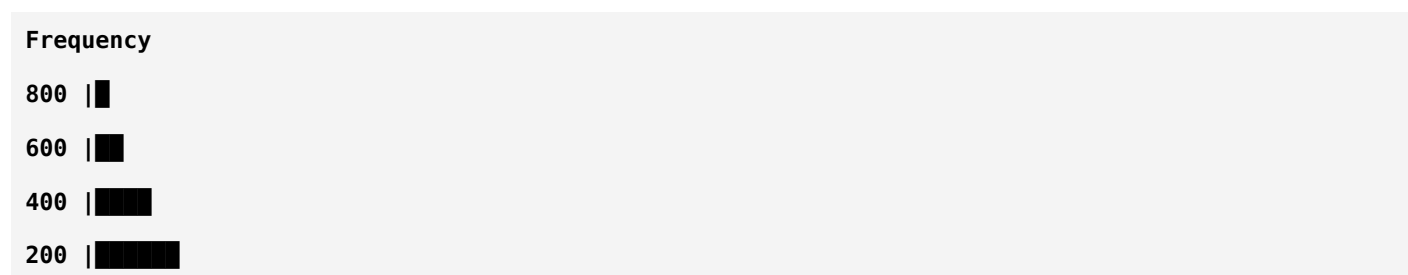
Cargo Ship



Fishing Boat



Submarine



A CNN can learn these visual patterns just like it learns patterns in photographs.

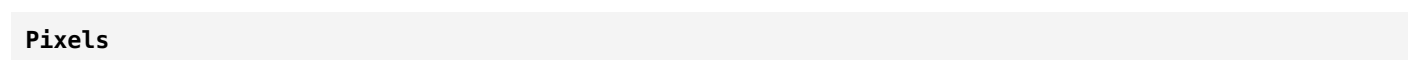
Why CNN Works So Well

A CNN does not know anything about sound.

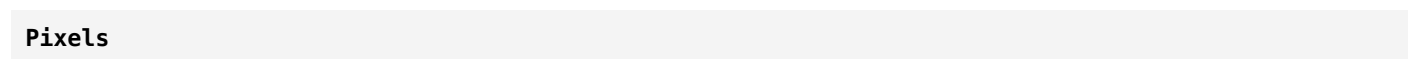
It only sees images.

Example

Cat Image



Spectrogram



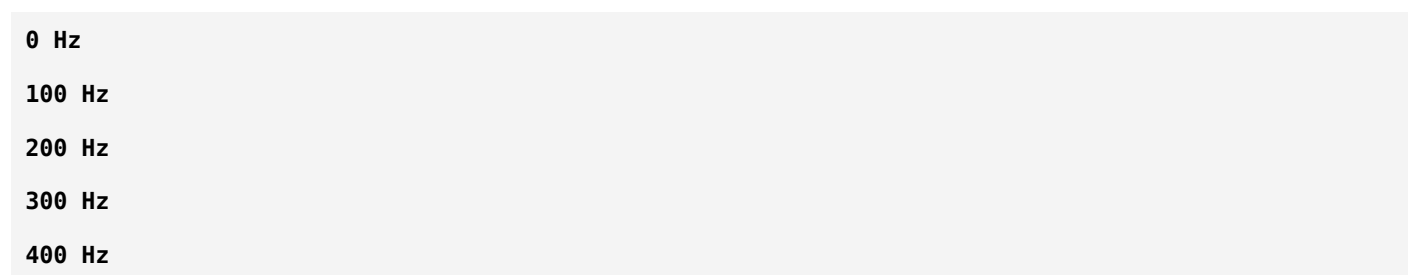
For the CNN, both are simply matrices of pixel values.

This is why spectrogram-based classification is so successful in audio tasks.

Spectrogram Types

1. Linear Spectrogram

Uses the original frequency scale.



Every frequency interval is equally spaced.

2. Log Spectrogram

Instead of raw magnitude

Magnitude

we display

Log(Magnitude)

Why?

Suppose

Frequency

100 Hz → 5

200 Hz → 1000

Without a logarithm

5

1000

The smaller signal is almost invisible.

Using a logarithmic scale makes both weak and strong components visible.

Most real-world audio systems use **log-magnitude spectrograms** because they reveal subtle details.

Where Spectrogram Fits

Raw Audio

↓

Preprocessing

↓

Windowing

↓

STFT

↓

Magnitude

↓

Log

↓

Spectrogram

↓

Mel Filter

↓

Mel Spectrogram

↓

CNN

Notice

Spectrogram is simply

$\text{Magnitude}(\text{STFT})$

Sometimes

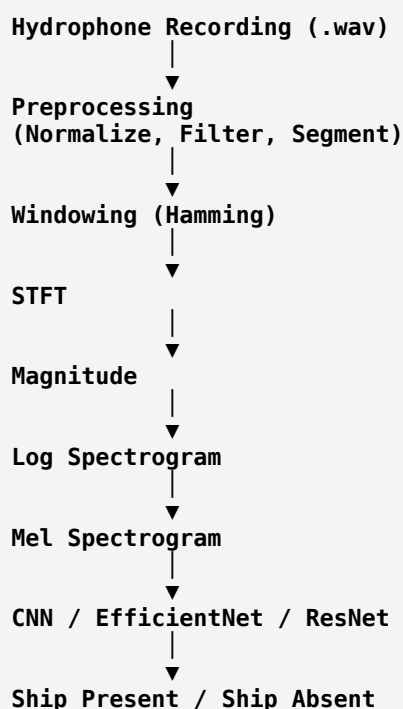
$\text{Log}(\text{Magnitude}(\text{STFT}))$

FFT vs STFT vs Spectrogram

FFT	STFT	Spectrogram
One transform on the whole signal	Many FFTs over short windows	Image of the STFT magnitude
Output: 1D frequency spectrum	Output: 2D time-frequency matrix	Output: 2D image
No timing information	Preserves timing	Easy for humans and CNNs to interpret

Spectrogram in Your Passive Sonar Project

Your processing pipeline will look like this:



Key Takeaways

- A **spectrogram is not a new transform**; it is a **visual representation of the STFT**.
- It answers three questions at once:
 - **When** did something happen? (Time)
 - **What frequency** was present? (Frequency)
 - **How strong** was it? (Color intensity)
- Spectrograms convert audio into an image, making them ideal inputs for deep learning models such as CNNs.

What's Next?

The next topic is **Mel Spectrogram**, which is the feature representation most commonly used in audio AI. It takes a regular spectrogram and transforms the frequency axis into the **Mel scale**, creating a representation that is often more compact and effective for machine learning. While the Mel scale was inspired by human hearing, it can also be a useful feature representation for sonar because it emphasizes broader spectral patterns and reduces dimensionality. We'll also discuss when a **standard spectrogram** or **LOFARGRAM** may be more appropriate for passive sonar applications.

ok next tell about Mel spectrogram

Excellent! Now we've reached one of the most important features in audio AI.

So far you've learned:

```
Raw Audio
  ↓
FFT
  ↓
STFT
  ↓
Spectrogram
```

Now we'll learn:

```
Spectrogram
  ↓
Mel Filter Bank
  ↓
Mel Spectrogram
```

Notice something important:

A Mel Spectrogram is NOT a completely new transform.

It is simply a **Spectrogram** whose frequency axis has been transformed using the **Mel scale**.

Why Do We Need Mel Spectrogram?

Suppose our STFT produces frequencies from

```
0 Hz ----- 8000 Hz
```

Imagine there are **256 frequency bins**.

```
0
31
62
93
124
...
8000 Hz
```

Every frequency interval is equally spaced.

This is called a **Linear Frequency Scale**.

Problem

Suppose these frequencies are present:

```
100 Hz
110 Hz
120 Hz
130 Hz
```

and

```
6000 Hz
6010 Hz
6020 Hz
6030 Hz
```

Do we really need to distinguish every 10 Hz difference at high frequencies?

Often, **no**.

Many nearby frequencies contain very similar information.

Instead of keeping every frequency separately, we can group them.

This reduces:

- Noise
- Data size
- Model complexity

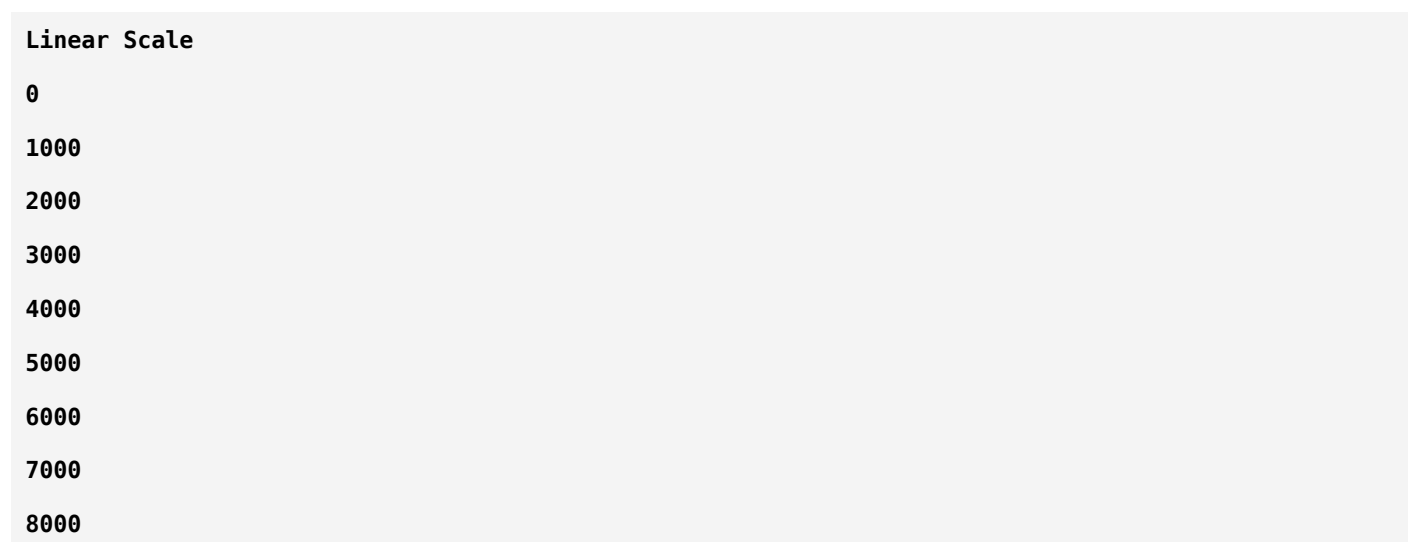
What is the Mel Scale?

The **Mel scale** is a frequency scale that is:

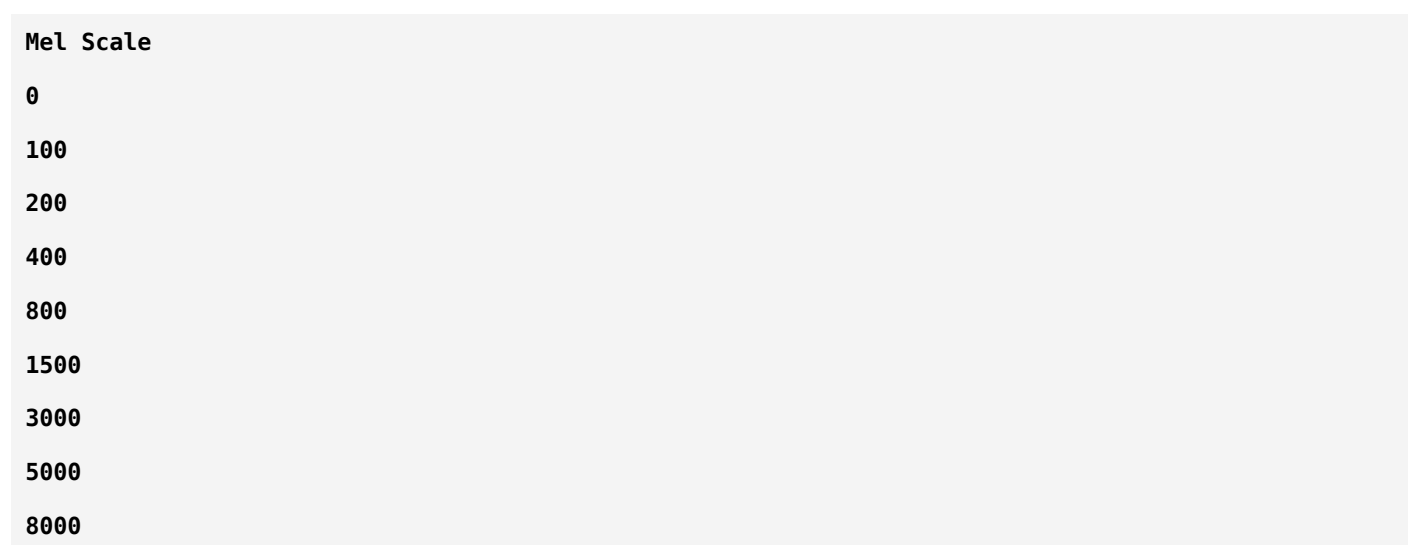
- More detailed at **low frequencies**

- More compressed at **high frequencies**

Instead of



we get something like



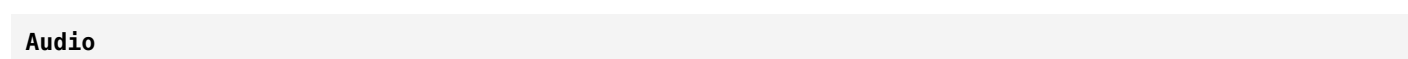
Notice:

- Low frequencies are represented with finer resolution.
- High frequencies are grouped into wider bands.

How is a Mel Spectrogram Created?

Step 1

Load audio



Step 2

Compute STFT

Time × Frequency Matrix



Step 3

Apply Mel Filter Bank



Step 4

Take logarithm



Step 5

Mel Spectrogram

What is a Mel Filter Bank?

This is the most important idea.

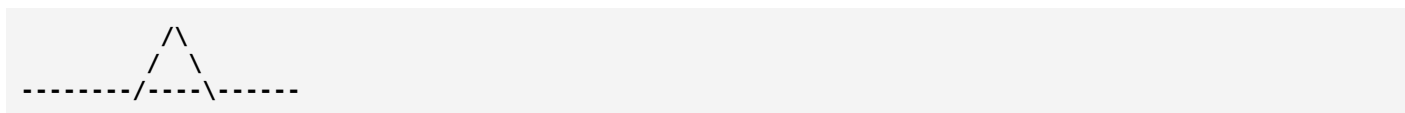
Suppose the STFT gives:

0
100
200
300
400
500
600
700
800 Hz

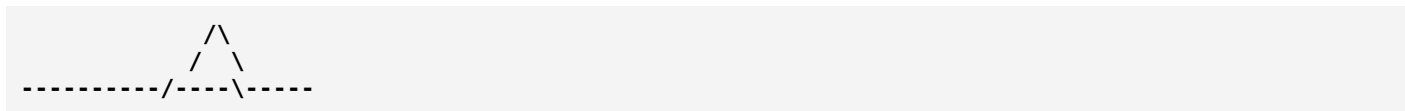
Instead of keeping every frequency separately,
we place overlapping triangular filters over the spectrum.
Example:



Another filter



Another



Each triangle combines nearby frequencies into one value.

Instead of:

256 frequency bins

we might keep only:

64 Mel bands

or

128 Mel bands

Visual Example

Suppose STFT gives:

Frequency	Energy
100	10
200	25
300	30
400	20
500	15
600	40
700	35
800	10

Instead of keeping all eight values,

Mel filters combine them:

Mel Band 1

100–250 Hz

↓

Average Energy

18

Mel Band 2

250–450 Hz

↓

Average

25

Mel Band 3

450–700 Hz

↓

Average

30

Now we have fewer, smoother values.

Mel Spectrogram

Normal Spectrogram

Frequency

800

700

600

500

400

300

200

100

Mel Spectrogram

Mel Band

64

63

62

...

2

1

Instead of frequency values,
the Y-axis now represents **Mel bands**.

Why Use Log Scale?

After applying Mel filters,
we take

Log (Energy)

Suppose

Energy

5

5000

Without logarithm
the smaller value becomes almost invisible.

Using logarithms

Log (5)

Log (5000)

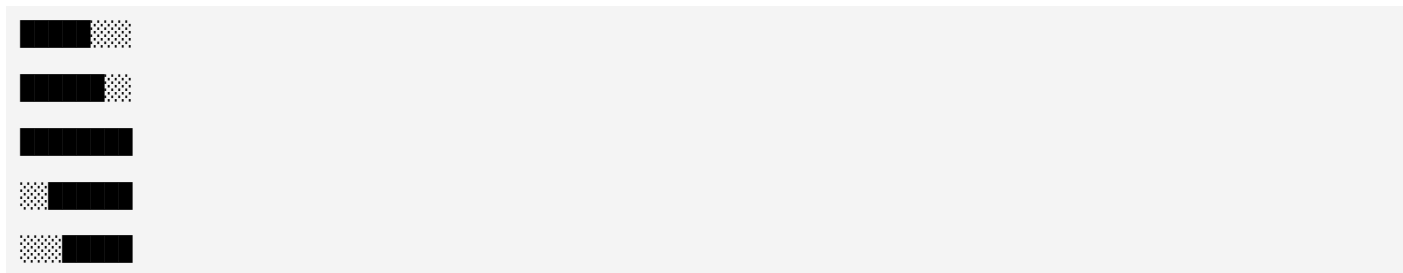
compresses the dynamic range, making both weak and strong components easier to analyze.

Complete Pipeline

```
Raw Audio
  ↓
Preprocessing
  ↓
Windowing
  ↓
STFT
  ↓
Magnitude
  ↓
Mel Filter Bank
  ↓
Log
  ↓
Mel Spectrogram
```

Why CNN Likes Mel Spectrograms

Suppose this is your Mel Spectrogram.



To a CNN,

this is just an image.

The CNN learns patterns like:

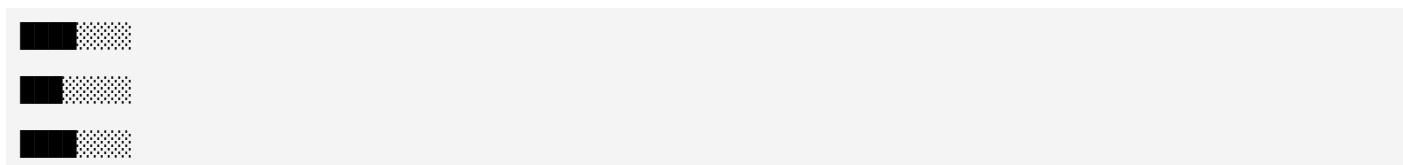
- Horizontal lines
- Vertical lines
- Bright regions
- Shapes

Different ships produce different Mel Spectrogram patterns.

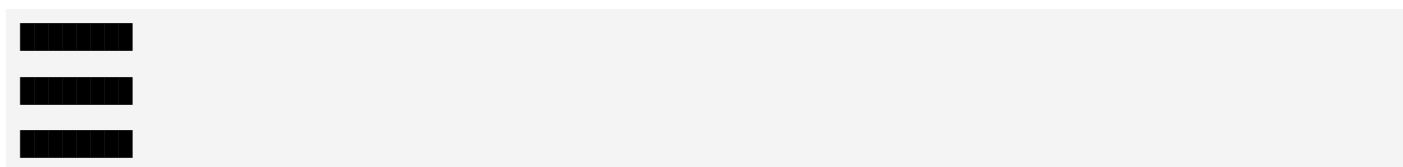
Mel Spectrogram in Ship Detection

Suppose

Ocean Noise



Ship Present



The CNN learns these visual differences.

Is Mel Spectrogram Good for Passive Sonar?

This is a very important question.

For **speech recognition**, Mel spectrograms are excellent because the Mel scale is designed to approximate how humans perceive pitch.

For **passive sonar**, the answer is more nuanced:

Advantages

- Reduces the number of frequency bins (lower dimensionality).
- Smooths noisy spectra.
- Often works well with CNN-based classifiers.
- Computationally efficient.

Disadvantages

A ship's machinery produces **precise tonal frequencies** (for example, engine harmonics and propeller blade rates). Compressing the frequency axis with Mel filters can reduce the resolution of these narrowband features, potentially losing information that is valuable for sonar analysis.

Because of this, many passive sonar systems use:

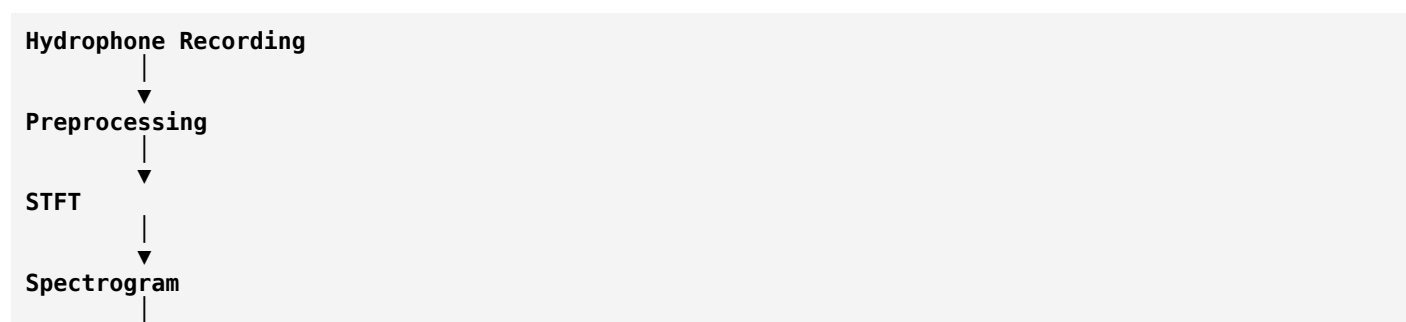
- **STFT Spectrograms**
- **LOFARGRAMs**
- **Constant-Q Transform (in some cases)**

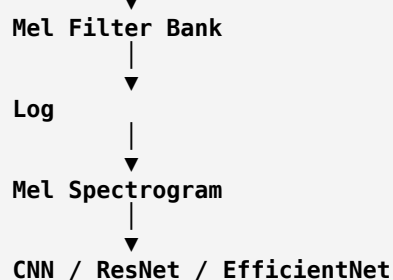
instead of, or alongside, Mel spectrograms.

Comparison

Feature	FFT	STFT	Spectrogram	Mel Spectrogram
Output	Frequency Spectrum	Time-Frequency Matrix	Image	Compressed Image
Time Information	✗	✓	✓	✓
Frequency Scale	Linear	Linear	Linear	Mel (nonlinear)
Good for CNN	✗	✗	✓	✓
Widely Used	DSP	DSP	Audio AI	Speech & Audio AI

Where Does Mel Spectrogram Fit?





For Your Passive Sonar Project

Since your goal is **binary classification (Ship Present vs Ship Absent)**, I recommend experimenting with multiple feature representations rather than assuming one is best:

1. **Log Spectrogram** (strong baseline).
2. **Mel Spectrogram** (compact and CNN-friendly).
3. **LOFARGRAM** (traditionally favored in passive sonar because it preserves narrowband tonal signatures).

You can train the same CNN or EfficientNet model on each representation and compare accuracy, precision, recall, and F1-score. This comparison itself can become a valuable part of your project or research paper.

Next Step

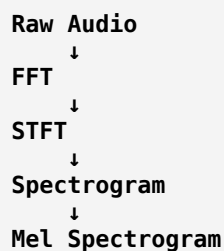
The next concept is **MFCC (Mel-Frequency Cepstral Coefficients)**.

MFCCs are **not images** like spectrograms. Instead, they compress a Mel spectrogram into a small set of numerical coefficients that summarize the spectral characteristics of each audio frame. They're widely used in speech recognition and can also be evaluated as features for passive sonar classification.

next MFcc

Excellent! You're now reaching the most advanced traditional audio feature. Once you understand **MFCC**, you'll have covered the core digital audio processing pipeline used in many audio AI systems.

So far you've learned:



Now we move to:



DCT (Discrete Cosine Transform)



MFCC

Notice something important:

MFCC is NOT another spectrogram.

It is a **compressed numerical representation** of a Mel Spectrogram.

Why Do We Need MFCC?

Suppose your Mel Spectrogram has

128 Mel Bands

For every time frame, you have

128 numbers

If your recording has 500 time frames,
your data size becomes

**500 × 128
=
64,000 values**

Many of these values are redundant because neighboring Mel bands are highly correlated.
Instead of storing all 128 values, MFCC compresses them.

The Main Idea

Think of a photograph.

Original image

4000 × 3000 pixels

Compress it into JPEG.

The image still looks almost the same,
but occupies much less memory.

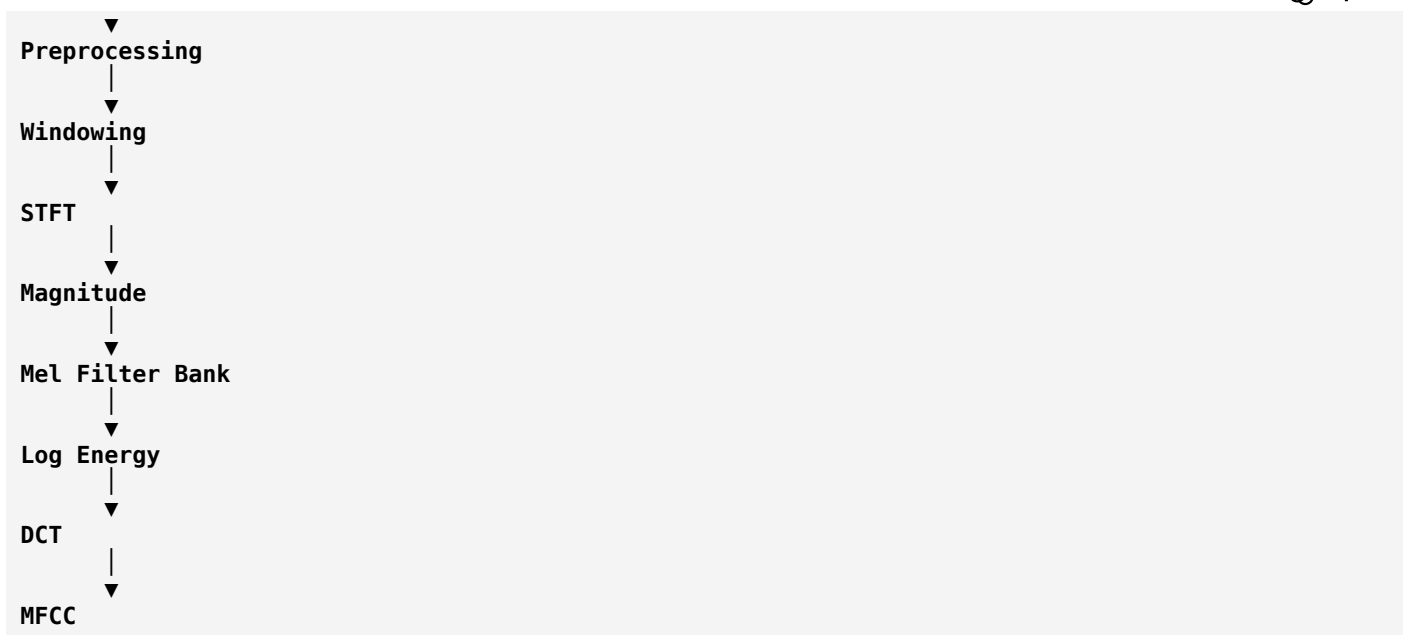
MFCC does something similar.

It compresses the Mel Spectrogram while preserving its most important information.

Complete MFCC Pipeline

Raw Audio





Everything is the same as the Mel Spectrogram pipeline until the final step.
The only new operation is the **Discrete Cosine Transform (DCT)**.

What is DCT?

Suppose the Mel Spectrogram gives

```
Frame 1
[10,12,15,16,17,19,20,22]
```

These numbers are very similar.
Instead of keeping all of them,
DCT transforms them into

```
[55,8,2,1,0,0,0,0]
```

Notice
Most important information is concentrated in the first few numbers.
The remaining coefficients are very small.
Therefore,
instead of storing

```
128 coefficients
```

we keep only

```
13 coefficients
```

This is the key idea behind MFCC.

Why DCT?

The DCT "decorrelates" the Mel-band energies.

Imagine you have marks in similar subjects:

Subject	Marks
Physics	90
Chemistry	91
Electronics	92

These values are closely related.

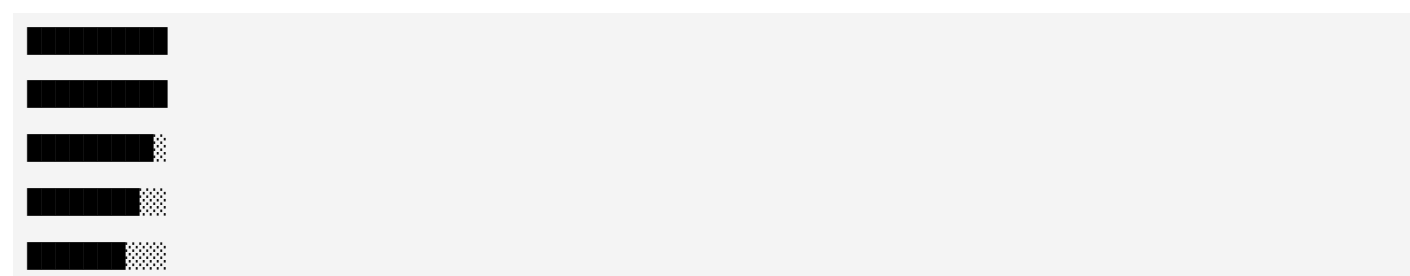
Instead of storing all three separately, you might summarize them as:

- Average ≈ 91
- Small differences

Similarly, DCT summarizes correlated Mel-band energies into a compact set of coefficients.

Visual Example

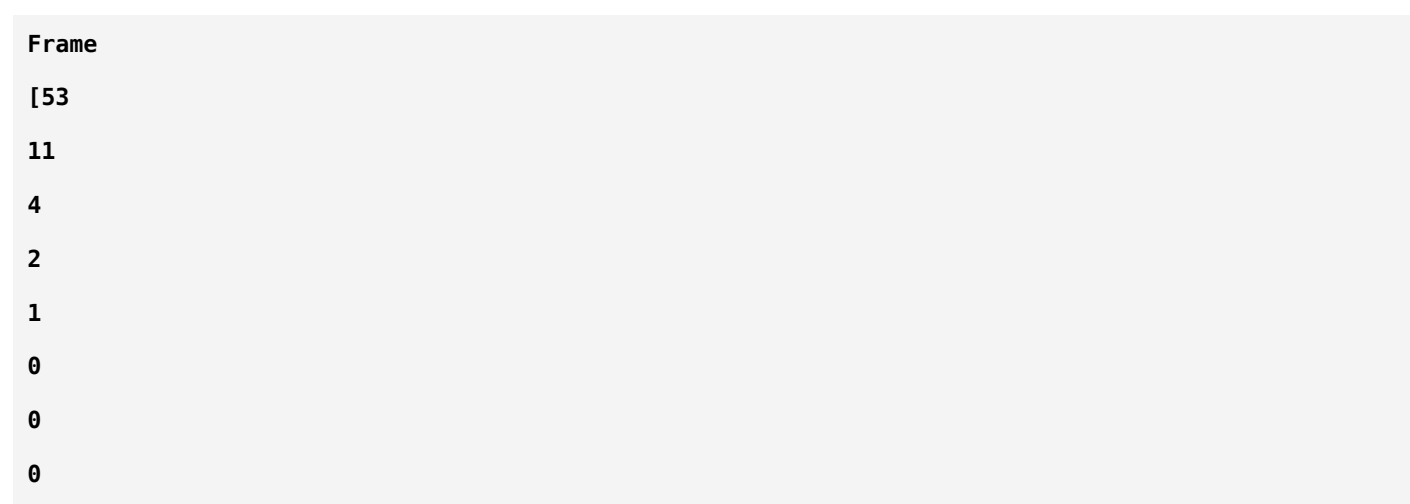
Mel Spectrogram

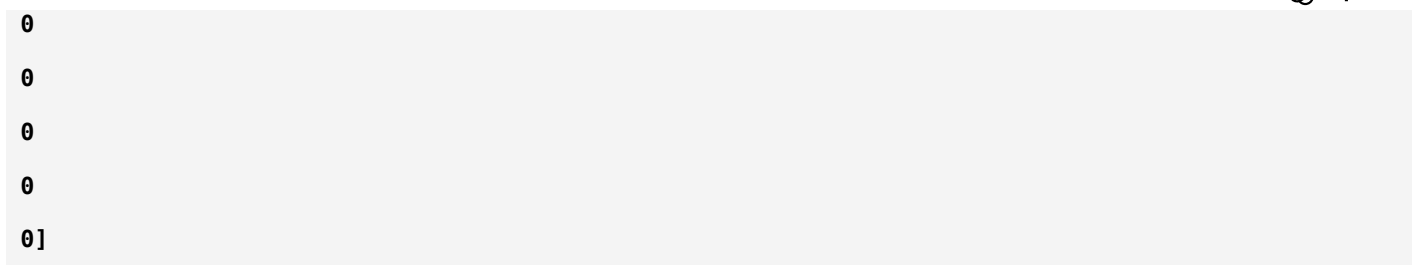


Each frame contains

128 values

After DCT





Only the first coefficients carry most of the useful information.

What Does MFCC Look Like?

Unlike a spectrogram,

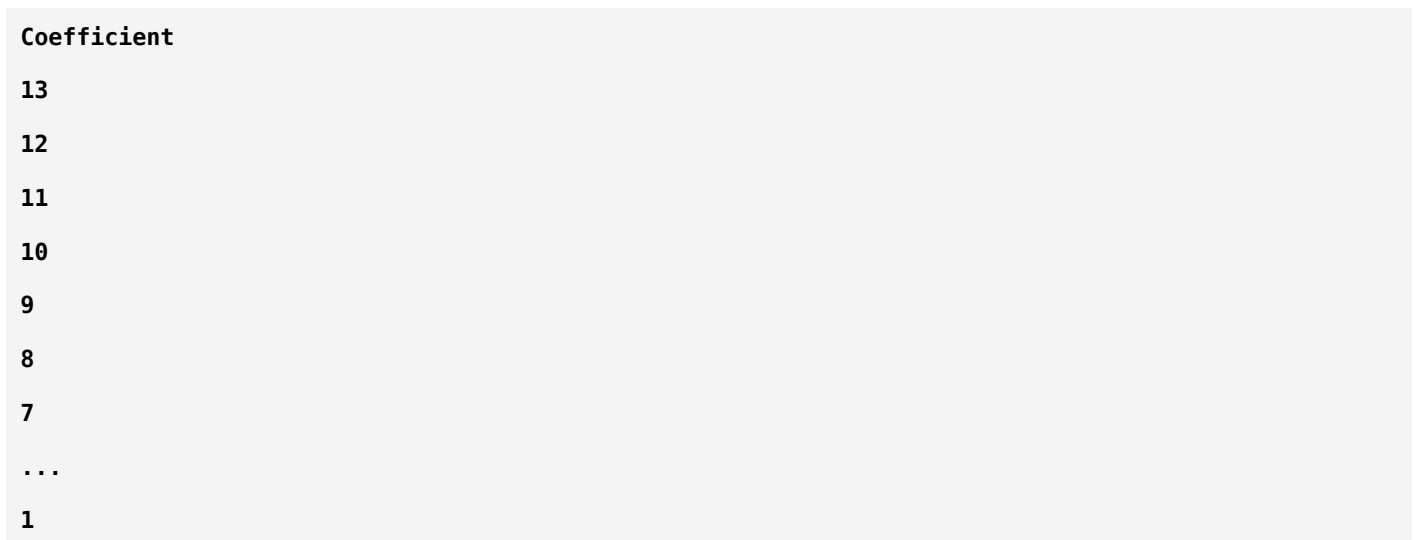
MFCC is **not displayed as frequencies**.

Instead

Y-axis



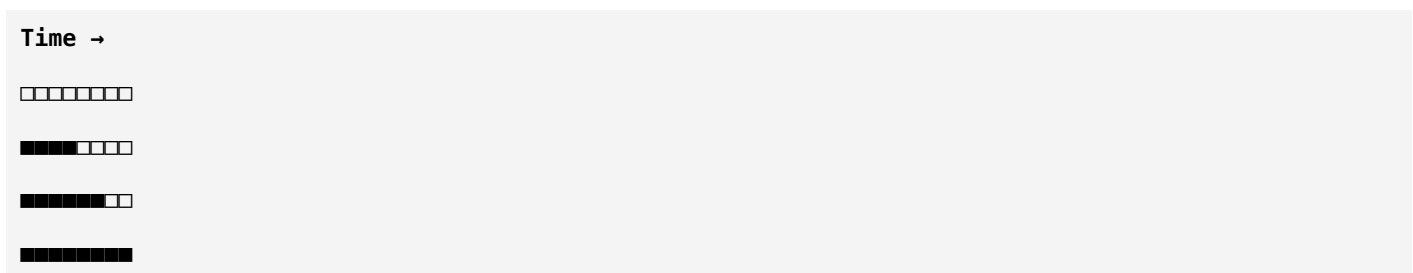
Coefficient Number



X-axis



Time



Each row is an MFCC coefficient rather than a frequency band.

Typical Number of MFCCs

Most applications keep

13

or

20

coefficients.

For example

Instead of

128 Mel Bands

↓

Store

13 MFCCs

This greatly reduces storage and computation.

Why MFCC Became Popular

In the early days of speech recognition,

computers had very limited memory.

Instead of processing huge spectrograms,

researchers compressed them into MFCCs.

Advantages:

- Smaller feature vectors.
- Faster machine learning.
- Reduced noise.
- Lower memory usage.

MFCCs became the standard feature for speech recognition for many years.

MFCC Example

Suppose your Mel Spectrogram contains

Frame 1

10

15
18
22
25
28
30
32

After DCT

MFCC
54
9
3
1
0
0
0
0

Instead of eight numbers,
only the first few coefficients are significant.

MFCC in Ship Detection

Imagine

Ocean Noise

MFCC
12
2
1
0
0
0

Ship Present

MFCC
45

15
8
4
2
1

Machine learning algorithms can learn these numerical patterns.

Why CNNs Prefer Spectrograms

Modern deep learning models (CNNs, ResNet, EfficientNet) are very good at learning directly from images.

Because of this,

many current audio applications use

MeL Spectrogram

instead of

MFCC

A CNN can discover useful features on its own without needing the compressed representation.

Is MFCC Good for Passive Sonar?

This is an important consideration.

Advantages

- Compact feature vectors.
- Fast to compute.
- Lower storage requirements.
- Good for classical machine learning models such as SVM, Random Forest, and KNN.

Limitations

MFCC was originally developed for **speech**, where the overall spectral envelope is more important than precise tonal frequencies.

In passive sonar, narrowband tonal components from engines, propellers, and gearboxes are often critical. Because MFCC compresses the spectrum, some of these fine details can be lost.

For this reason, many sonar researchers prefer:

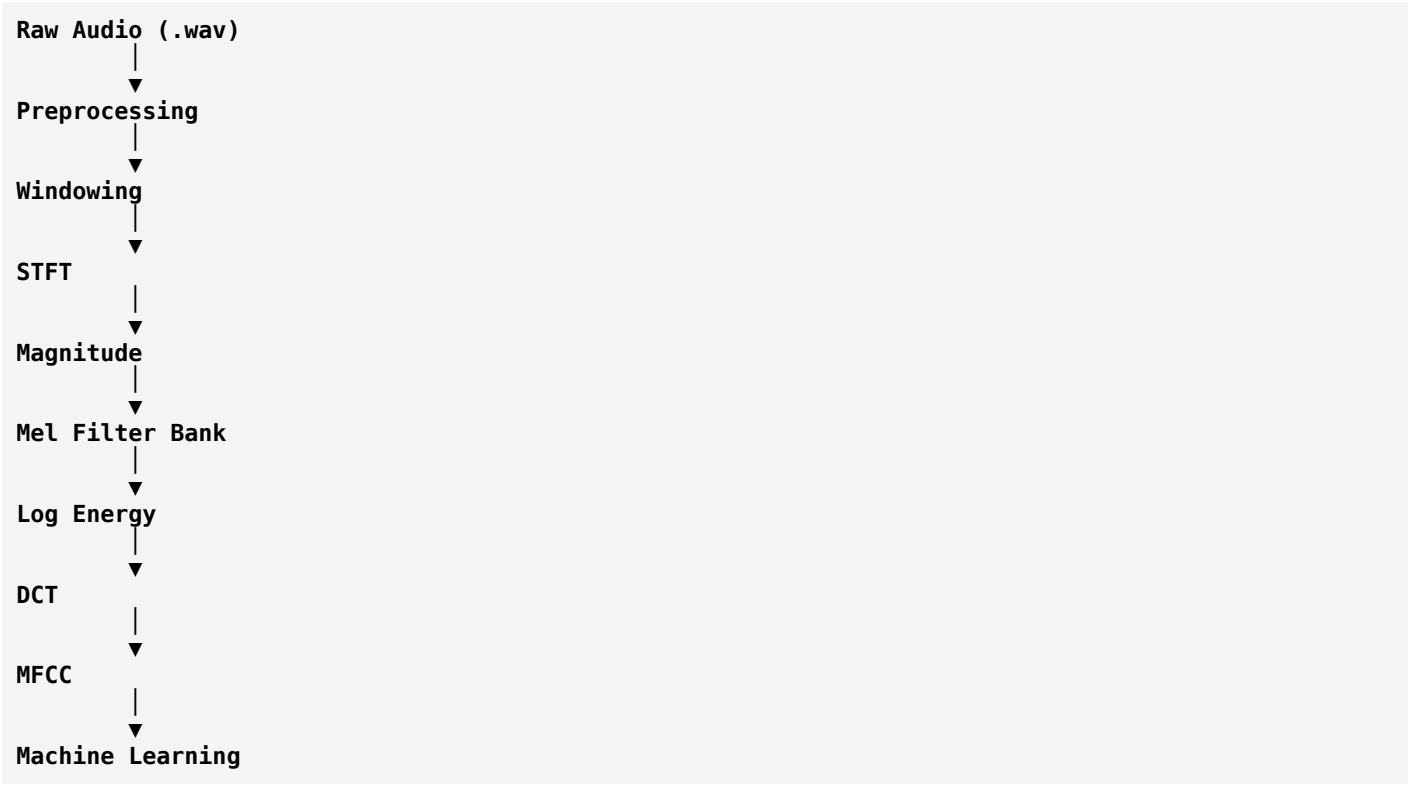
- Log Spectrograms.
- LOFARGRAMs.

- High-resolution STFT representations.

Comparison

Feature	Output	Good For	Information Retained
FFT	1D Spectrum	Frequency analysis	High
STFT	Time-Frequency Matrix	Non-stationary signals	High
Spectrogram	Image	CNNs	High
Mel Spectrogram	Compressed frequency image	Deep learning	Moderate to High
MFCC	Numerical coefficients	Classical ML	Moderate

Complete Audio Processing Pipeline



Which Features Should You Use?

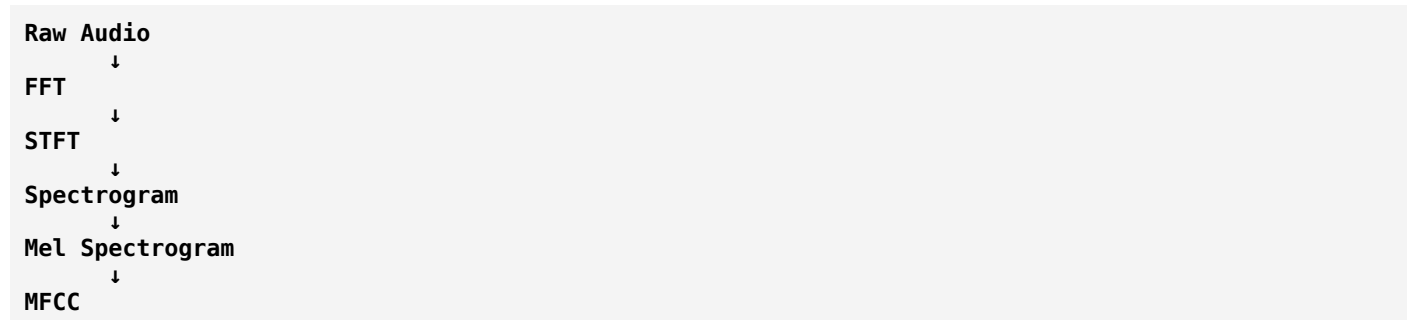
For your **Passive Sonar-Based Ship Presence Detection** project, here's a practical recommendation:

Feature	Recommendation	Reason
FFT	✗ Not as direct model input	Lacks time information
STFT	✓ Foundation	Needed to create later features
Log Spectrogram	★★★★★	Excellent baseline for CNNs

Feature	Recommendation	Reason
Mel Spectrogram	★★★★☆	Good balance of size and performance
MFCC	★★★★☆☆	Useful as a baseline with traditional ML models
LOFARGRAM	★★★★★★	Specifically designed for passive sonar and preserves tonal signatures

Where You Are Now

You now understand the complete feature extraction chain:



The **next topic should be LOFARGRAM**, because unlike MFCC—which was designed for speech—LOFARGRAM was developed specifically for analyzing underwater acoustic signals. It's one of the most important feature representations in passive sonar research and is highly relevant to your project.

LOFARGRAM

Excellent. **LOFARGRAM** is probably the **most important feature representation** in passive sonar. Unlike MFCC (developed for speech), LOFARGRAM was specifically developed for **underwater acoustic analysis**.

If your project is:

Passive Sonar-Based Target Ship Classification (Ship Present / Ship Absent)

then understanding LOFARGRAM is essential.

What is LOFAR?

LOFAR stands for:

LOw Frequency Analysis and **Re**cording.

Originally, LOFAR systems were developed by naval forces to detect and classify submarines and ships using passive sonar.

Why Was LOFAR Developed?

Imagine a hydrophone records:

Ocean Noise
+
Ship Engine
+
Propeller
+
Pump
+
Gearbox
+
Marine Animals

All these sounds are mixed together.

A normal waveform looks like this:

Amplitude


Can you identify the ship?

✗ No.

Apply FFT:

Frequency
100 Hz
200 Hz
300 Hz
500 Hz

Better.

You now know which frequencies exist.

But...

When did they occur?

FFT cannot answer.

Apply STFT:

Time
↓
Frequency

Now you know:

- Which frequency
- At which time

Good.

But sonar engineers observed something interesting.

Ships produce **very stable narrowband tones**.

Example:

Engine

100 Hz

Propeller

240 Hz

Gearbox

620 Hz

These frequencies remain visible for a long time.

LOFARGRAM was designed to emphasize these stable tonal components.

Is LOFARGRAM Different from STFT?

This is the biggest misconception.

Many students think:

STFT

↓

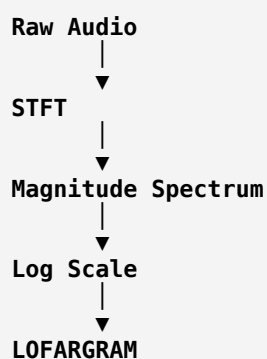
LOFARGRAM

are completely different.

Actually,

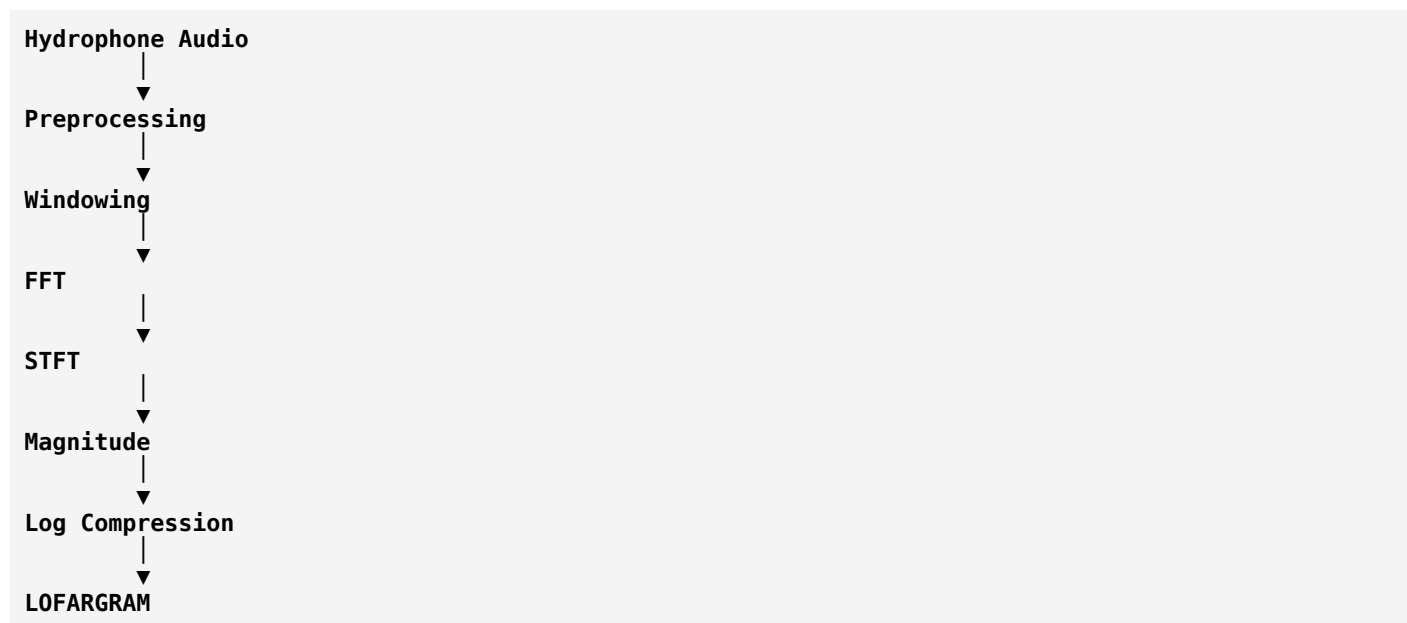
LOFARGRAM is **built using STFT**.

Think of it like this:



It is essentially a **log-scaled spectrogram** optimized for low-frequency passive sonar analysis.

LOFARGRAM Pipeline



Notice

Everything is identical to the spectrogram pipeline.

The difference lies in how the spectrum is displayed and interpreted.

Why "Low Frequency"?

Most ship machinery generates energy in lower frequencies.

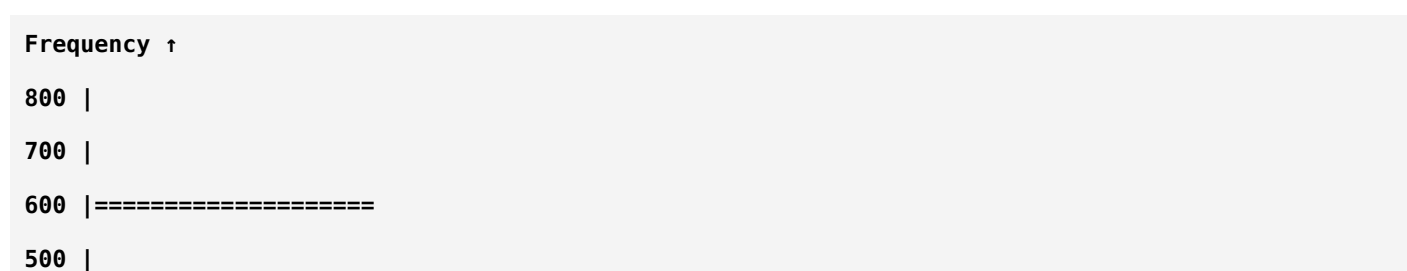
Typical examples:

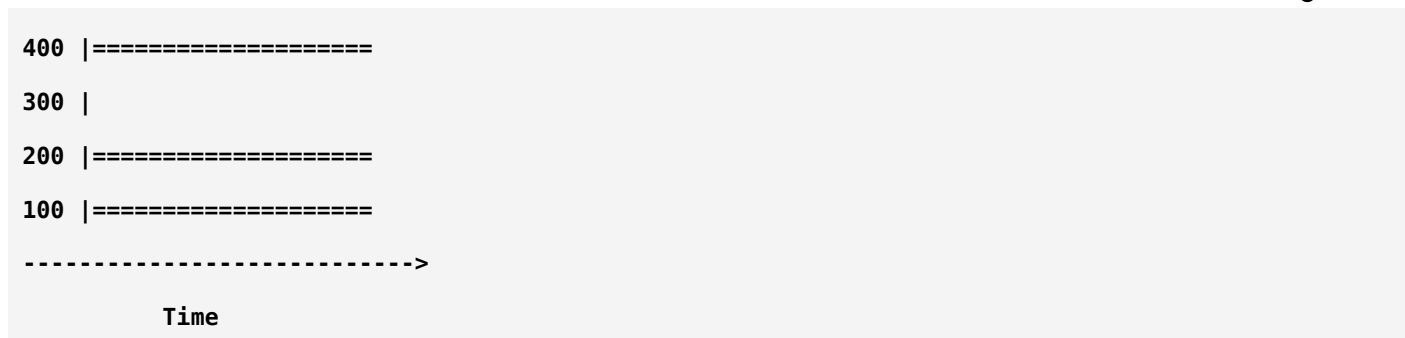
Source	Frequency Range
Engine	20–200 Hz
Propeller	50–500 Hz
Gearbox	100–1000 Hz
Pumps	200–1500 Hz

Because these low-frequency tones travel long distances underwater, LOFAR systems focus on them.

What Does a LOFARGRAM Look Like?

Imagine this image.





The horizontal bright lines represent stable frequencies.

Each line corresponds to a machine inside the ship.

Why Horizontal Lines?

Suppose the engine rotates at a constant speed.

It continuously produces

120 Hz

The frequency remains the same.

So in the LOFARGRAM,

120 Hz

appears as

A horizontal line.

Similarly,

Propeller

240 Hz

↓

Gearbox

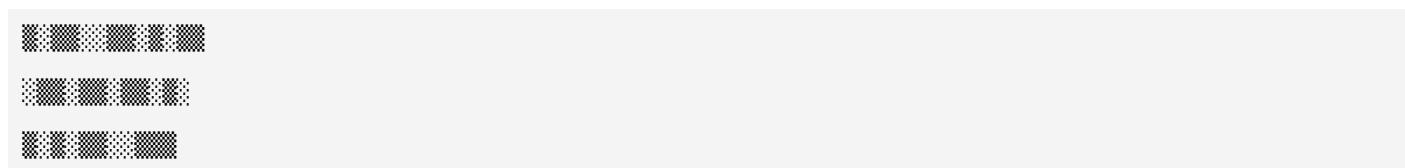
630 Hz

↓

Ocean Noise

Ocean noise is random.

Its LOFARGRAM looks like



There are no clear horizontal lines.

Ship Present



Notice

Stable horizontal bands.

These are called **tonal signatures**.

Why Are Tonal Signatures Important?

Every ship has

- Different engine
- Different propeller
- Different gearbox
- Different shaft speed

Therefore

every ship produces

a unique combination of tonal frequencies.

Think of it like a fingerprint.

Example

Ship A

100 Hz
250 Hz
620 Hz

Ship B

90 Hz
220 Hz
580 Hz

The CNN learns these patterns.

LOFARGRAM vs Spectrogram

A normal spectrogram displays all frequencies equally.

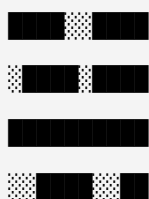
LOFARGRAM emphasizes:

- Low frequencies
- Narrowband tones
- Long-duration stable components
- Log-scaled intensity

making it more suitable for passive sonar.

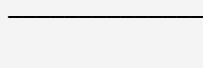
Example Comparison

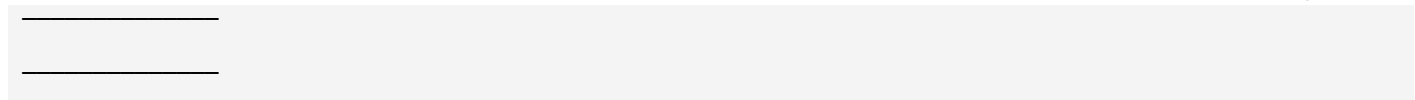
Spectrogram



Lots of texture.

LOFARGRAM





Very clean horizontal lines.

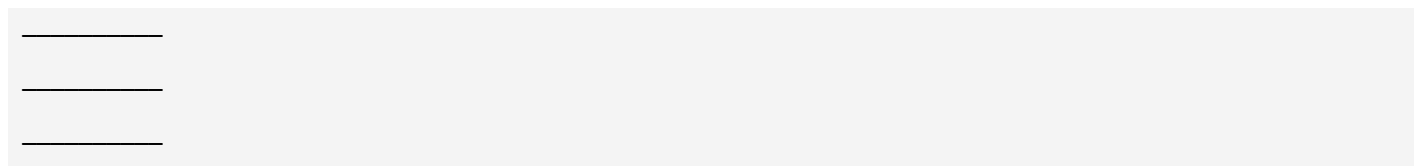
Those lines are exactly what sonar operators search for.

Why CNN Works So Well

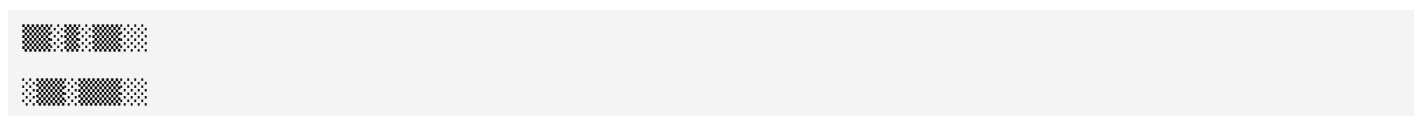
The CNN treats the LOFARGRAM like an image.

Example

Ship



No Ship



The CNN learns

- Horizontal patterns
- Bright lines
- Spacing between harmonics
- Energy distribution

without explicitly knowing they represent machinery.

Why LOFARGRAM is Better than MFCC for Sonar

MFCC	LOFARGRAM
Designed for speech	Designed for underwater acoustics
Compresses frequency information	Preserves narrowband tonal information
Loses some fine spectral details	Retains detailed tonal signatures
Good for speech recognition	Excellent for ship detection and classification

Where LOFARGRAM Fits

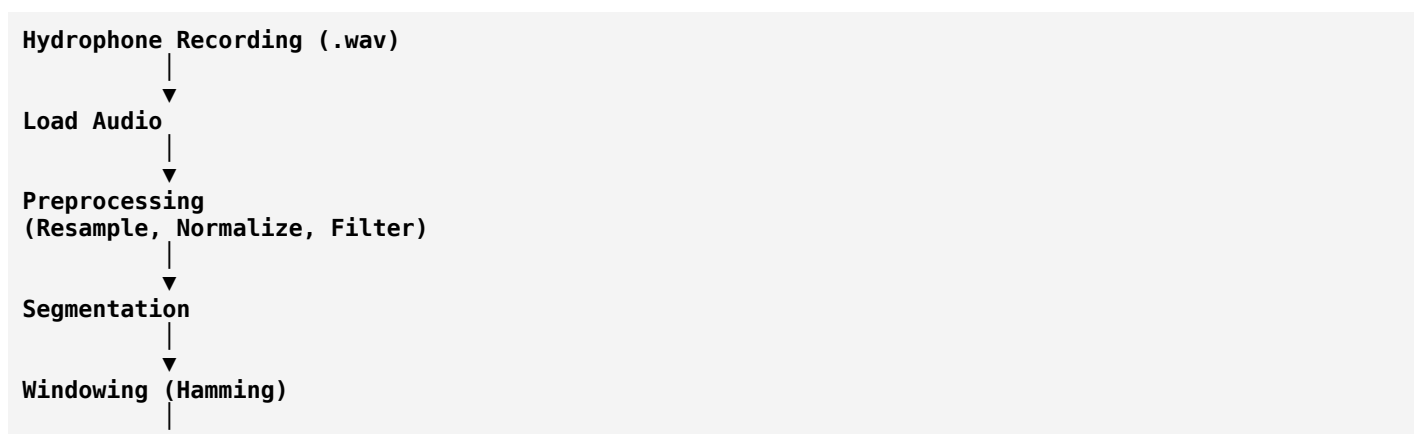


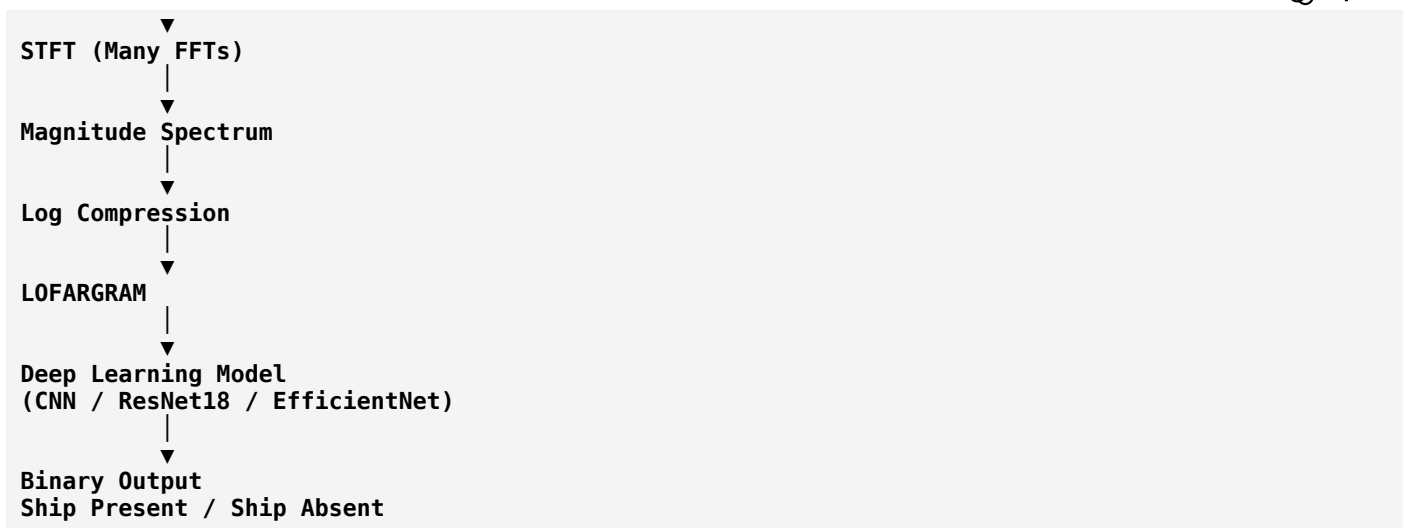
Which Feature Should You Choose?

For your project, here's how I'd prioritize them:

Feature	Suitability for Passive Sonar	Notes
FFT	★★★★☆	Good for understanding frequency content, but no time information.
STFT	★★★★★	Essential intermediate step for all time-frequency analysis.
Spectrogram	★★★★★	Strong baseline for CNN models.
Mel Spectrogram	★★★★☆	Useful, but may smooth away important narrowband tones.
MFCC	★★★☆☆	Better suited to speech than sonar.
LOFARGRAM	★★★★★★	Best traditional representation for highlighting ship tonal signatures.

Complete Passive Sonar Pipeline





One Important Clarification

In research papers, you may see slightly different definitions of a "LOFARGRAM." Some implementations include additional processing such as frequency averaging, background normalization, or tonal enhancement before visualization. However, the core idea remains the same:

- Start with **STFT**.
- Compute the **magnitude (or power) spectrum**.
- Apply **logarithmic scaling**.
- Emphasize stable **low-frequency tonal components** that characterize ships.

Understanding this foundation will make it much easier to read and implement different LOFARGRAM variants described in the passive sonar literature.

At this point, you've covered the complete digital audio feature extraction chain needed for your project. The next stage is deciding how to generate these features from your dataset in Python (using libraries such as `librosa` and `scipy`) and preparing them as inputs for your deep learning model.

Friday 6:19 AM

No as of now I will not go into any implementation now I want to learn basics of machine learning that I have to learn to understand and complete my project mainly focuses on key terms and everything so that I can easily understand

That's the best approach. **Don't jump into coding yet.** First build a strong conceptual foundation. Once you understand the terminology, reading research papers and implementing models becomes much easier.

Since your project is:

Passive Sonar-Based Binary Ship Classification (Ship Present / Ship Absent)

you don't need to learn *all* of machine learning. You need a focused roadmap.

Machine Learning Roadmap for Your Project

1. Machine Learning Basics
 - ↓
2. Data & Features
 - ↓
3. Model Basics
 - ↓
4. Training Process
 - ↓
5. Optimization
 - ↓
6. Evaluation Metrics
 - ↓
7. Deep Learning Basics
 - ↓
8. CNN Fundamentals
 - ↓
9. Transfer Learning
 - ↓
10. Binary Classification

Phase 1: Machine Learning Basics

Understand these concepts first:

- What is Artificial Intelligence (AI)?
- What is Machine Learning (ML)?
- What is Deep Learning (DL)?
- AI vs ML vs DL
- Types of Machine Learning:
 - Supervised Learning
 - Unsupervised Learning
 - Reinforcement Learning
- What is a Dataset?
- What is a Feature?
- What is a Label?
- Training Set
- Validation Set
- Test Set

Phase 2: Data Fundamentals

These are essential for your sonar project:

- Sample
- Feature
- Label

- Feature Vector
 - Dataset
 - Class
 - Binary Classification
 - Multi-class Classification
 - Class Imbalance
 - Data Augmentation
 - Normalization
 - Standardization
-

Phase 3: Model Fundamentals

Learn what a model actually is:

- What is a Model?
 - Parameters
 - Hyperparameters
 - Input
 - Output
 - Prediction
 - Inference
 - Decision Boundary
 - Feature Space
-

Phase 4: Training Concepts

This is one of the most important topics.

Learn:

- Training
- Epoch
- Batch
- Batch Size
- Iteration
- Forward Propagation
- Loss Function
- Cost Function

- Gradient
- Gradient Descent
- Backpropagation
- Optimizer
- Learning Rate

These concepts explain **how a model learns**.

Phase 5: Neural Network Basics

Understand:

- Artificial Neuron
 - Perceptron
 - Activation Function
 - ReLU
 - Sigmoid
 - Tanh
 - Hidden Layer
 - Input Layer
 - Output Layer
 - Weight
 - Bias
-

Phase 6: Deep Learning

Learn:

- Neural Network
- Deep Neural Network (DNN)
- CNN
- RNN
- LSTM
- Attention (basic idea)

For your project, **CNN is the most important**.

Phase 7: CNN Concepts

These are directly related to spectrogram and LOFARGRAM images:

- Image Representation
 - Pixel
 - Channel
 - Convolution
 - Kernel / Filter
 - Feature Map
 - Padding
 - Stride
 - Pooling
 - Max Pooling
 - Average Pooling
 - Flatten
 - Fully Connected Layer
 - Softmax
 - Sigmoid
-

Phase 8: Transfer Learning

Since you may use models like **ResNet18** or **EfficientNet**, understand:

- Pre-trained Model
 - Fine-tuning
 - Feature Extraction
 - Frozen Layers
 - Trainable Layers
-

Phase 9: Evaluation Metrics

For binary classification, learn:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1 Score
- Specificity

- Sensitivity
- ROC Curve
- AUC
- Precision-Recall Curve

These tell you **how well your model performs**.

Phase 10: Overfitting & Generalization

Very important concepts:

- Underfitting
 - Overfitting
 - Generalization
 - Bias
 - Variance
 - Regularization
 - Dropout
 - Early Stopping
 - Cross Validation
-

Phase 11: Optimization

Understand:

- SGD
 - Momentum
 - RMSProp
 - Adam
 - AdamW
 - Learning Rate Scheduler
-

Phase 12: Loss Functions

For your project:

- Binary Cross Entropy ★★★★★
 - Categorical Cross Entropy
 - Mean Squared Error (know why it's not commonly used for binary classification)
-

Phase 13: Practical Concepts

Know these terms because you'll see them everywhere:

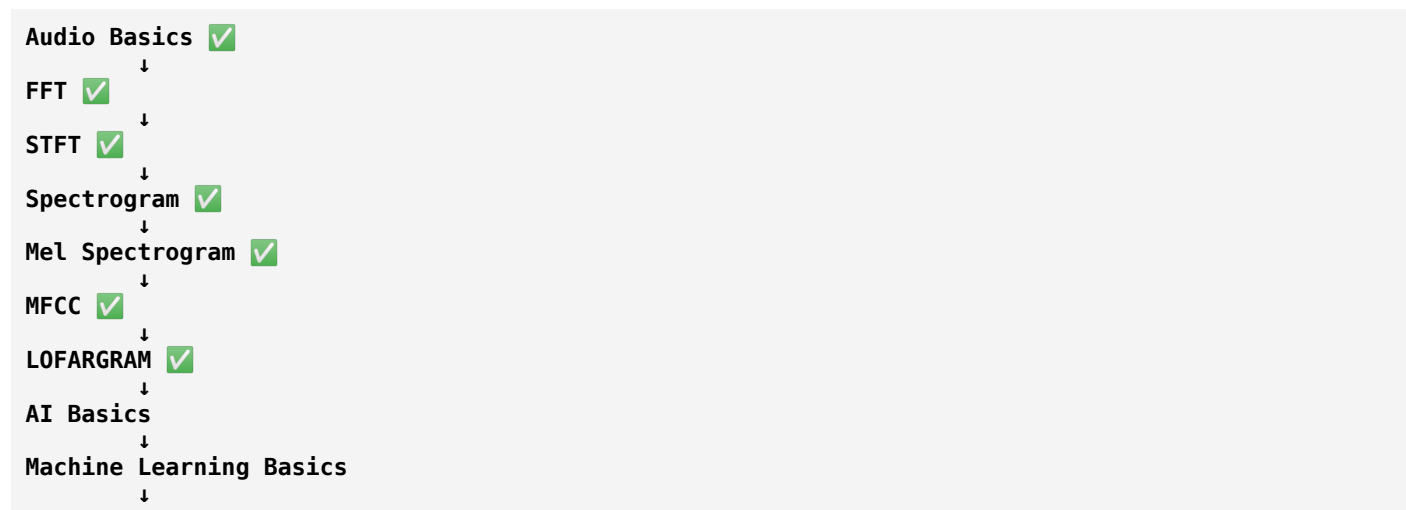
- Tensor
- Tensor Shape
- NumPy Array
- GPU
- CUDA
- TensorFlow
- PyTorch
- Model Checkpoint
- Inference
- Deployment

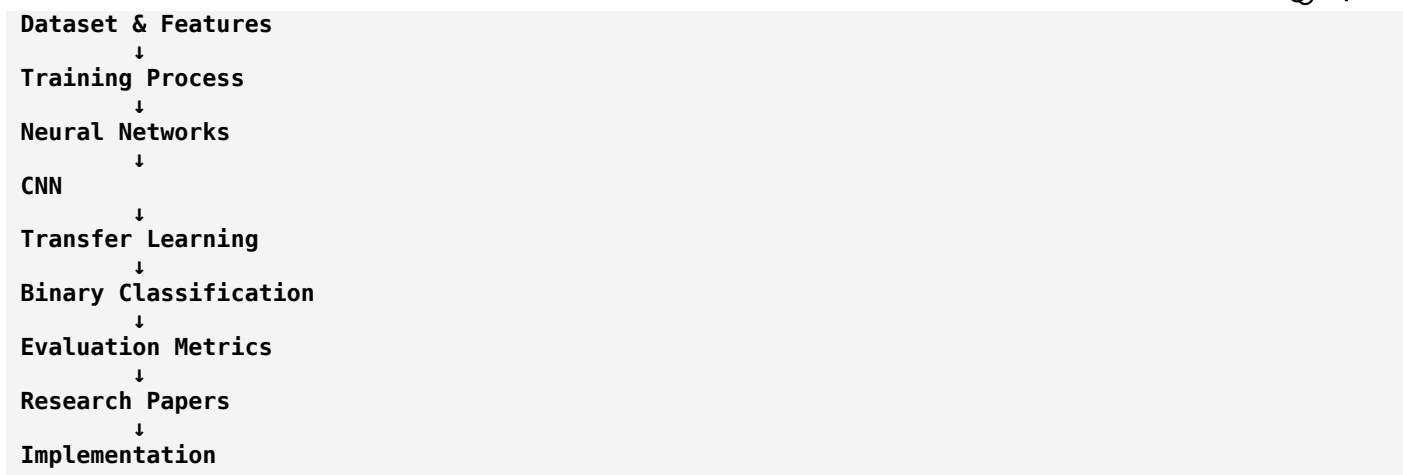
Phase 14: Research Concepts

When reading papers, you'll encounter:

- Baseline Model
- Proposed Method
- Ablation Study
- State-of-the-Art (SOTA)
- Benchmark Dataset
- Precision-Recall Trade-off
- Statistical Significance

Learning Order (Recommended)





How We'll Learn

Instead of learning random theory, we'll connect **every concept to your sonar project**.

For example:

- **Feature** → A spectrogram or LOFARGRAM extracted from a sonar clip.
- **Label** → "Ship Present" or "Ship Absent."
- **Epoch** → One complete pass through all sonar training samples.
- **CNN** → Learns patterns such as horizontal tonal lines in LOFARGRAMs.
- **Loss Function** → Measures how wrong the model's predictions are.
- **Precision** → Of all clips predicted as "Ship Present," how many actually contain a ship?

This way, every concept has a direct connection to your project rather than being an abstract definition.

My recommendation for the next topic

Start with "**What is Machine Learning?**" Then continue in this order:

1. AI vs Machine Learning vs Deep Learning
2. Supervised Learning (most important for your project)
3. Dataset, Sample, Feature, and Label
4. Binary Classification

Once these four topics are clear, the rest of machine learning becomes much easier because you'll already understand the overall learning process.

Ok the path you given exactly I need ok continue with it start phase 1

Perfect! We'll learn this like a university course, but **everything will be related to your Passive Sonar Ship Detection project**. We won't memorize definitions—we'll understand why each concept matters.

Phase 1: Machine Learning Basics

Topic 1: What is Artificial Intelligence (AI)?

Definition

Artificial Intelligence (AI) is the broad field of creating machines that can perform tasks that normally require human intelligence.

These tasks include:

- Recognizing images
- Understanding speech
- Making decisions
- Playing games
- Driving vehicles
- Detecting ships from sonar signals

Think of AI as the **big umbrella**.

Example in Your Project

Imagine a naval officer listens to a sonar recording.

They may say:

"I hear a ship."

That decision is based on years of experience.

If we build a system that can make the same decision automatically:

```

Sonar Audio
  ↓
AI System
  ↓
Ship Present
    
```

That system is an **Artificial Intelligence** system.

Topic 2: What is Machine Learning (ML)?

Machine Learning is a **subset of AI**.

Instead of writing explicit rules, we let the computer **learn patterns from data**.

Traditional Programming

```
Rules + Data
  ↓
Program
  ↓
Answer
```

Example:

If temperature > 38°C, print "Fever."

You write the rule.

Machine Learning

```
Data + Correct Answers
  ↓
Learning Algorithm
  ↓
Model
```

The computer discovers the rules by itself.

Sonar Example

Suppose you collect:

Sonar Recording	Label
Audio 1	Ship Present
Audio 2	No Ship
Audio 3	Ship Present
Audio 4	No Ship

Instead of telling the computer:

"Look for 120 Hz and 250 Hz."

You simply provide many examples.

The model learns the patterns automatically.

Why is it Called "Learning"?

Imagine teaching a child to recognize dogs.

You don't explain every rule.

You show many examples.

```
Dog 🐶
Dog 🐶
Dog 🐶
```

Cat 🐱
Dog 🐶

Eventually, the child learns the concept of "dog."

Machine learning works similarly.

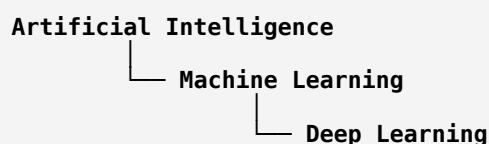
Instead of children, we train a model.

Instead of dogs, we use sonar recordings.

Topic 3: What is Deep Learning (DL)?

Deep Learning is a **subset of Machine Learning**.

It uses **Artificial Neural Networks** with many layers to automatically learn complex patterns.

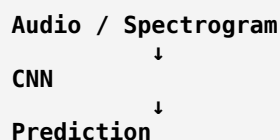


Traditional ML vs Deep Learning

Traditional ML:



Deep Learning:



The neural network learns many useful features automatically.

In Your Project

Input:

LOFARGRAM Image

↓

CNN

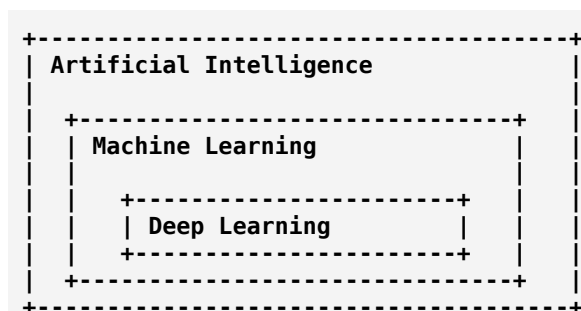
↓

Output:

Ship PresentThis is **Deep Learning**.

Topic 4: AI vs ML vs DL

Imagine three circles:



Artificial Intelligence

Goal: Build intelligent systems.

Examples:

- Chatbots
- Self-driving cars
- Sonar target detection

Machine Learning

Method: Learn from data.

Examples:

- Spam detection
- Disease prediction
- Ship detection

Deep Learning

Method: Learn using deep neural networks.

Examples:

- Image recognition
- Speech recognition

- Sonar image classification

Topic 5: Types of Machine Learning

There are three major types.

1. Supervised Learning ★★★★★

The model learns from **inputs and correct answers (labels)**.

Example:

Input	Label
Sonar Clip A	Ship Present
Sonar Clip B	No Ship
Sonar Clip C	Ship Present

The model learns the relationship between the input and the label.

Your project belongs to this category.

2. Unsupervised Learning

Here, you only provide the data.

No labels.

Example:

1000 sonar recordings

The algorithm groups similar recordings together.

It might discover:

- Group A: Similar engine sounds
- Group B: Similar background noise
- Group C: Marine life

No one tells it what these groups mean.

3. Reinforcement Learning

The model learns by interacting with an environment and receiving rewards or penalties.

Example:

An underwater robot:

Move Left

Reward: +10

Move Right

Penalty: -5

Over time, it learns the best actions.

This is **not used** in your project.

Which Type Does Your Project Use?

```

Sonar Recording
  ↓
Feature (LOFARGRAM)
  ↓
Label
(Ship Present / Ship Absent)
  ↓
Train CNN
  
```

Since you have known labels, it is **Supervised Learning**.

Phase 1 Summary

Concept	Meaning	Example in Your Project
AI	Making machines perform intelligent tasks	Automatic ship detection
ML	Learning patterns from data	Learning from labeled sonar recordings
DL	ML using deep neural networks	CNN classifying LOFARGRAMs
Supervised Learning	Data + labels	Ship / No Ship dataset
Unsupervised Learning	Data only	Grouping similar underwater sounds
Reinforcement Learning	Learning by rewards	Not used for your project

Real-World Analogy

Imagine you're training a new sonar operator.

You give them 10,000 recordings.

For each recording, you also provide the answer:

```

Recording 1 → Ship
Recording 2 → No Ship
Recording 3 → Ship
  
```


Recording 4 → No Ship

After enough practice, they begin recognizing ships on their own.

A machine learning model learns in exactly the same way—except instead of ears and experience, it uses mathematical algorithms and data.

What We'll Learn Next (Still in Phase 1)

The next lesson is **Dataset Fundamentals**, where we'll cover:

- What is a Dataset?
- What is a Sample?
- What is a Feature?
- What is a Label?
- What is a Feature Vector?
- What are Classes?
- Why are these concepts the foundation of every machine learning project?

These are the most frequently used terms in ML papers, tutorials, and implementations, so understanding them well will make everything that follows much easier.

Ok next

Excellent! This is one of the **most important topics in Machine Learning**. If you understand these concepts well, then almost every research paper and ML tutorial will make sense.

Phase 1 (Part 2): Dataset Fundamentals

We'll learn these in order:

1. Dataset
2. Data Point / Sample
3. Feature
4. Label (Target)
5. Feature Vector
6. Class
7. Binary Classification
8. Multi-class Classification

We'll use **your sonar project** as the example throughout.

1. What is a Dataset?

A **dataset** is simply a collection of data used to train or test a machine learning model.

Think of it as a **textbook** from which the model learns.

Example

Suppose you collect 10,000 sonar recordings.

```
Dataset
Recording 1
Recording 2
Recording 3
...
Recording 10000
```

This entire collection is called the **dataset**.

Sonar Example

Recording	Label
audio001.wav	Ship Present
audio002.wav	No Ship
audio003.wav	Ship Present
audio004.wav	No Ship

This table is your dataset.

2. What is a Sample (Data Point)?

A **sample** is **one individual example** from the dataset.

If your dataset has:

```
10,000 sonar recordings
```

Then

```
1 recording = 1 sample
```

Example:

```
audio003.wav
```

This single recording is one sample.

Analogy

Imagine a classroom.

There are **60 students**.

The classroom is the **dataset**.

Each student is one **sample**.

In Your Project

One sample could be:

3-second sonar clip

or

One LOFARGRAM image

Depending on your pipeline.

3. What is a Feature?

This is one of the most important concepts.

A **feature** is a measurable property or characteristic of a sample that helps the model make a prediction.

Think of it as a **clue**.

Human Example

Suppose you want to identify a person.

Useful features include:

- Height
- Weight
- Eye color
- Hair color
- Age

These are all features.

Sonar Example

Suppose your sample is a sonar recording.

Possible features are:

- FFT values
- Spectrogram
- Mel Spectrogram
- MFCC
- LOFARGRAM
- Energy
- Frequency peaks

These features contain information that helps distinguish:

- Ship Present
- Ship Absent

Important Point

The **raw audio** itself is usually **not** the feature.

We process the audio to extract informative features.

For example:

```
Raw Audio
  ↓
STFT
  ↓
LOFARGRAM
```

The **LOFARGRAM** becomes the feature given to the model.

4. What is a Label (Target)?

A **label** is the correct answer that the model should learn to predict.

Example:

Sample	Label
audio001.wav	Ship Present
audio002.wav	No Ship

The label tells the model whether its prediction is correct.

In Your Project

Sample:

LOFARGRAM Image

Label:

Ship Present

or

Ship Absent

Without labels, the model doesn't know the correct answer during supervised training.

5. What is a Feature Vector?

A **feature vector** is the numerical representation of all features for one sample.

Machine learning models work with numbers, not concepts like "ship" or "noise."

Example

Suppose a sample has these extracted features:

Feature	Value
Energy	0.85
Peak Frequency	120 Hz
Spectral Centroid	340 Hz
Bandwidth	180 Hz

The feature vector is:

[0.85, 120, 340, 180]

This list of numbers is what the model actually receives.

In Deep Learning

If you use a **LOFARGRAM**, the feature vector is no longer a simple list.

Instead, it is a **matrix of pixel values**.

Example:

128 × 128

The CNN processes this image directly.

6. What is a Class?

A **class** is one possible category that a sample can belong to.

In Your Project

There are two classes:

Class 0

No Ship

Class 1

Ship Present

Every sample belongs to exactly one class.

7. What is Binary Classification?

Binary means **two**.

A binary classification problem has exactly two classes.

Examples:

Problem	Classes
Spam Detection	Spam / Not Spam
Medical Diagnosis	Disease / Healthy
Face Unlock	Authorized / Unauthorized
Your Project	Ship Present / Ship Absent

Your project is a **binary classification** problem.

8. What is Multi-class Classification?

Here, there are more than two classes.

Example:

Instead of just detecting a ship, suppose you want to identify its type:

Sample	Label
Audio 1	Cargo Ship
Audio 2	Fishing Boat
Audio 3	Submarine
Audio 4	Oil Tanker

Now the model chooses one of several categories.

That is **multi-class classification**.

Putting Everything Together

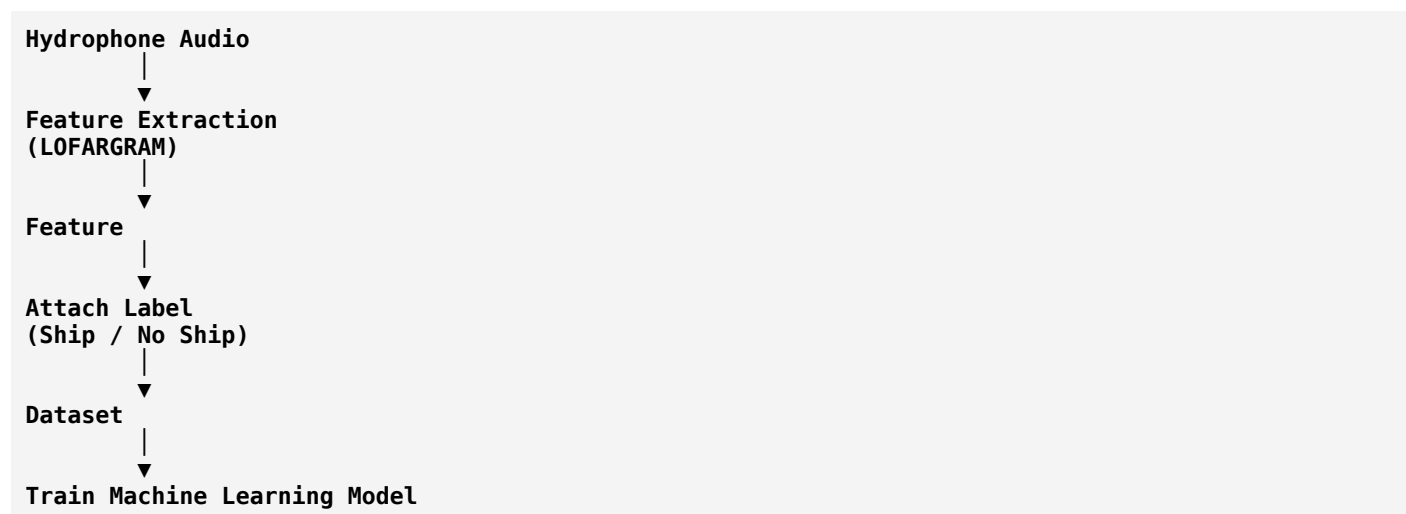
Imagine this dataset:

Sample	Feature	Label
audio001.wav	LOFARGRAM	Ship Present
audio002.wav	LOFARGRAM	No Ship
audio003.wav	LOFARGRAM	Ship Present

Here:

- The whole table is the **dataset**.
- Each row is one **sample**.
- The LOFARGRAM is the **feature**.
- "Ship Present" or "No Ship" is the **label**.
- There are **2 classes**, making it a **binary classification** problem.

Visual Flow



Summary Table

Term	Meaning	Example in Your Project
Dataset	Collection of all data	All sonar recordings
Sample	One data item	One sonar clip
Feature	Information used for learning	LOFARGRAM, Spectrogram, MFCC
Label	Correct answer	Ship Present / Ship Absent
Feature Vector	Numerical representation of features	Pixel values of a LOFARGRAM or extracted numeric features

Term	Meaning	Example in Your Project
Class	Possible output category	Ship, No Ship
Binary Classification	Two output classes	Ship Present vs Ship Absent
Multi-class Classification	More than two output classes	Cargo Ship, Fishing Boat, Submarine, etc.

Quiz (Try Without Looking)

1. In your project, what is the **dataset**?
2. What is **one sample**?
3. What is the **feature** if you use LOFARGRAM?
4. What is the **label**?
5. Why is your project called **binary classification**?

If you can answer these confidently, you've understood one of the most fundamental parts of machine learning.

Next Lesson

Next we'll cover another crucial topic:

Training Set, Validation Set, and Test Set

You'll learn:

- Why we don't train on all the data.
- Why we split the dataset.
- What **data leakage** is.
- How models are evaluated fairly.

This topic is essential because every machine learning project—including yours—depends on splitting data correctly before training.

Next

Excellent! This is one of the **most important concepts in Machine Learning**.

Many beginners think:

"I have 10,000 sonar recordings. I'll train the model on all of them."

This seems logical, but it's actually **a mistake**.

Let's understand why.

Phase 1 (Part 3): Training Set, Validation Set, and Test Set

Why Can't We Train on All the Data?

Imagine you are preparing for an exam.

Your teacher gives you **100 questions**.

You memorize every answer.

During the exam, the teacher asks **the exact same 100 questions**.

You score **100/100**.

Does that mean you truly understand the subject?

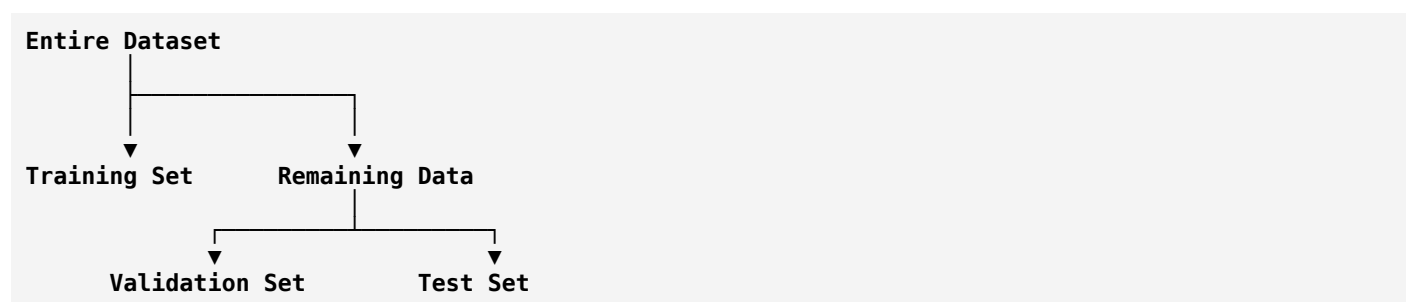
No.

It only means you memorized the questions.

Machine learning models can do the same thing—they can **memorize** the training data instead of learning general patterns.

The Solution

We divide the dataset into three parts:



Each part has a different purpose.

1. Training Set

The **training set** is the data used to **teach the model**.

Example:

Suppose you have

10,000 sonar recordings

You might use

8,000 recordings

for training.

The model repeatedly looks at these samples and learns patterns.

Example:

```

LOFARGRAM
  ↓
CNN
  ↓
Prediction
  ↓
Compare with Label
  ↓
Improve

```

This learning process happens only on the training set.

2. Validation Set

Now suppose the model has learned from the training data.

How do we know if it is improving?

We test it on **new data that it has never seen during training**.

This is the **validation set**.

Example:

```
1,000 sonar recordings
```

The model predicts:

```
Ship Present
```

Actual label:

```
Ship Present
```

Good!

If the predictions improve over time, training is progressing well.

Why Validation?

The validation set helps us answer questions like:

- Should we train for more epochs?
- Should we stop training?
- Which model performs better?
- Which learning rate is best?

It guides decisions **during development**.

3. Test Set

The **test set** is the final exam.

The model has **never seen** these samples during:

- Training
- Validation
- Hyperparameter tuning

Example:

1,000 completely unseen recordings

Now we evaluate the model.

This tells us how well it is likely to perform in the real world.

Visual Example

Suppose your dataset contains **100 sonar recordings**.

Split it like this:

```

100 Recordings
├── 70 → Training
├── 15 → Validation
└── 15 → Test
  
```

Typical real-world splits are:

- **70% / 15% / 15%**
- **80% / 10% / 10%**

The exact percentages are less important than keeping the sets separate.

Real-Life Analogy

Imagine preparing for a driving license.

Training

You practice driving every day.

↓

Training Set

Validation

Your instructor gives you practice tests.



Validation Set

Test

Government driving test.



Test Set

You don't practice during the real exam.

Your Sonar Project Example

Suppose you have

12,000 sonar clips

Split them as:

Training

9,600

Validation

1,200

Test

1,200

Now:

Training



Model learns

Validation



Model tuning

Test



Final performance

What is Data Leakage?

This is a very important concept.

Imagine:

```
Training
audio001.wav
```

and

```
Test
audio001.wav
```

The same recording appears in both sets.

The model already knows the answer!

It will appear to perform extremely well, but this performance is misleading.

This problem is called **data leakage**.

Rule

Never allow information from the training set to leak into the validation or test sets.

Another Example

Suppose you split one long recording into many overlapping clips.

If clips from the same original recording appear in both training and testing, the model may recognize recording-specific characteristics instead of learning true ship signatures.

This is another form of data leakage and is especially important in audio projects.

Why Keep the Test Set Hidden?

Suppose after testing you get:

```
Accuracy
80%
```

You modify the model.

Test again.

Now:

```
Accuracy
82%
```

Modify again.

Eventually, you may unintentionally optimize specifically for the test set.

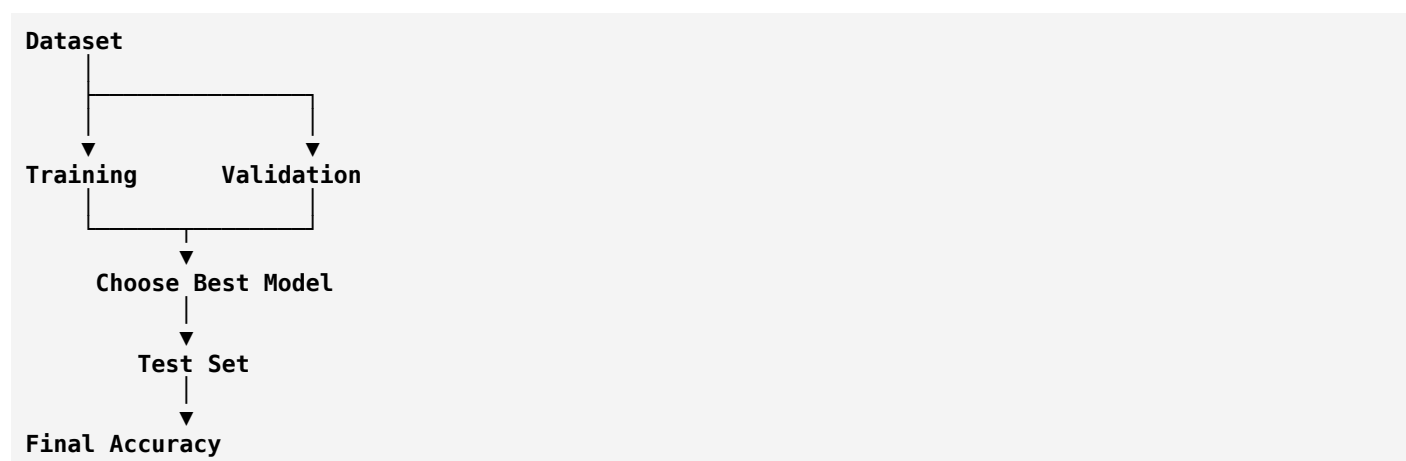
Then the test set is no longer an unbiased evaluation.

That's why many researchers evaluate on the test set **only once**, after all development is complete.

Summary Table

Dataset Split	Purpose	Used for Learning?
Training Set	Teach the model	✓ Yes
Validation Set	Tune the model and monitor training	✗ No
Test Set	Final evaluation	✗ No

Complete Workflow



In Your Passive Sonar Project

Imagine your dataset contains **LOFARGRAM** images:

- **Training Set:** Used to teach the CNN what "Ship Present" and "Ship Absent" look like.
- **Validation Set:** Used during training to decide if the model is improving or starting to overfit.
- **Test Set:** Used only at the end to estimate how well the model will classify completely new sonar recordings.

Key Terms to Remember

- **Training Set** → Used to learn.
- **Validation Set** → Used to tune and monitor.
- **Test Set** → Used for the final evaluation.
- **Data Leakage** → Any information from the test/validation data reaching the training process.

Next Lesson

Next we'll begin **Phase 2: Model Fundamentals**, starting with:

1. **What is a Machine Learning Model?**
2. **What are Parameters?**
3. **What are Hyperparameters?**
4. **What is Prediction (Inference)?**

These concepts explain what a model actually is and how it turns a LOFARGRAM into the prediction **"Ship Present"** or **"Ship Absent."** This is where you'll start understanding how learning actually happens inside the model.

Next

Excellent! Now we reach the heart of Machine Learning.

Up to now you know:

- ☒ What AI is
- ☒ What Machine Learning is
- ☒ What Deep Learning is
- ☒ Dataset
- ☒ Sample
- ☒ Feature
- ☒ Label
- ☒ Training/Validation/Test sets

Now the biggest question is:

What exactly is a Machine Learning Model?

This is the foundation for everything that follows.

Phase 2: Model Fundamentals

Topic 1: What is a Machine Learning Model?

Definition

A **Machine Learning Model** is a mathematical function that learns patterns from data and uses those patterns to make predictions on new data.

Think of it as a **decision-maker**.

Example

Imagine you train a new naval officer.

After listening to thousands of sonar recordings, they can identify whether a ship is present.

Now imagine replacing that officer with a computer.

The trained computer program is called the **model**.

```
Sonar Recording
  ↓
Trained Model
  ↓
Ship Present
```

The model is the part that has "learned."

Before Training

Imagine a student on the first day of class.

You play a sonar recording.

Student:

"I don't know."

Similarly, before training:

```
Input
  ↓
Model
  ↓
Random Guess
```

The model has not learned anything yet.

After Training

Now the student has listened to 10,000 labeled recordings.

You play a new recording.

Student:

"This sounds like a ship."

Similarly:

```
Input
  ↓
Trained Model
  ↓
Ship Present
```


The model has learned patterns from experience.

What Does the Model Actually Learn?

It **does not memorize** recordings.

Instead, it learns relationships like:

- Certain frequency patterns
- Certain tonal signatures
- Energy distributions
- Shapes in the LOFARGRAM

For example:

Horizontal bright lines
↓
Likely Ship

These learned relationships are stored inside the model.

Topic 2: What are Parameters?

This is one of the most important concepts.

Definition

Parameters are the internal values that the model learns automatically during training.

They represent the model's knowledge.

Analogy

Imagine you're adjusting a radio.

You turn the tuning knob until the signal becomes clear.

The knob position is like a **parameter**.

The model continuously adjusts millions of such values while learning.

In a Neural Network

Every connection has a **weight**.

Example:

```

Input
|
(weight = 0.8)
|
Neuron

```

Initially:

```
Weight = 0.23
```

After learning:

```
Weight = 1.87
```

These learned weights are the **parameters**.

Example

Suppose the model sees:

```
LOFARGRAM
```

Initially it predicts:

```
No Ship
```

Wrong!

The model adjusts its parameters.

Next time it predicts:

```
Ship Present
```

Better.

This adjustment happens thousands of times until the parameters represent useful knowledge.

Topic 3: What are Hyperparameters?

This is where many beginners get confused.

Let's compare.

Parameters

- Learned automatically.
- Change during training.

Example:

Weight = 0.83

The model updates this by itself.

Hyperparameters

Hyperparameters are **chosen by you before training**.

The model does **not** learn them.

Examples:

- Learning Rate
- Batch Size
- Number of Epochs
- Optimizer
- Number of Layers

These are settings that control how the model learns.

Analogy

Imagine cooking rice.

Parameters:

- The actual taste of the rice after cooking.

Hyperparameters:

- Cooking time
- Water quantity
- Flame level

You choose these before cooking.

Parameter vs Hyperparameter

Parameter	Hyperparameter
Learned automatically	Set by the developer
Changes during training	Usually fixed before training
Example: Weights, Biases	Example: Learning Rate, Batch Size

Topic 4: What is Prediction?

Once a model has been trained, we can use it to make predictions.

Example:

```
New Sonar Recording
  ↓
Model
  ↓
Ship Present
```

The output is called a **prediction**.

Topic 5: What is Inference?

Inference is simply the process of using a trained model to make predictions.

Training:

```
Model learns
```

Inference:

```
Model uses what it learned
```

Example

During training:

```
10,000 recordings
```

↓

Model learns.

Later, the navy receives a new sonar recording.

```
New Recording
  ↓
Model
  ↓
Prediction
```

This prediction process is called **inference**.

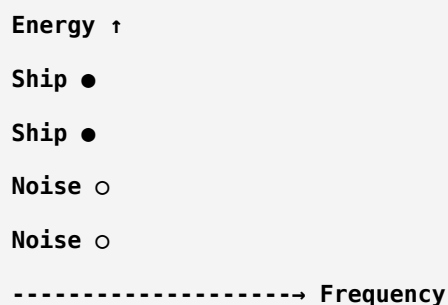
Topic 6: Feature Space (Basic Idea)

Imagine each sonar recording has two features:

- Engine Frequency

- Signal Energy

We can plot each recording as a point.

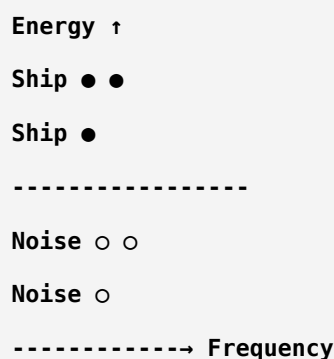


This graph is called the **feature space**.

The model tries to separate the two groups.

Topic 7: Decision Boundary

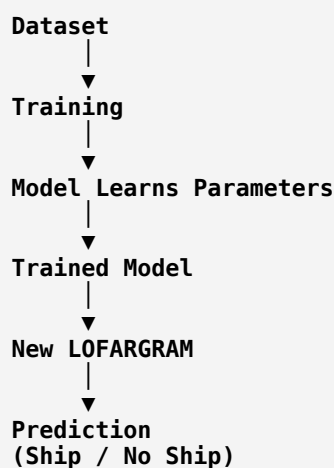
Imagine drawing a line that separates ships from noise.



That separating line is called the **decision boundary**.

A good model learns a decision boundary that correctly separates the classes.

Complete Flow



Summary

Concept	Meaning	Example in Your Project
Model	Learns patterns from data	CNN trained on LOFARGRAMs
Parameter	Values learned during training	Weights and biases
Hyperparameter	Settings chosen before training	Learning rate, batch size
Prediction	Model's output	Ship Present
Inference	Using the trained model	Classifying a new sonar clip
Feature Space	Space where samples are represented	LOFARGRAM features
Decision Boundary	Separation between classes	Ship vs No Ship

One Important Clarification

Many people think:

Model = Algorithm

They are related but **not the same**.

- **Algorithm** → The learning procedure (e.g., how a CNN updates its weights during training).
- **Model** → The trained result after learning.

For example:

- Before training: an untrained CNN architecture.
- After training: a CNN model that has learned to detect ships.

Next Topic (Very Important)

Next we'll learn **how a model actually learns**, step by step.

We'll cover:

1. **Epoch**
2. **Batch**
3. **Batch Size**
4. **Iteration**
5. **Forward Propagation**
6. **Loss Function**

These concepts describe the complete training process and are essential before moving on to gradient descent, backpropagation, and neural networks. This is where machine learning starts becoming truly intuitive.

Next

Excellent! This is the **most important phase** in Machine Learning.

After this lesson, you'll understand **how a model actually learns**.

This is the concept that every ML engineer, researcher, and deep learning practitioner must understand.

Phase 3: How Does a Model Learn?

Imagine you're teaching a student to identify ships from sonar recordings.

The learning process is:

```

See Example
  ↓
Guess
  ↓
Check Answer
  ↓
Correct Mistake
  ↓
Learn
  ↓
Repeat Thousands of Times
  
```

A machine learning model learns in exactly the same way.

Complete Training Cycle

```

Dataset
  ↓
Take One Batch
  ↓
Model Makes Prediction
  ↓
Calculate Error (Loss)
  ↓
Improve Parameters
  ↓
Next Batch
  ↓
Repeat
  
```

Every concept we'll learn fits into this cycle.

1. What is an Epoch?

Definition

An **Epoch** is **one complete pass through the entire training dataset**.

Example

Suppose your training dataset contains:

1000 sonar recordings

If the model has seen all 1000 recordings once,
that is:

1 Epoch

Example

Training Data

Recording 1
Recording 2
...
Recording 1000

After seeing all of them

↓

Epoch 1 Complete ✓

If it sees them again

↓

Epoch 2 Complete ✓

Again

↓

Epoch 3 Complete ✓

Why Multiple Epochs?

A student doesn't learn after reading a book once.

They revise.

Similarly,

the model needs multiple passes.

Example

Epoch 1
Accuracy = 55%

Epoch 5
Accuracy = 80%

Epoch 20
Accuracy = 95%

The model gradually improves.

2. What is a Batch?

Imagine you have

10,000 sonar recordings

Should the computer process all 10,000 at once?

Usually **no**, because it would require too much memory.

Instead, we divide the dataset into smaller groups.

Each group is called a **batch**.

Example

Training Data

1000 Samples

Divide into

100
100
100
100
100
100
100
100
100
100
100

Each group of 100 samples is a **batch**.

3. What is Batch Size?

The number of samples in one batch is called the **Batch Size**.

Example

Batch Size = 32

means

the model learns from **32 samples at a time**.

Common Batch Sizes

16
32
64
128
256

There is no universally "best" batch size. It depends on the dataset, model, and available GPU memory.

Example

Suppose

Training Samples
1000

Batch Size

100

Then

$1000 / 100 = 10$ batches

The model processes

Batch 1

↓

Batch 2

↓

Batch 3

↓

...

↓

Batch 10

Then

Epoch 1 finishes.

4. What is an Iteration?

Many beginners confuse **Epoch** and **Iteration**.

Let's clarify.

Suppose

Training Samples

1000

Batch Size

100

Therefore

10 batches

Each time the model processes **one batch**, it completes **one iteration**.

So

10 Iterations
=
1 Epoch

Formula

Iterations
=
Training Samples
÷
Batch Size

Example

Training Samples = 2000
Batch Size = 100
Iterations = 20

If you train for

30 Epochs

Total iterations

20 × 30
=

600

Visual Example

Training Data

□□□□□□□□

Batch 1

■

Iteration 1

↓

Batch 2

■

Iteration 2

↓

...

↓

Batch 10

■

Iteration 10

↓

Epoch Finished

Sonar Project Example

Suppose

Training Images

8000 LOFARGRAMS

Batch Size

32

Iterations

8000 / 32

=

250

Therefore

250 Iterations

=

1 Epoch

Train for

20 Epochs

↓

Total iterations

5000

Summary Table

Term	Meaning
Epoch	One complete pass through the training dataset
Batch	A small group of samples processed together
Batch Size	Number of samples in one batch
Iteration	One update of the model after processing a batch

Real-Life Analogy

Imagine a teacher has **300 answer sheets**.

Instead of checking all 300 at once,
they check **30 papers at a time**.

- **30 papers** → Batch
- **Batch Size = 30**
- After checking one batch → **1 Iteration**
- After checking all 300 papers → **1 Epoch**

If the teacher reviews all papers **5 times**, that's **5 Epochs**.

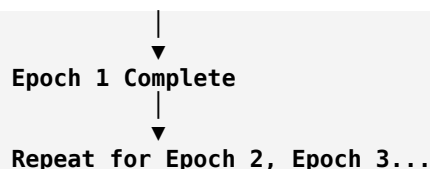
Complete Learning Flow

8000 LOFARGRAM Images

↓
Split into Batches

↓
Batch 1 → Iteration 1
Batch 2 → Iteration 2
Batch 3 → Iteration 3
...

Batch 250 → Iteration 250



Quiz (Answer Without Looking)

1. If you have **1,000 training samples** and a **batch size of 100**, how many **iterations** are in one epoch?
2. What is the difference between a **batch** and a **batch size**?
3. If you train for **10 epochs**, how many times has the model seen each training sample?

If you answer:

1. **10 iterations**
2. **A batch is the group; batch size is the number of samples in that group**
3. **10 times**

then you've understood this topic correctly.

Next Lesson (The Heart of Deep Learning)

Now that you know **how data is fed into the model**, the next lesson explains **what happens inside the model for each batch**:

1. **Forward Propagation**
2. **Prediction**
3. **Loss Function**
4. **Cost Function**
5. **Why the model makes mistakes**
6. **How those mistakes are measured**

This is where you'll finally understand how a neural network begins learning from its errors.

Next

Excellent! ★

This is the **heart of Machine Learning**.

If you understand this lesson, you'll understand **how every deep learning model (CNN, ResNet, EfficientNet, etc.) learns**.

Everything that happens during training can be summarized like this:

Input
↓

```

Prediction
↓
Compare with Correct Answer
↓
Calculate Error
↓
Improve Model
↓
Repeat

```

Let's understand each step.

Phase 3 (Part 2): Forward Propagation, Prediction & Loss Function

Step 1: Forward Propagation

Definition

Forward Propagation (Forward Pass) is the process where the input data passes through the model to produce an output (prediction).

Nothing is learned in this step.

The model is simply making a guess using its current parameters.

Your Sonar Project

Input:

LOFARGRAM Image

↓

CNN

↓

Output:

Ship Present = 0.82
Ship Absent = 0.18

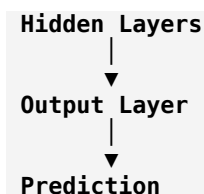
This entire process is called **Forward Propagation**.

Visual Flow

```

LOFARGRAM
↓
Convolution Layers
↓

```



Think of forward propagation as:

The model looks at the input and gives its current best answer.

Step 2: Prediction

The output produced during forward propagation is called the **prediction**.

Example:

Actual Label:

Ship Present

Model predicts:

Ship Present

Correct!

Another example:

Actual Label:

Ship Present

Prediction:

Ship Absent

Wrong!

The model now needs to learn from this mistake.

Step 3: Probability

Modern classifiers don't usually say only:

Ship Present

Instead they output probabilities.

Example:

Ship Present = 0.95

Ship Absent = 0.05

Meaning:

"I'm 95% confident that a ship is present."

Another example:

Ship Present = 0.52

Ship Absent = 0.48

The model is uncertain.

Step 4: What is Loss?

Suppose the correct answer is

Ship Present

Model predicts

Ship Absent

Clearly,

the model made a mistake.

But...

How big is the mistake?

We need a way to measure it.

That measurement is called the **Loss**.

Definition

A **Loss Function** measures how wrong the model's prediction is for **one sample or one batch**.

Small Loss

↓

Good Prediction

Large Loss

↓

Poor Prediction

Example

Actual:

Ship Present

Prediction:

Ship Present = 0.99

Loss

↓

Very Small

Another example

Actual:

Ship Present

Prediction:

Ship Present = 0.20

Loss

↓

Very Large

The larger the loss, the more the model needs to improve.

Student Analogy

Imagine a math test.

Question:

5 + 5 = ?

Student writes

10

Error

↓

0

Excellent.

Another student writes

17

Large error.

Similarly,

loss measures **how far the prediction is from the correct answer**.

Step 5: What is a Loss Function?

A **Loss Function** is simply the mathematical formula used to calculate the loss.

Different problems use different loss functions.

For your binary classification project, the most common choice is:

Binary Cross-Entropy (BCE) Loss.

For now, you only need to remember:

Loss Function = Formula that measures prediction error.

We'll study Binary Cross-Entropy in detail later.

Step 6: Loss vs Accuracy

Many beginners think these are the same.

They are not.

Example:

100 test samples

Model correctly predicts 90.

Accuracy:

90%

Now look at the wrong predictions.

Case 1:

The model was almost correct.

Ship = 0.49

No Ship = 0.51

Case 2:

The model was extremely confident but wrong.

Ship = 0.01

No Ship = 0.99

Both count as **one wrong prediction** for accuracy.

But the **loss** is much larger in Case 2 because the model was confidently incorrect.

That's why training uses **loss**, not accuracy, to improve the model.

Step 7: What is Cost Function?

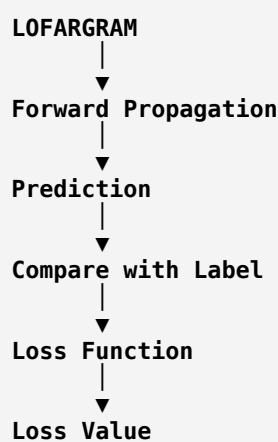
You may hear both **Loss Function** and **Cost Function**.

The distinction is:

- **Loss** → Error for one sample (or sometimes one batch, depending on context).
- **Cost** → Average loss over many samples.

In modern deep learning, people often use the words interchangeably, so don't be surprised if a paper calls the training objective "loss."

Complete Training Flow



At this point,

the model knows:

"I made a mistake."

But it still doesn't know **how to fix it**.

That comes next.

Sonar Example

Input:

LOFARGRAM

Actual Label:

Ship Present

Prediction:

Ship Absent

Loss:

High

The model realizes:

"My prediction is poor. I need to adjust my internal parameters."

How does it adjust them?

Using:

- **Gradient**
- **Gradient Descent**
- **Backpropagation**

These are the next topics.

Summary

Concept	Meaning	Example
Forward Propagation	Input moves through the model to produce an output	LOFARGRAM → CNN → Prediction
Prediction	The model's output	Ship Present (0.92)
Probability	Model's confidence	92% Ship Present
Loss	Measures prediction error	High if the prediction is wrong
Loss Function	Formula to calculate loss	Binary Cross-Entropy
Cost Function	Average loss over many samples	Used to evaluate training progress

Real-Life Analogy

Imagine you're learning archery.

```

Shoot Arrow
↓
See Where It Lands
↓
Measure Distance from Bullseye
↓
Adjust Your Aim
↓
Shoot Again

```

- **Arrow** → Prediction

- **Bullseye** → Correct Label
- **Distance from Bullseye** → Loss
- **Adjusting Your Aim** → Learning

The model learns in exactly the same way.

Next Lesson (The Core of Learning)

Now comes the most important lesson in deep learning:

1. **Gradient**
2. **Gradient Descent**
3. **Learning Rate**
4. **Backpropagation**
5. **Optimizer (SGD, Adam)**

These concepts explain **how the model changes its parameters to reduce the loss**. Once you understand them, you'll know exactly how a CNN learns to distinguish **Ship Present** from **Ship Absent**.

Ok next

Excellent! 🌟

This is the **most important topic in all of Machine Learning**.

After this lesson, you'll understand **how a model actually learns from its mistakes**.

Every neural network (CNN, ResNet, EfficientNet, Transformer, etc.) learns using these concepts.

Phase 3 (Part 3): Gradient, Gradient Descent, Learning Rate & Backpropagation

Let's first ask a simple question.

The Problem

Suppose the model predicts:

Ship Absent

But the correct answer is:

Ship Present

The model calculates a **high loss**.

Now the question is:

How does the model know which parameters (weights) to change, and by how much?

The answer involves **Gradient** and **Backpropagation**.

Step 1: What is a Gradient?

Definition

A **gradient** tells the model:

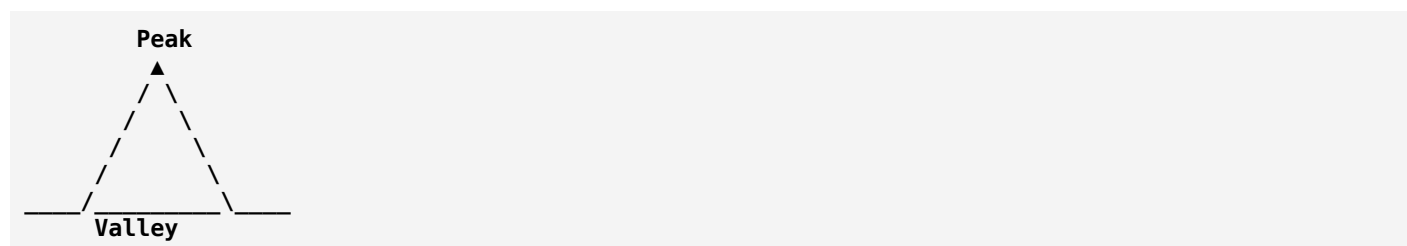
- Which direction to change the parameters.
- How much changing a parameter will increase or decrease the loss.

Think of it as a **guide**.

Mountain Analogy

Imagine you're standing on a mountain.

Your goal is to reach the **lowest point** (the valley).



You don't know the path.

But you can look at the slope.

The slope tells you:

"Walk downhill."

In machine learning:

- Mountain height = **Loss**
- Valley = **Minimum Loss**
- Slope = **Gradient**

The gradient tells the model which way reduces the error.

Step 2: What is Gradient Descent?

Definition

Gradient Descent is the optimization algorithm that updates the model's parameters to reduce the loss.

Simple idea:

```

High Loss
  ↓
Compute Gradient
  ↓
Update Parameters
  ↓
Lower Loss
  
```

Repeat this process many times until the model performs well.

Sonar Example

Initially:

```

Prediction
Ship Absent ❌
  
```

Loss:

```

0.95
  
```

Gradient says:

"Increase some weights, decrease others."

The model updates its parameters.

Next prediction:

```

Ship Present ✅
  
```

Loss:

```

0.20
  
```

The model has improved.

Step 3: What is a Learning Rate?

This is one of the most important **hyperparameters**.

Definition

The **learning rate** controls **how big a step the model takes** when updating its parameters.

Mountain Analogy

Imagine you're walking downhill.

Very Small Steps

```
.
.
.
.
.
Valley
```

You'll eventually reach the valley, but it takes a long time.

Very Large Steps

```
Valley
← Jump →
← Jump →
← Jump →
```

You may keep jumping past the valley and never settle there.

Medium Steps

```
Step
Step
Step
Valley ✓
```

This is usually what we want.

Learning Rate Values

Examples:

```
0.1
0.01
0.001
0.0001
```

Common starting values (depending on the optimizer) are:

- **0.001** (often used with Adam)
- **0.01** (common with SGD, but not universal)

The best value depends on the model and dataset.

Too Small vs Too Large

Learning Rate	Result
Too Small	Training is very slow
Too Large	Training may become unstable or fail to converge
Well Chosen	Faster and stable learning

Step 4: What is Backpropagation?

This is one of the most famous terms in deep learning.

Definition

Backpropagation is the algorithm that computes the gradients for all parameters by propagating the error backward through the network.

It tells every weight:

"Here's how much you contributed to the error."

Why "Back" Propagation?

Remember:

Forward Propagation

```

Input
↓
Hidden Layers
↓
Prediction

```

Now we know the prediction is wrong.

The error travels in the opposite direction.

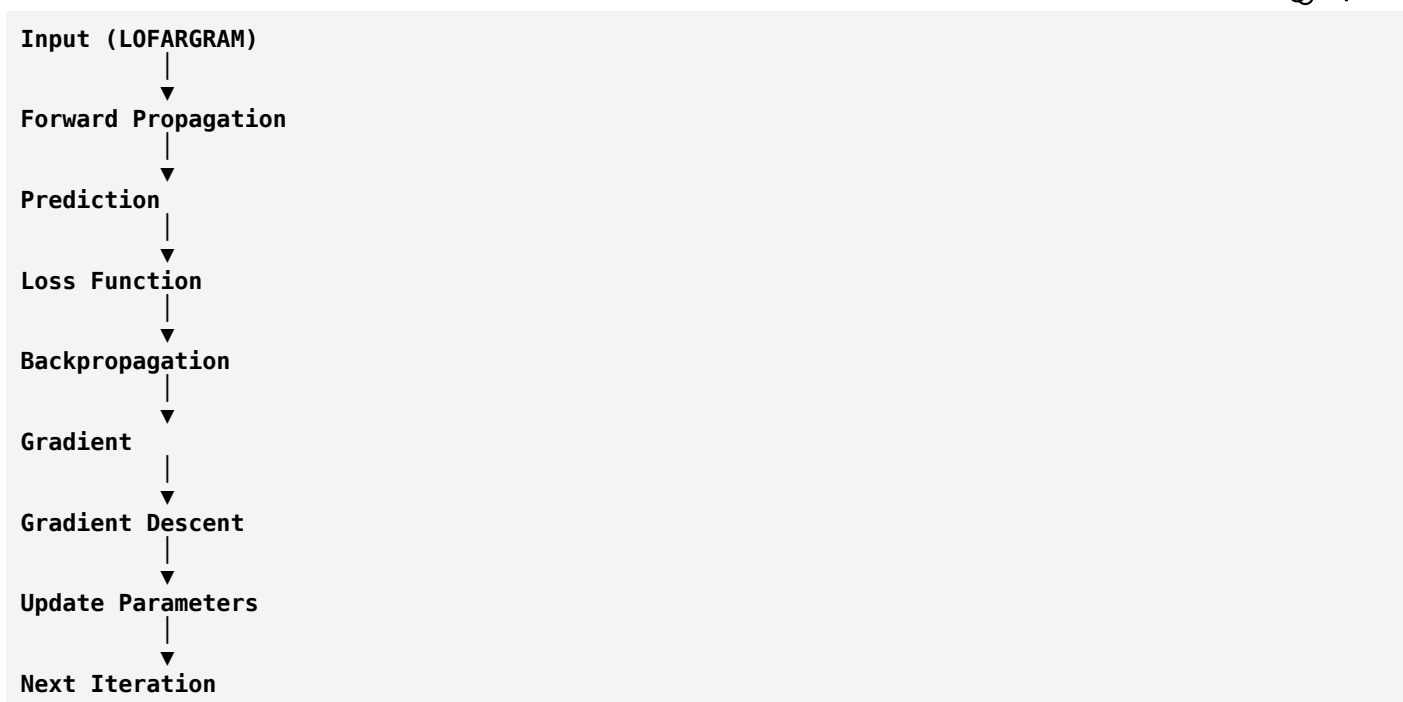
```

Loss
↑
Hidden Layers
↑
Input

```

This backward flow computes gradients for every parameter.

Complete Learning Cycle



This cycle repeats thousands of times during training.

Step 5: What is an Optimizer?

An **optimizer** is the algorithm that uses the gradients to update the model's parameters.

You can think of it as the **driver** that moves the model toward lower loss.

Common optimizers include:

- **SGD (Stochastic Gradient Descent)** – Simple and effective.
- **Adam (Adaptive Moment Estimation)** – One of the most popular optimizers because it adapts the learning process automatically.
- **RMSProp** – Often useful in some sequence-learning tasks.

For many deep learning projects, **Adam** is a common starting choice.

Sonar Project Example

Suppose the model sees:

LOFARGRAM

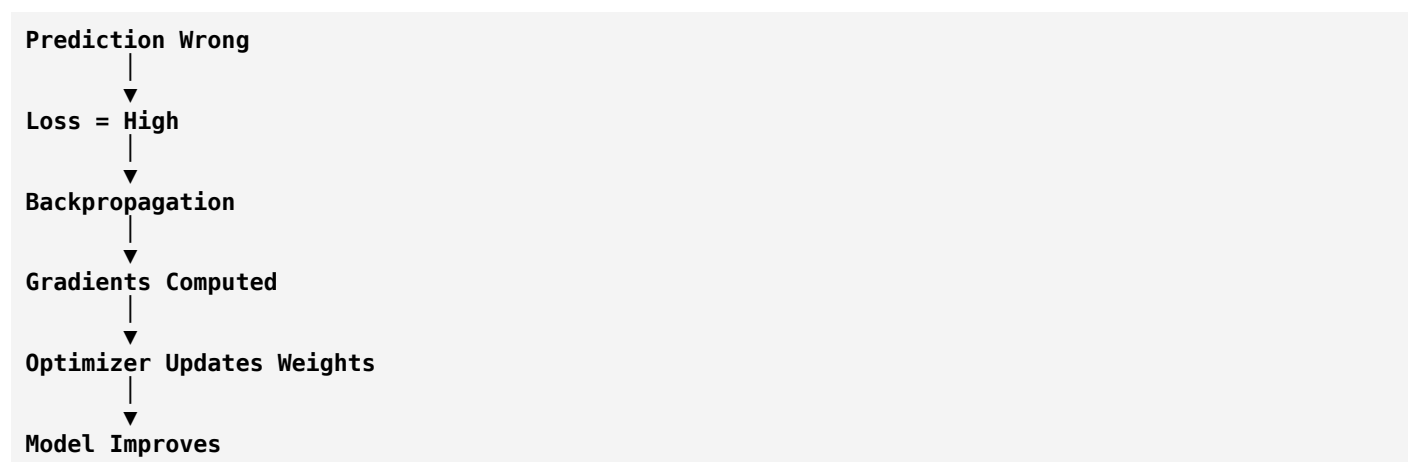
Correct Label:

Ship Present

Prediction:

Ship Absent

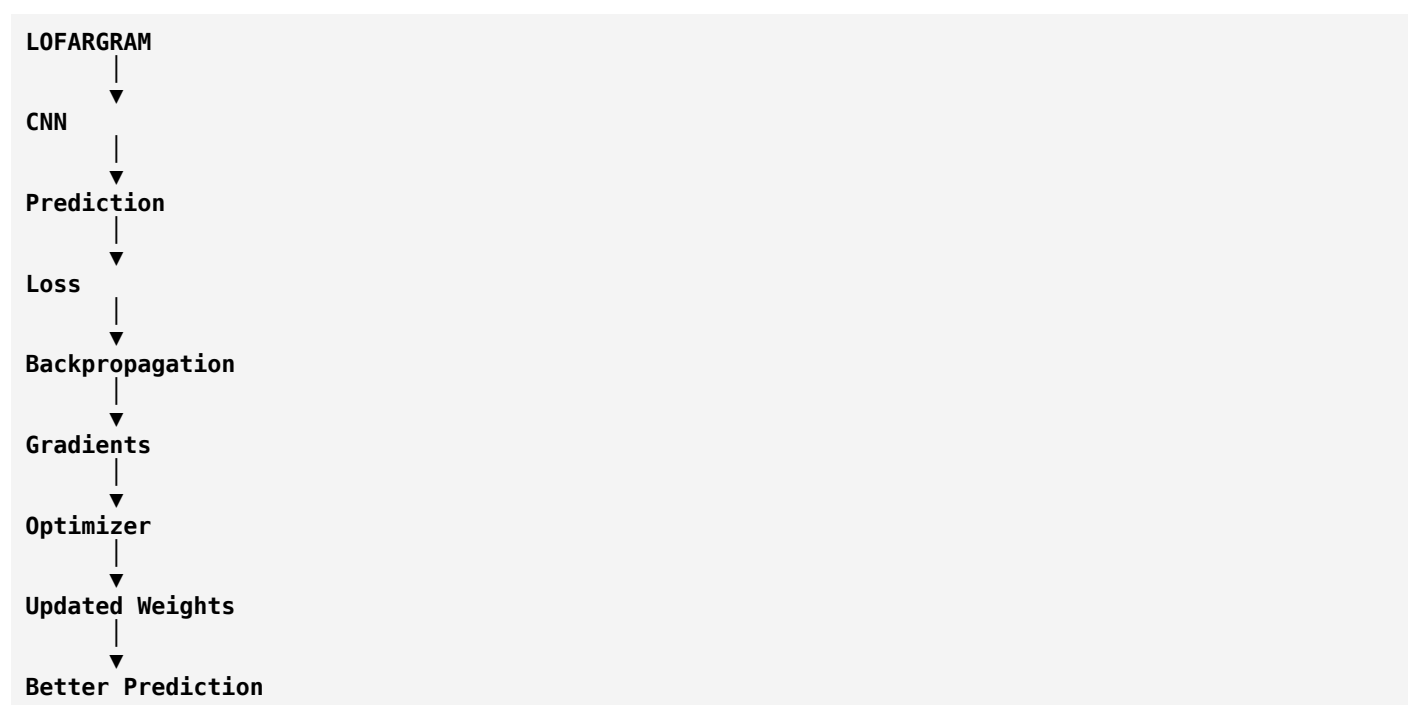
The training process is:



Repeat this for thousands of sonar samples.

Eventually the model learns to recognize ship signatures.

Visual Summary



Real-Life Analogy

Imagine you're learning to throw a ball into a basket.

Attempt 1:

✗ Missed to the left.

You realize:

"I need to throw slightly to the right."

Attempt 2:

✗ Too far.

You adjust again.

Attempt 3:

✓ Basket!

In this analogy:

- **Prediction** → Your throw.
- **Loss** → Distance from the basket.
- **Gradient** → Which direction to adjust.
- **Learning Rate** → How much to adjust.
- **Backpropagation** → Figuring out what caused the miss.
- **Optimizer** → Applying the adjustment.

Summary Table

Concept	Meaning	Simple Analogy
Gradient	Direction and rate of change of the loss	Slope of a mountain
Gradient Descent	Method to reduce loss	Walking downhill
Learning Rate	Step size for parameter updates	Length of each step
Backpropagation	Computes gradients for all parameters	Tracing mistakes back to their causes
Optimizer	Updates parameters using gradients	Driver following the downhill path

One Important Clarification

People often say:

"Backpropagation trains the model."

More precisely:

- **Backpropagation** computes the gradients.
- **The optimizer** uses those gradients to update the parameters.
- **Gradient Descent** is the underlying optimization idea that many optimizers are based on.

They work together as parts of one learning process.

What Comes Next?

You now understand **how the model learns**.

The next phase is **Neural Network Fundamentals**, where we'll build up from a single artificial neuron to a complete neural network:

1. Artificial Neuron
2. Perceptron
3. Weights & Biases (in depth)
4. Activation Functions (ReLU, Sigmoid, Tanh)
5. Input, Hidden, and Output Layers
6. How multiple neurons form a neural network

Once you understand these, CNNs will become much easier to learn because a CNN is built from these same fundamental building blocks.

Ok next

Excellent! 🌟

You now understand **how a model learns**.

The next step is understanding **what the model is made of**.

Think of it this way:

- Previous lesson → **How the brain learns**
- This lesson → **How the brain is built**

This is the foundation of every deep learning model, including **CNN**, **ResNet**, and **EfficientNet**.

Phase 4: Neural Network Fundamentals

We'll cover:

1. Artificial Neuron
2. Perceptron
3. Weights
4. Bias
5. Activation Function
6. Input Layer
7. Hidden Layer
8. Output Layer

1. What is a Neuron?

Let's start with the human brain.

Your brain contains billions of **neurons**.

A neuron:

- Receives information.
- Processes it.
- Sends information to other neurons.

Example:

```

Eyes
↓
Brain Neuron
↓
Decision
  
```

Artificial Intelligence was inspired by this idea.

2. What is an Artificial Neuron?

An **Artificial Neuron** is the basic building block of a neural network.

It does three things:

1. Receives inputs.
2. Performs mathematical calculations.
3. Produces an output.

Think of it as a **small decision-making unit**.

Sonar Example

Inputs:

```

Feature 1 → Engine Frequency
Feature 2 → Signal Energy
Feature 3 → Spectral Power
  
```

↓

Artificial Neuron

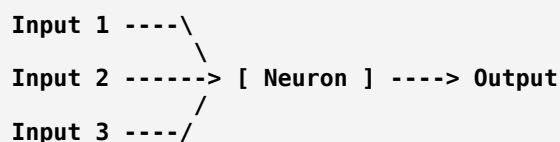
↓

Output:

```

Ship Probability
  
```

Structure of an Artificial Neuron



Every neuron works like this.

3. What are Weights?

Every input has an associated **weight**.

A weight tells the neuron:

"How important is this input?"

Example

Suppose a neuron receives:

Engine Frequency
Signal Energy
Background Noise

Maybe the model learns:

Feature	Weight
Engine Frequency	0.9
Signal Energy	0.8
Background Noise	0.2

This means:

- Engine frequency is very important.
- Signal energy is important.
- Background noise is less useful.

These weights are **learned during training**.

4. What is Bias?

Bias is another value added before the neuron produces its output.

It allows the neuron to shift its decision threshold.

Think of it like adjusting the starting point of a weighing scale before you begin measuring.

Without a bias, the neuron would be much less flexible.

Simple Formula

A neuron computes something like:

```
Output = Activation((Input × Weight) + Bias)
```

Don't worry about the math right now.

Just remember:

- **Weights** control the importance of each input.
- **Bias** shifts the neuron's decision.

5. What is an Activation Function?

Imagine the neuron calculates:

```
Value = 2.8
```

Should this mean:

- Ship?
- No Ship?

The neuron needs a rule to decide.

That rule is called the **Activation Function**.

It determines the neuron's output.

Why Do We Need Activation Functions?

Without activation functions,

a neural network would behave like a simple linear calculator.

It wouldn't be able to learn complex patterns such as:

- Ship engine harmonics.
- Different tonal signatures.
- Complex LOFARGRAM textures.

Activation functions allow neural networks to learn these nonlinear relationships.

Common Activation Functions

1. ReLU (Rectified Linear Unit)

Rule:

```
If  $x > 0$ 
Output =  $x$ 
Else
Output = 0
```

Example:

Input	Output
-4	0
-1	0
3	3
7	7

ReLU is the most commonly used activation function inside modern CNNs because it is simple and effective.

2. Sigmoid

The output is always between:

```
0
and
1
```

Example:

```
0.95
```

means

95% probability.

For binary classification, the final output layer often uses a sigmoid activation.

Example:

```
0.92
```

↓

Ship Present

```
0.08
```

↓

Ship Absent

3. Tanh

Output range:

-1
to
+1

It is used in some neural network architectures, especially certain recurrent networks, but is less common than ReLU in CNN hidden layers.

6. Input Layer

This is where the data enters the network.

In your project:

LOFARGRAM Image

↓

Input Layer

The input layer simply passes the data into the network.

7. Hidden Layers

These are the layers where learning happens.

Example:

Input

↓

Hidden Layer 1

↓

Hidden Layer 2

↓

Hidden Layer 3

↓

Output

Each hidden layer learns increasingly complex patterns.

For a LOFARGRAM:

- Early layers may detect edges or simple lines.
- Middle layers may detect tonal bands and harmonics.
- Deeper layers may recognize ship-specific patterns.

8. Output Layer

This produces the final prediction.

For your binary classification project:

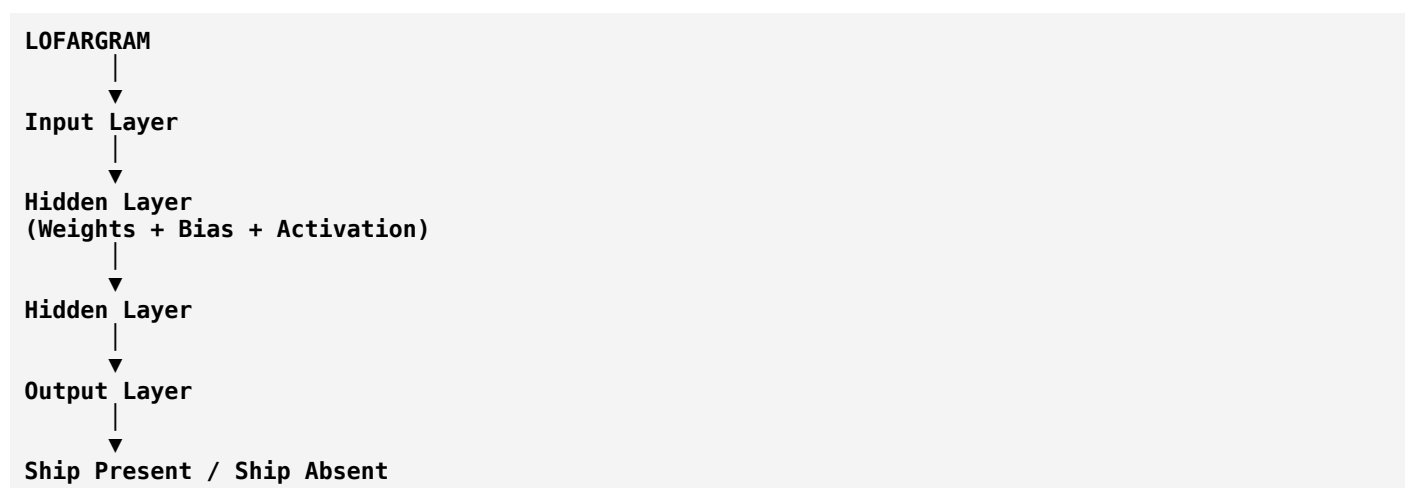
Ship Present = 0.94

or

Ship Absent = 0.06

The output layer gives the model's final decision.

Complete Neural Network



Real-Life Analogy

Imagine hiring people to inspect sonar recordings.

- **Input Layer** → Receives the recording.
- **Hidden Layers** → Teams of experts analyze different clues.
- **Weights** → Importance each expert gives to a clue.
- **Bias** → Personal adjustment to decision-making.
- **Activation Function** → Decides whether the evidence is strong enough.
- **Output Layer** → Final verdict: "Ship Present" or "Ship Absent."

Summary Table

Concept	Meaning	Example in Your Project
Artificial Neuron	Basic computing unit	Processes sonar features
Weight	Importance of an input	Engine frequency may get a higher weight
Bias	Adjustable offset	Helps shift the neuron's decision threshold
Activation Function	Decides the neuron's output	ReLU in hidden layers, Sigmoid in output
Input Layer	Receives data	LOFARGRAM image
Hidden Layer	Learns patterns	Detects tonal signatures and harmonics
Output Layer	Produces prediction	Ship Present / Ship Absent

One Important Clarification

Many beginners think:

One neuron = One complete AI model.

That's not true.

A single neuron can solve only very simple problems.

Real deep learning models contain **thousands to millions of neurons** working together. For example, CNNs like **ResNet** and **EfficientNet** have many layers with millions of trainable parameters.

What Comes Next?

Now that you understand the basic building blocks, we'll move to the topic that is **directly related to your project**:

Convolutional Neural Networks (CNNs)

We'll learn:

1. Why ordinary neural networks are not ideal for images.
2. What a convolution is.
3. What kernels (filters) are.
4. Feature maps.
5. Padding.
6. Stride.
7. Pooling.
8. Flatten.
9. Fully Connected Layers.

10. Why CNNs are perfect for **LOFARGRAMs and spectrograms**.

This is the most important deep learning topic for your passive sonar classification project.

Ok next

Excellent! ★

This is **the most important topic for your project**.

Your model will most likely take a **LOFARGRAM or Spectrogram** as input, and the model that processes these images is usually a **Convolutional Neural Network (CNN)**.

If you understand CNNs well, you'll understand almost every research paper related to sonar image classification.

Phase 5: Convolutional Neural Networks (CNN)

Before learning CNNs, let's answer one question.

Why Can't We Use a Normal Neural Network?

Suppose your LOFARGRAM image is:

128 × 128 pixels

Total pixels:

128 × 128 = 16,384

A normal neural network would connect **every pixel to every neuron**.

Example:

16,384 Inputs
↓
1000 Neurons

That means **millions of connections (weights)**.

Problems:

- Very high memory usage.
- Slow training.
- More prone to overfitting.
- Ignores the fact that nearby pixels are related.

CNNs solve these problems.

What is a CNN?

A **Convolutional Neural Network (CNN)** is a neural network designed specifically for image-like data. Instead of looking at the whole image at once, it looks at **small regions** and gradually learns patterns. Think of how a human examines a picture:

- First, you notice simple edges.
- Then shapes.
- Then complete objects.

A CNN works in a similar hierarchical way.

Your Project

Input:

LOFARGRAM Image

↓

CNN

↓

Prediction:

Ship Present

or

Ship Absent

What is Convolution?

This is the core operation of a CNN.

Instead of processing the whole image, a small matrix called a **kernel (or filter)** slides over the image. At every position, it performs a mathematical operation and produces a new value.

This process helps detect patterns.

Analogy

Imagine reading a large book.

You don't look at all pages simultaneously.

You read:

- Line 1
- Then Line 2
- Then Line 3

Similarly, a CNN examines an image **piece by piece**.

What is a Kernel (Filter)?

A **kernel** is a small matrix, such as:

3×3

or

5×5

It moves across the image searching for specific patterns.

Different kernels learn different features.

Examples:

- Horizontal lines
- Vertical lines
- Edges
- Corners
- Curves
- Textures

During training, the CNN learns the best kernel values automatically.

Sonar Example

Suppose a ship creates strong horizontal tonal lines in a LOFARGRAM.

A learned kernel may become very good at detecting those horizontal structures.

Another kernel may learn to detect:

- Engine harmonics
- Broadband noise
- Repeating frequency patterns

Each kernel specializes in a different feature.

What is a Feature Map?

After a kernel scans the image, it produces a new image.

This new image is called a **Feature Map** (also called an activation map).

Original Image



Kernel scans



Feature Map

The feature map highlights where the kernel found its pattern.

Example

Input:

LOFARGRAM

Kernel:

"Detect horizontal lines"



Feature Map:

Bright regions where horizontal tonal lines are present.

This makes it easier for later layers to recognize ships.

Multiple Kernels

A CNN doesn't use just one kernel.

It may learn dozens or even hundreds.

For example:

Kernel 1 → Horizontal lines

Kernel 2 → Vertical edges

Kernel 3 → Harmonics

Kernel 4 → Curved patterns

Each kernel creates its own feature map.

Together, they provide a rich description of the input image.

What is Stride?

Stride is the number of pixels the kernel moves each time.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Example:

Stride = 1

The kernel moves one pixel at a time.

Stride = 2

The kernel jumps two pixels at a time.

Effect

- Smaller stride → More detailed analysis, but slower computation.
- Larger stride → Faster computation, but some detail may be skipped.

What is Padding?

Suppose the kernel reaches the edge of the image.

Without padding, the output image becomes smaller after each convolution.

Padding solves this by adding extra pixels (usually zeros) around the border before applying the kernel.

Benefits:

- Preserves edge information.
- Helps maintain image size when desired.

What is Pooling?

After convolution, CNNs often perform **pooling**.

Pooling reduces the size of the feature map while keeping the most important information.

This makes computation faster and reduces overfitting.

Max Pooling

This is the most common type.

Suppose we have:

```
2 5
3 1
```

Max Pooling keeps:

```
5
```

It selects the maximum value from a small region.

Think of it as asking:

"What is the strongest feature in this area?"

Why is Pooling Useful for Sonar?

Imagine a ship's tonal line shifts slightly because of noise or movement.

Even if its position changes a little, pooling helps preserve the important feature.

This makes the CNN more robust to small variations.

What is Flatten?

After many convolution and pooling layers, the feature maps are still arranged as images.

Eventually, we convert them into a single long vector.

This process is called **Flattening**.

Example:

```
Feature Maps
  ↓
Flatten
  ↓
[0.2, 0.8, 0.4, ...]
```

This vector is then passed to the final classifier.

Fully Connected Layer

The flattened vector goes into one or more **Fully Connected (Dense) Layers**.

These layers combine all the extracted features to make the final decision.

Example:

```
Extracted Features
  ↓
Dense Layer
  ↓
Output
```

Final Output Layer

For your binary classification project, the output layer usually contains **one neuron** with a **Sigmoid** activation.

Example:

```
0.97
```

Interpretation:

97% probability that a ship is present.

Complete CNN Pipeline for Your Project



How a CNN "Sees" a LOFARGRAM

Instead of seeing:

"This is a ship."

It learns gradually:

- Layer 1 → Simple edges and lines.
- Layer 2 → Tonal bands and harmonics.
- Layer 3 → Complex frequency patterns.
- Layer 4 → Ship-specific acoustic signatures.
- Final Layer → Ship Present or Ship Absent.

This hierarchical learning is what makes CNNs so powerful.

Summary Table

Concept	Meaning	Role in Your Project
CNN	Neural network for image data	Processes LOFARGRAMs
Convolution	Scans the image with kernels	Detects local patterns
Kernel (Filter)	Small learnable matrix	Learns ship-related features
Feature Map	Output of convolution	Highlights detected patterns

Concept	Meaning	Role in Your Project
Stride	Kernel movement step	Controls scanning speed/detail
Padding	Adds border pixels	Preserves edge information
Pooling	Reduces feature map size	Keeps important features, reduces computation
Flatten	Converts feature maps into a vector	Prepares features for classification
Fully Connected Layer	Final decision-making layer	Combines learned features
Sigmoid	Produces probability (0–1)	Outputs Ship Present / Ship Absent

Why CNNs Are Ideal for Passive Sonar

Your LOFARGRAM is essentially an **image**, where:

- **X-axis** = Time
- **Y-axis** = Frequency
- **Pixel intensity** = Signal strength

Ship engines produce characteristic visual patterns—such as horizontal tonal lines and harmonic structures—that CNNs are very good at learning automatically. This is why CNN-based approaches are widely used in passive sonar image classification.

Next Topic

Now that you understand CNNs, we'll move to **Transfer Learning**, where you'll learn:

1. What is a pre-trained model?
2. Why use **ResNet-18** or **EfficientNet** instead of building a CNN from scratch?
3. Feature extraction vs fine-tuning.
4. Frozen vs trainable layers.
5. Why transfer learning is especially useful when your sonar dataset is relatively small.

This is the next key concept because many modern passive sonar classification systems are built by adapting pre-trained CNN models rather than training an entirely new network from scratch.

Ok next

Excellent! ★

Now you've reached one of the **most practical topics** in Deep Learning.

If you read recent research papers on passive sonar, you'll often see models like:

- **ResNet-18**
- **EfficientNet**

- **MobileNet**
- **DenseNet**
- **VGG16**

You may wonder:

"Why don't researchers build their own CNN from scratch?"

The answer is **Transfer Learning**.

Phase 6: Transfer Learning

First, What is the Problem?

Suppose you want to build a CNN from scratch.

Your dataset contains:

2,000 LOFARGRAM images

Is this enough?

Usually **no**.

Modern CNNs have **millions of parameters**. Training them well often requires **tens or hundreds of thousands of images** (or more, depending on the model and task).

With only a small dataset, the model may **overfit**—it memorizes the training data instead of learning patterns that generalize.

The Solution: Transfer Learning

Definition

Transfer Learning means taking a model that has already learned useful visual features from a large dataset and adapting it to solve your new task.

Instead of starting from zero, you start with an experienced model.

Human Analogy

Imagine two people.

Person A

Never learned to drive.

Teaching them takes months.

Person B

Already knows how to drive a car.

Now teach them to drive a truck.

They learn much faster because they already understand:

- Steering
- Braking
- Traffic rules

This is **transfer learning**.

Existing knowledge is transferred to a new task.

Deep Learning Example

Suppose a CNN has already learned from millions of images.

It already understands basic visual concepts like:

- Edges
- Lines
- Curves
- Shapes
- Textures

Instead of training a new CNN,
you reuse this knowledge.

Then teach it:

Ship Present

Ship Absent

Only the final part of the network needs to adapt to your specific problem.

Pre-trained Model

A **pre-trained model** is a model that has already been trained on a large dataset.

It has already learned useful features.

Examples include:

- ResNet-18
- EfficientNet

- MobileNet
- DenseNet

These models are commonly pre-trained on a large image dataset called **ImageNet**, which contains millions of labeled images.

Even though ImageNet contains everyday images (animals, vehicles, objects), the early layers learn general features like edges and textures that are useful for many vision tasks—including spectrograms and LOFARGRAMs.

Why Can It Help with Sonar?

You might ask:

"Ships don't look like cats or cars. Why use an ImageNet model?"

Great question.

The first layers of a CNN don't recognize ships or cats.

They learn **basic visual patterns** such as:

- Edges
- Horizontal lines
- Vertical lines
- Curves
- Textures

A LOFARGRAM also contains these kinds of visual structures.

So those learned low-level features can still be valuable.

The deeper layers are then adapted to recognize sonar-specific patterns.

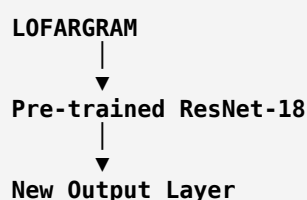
Feature Extraction

One way to use transfer learning is **Feature Extraction**.

Here:

- Keep almost all of the pre-trained model unchanged.
- Replace the final classification layer with a new one for your task.

Example:



↓
Ship / No Ship

Only the new output layer is trained.

Fine-Tuning

Another approach is **Fine-Tuning**.

Here:

- Start with a pre-trained model.
- Continue training some (or all) of its layers using your sonar dataset.

The model gradually adjusts its existing knowledge to your specific domain.

Feature Extraction vs Fine-Tuning

Feature Extraction	Fine-Tuning
Most pre-trained layers are kept fixed	Some or all layers continue learning
Faster training	Slower training
Needs less data	Usually benefits from more data
Lower risk of overfitting	More flexible, but higher overfitting risk if data is limited

Frozen Layers

A **frozen layer** is a layer whose parameters are **not updated** during training.

It keeps the knowledge learned from pre-training.

Example:

Layer 1

Frozen

↓

Layer 2

Frozen

↓

Layer 3

Frozen



Output Layer

Trainable

Trainable Layers

These layers continue learning.

Their parameters change during training.

Example:

Last Few Layers

Trainable

They gradually adapt to sonar data.

Which Strategy is Better?

It depends on your dataset.

Small Dataset

Use:

- Feature Extraction
- Freeze most layers

This reduces the risk of overfitting.

Larger Dataset

Use:

- Fine-Tuning

The model can learn more sonar-specific features.

Sonar Example

Suppose you have:

4,000 LOFARGRAM images

A common approach is:

```

ResNet-18
|
Freeze Early Layers
|
Fine-Tune Later Layers
|
Train New Classifier
|
Ship Present / Ship Absent

```

The early layers keep their general visual knowledge.

The later layers specialize in detecting sonar signatures.

Why Researchers Use ResNet-18

ResNet-18 is popular because it:

- Is relatively lightweight.
- Trains faster than deeper ResNet variants.
- Often performs well on smaller datasets.
- Uses **skip (residual) connections**, which help very deep networks learn more effectively.

We'll discuss residual connections when we study model architectures.

Why Researchers Use EfficientNet

EfficientNet is designed to achieve strong accuracy while using fewer parameters and less computation.

It balances:

- Network depth
- Width
- Input image resolution

This often makes it a good choice when computational resources are limited.

Complete Transfer Learning Pipeline

```

LOFARGRAM
|
▼
Pre-trained CNN
(ResNet-18 / EfficientNet)
|
▼
Feature Extraction
|
▼
Fine-Tuning (Optional)
|
▼
New Classifier
|

```

Summary Table

Concept	Meaning	Role in Your Project
Transfer Learning	Reusing knowledge from a pre-trained model	Faster and better training
Pre-trained Model	Model already trained on a large dataset	ResNet-18, EfficientNet
Feature Extraction	Keep most layers fixed and train a new classifier	Good for small datasets
Fine-Tuning	Continue training some or all layers	Adapts the model to sonar data
Frozen Layers	Parameters do not change	Preserve pre-trained knowledge
Trainable Layers	Parameters continue updating	Learn sonar-specific patterns

Real-Life Analogy

Imagine hiring an experienced music teacher to teach a student to recognize ship engine sounds.

The teacher already understands:

- Sound patterns
- Rhythm
- Frequency

Now they only need to learn:

- What a ship engine sounds like.

That's much easier than training someone with no experience at all.

Transfer learning works the same way.

Congratulations! 🎉

At this point, you've built a solid conceptual foundation in:

- ☒ AI, ML, and Deep Learning
- ☒ Datasets, Features, and Labels
- ☒ Training, Validation, and Test Sets
- ☒ Models, Parameters, and Hyperparameters
- ☒ Epochs, Batches, and Iterations
- ☒ Forward Propagation and Loss
- ☒ Gradient Descent and Backpropagation
- ☒ Neural Networks

-  CNNs
-  Transfer Learning

Next Phase

The next topic is **Model Evaluation**, where you'll learn how to answer:

"Is my ship classifier actually good?"

We'll cover:

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score
- Sensitivity
- Specificity
- ROC Curve
- AUC
- Precision-Recall Curve

These metrics are essential because **accuracy alone can be misleading**, especially if your sonar dataset is imbalanced (for example, if "No Ship" recordings greatly outnumber "Ship Present" recordings). Understanding these metrics will help you correctly evaluate and compare different models.

Ok next

Excellent! 

This is one of the **most important phases** of Machine Learning.

Many beginners think:

"My model has 98% accuracy, so it's excellent."

Not necessarily.

A model can have **98% accuracy and still be almost useless**.

Let's understand why.

Phase 7: Model Evaluation Metrics

Suppose your trained model predicts:

LOFARGRAM
↓
CNN
↓
Ship Present

How do we know if the prediction is good?

We use **evaluation metrics**.

The most important one is the **Confusion Matrix**.

1. What is a Confusion Matrix?

A **Confusion Matrix** is a table that compares:

- What the model predicted
- What the correct answer actually was

For a binary classification problem like yours:

Actual \ Predicted	Ship Present	Ship Absent
Ship Present	TP	FN
Ship Absent	FP	TN

This table is the foundation of almost every evaluation metric.

2. Understanding TP, TN, FP, FN

These four terms are extremely important.

True Positive (TP)

A ship is actually present.

The model predicts:

Ship Present

Correct!

Example:

Actual	Prediction
Ship	Ship

This is **True Positive**.

True Negative (TN)

No ship is actually present.

The model predicts:

Ship Absent

Correct.

Actual	Prediction
No Ship	No Ship

This is **True Negative**.

False Positive (FP)

There is **no ship**.

The model predicts:

Ship Present

Wrong.

This is also called a **False Alarm**.

In sonar systems, false alarms can waste time and resources because operators investigate something that isn't a ship.

False Negative (FN)

A ship **is present**.

The model predicts:

Ship Absent

Wrong.

This means the system **missed a real ship**.

For many defense applications, this is often considered the most serious type of error.

Visual Summary

Actual	Prediction	Result
Ship	Ship	True Positive (TP) 

Actual	Prediction	Result
No Ship	No Ship	True Negative (TN) ✓
No Ship	Ship	False Positive (FP) ✗
Ship	No Ship	False Negative (FN) ✗

Example

Suppose you test the model on **100 sonar recordings**.

Results:

TP = 40

TN = 50

FP = 5

FN = 5

Now let's calculate the metrics.

3. Accuracy

Definition

Accuracy tells us:

How many predictions were correct overall?

Formula:

Accuracy

=

(TP + TN)

/

(TP + TN + FP + FN)

Example:

(40 + 50)

/

100

=

90%

The model is correct **90% of the time**.

Problem with Accuracy

Imagine this dataset:

990
No Ship
10
Ship

Now imagine a lazy model that always predicts:

No Ship

Correct predictions:

990

Wrong predictions:

10

Accuracy:

99%

Looks amazing!

But the model **never detects a ship**.

So accuracy alone can be very misleading, especially with **imbalanced datasets**.

4. Precision

Definition

Precision answers:

When the model predicts "Ship Present," how often is it correct?

Formula:

Precision
=
TP
/
(TP + FP)

Example:

TP = 40

FP = 5

Precision:

40 / 45

=

88.9%

High precision means **few false alarms**.

Sonar Meaning

High Precision

↓

If the system says

Ship Present

it is usually correct.

Operators waste less time checking false detections.

5. Recall (Sensitivity)

Definition

Recall answers:

Of all the real ships, how many did the model successfully detect?

Formula:

Recall

=

TP

/

(TP + FN)

Example:

TP = 40

FN = 5

Recall:

40 / 45

=

88.9%

Sonar Meaning

High Recall

↓

The system misses very few ships.

This is often extremely important in surveillance applications.

Precision vs Recall

Imagine a radar operator.

High Precision

Whenever they report a ship,
they are almost always correct.

But they may miss some ships.

High Recall

They detect almost every ship.

But sometimes they also report false alarms.

There is often a trade-off between precision and recall.

6. F1 Score

Sometimes we want one number that balances both precision and recall.

That is the **F1 Score**.

It becomes high only when **both precision and recall are high**.

Example:

Precision:

90%

Recall:

90%

↓

High F1 Score.

If one is very low, the F1 Score also drops.

7. Specificity

Specificity answers:

Of all the recordings with no ship, how many did the model correctly identify as "No Ship"?

Formula:

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

High specificity means the model produces fewer false alarms.

8. ROC Curve

A classifier often outputs probabilities, such as:

Ship Present = 0.82

You choose a threshold (for example, 0.5) to decide whether to classify it as "Ship Present."

By changing this threshold, the balance between true positives and false positives changes.

A **ROC (Receiver Operating Characteristic) Curve** shows this trade-off across many thresholds.

9. AUC

AUC (Area Under the ROC Curve) summarizes the ROC curve into a single number.

General interpretation:

- **1.0** → Excellent separation between classes.
- **0.5** → About as good as random guessing.

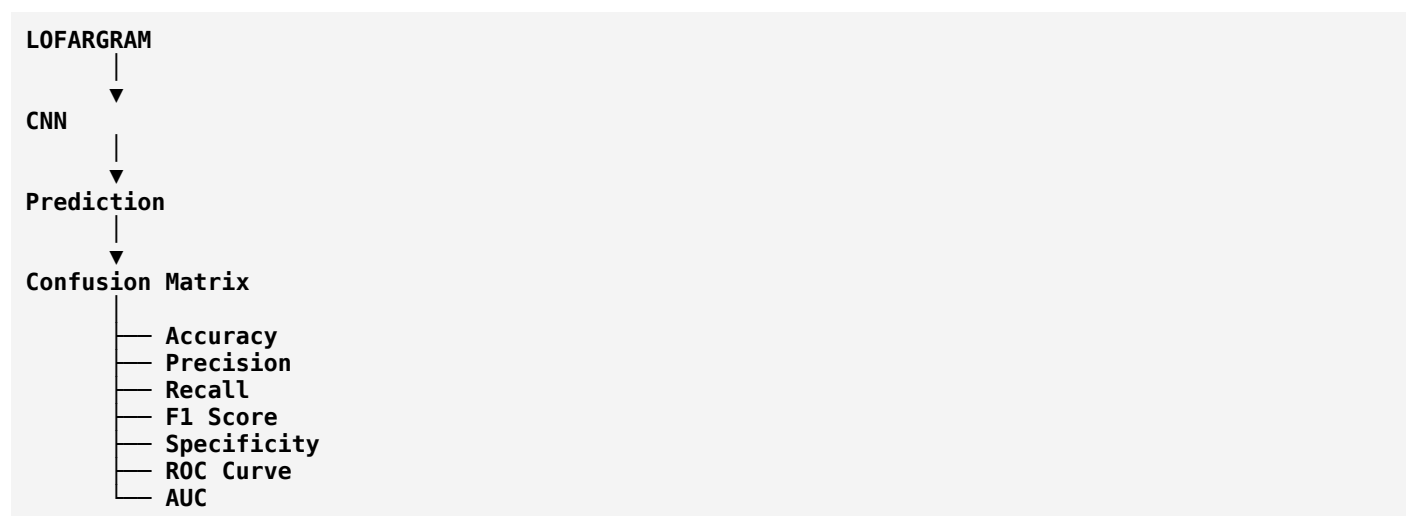
Higher AUC generally indicates better discrimination between "Ship Present" and "Ship Absent."

10. Precision-Recall Curve

When your dataset is **imbalanced** (for example, far more "No Ship" than "Ship Present" samples), the **Precision-Recall (PR) Curve** is often more informative than the ROC curve.

Since many passive sonar datasets can be imbalanced, researchers frequently report both ROC-AUC and PR-AUC.

Complete Evaluation Flow



Summary Table

Metric	Answers the Question	Why It Matters
Confusion Matrix	What was predicted vs what was correct?	Foundation of all metrics
Accuracy	How many predictions were correct overall?	Good for balanced datasets
Precision	When the model predicts "Ship," how often is it right?	Reduces false alarms
Recall (Sensitivity)	How many real ships were detected?	Reduces missed ships
F1 Score	Is there a good balance between precision and recall?	Useful when both are important
Specificity	How well are "No Ship" recordings identified?	Measures rejection of false alarms
ROC Curve	How performance changes with the threshold	Evaluates classifiers across thresholds
AUC	Overall ability to separate classes	Higher is generally better
PR Curve	Precision vs Recall across thresholds	Especially useful for imbalanced datasets

Which Metrics Matter Most for Your Project?

For a **Passive Sonar Ship Detection** system, these are especially important:

1. **Recall (Sensitivity)** – Missing a real ship (False Negative) can be very costly.
2. **Precision** – Too many false alarms reduce operator trust and waste resources.

3. **F1 Score** – Good overall balance when both missed detections and false alarms matter.

4. **ROC-AUC / PR-AUC** – Useful for comparing different models and thresholds.

Accuracy is still useful, but it **should not be the only metric** you rely on.

Congratulations! 🎉

You now understand the complete conceptual pipeline from:

- Digital Audio Processing (FFT, STFT, Spectrogram, Mel Spectrogram, MFCC, LOFARGRAM)
- Machine Learning fundamentals
- Neural Networks
- CNNs
- Transfer Learning
- Model Evaluation

You now have the background needed to start reading passive sonar research papers with much greater confidence. The next natural step is to study **popular CNN architectures** (ResNet, EfficientNet, MobileNet, DenseNet, VGG) and understand **why researchers choose one architecture over another** for sonar classification tasks. That will make it much easier to understand the models used in your project.

What about deep learning basics? have we learned everything that we have to learn or to know?

That's a very good question. The answer is:

No. We have built a strong foundation, but we haven't learned everything yet.

What we've covered is roughly **50–60% of the deep learning concepts** you'll need for your project. We now understand *how* models learn, but not yet *why different architectures work* or *how they're designed*.

Here's where you stand:

✅ Completed (Strong Foundation)

1. Digital Audio Processing

- ✅ Fourier Transform
- ✅ FFT
- ✅ STFT
- ✅ Spectrogram
- ✅ Mel Spectrogram
- ✅ MFCC

- ☒ LOFARGRAM
-

2. Machine Learning Basics

- ☒ AI vs ML vs DL
 - ☒ Supervised Learning
 - ☒ Dataset
 - ☒ Sample
 - ☒ Feature
 - ☒ Label
 - ☒ Binary Classification
 - ☒ Training / Validation / Test Sets
-

3. Model Fundamentals

- ☒ Model
 - ☒ Parameters
 - ☒ Hyperparameters
 - ☒ Prediction
 - ☒ Inference
-

4. Training Process

- ☒ Epoch
 - ☒ Batch
 - ☒ Batch Size
 - ☒ Iteration
 - ☒ Forward Propagation
 - ☒ Loss Function
 - ☒ Gradient
 - ☒ Gradient Descent
 - ☒ Learning Rate
 - ☒ Backpropagation
 - ☒ Optimizer
-

5. Neural Networks

- ☒ Artificial Neuron
 - ☒ Perceptron
 - ☒ Weights
 - ☒ Bias
 - ☒ Activation Functions
 - ☒ Hidden Layers
 - ☒ Output Layer
-

6. CNN Basics

- ☒ Convolution
 - ☒ Kernel
 - ☒ Feature Map
 - ☒ Stride
 - ☒ Padding
 - ☒ Pooling
 - ☒ Flatten
 - ☒ Fully Connected Layer
-

7. Transfer Learning

- ☒ Pre-trained Models
 - ☒ Feature Extraction
 - ☒ Fine-Tuning
 - ☒ Frozen Layers
 - ☒ Trainable Layers
-

8. Evaluation

- ☒ Confusion Matrix
- ☒ Accuracy
- ☒ Precision
- ☒ Recall

-  F1 Score
-  Specificity
-  ROC
-  AUC

Still Remaining (Very Important)

These topics are commonly used in research papers and practical deep learning.

Phase A: Deep Learning Architectures ★★★★★

- ResNet
- EfficientNet
- VGG
- MobileNet
- DenseNet
- Inception
- AlexNet

These explain **why researchers choose one model over another.**

Phase B: Regularization ★★★★★

These help prevent overfitting.

- Overfitting
- Underfitting
- Bias vs Variance
- Dropout
- Weight Decay (L2 Regularization)
- Early Stopping
- Data Augmentation

Phase C: Optimization ★★★★★

Go a bit deeper into:

- SGD

- Adam
 - RMSProp
 - AdamW
 - Momentum
 - Learning Rate Scheduler
 - Warm-up
-

Phase D: Loss Functions ★★★★★

Understand when to use:

- Binary Cross-Entropy
 - Categorical Cross-Entropy
 - Focal Loss
 - Dice Loss (good to know, common in segmentation)
-

Phase E: Practical Deep Learning ★★★★★

These are essential when implementing models.

- Tensor
 - Tensor Shape
 - Channels
 - Batch Dimension
 - GPU
 - CUDA
 - TensorFlow vs PyTorch
 - Model Checkpoint
 - Saving & Loading Models
-

Phase F: Reading Research Papers ★★★★★

Learn terms like:

- Baseline
- Proposed Model
- State-of-the-Art (SOTA)

- Ablation Study
- Benchmark Dataset
- Generalization

Phase G: Sonar-Specific Deep Learning ★★★★★

This is directly related to your project.

- Why CNN works on spectrograms
- Why ResNet-18 is popular in sonar
- Why EfficientNet performs well
- Attention Mechanisms (basic idea)
- LSTM / Bi-LSTM for temporal information
- CNN + LSTM hybrid models
- Vision Transformers (ViT) (basic understanding)

What We DON'T Need (For Your Current Project)

You don't need to master everything in deep learning right now, such as:

- Generative Adversarial Networks (GANs)
- Diffusion Models
- Large Language Models (LLMs)
- Reinforcement Learning
- Graph Neural Networks (GNNs)
- Object Detection models (YOLO, Faster R-CNN)
- Image Segmentation models (U-Net, Mask R-CNN)

These are valuable topics, but they aren't necessary for a passive sonar binary classification project.

My Recommended Learning Order

This is the path I'd suggest from here:

- Phase 1  Digital Audio Processing
- Phase 2  Machine Learning Basics
- Phase 3  Training Process
- Phase 4  Neural Networks
- Phase 5  CNN

Phase 6  Transfer Learning

Phase 7  Evaluation Metrics

NOW CONTINUE WITH

Phase 8 → Overfitting & Underfitting

↓

Phase 9 → Regularization Techniques

↓

Phase 10 → Deep CNN Architectures
(ResNet, EfficientNet, etc.)

↓

Phase 11 → Practical Deep Learning Concepts

↓

Phase 12 → Sonar-Specific Deep Learning

↓

Phase 13 → Reading Research Papers

↓

Phase 14 → Implementation (PyTorch/TensorFlow)

My recommendation

Before jumping into **ResNet** and **EfficientNet**, I recommend learning:

1. **Overfitting & Underfitting**
2. **Bias & Variance**
3. **Dropout**
4. **Early Stopping**
5. **Data Augmentation**

The reason is that these concepts explain **why models succeed or fail**. Once you understand them, the design choices in architectures like ResNet and EfficientNet will make much more sense, and you'll be able to understand research papers much more easily.

After that, we'll study **ResNet-18** and **EfficientNet** in depth, since those are the architectures most relevant to your passive sonar classification project.

Ok start next

Perfect. From this point onward, we'll focus on the remaining deep learning concepts that are **directly useful for your passive sonar project**.

Phase 8: Overfitting, Underfitting & Bias-Variance Tradeoff

This is one of the **most important concepts** in deep learning.

Many students build a CNN, get **99% training accuracy**, and think:

"My model is amazing!"

But when they test it on new data, the accuracy drops to **60%**.

Why?

The answer is **Overfitting**.

Imagine This Scenario

Suppose you have:

- **Training Dataset:** 8,000 LOFARGRAM images
- **Test Dataset:** 2,000 LOFARGRAM images

After training:

```
Training Accuracy = 99%  
Test Accuracy = 58%
```

This looks strange.

If the model is so good, why is the test accuracy so low?

Because it **memorized** the training data instead of learning general patterns.

This is called **Overfitting**.

1. What is Underfitting?

Definition

Underfitting occurs when the model is **too simple** to learn the patterns in the data.

It performs poorly on both the training and test datasets.

Example

```
Training Accuracy = 60%  
Test Accuracy = 58%
```

The model cannot even learn the training data well.

Student Analogy

A student studies only the chapter titles before an exam.

Result:

- Performs poorly in practice tests.
- Performs poorly in the final exam.

The student didn't learn enough.

That's **underfitting**.

2. What is Overfitting?

Definition

Overfitting occurs when the model memorizes the training data instead of learning general patterns. It performs very well on the training set but poorly on unseen data.

Example

Training Accuracy = 99%
Test Accuracy = 62%

The model remembers the exact training examples but struggles with new recordings.

Student Analogy

Imagine a student memorizes the answers to last year's question paper.

If the same questions appear:

✓ 100%

If new questions appear:

✗ Poor performance.

The student memorized instead of understanding.

This is exactly what an overfit model does.

3. What is Good Fitting?

This is what we want.

Example:

Training Accuracy = 95%
Test Accuracy = 93%

The model has learned the underlying patterns rather than memorizing the data.

Visual Comparison

Underfitting

Training = 60%
Testing = 58%

Model hasn't learned enough.

Good Fit

Training = 95%
Testing = 93%

Model generalizes well.

Overfitting

Training = 99%
Testing = 60%

Model memorized the training data.

Why Does Overfitting Happen?

Common reasons include:

- Dataset is too small.
- Model is too complex.
- Training for too many epochs.
- Too many parameters.
- Too much noise in the data.

For sonar data, overfitting is common because collecting large, labeled underwater datasets is difficult.

Why Is Overfitting Dangerous?

Imagine your sonar system is deployed at sea.

If it only recognizes ships that look like the training examples, it may fail to detect ships in different ocean conditions or with different noise patterns.

A good model must work well on **new** data, not just the data it has already seen.

4. What is Bias?

In machine learning (not the neuron bias we discussed earlier), **bias** refers to error caused by an overly simple model.

High bias means:

"The model cannot capture the true relationship in the data."

Characteristics:

- Low training accuracy.
- Low test accuracy.
- Underfitting.

5. What is Variance?

Variance refers to how much a model changes when trained on different datasets.

High variance means:

"The model is too sensitive to the training data."

Characteristics:

- Very high training accuracy.
- Poor test accuracy.
- Overfitting.

Bias vs Variance

High Bias	High Variance
Model is too simple	Model is too complex
Underfitting	Overfitting
Misses important patterns	Memorizes training data

The goal is to find a balance between the two.

How Do We Detect Overfitting?

Compare the training and validation performance during training.

Example:

Epoch	Training Accuracy	Validation Accuracy
1	65%	64%
5	82%	81%
10	92%	91%
20	99%	85%
30	100%	78%

Notice:

- Training accuracy keeps increasing.
- Validation accuracy starts decreasing.

This is a classic sign of **overfitting**.

Sonar Example

Suppose your CNN learns:

- Exact background noise from one recording location.
- Exact ocean conditions from your training set.

When tested in a different sea region, performance drops.

The model learned **location-specific noise** instead of **ship characteristics**.

That's overfitting.

How Can We Reduce Overfitting?

We'll study these in detail next:

- ☒ More training data
- ☒ Data Augmentation
- ☒ Dropout
- ☒ Early Stopping
- ☒ Weight Decay (L2 Regularization)
- ☒ Transfer Learning
- ☒ Batch Normalization (good to know)

Summary Table

Concept	Meaning	Result
Underfitting	Model is too simple	Poor training and test performance
Good Fit	Learns general patterns	High training and test performance
Overfitting	Memorizes training data	High training, poor test performance
High Bias	Can't learn enough	Underfitting
High Variance	Learns too specifically	Overfitting

Key Takeaways

- **Training accuracy alone is not enough.**
- **Validation performance tells you how well the model generalizes.**
- The ultimate goal is **good performance on unseen sonar recordings**, not perfect performance on the training set.

Next Lesson

Now that you know what overfitting is, the next lesson is:

Regularization Techniques

We'll cover:

1. **Dropout** (one of the most important techniques in deep learning)
2. **Early Stopping**
3. **Data Augmentation**
4. **Weight Decay (L2 Regularization)**
5. **Batch Normalization**

These techniques are used in almost every modern CNN—including ResNet and EfficientNet—to improve generalization and reduce overfitting.

Ok next

Excellent! ★

This is one of the **most practical topics** in Deep Learning.

Almost every research paper you read will mention one or more of these techniques.

These methods don't make the model "smarter"—they make it **generalize better**, meaning it performs well on **new sonar recordings**, not just the training data.

Phase 9: Regularization Techniques

What is Regularization?

Definition

Regularization refers to techniques used to **reduce overfitting** and improve a model's ability to generalize to unseen data.

Think of it as preventing a student from memorizing answers and instead encouraging them to understand the subject.

1. Dropout

What is Dropout?

Imagine a classroom where the same few students always answer every question.

The others never learn.

A teacher decides that in each class, some of the active students must stay silent so others are forced to participate.

This helps the whole class learn better.

That's the idea behind **Dropout**.

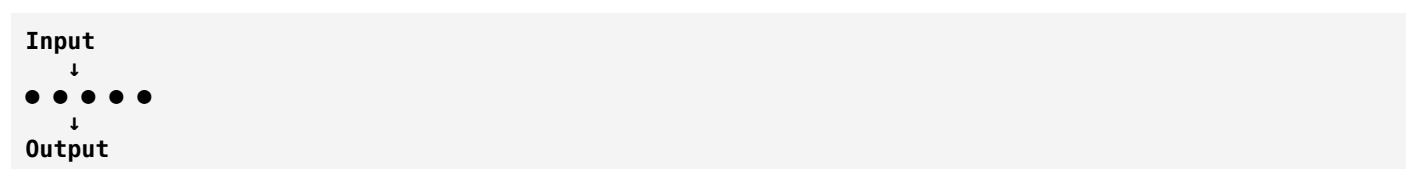
How Dropout Works

During training:

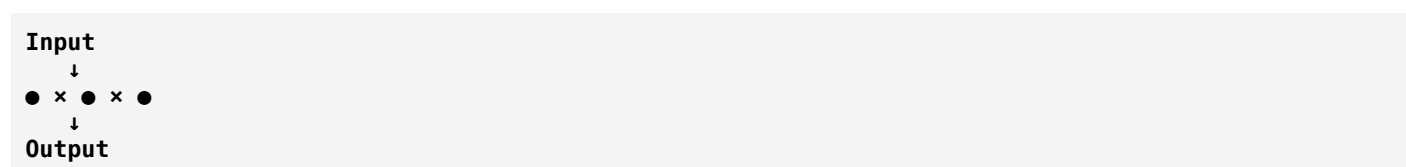
Some neurons are **temporarily turned off (dropped)**.

Example:

Without Dropout



With Dropout



- • = Active neuron
- x = Dropped neuron

The dropped neurons do **not** participate in that training step.

In the next batch, a different set of neurons may be dropped.

Why Use Dropout?

Without dropout:

- The network may rely too much on a few neurons.
- It can memorize the training data.

With dropout:

- The network learns more robust features.
 - It becomes less dependent on any single neuron.
 - Overfitting is reduced.
-

Dropout Rate

The **dropout rate** is the fraction of neurons dropped during training.

Examples:

- 0.2 → Drop 20% of neurons.
- 0.5 → Drop 50% of neurons.

Common values are **0.2 to 0.5**, but the best choice depends on the model and dataset.

During Testing

An important point:

Dropout is used only during training.

During validation and testing:

- All neurons are active.
- The full network is used to make predictions.

2. Early Stopping

The Problem

Suppose you train for 100 epochs.

Training accuracy keeps improving.

But validation accuracy behaves like this:

Epoch	Validation Accuracy
5	80%
10	88%
15	92%
20	93% 
25	92%
30	90%
40	86%

After epoch 20, the model starts to overfit.

Solution

Stop training when validation performance stops improving.

This is called **Early Stopping**.

Instead of training for 100 epochs, you might stop at epoch 20 and keep the best-performing model.

Analogy

A student studies:

- Day 1 → Learns.
- Day 2 → Learns more.
- Day 3 → Learns more.
- Day 20 → Fully prepared.

After that, they keep rereading the same notes without gaining new understanding.

Stopping at the right time is more effective.

3. Data Augmentation

What is Data Augmentation?

Instead of collecting more real data, we create **new training samples** by applying transformations to existing ones.

For ordinary images, examples include:

- Rotation
- Cropping
- Brightness adjustment
- Horizontal flipping (when appropriate)

This helps the model see more variation.

Sonar-Specific Note

For **LOFARGRAMs** and **spectrograms**, augmentation must be chosen carefully.

Some common image augmentations (like arbitrary rotations or horizontal flips) may change the meaning of the data because the axes represent **time** and **frequency**.

Researchers often use augmentations that preserve the physical meaning of the signal, such as:

- Small time shifts

- Controlled noise addition
- Time masking
- Frequency masking (similar to SpecAugment)
- Slight amplitude scaling

The right augmentation depends on the application and should reflect realistic sonar conditions.

Why It Helps

More variation means:

- Less memorization.
 - Better generalization.
 - Better performance on unseen recordings.
-

4. Weight Decay (L2 Regularization)

The Idea

During training, some weights may become unnecessarily large.

Very large weights can make the model fit the training data too closely.

Weight Decay adds a penalty for large weights.

This encourages the model to keep weights smaller unless they are truly needed.

Analogy

Imagine a company gives bonuses based on performance.

But if someone asks for an extremely large bonus, there's a penalty.

Employees are encouraged to make reasonable choices.

Similarly, weight decay discourages excessively large parameter values.

5. Batch Normalization

The Problem

As data flows through many layers, its distribution changes.

This can make training slower and less stable.

Solution

Batch Normalization normalizes the activations within a mini-batch before passing them to the next layer.

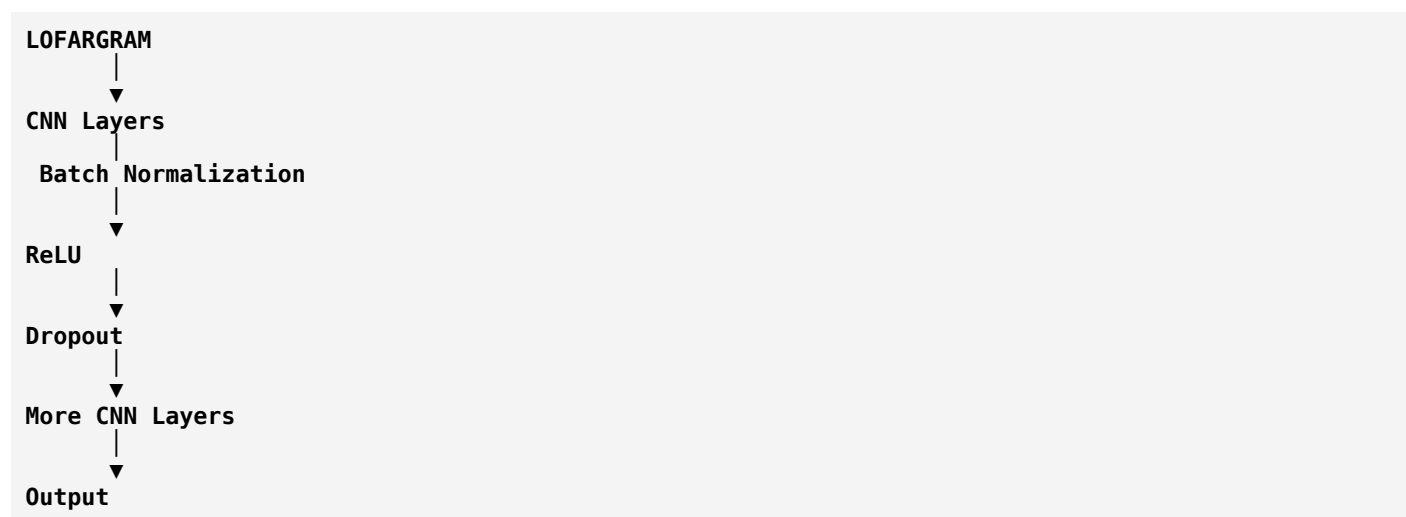
Benefits include:

- Faster training.
- More stable optimization.
- Often allows higher learning rates.
- Can provide a small regularization effect.

Batch Normalization does **not** replace dropout—they solve different problems and are often used together.

Putting It All Together

Imagine training a CNN for your sonar project.



During training:

- Batch Normalization stabilizes learning.
- Dropout reduces overfitting.
- Weight Decay keeps weights under control.
- Early Stopping prevents training too long.
- Data Augmentation provides more diverse training examples.

Together, these techniques help produce a model that performs well on **new sonar recordings**.

Summary Table

Technique	Purpose	Main Benefit
Dropout	Randomly disable neurons during training	Reduces overfitting
Early Stopping	Stop when validation stops improving	Prevents over-training
Data Augmentation	Create realistic variations of training data	Improves generalization
Weight Decay (L2)	Penalize very large weights	Encourages simpler models
Batch Normalization	Normalize activations during training	Faster, more stable training

Real-Life Analogy

Preparing for an exam:

- **Dropout** → Practice without always relying on your strongest friend.
- **Early Stopping** → Stop revising once you're well prepared instead of studying endlessly.
- **Data Augmentation** → Solve many different types of practice questions.
- **Weight Decay** → Avoid overcomplicating your answers.
- **Batch Normalization** → Keep your learning process organized and consistent.

Final Takeaway

A good deep learning model is **not** the one with the highest training accuracy.

A good model is the one that performs reliably on **new, unseen data**.

Regularization techniques help achieve that goal.

Next Phase (One of the Most Important)

Now we'll move to **Deep CNN Architectures**, where you'll learn:

1. **AlexNet** – The CNN that sparked the modern deep learning revolution.
2. **VGG16** – Simplicity through deeper networks.
3. **GoogLeNet (Inception)** – Efficient multi-scale feature extraction.
4. **ResNet** – Residual (skip) connections and why they solved the vanishing gradient problem.
5. **DenseNet** – Dense connectivity between layers.
6. **MobileNet** – Lightweight CNNs for mobile and embedded devices.
7. **EfficientNet** – Balancing depth, width, and resolution for excellent accuracy and efficiency.

These architectures are especially important because research papers on passive sonar classification frequently compare or build upon them, particularly **ResNet** and **EfficientNet**.

Ok let's go for these deep CNN architecture but this time don't tell about every architecture at once tell about the each one by one and give me clear much more clarity I have to learn indepth working of each it's upto you how you will make me to get the clarity

Perfect. I actually **prefer** teaching it this way.

If I explain all architectures (AlexNet, VGG, ResNet, DenseNet, MobileNet, EfficientNet) in one lesson, you'll only memorize names.

Instead, we'll study them **like a university course**, where each architecture builds on the limitations of the previous one.

This way, when you read a research paper that says:

"We used ResNet-18 because of residual learning..."

you'll immediately understand **why**, not just **what**.

Our New Roadmap (Deep CNN Architectures)

We'll spend several lessons on each architecture.

```
Part 1 → History of CNN Evolution
Part 2 → AlexNet (3-4 lessons)
Part 3 → VGGNet (2-3 lessons)
Part 4 → GoogLeNet/Inception (3-4 lessons)
Part 5 → ResNet (5-6 lessons)
Part 6 → DenseNet (3 lessons)
Part 7 → MobileNet (3 lessons)
Part 8 → EfficientNet (4-5 lessons)
Part 9 → Comparison of all architectures
Part 10 → Which architecture is best for your sonar project?
```

We'll understand:

- Why it was invented.
- What problem it solved.
- Its architecture.
- How data flows inside it.
- Advantages.
- Limitations.
- Why researchers still use it (or don't).
- Why it matters for your passive sonar project.

Lesson 1: The Evolution of CNNs (Before AlexNet)

Before studying AlexNet, you need to understand **why AlexNet changed the world**.

Every architecture exists because the previous one had a limitation.

Deep learning evolved by solving one problem after another.

Before 2012: Computer Vision Was Very Different

Imagine you want to detect a ship from a LOFARGRAM today.

You would probably do:

```
LOFARGRAM
  ↓
CNN
  ↓
Prediction
```

Simple.

But before 2012, researchers couldn't do this.

Instead, they followed a long pipeline.

Traditional Machine Learning Pipeline

```
Raw Signal
  ↓
FFT
  ↓
Spectrogram
  ↓
Handcrafted Features
(MFCC, HOG, SIFT, etc.)
  ↓
Classifier
(SVM, Random Forest)
  ↓
Prediction
```

Notice something important.

The computer **did not learn the features**.

Humans manually designed them.

What are Handcrafted Features?

Suppose you want to detect ships.

An engineer might say:

"Ship engines usually have strong harmonic frequencies."

So they manually calculate:

- Energy
- MFCC
- Spectral Centroid

- Zero Crossing Rate
- Band Energy

Then they feed these numbers into an SVM.

The success of the system depends heavily on whether the engineer chose good features.

The Biggest Problem

Imagine you're a doctor.

Every patient is different.

Would you want to manually write rules for every disease?

Probably not.

You'd rather have a system that learns patterns automatically.

The same applies to images and sonar.

Researchers wanted a model that could say:

"Don't tell me which features to use. I'll learn them myself."

That idea led to deep learning.

Why Was It Difficult?

People already knew about neural networks in the 1980s and 1990s.

So why didn't they use them?

Because of three major problems.

Problem 1: Small Datasets

Neural networks need a lot of data.

Back then, image datasets were tiny.

Example:

```
500 images  
1000 images  
2000 images
```

That's nowhere near enough to train a deep network.

Problem 2: Slow Computers

Training a deep network requires billions of calculations.

CPUs were simply too slow.

Training one large model could take weeks or months.

Then researchers realized something important:

Graphics cards (GPUs) are extremely good at performing many calculations in parallel.

This changed everything.

Problem 3: Better Training Techniques Were Needed

Researchers gradually developed:

- Better activation functions (like ReLU)
- Better optimization algorithms
- Better initialization methods
- Better regularization techniques (Dropout)
- Larger labeled datasets

These advances made deep networks practical.

Then Came ImageNet

One of the biggest breakthroughs was the creation of **ImageNet**.

ImageNet contains **millions of labeled images** across thousands of object categories.

For the first time, researchers had enough data to train deep CNNs effectively.

This dataset became the benchmark for computer vision research.

The ImageNet Challenge (ILSVRC)

Every year, researchers competed to build the best image classification model.

Before 2012, progress was gradual.

Then something incredible happened.

The Revolution

In **2012**, a CNN called **AlexNet** dramatically outperformed traditional computer vision methods in the ImageNet competition.

Its improvement was so significant that it convinced much of the research community that deep learning was the future of computer vision.

This event is widely regarded as the beginning of the modern deep learning era.

Why AlexNet Won

AlexNet wasn't just "a bigger CNN."

It combined several key ideas:

- A deeper architecture than many earlier successful models.
- **ReLU** activation instead of slower activation functions like sigmoid.
- **Dropout** to reduce overfitting.
- GPU-based training.
- Training on the large ImageNet dataset.

Each of these contributed to its success.

We'll study each one in detail.

Why Should You Care?

Your passive sonar project also follows the same philosophy.

Old approach:

```
Signal
↓
MFCC
↓
SVM
```

Modern approach:

```
Signal
↓
LOFARGRAM
↓
CNN
```

Instead of manually deciding which features are important, the CNN learns useful features directly from the LOFARGRAM.

That is exactly the revolution AlexNet helped popularize.

Summary

Before AlexNet	After AlexNet
Handcrafted features	Automatic feature learning
Traditional ML (SVM, etc.)	Deep CNNs
CPU-based training	GPU-accelerated training
Shallow models	Deep neural networks
Feature engineering by humans	Feature learning by the model

What We'll Learn Next (AlexNet Part 1)

The next lesson is entirely about **AlexNet**.

We'll answer questions like:

- What exactly is AlexNet?
- Who invented it?
- Why did it outperform other methods?
- What is its complete architecture?
- Why does it have 8 layers?
- How does an image travel from input to output?
- What is happening inside each layer?

We'll go layer by layer so that by the end, you'll be able to explain AlexNet from memory and understand why it became one of the most influential CNN architectures in history.

From this point onward, we're entering the architecture level of deep learning—the same concepts you'll encounter in research papers on passive sonar classification.

Excellent. You're now entering the level where you'll understand **why there are so many ResNet versions** and why almost every sonar paper mentions **ResNet-18** or **ResNet-50**.

This lesson is one of the most important for your project.

Deep CNN Architectures

Chapter 3: ResNet (Part 3)

Understanding ResNet-18, ResNet-34, ResNet-50, ResNet-101 & ResNet-152

First Question

Open any deep learning research paper and you'll see names like:

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

Students often think:

"These must be completely different networks."

They are not.

They all belong to the **ResNet family**.

The main difference is **how many layers (or residual blocks) they contain**.

Think of them like different models of the same car.

Real-Life Analogy

Imagine buying a laptop.

You have:

Laptop Series

↓

i3

↓

i5

↓

i7

↓

i9

All are laptops.

The difference is mainly in **power and complexity**.

Similarly:

ResNet Family

↓

ResNet-18

↓

ResNet-34

↓

ResNet-50

↓

ResNet-101

↓

ResNet-152

All use the **same core idea: Residual Learning**.

What Does the Number Mean?

The number roughly represents the **number of learnable layers** in the network.

Examples:

Model	Learnable Layers
ResNet-18	18
ResNet-34	34
ResNet-50	50
ResNet-101	101
ResNet-152	152

Notice something.

The architecture becomes deeper.

Why Make the Network Deeper?

Imagine identifying ships.

With a small network,

the model may only recognize:

- Engine harmonics
- Tonal lines

A deeper network can also learn:

- Complex engine modulation
- Cavitation patterns
- Higher-level acoustic signatures

More layers generally mean **greater representational power**.

But deeper isn't always better—you need enough data and computation to benefit from the extra depth.

How is ResNet Built?

Instead of thinking in terms of individual layers,

think in terms of **Residual Blocks**.

A Residual Block is the building block of the network.

Imagine LEGO.

One LEGO brick:

[Residual Block]

Now build a wall.

[Block]

↓

[Block]

↓

[Block]

↓

[Block]

The more blocks,
the deeper the network.

ResNet-18

ResNet-18 contains fewer residual blocks.

It is:

- Relatively lightweight
- Fast
- Easy to train
- Less computationally expensive

That's why it's extremely popular in research with limited datasets.

ResNet-152

Now imagine:

Many more residual blocks.

The network becomes much deeper.

Advantages:

- Can learn more complex features.

Disadvantages:

- Requires much more computation.
- Needs more memory.
- Usually benefits from larger datasets.

Think Like a Student

Suppose two students prepare for an exam.

Student A studies:

10 chapters.

Student B studies:

100 chapters.

Who performs better?

You might say:

Student B.

But what if:

- The exam only covers 10 chapters?

Or

- Student B only had one day to study?

Then studying more wasn't necessarily helpful.

The same idea applies to deep networks.

The model depth should match the problem and the available data.

Why ResNet-18 is Popular in Sonar Research

Now let's connect this to your project.

Imagine your dataset.

You are **not** working with ImageNet.

You are working with sonar data, where datasets are often much smaller.

Using ResNet-152 might:

- Overfit more easily.
- Require much longer training.
- Offer little or no improvement.

ResNet-18 often provides an excellent balance between:

- Accuracy.
- Speed.
- Generalization.

That's one reason why it appears so often in passive sonar classification papers.

Two Types of Residual Blocks

This is something many beginners don't know.

Not every ResNet uses the same kind of block.

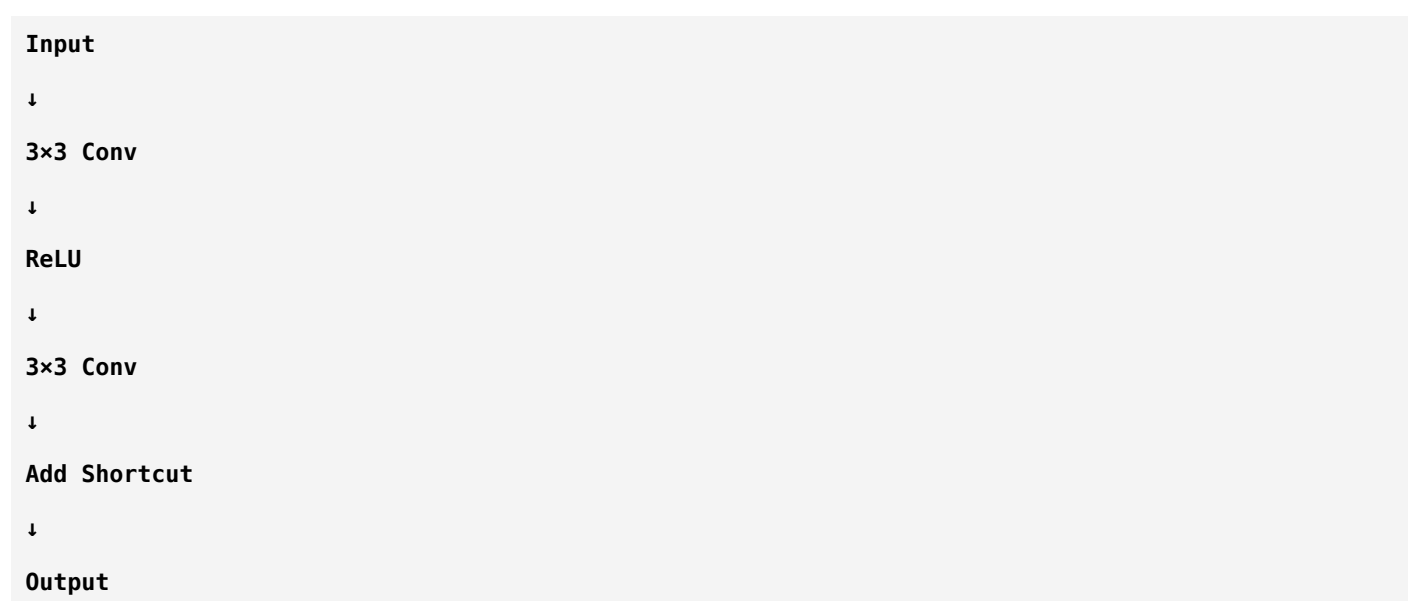
There are **two main block types**.

Type 1: Basic Block

Used in:

- ResNet-18
- ResNet-34

Structure:



Simple.

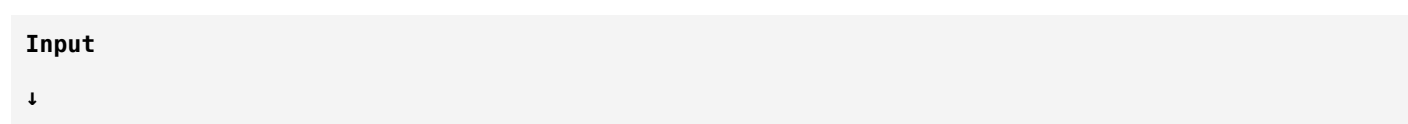
Easy to understand.

Type 2: Bottleneck Block

Used in:

- ResNet-50
- ResNet-101
- ResNet-152

Structure:



```

1×1 Conv
↓
3×3 Conv
↓
1×1 Conv
↓
Add Shortcut
↓
Output

```

This looks more complicated.

Why?

Because researchers wanted to make very deep networks more computationally efficient.

We'll study bottleneck blocks in detail in the next lesson.

Why Introduce 1×1 Convolutions?

At first glance, a **1×1 convolution** seems useless.

Students ask:

"How can a filter that only looks at one pixel help?"

Excellent question.

The answer is:

It doesn't look across neighboring pixels.

Instead, it learns to **mix and transform information across the channel dimension**.

It can also reduce or increase the number of channels, making later computations much cheaper.

This is one of the clever ideas behind deeper ResNets.

We'll cover it thoroughly next.

Comparing the Models

Model	Block Type	Advantages	Trade-offs
ResNet-18	Basic Block	Fast, lightweight	Lower capacity
ResNet-34	Basic Block	More expressive than 18	More computation
ResNet-50	Bottleneck	Strong accuracy, efficient for its depth	More complex
ResNet-101	Bottleneck	Very powerful	Heavy computation
ResNet-152	Bottleneck	Extremely deep	High memory and compute requirements

Which One Should You Use?

Suppose you're classifying:

- Dogs.
- Cats.
- Cars.

With millions of images.

A deeper model like ResNet-50 or ResNet-101 might make sense.

Now consider your passive sonar project.

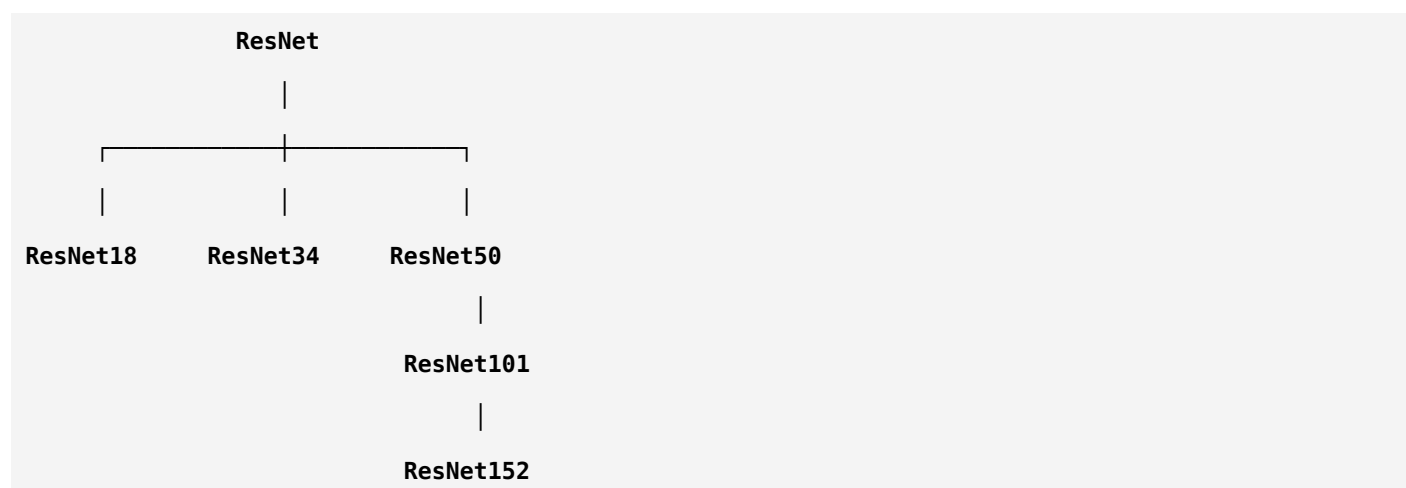
Typical datasets are much smaller.

In many cases:

ResNet-18 or ResNet-34 is a very sensible starting point because they provide strong performance without requiring enormous amounts of data or computation.

Of course, the best choice depends on your dataset size, hardware, and performance goals.

Visualizing the Family



The concept is the same.

Only the depth and block design change.

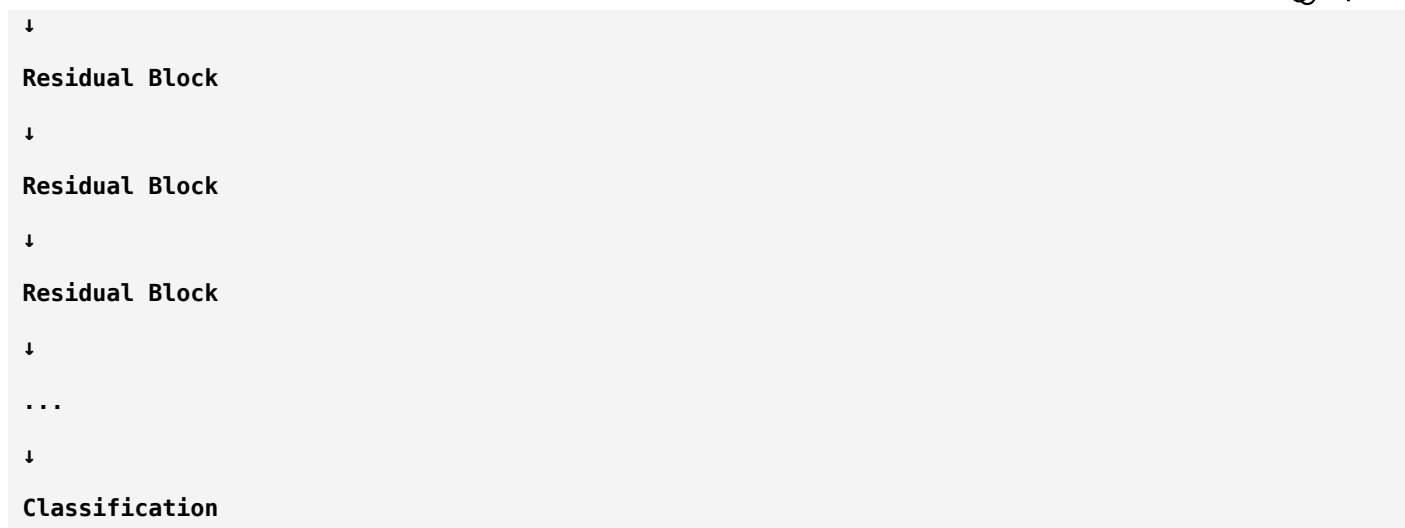
How Does a LOFARGRAM Flow Through ResNet?

Imagine your input.

LOFARGRAM

↓

Conv Layer



Each block gradually improves the learned representation.

Instead of relearning everything,

each block asks:

"What useful information should I add to what I already know?"

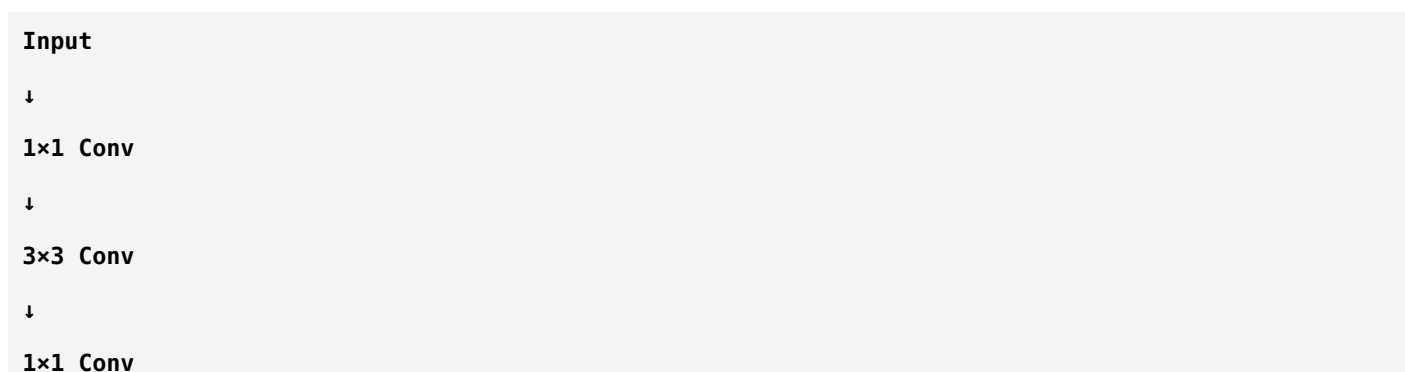
That's the heart of residual learning.

Summary

Concept	Key Idea
ResNet Family	Same core architecture, different depths
ResNet-18	Lightweight, Basic Blocks
ResNet-34	Deeper Basic Blocks
ResNet-50+	Uses Bottleneck Blocks
Deeper Networks	More capacity but higher computational cost
Sonar Applications	ResNet-18 is a common and practical choice for many datasets

One Thought Before the Next Lesson

Today we saw this block:



↓

Output

It probably looks strange.

You may be wondering:

"Why would anyone use a 1×1 convolution?"

That question led to another important breakthrough in CNN design.

Next Lesson (ResNet Part 4)

In the next lesson, we'll go deep into the **Bottleneck Block**, where you'll learn:

- What exactly is a **1×1 convolution**?
- How can it reduce computation without throwing away useful information?
- Why does ResNet-50 use bottleneck blocks instead of basic blocks?
- How does the bottleneck design make very deep networks practical?
- Why do modern architectures like **DenseNet**, **EfficientNet**, and many others also use **1×1 convolutions**?

By the end of that lesson, you'll understand one of the smartest design ideas in modern CNNs.

Next

Excellent! This lesson is one of the **deepest concepts in CNN architecture**.

When students first see a **1×1 convolution**, almost everyone says:

"This makes no sense."

Because they think:

"A convolution should look at neighboring pixels."

A 1×1 filter doesn't even look at neighbors.

So...

Why does it exist?

Today we'll answer that.

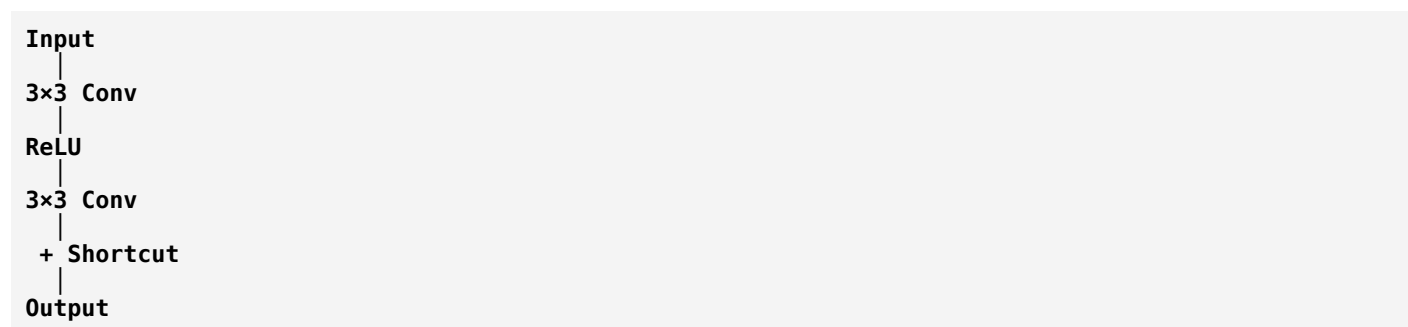
Deep CNN Architectures

Chapter 3: ResNet (Part 4)

The Bottleneck Block & the Mystery of 1×1 Convolutions

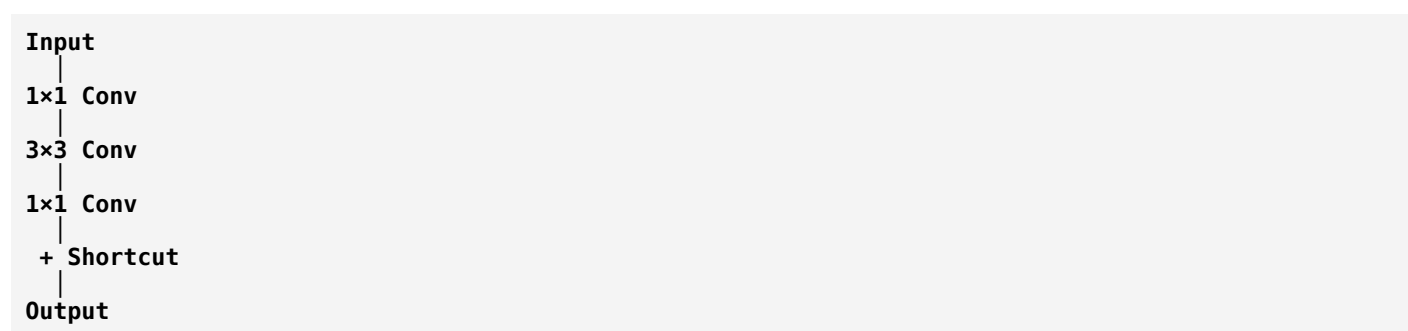
First, Let's Recall the Basic Block

In ResNet-18, one block looks like:



Simple.

Now look at ResNet-50.



Students immediately ask:

"Why add two extra layers?"

Good question.

Imagine a Big Library

Suppose you enter a library with **1000 books**.

You only need books about:

- Ships
- Ocean Acoustics
- Signal Processing

Would you carry all 1000 books everywhere?

No.

You would first select only the useful books.

Then study them.

Then organize your final notes.

That is exactly what a Bottleneck Block does.

The Secret of Channels

Remember from previous lessons:

After several convolution layers, your feature map may look like:

56 × 56 × 256

Let's understand this carefully.

- Height = 56
- Width = 56
- Channels = 256

Those **256 channels** represent **256 different learned features**.

Think of them as 256 experts.

Expert 1:

Looks for harmonic lines.

Expert 2:

Looks for broadband noise.

Expert 3:

Looks for engine modulation.

...

Expert 256:

Looks for another acoustic pattern.

The Problem

Now suppose we apply a **3×3 convolution** directly.

Every filter has to process **all 256 channels**.

That is computationally expensive.

Imagine asking one person to interview **256 experts** before making a decision.

Slow.

The Brilliant Idea

Instead of using all 256 channels,

what if we first reduce them?

Example:

56 × 56 × 256

↓

1×1 Conv

↓

56 × 56 × 64

Notice something.

Height:

Still 56.

Width:

Still 56.

Only the channels changed:

256

↓

64

The network has compressed the information.

Wait...

How Can 1×1 Convolution Reduce Channels?

Excellent question.

This is where most tutorials stop.

Let's understand it.

Think About Mixing Colors

Imagine you have:

- Red
- Green
- Blue

You mix them.

The result is a new color.

Did you look at neighboring pixels?

No.

You only mixed information at the **same pixel location**.

A **1×1 convolution** does the same thing.

It combines information **across channels**, not across space.

Example

Suppose one pixel has values:

Channel 1 = 10

Channel 2 = 7

Channel 3 = 15

Channel 4 = 3

A 1×1 filter learns weights like:

0.2

0.5

-0.3

0.8

It combines those channel values into a new feature.

The filter is learning:

"How important is each channel?"

This is why people often describe 1×1 convolutions as **channel mixers** or **channel projections**.

Three Stages of a Bottleneck Block

Now let's understand the complete design.

Stage 1: Reduce

Input:

56 × 56 × 256

↓

1×1 Conv

↓

56 × 56 × 64

Purpose:

Reduce the number of channels.

This saves computation.

Stage 2: Learn

Now apply:

3×3 Conv

But notice:

Instead of processing **256 channels**,

it only processes **64 channels**.

Much cheaper.

This is where the main spatial feature learning happens.

Stage 3: Expand

Finally,

another:

1×1 Conv

restores the channel dimension.

Example:

56 × 56 × 64

↓

56 × 56 × 256

Now the output has the expected shape again, so it can be combined with the shortcut path.

Real-Life Analogy

Imagine writing a report.

Step 1:

Remove unnecessary details.

↓

Step 2:

Analyze the important information.

↓

Step 3:

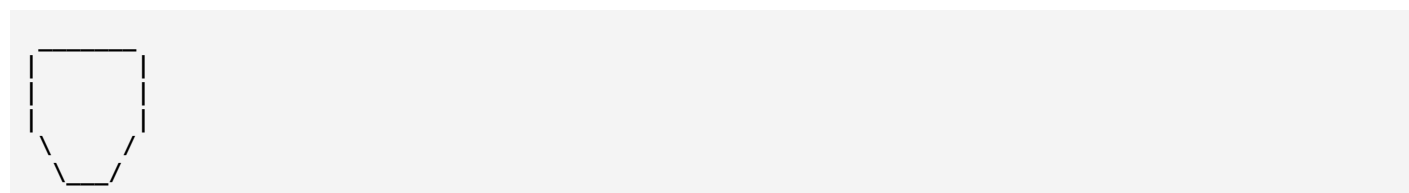
Write the complete report again.

You don't analyze everything.

You analyze the essential parts.

Why Is This Called a Bottleneck?

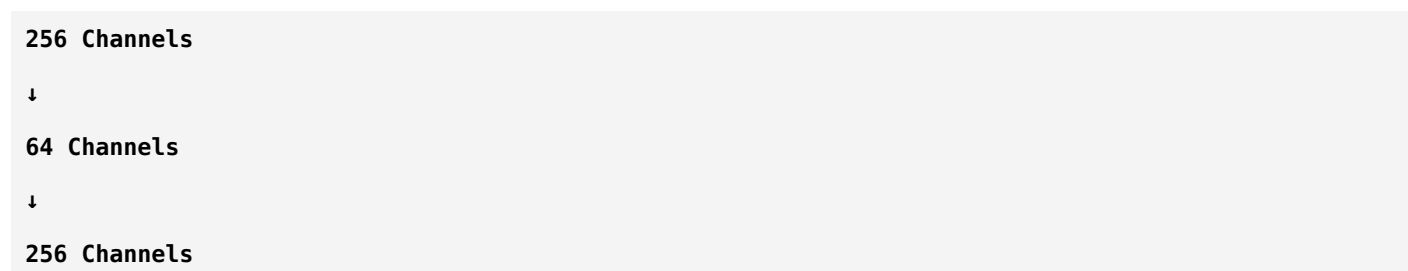
Think of a water bottle.



The neck is narrow.

Everything passes through that narrow region.

Similarly:



The information is squeezed into a smaller representation before expanding again.

That's why it's called a **Bottleneck Block**.

Why Doesn't Information Get Lost?

Students often worry:

"If we reduce 256 channels to 64, aren't we throwing away information?"

A reasonable concern.

The answer is:

The network **learns** which channel combinations are most useful.

If reducing too aggressively hurts performance, training will adjust the weights to preserve important information.

So while compression can discard some redundancy, the goal is to keep the information that matters most for the task.

Sonar Example

Imagine your LOFARGRAM produces channels like:

- Engine harmonics
- Propeller tones
- Broadband noise
- Cavitation
- Ambient ocean noise
- Machinery modulation
- Other learned patterns

Not every channel is equally important for every decision.

The 1×1 convolution learns how to combine and emphasize the most informative channel patterns before the more expensive 3×3 convolution.

Why Is Bottleneck So Powerful?

Without a bottleneck:

3×3 Conv

↓

Huge Computation

With a bottleneck:

Reduce Channels

↓

3×3 Conv

↓

Expand Channels

The expensive spatial convolution operates on fewer channels, making the network much more computationally efficient.

This is one reason ResNet-50 can be much deeper than ResNet-18 while remaining practical.

Basic Block vs Bottleneck Block

Basic Block	Bottleneck Block
$3 \times 3 \rightarrow 3 \times 3$	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
Used in ResNet-18 & 34	Used in ResNet-50, 101 & 152
Simpler	More computationally efficient for deep networks
Good for shallower models	Better suited for very deep models

Why This Matters Beyond ResNet

The idea of **1×1 convolutions** didn't stop with ResNet.

You'll encounter them in many important architectures, including:

- **GoogLeNet (Inception)**
- **DenseNet**
- **MobileNet**
- **EfficientNet**

So understanding this concept now will make those architectures much easier to learn.

Key Takeaways

1. **1×1 convolutions do not look across neighboring pixels.**
2. **They learn relationships between channels.**
3. **They can reduce or increase the number of channels.**
4. **Bottleneck blocks reduce computation while preserving strong performance.**
5. **That's why ResNet-50 and deeper models use bottleneck blocks instead of basic blocks.**

Think Like an Engineer

Suppose you're building a passive sonar classifier.

Your feature maps have **512 channels**.

Would you:

- Run expensive 3×3 convolutions on all 512 channels?

Or

- First compress the channels to a smaller, informative representation, perform the expensive operation there, and then expand them again?

Modern CNN designers usually choose the second approach because it provides a much better balance between computational cost and representational power.

Next Lesson (ResNet Part 5 – Final)

In the final ResNet lesson, we'll connect everything you've learned and answer:

- **How does a complete LOFARGRAM travel through ResNet-18?**
- **What happens at every stage of the network?**
- **Why is ResNet-18 one of the most popular choices for passive sonar target classification?**
- **When should you choose ResNet-18 vs ResNet-50?**
- **How is ResNet used with transfer learning in real research papers?**

That lesson will tie the entire ResNet chapter directly to **your passive sonar project** and prepare you for understanding modern sonar classification research.

Ok next

Excellent. This is the **final chapter on ResNet**, where we'll connect everything directly to **your passive sonar target classification project**.

After this lesson, you won't just know ResNet—you'll understand **how researchers actually use it** in real-world sonar systems.

Deep CNN Architectures

Chapter 3: ResNet (Part 5 – Final)

ResNet in Passive Sonar Target Classification

The Big Picture

Let's assume your goal is:

Binary Classification

Input: Passive sonar recording

Output: Ship Present  or Ship Absent 

At first glance, this seems impossible.

How can a neural network understand an audio signal?

The answer is:

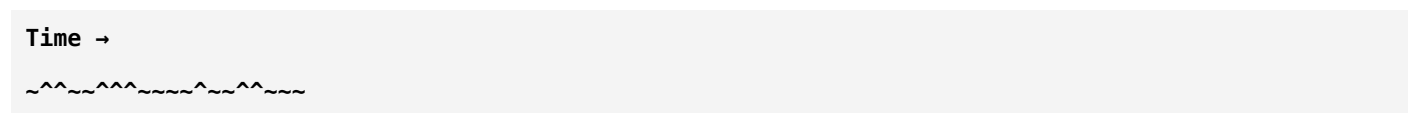
It doesn't work directly on raw audio in most traditional CNN pipelines.

Instead, we first convert the audio into a visual representation.

Step 1: Passive Sonar Signal

Imagine a hydrophone records sound.

The raw waveform looks like this:

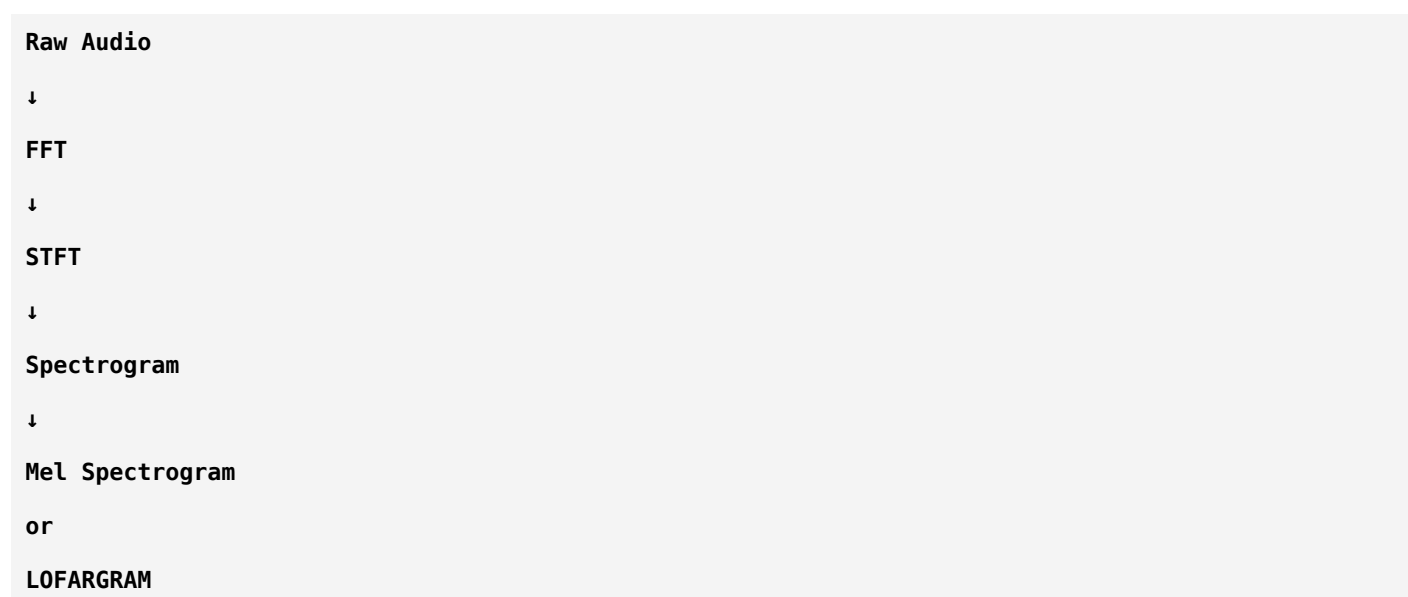


This waveform is difficult for a CNN to process directly.

So we transform it.

Step 2: Signal Processing

You already learned these techniques:

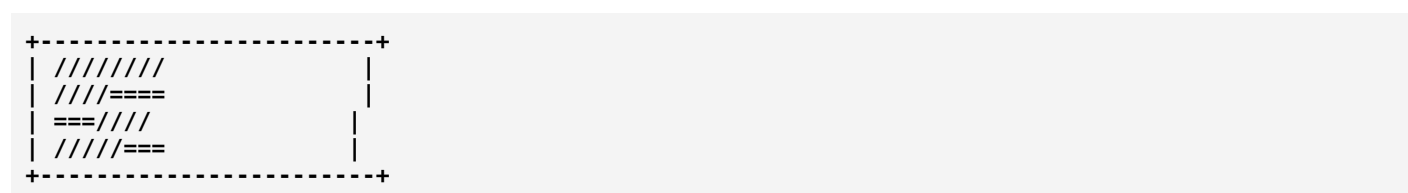


For passive sonar, **LOFARGRAM** is one of the most common choices because it highlights frequency components over time that are useful for detecting ship signatures.

Now the signal becomes an image-like representation.

Step 3: Feed the LOFARGRAM to ResNet

Imagine your LOFARGRAM:



To a human,
this looks like a heatmap.

To ResNet,
it's simply an image.

What Does the First Layer Learn?

The first convolution sees only small regions.

It learns simple patterns like:

- Horizontal lines
- Vertical lines
- Bright frequency bands
- Small edges

In sonar, these may correspond to:

- Narrow tonal lines
- Local changes in energy
- Short-duration events

These are the building blocks.

Middle Layers

The next residual blocks combine those simple patterns.

Instead of individual frequency lines,
they begin recognizing:

- Harmonic groups
- Repeating tonal structures
- Frequency trajectories

These are much more informative than isolated pixels.

Deep Layers

Now ResNet starts recognizing high-level acoustic patterns.

For example:

- Engine acoustic signatures
- Propeller-related harmonics
- Stable frequency structures associated with ships

At this point, the network isn't "seeing lines" anymore.
It's learning abstract representations that help distinguish:

Ship Present vs **Ship Absent**.

Final Classification

After many residual blocks:

```
LOFARGRAM
↓
Residual Blocks
↓
Global Average Pooling
↓
Fully Connected Layer
↓
Softmax / Sigmoid
↓
Prediction
```

For your binary problem, the final output might be:

```
Ship Present = 0.97
Ship Absent = 0.03
```

The network predicts **Ship Present** because it has the higher probability.

Why Global Average Pooling?

Earlier architectures like AlexNet and VGG used very large **fully connected layers**, which introduced millions of parameters.

ResNet instead uses **Global Average Pooling (GAP)** before the final classifier.

Imagine the last feature map:

```
7 × 7 × 512
```

Instead of flattening everything,

GAP computes the **average value of each channel**.

So:

```
7×7×512
```

↓

512 numbers

Each number summarizes one feature channel.

Advantages:

- Far fewer parameters.
- Lower risk of overfitting.
- Better generalization.
- Less memory usage.

This is one reason ResNet is much more efficient than VGG.

Why is ResNet-18 So Popular?

Suppose you have:

- 2,000 sonar recordings.

Would ResNet-152 be a good choice?

Probably not.

It has very high capacity and may overfit if the dataset is too small.

ResNet-18 often works well because:

- It's easier to train.
- It's computationally efficient.
- It generalizes well on many medium-sized datasets.
- It has significantly fewer parameters than deeper ResNets.

This is why you'll frequently see **ResNet-18** in passive sonar research.

Transfer Learning

This is one of the most important ideas for your project.

Imagine you train ResNet from scratch.

The network must learn:

- Basic edges.
- Frequency patterns.
- Intermediate structures.
- High-level representations.

That requires a lot of data.

Instead, researchers often start with a **ResNet pretrained on ImageNet**.

You might wonder:

"ImageNet contains natural images, not sonar. Why would this help?"

Because the **early layers** learn very general features like edges, gradients, and simple textures.

Those low-level feature extraction capabilities can still be useful when fine-tuned on sonar images.

Then researchers replace the final classification layer and train the network on their sonar dataset.

This process is called **Transfer Learning**.

Your Complete Pipeline

Now you can finally understand an end-to-end passive sonar classifier.



This is very close to the architecture used in many research papers.

Why ResNet Fits Your Project

Your project involves recognizing subtle acoustic patterns.

ResNet helps because:

- Residual connections improve optimization.
- It learns hierarchical features automatically.
- It is robust enough for complex acoustic images.
- It works well with transfer learning.
- ResNet-18 offers an excellent balance of accuracy and efficiency.

Comparing AlexNet, VGG, and ResNet

Feature	AlexNet	VGG	ResNet
Main Idea	First successful deep CNN	Showed the power of depth	Solved the degradation problem with residual learning
Learnable Layers	8	16 / 19	18-152+
Residual Connections	✗	✗	✓
Global Average Pooling	✗	✗	✓
Large Fully Connected Layers	✓	✓	Minimal
Common in Modern Sonar Research	Rare	Sometimes	Very common

A Researcher's Thought Process

When reading a sonar paper that says:

"We used ResNet-18."

You should now ask:

- Why ResNet-18 instead of VGG?
- How large was their dataset?
- Did they use transfer learning?
- What input representation did they use? (LOFARGRAM, spectrogram, Mel spectrogram?)
- Why was ResNet-18 sufficient?

These are the kinds of questions that distinguish someone who understands the field from someone who only knows the model names.

What You've Learned So Far

You now understand:

Digital Audio Processing

- FFT
- STFT
- Spectrogram
- Mel Spectrogram
- MFCC
- LOFARGRAM

Machine Learning

- Features
- Labels
- Training
- Validation
- Testing
- Loss
- Gradient Descent
- Backpropagation
- Overfitting
- Regularization

Deep Learning

- Neurons
- Activation Functions
- CNNs
- Convolution
- Pooling
- Feature Maps
- ReLU
- Dropout
- Batch Normalization

CNN Architectures

- AlexNet
- VGG
- ResNet

You've built a solid foundation.

Where Do We Go Next?

Now we move from **classic CNNs** to **modern architectures**.

The order I recommend is:

1. **DenseNet** (understand dense connections and feature reuse)
2. **Inception / GoogLeNet** (multi-scale feature extraction)

3. **EfficientNet** (compound scaling; very relevant to your project)
 4. **MobileNet** (lightweight models)
 5. **Vision Transformers (ViT)** (modern alternative to CNNs)
-

★ My recommendation

Since your project is **Passive Sonar Based Target Ship Classification**, the next architecture should be **DenseNet**.

Why?

Because many sonar classification papers compare:

- ResNet
- DenseNet
- EfficientNet

DenseNet introduces a different idea from ResNet:

Instead of connecting only the input to the output of a block, what if every layer could directly access the outputs of all previous layers?

That simple idea led to another major breakthrough in deep learning.

From here onward, you'll start recognizing the design philosophy behind almost every modern CNN architecture.

Today 9:46 AM

ok next

Excellent. Welcome to **DenseNet**.

In my opinion, **DenseNet is the second most important architecture** for your passive sonar project after ResNet.

Many sonar research papers compare:

- ResNet-18
- DenseNet-121
- EfficientNet-B0

So if you understand DenseNet deeply, you'll understand a large portion of the current literature.

Deep CNN Architectures

Chapter 4: DenseNet (Part 1)

Why Was DenseNet Invented?

Let's first think like researchers again.

The Story Begins

In 2015, ResNet solved a huge problem.

It allowed researchers to train very deep networks.

Everyone celebrated.

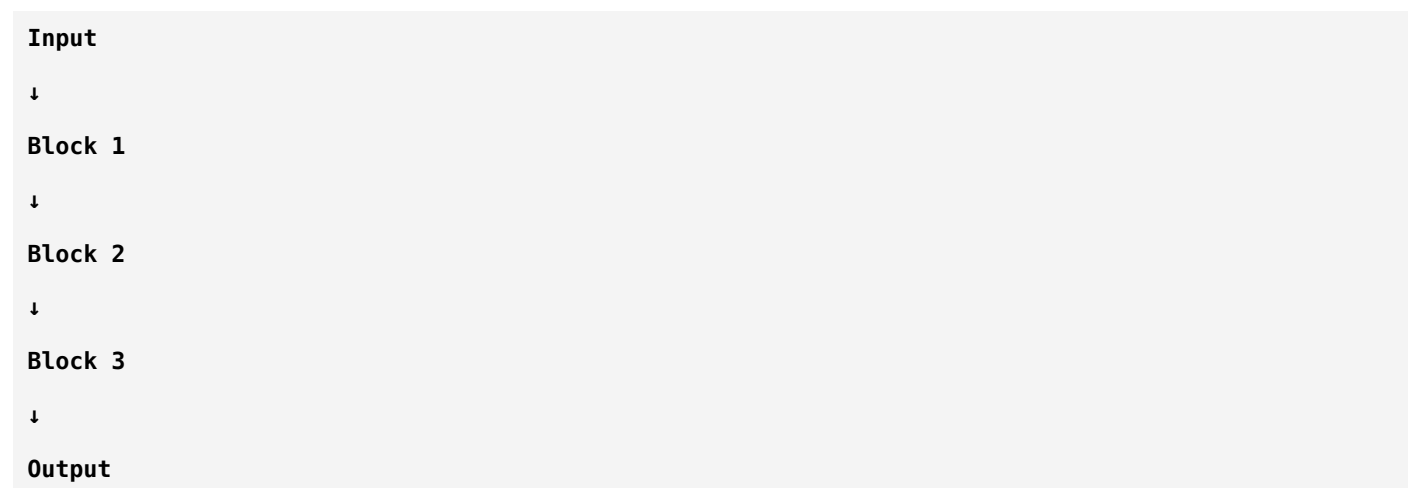
But after some time, researchers asked another question.

"Even with skip connections... are we still throwing away useful information?"

This question led to DenseNet.

Let's Recall ResNet

Imagine three residual blocks.



Inside each block, the input is added to the output.

For one block:

$$\text{Output} = x + F(x)$$

Very clever.

But researchers noticed something.

Imagine a Classroom

Suppose there are five students.

Student 1 discovers an important formula.

Student 2 learns it.

Student 3 learns from Student 2.

Student 4 learns from Student 3.

Student 5 learns from Student 4.

Notice something.

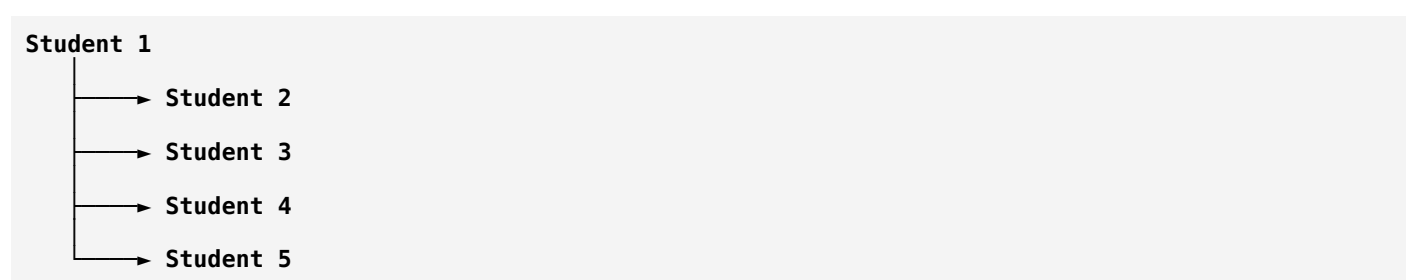
Student 5 never talks directly to Student 1.

Information has to pass through everyone in between.

Some details may become weaker or less useful.

What If Everyone Could Talk Directly?

Imagine a different classroom.



Now every student can directly use the original knowledge.

Nothing has to be rediscovered.

This is exactly the idea behind DenseNet.

The Main Philosophy

ResNet says:

"Add the previous information."

DenseNet says:

"Don't just add it. Keep it and make it available to every future layer."

That is the biggest difference.

What Does "Dense" Mean?

The word **Dense** means:

Every layer is connected to every later layer.

Not just the next one.

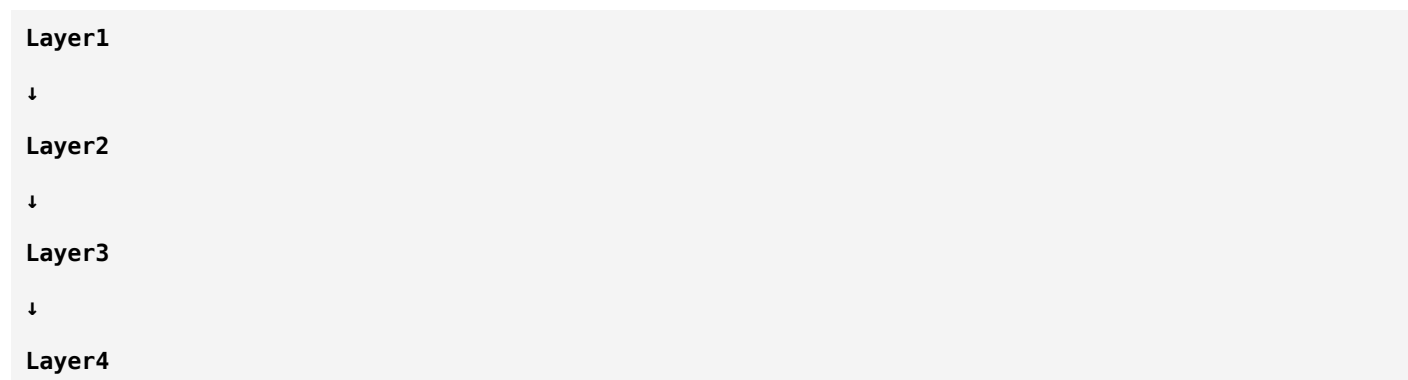
Every layer shares its features with all future layers.

That's why it is called:

Dense Network.

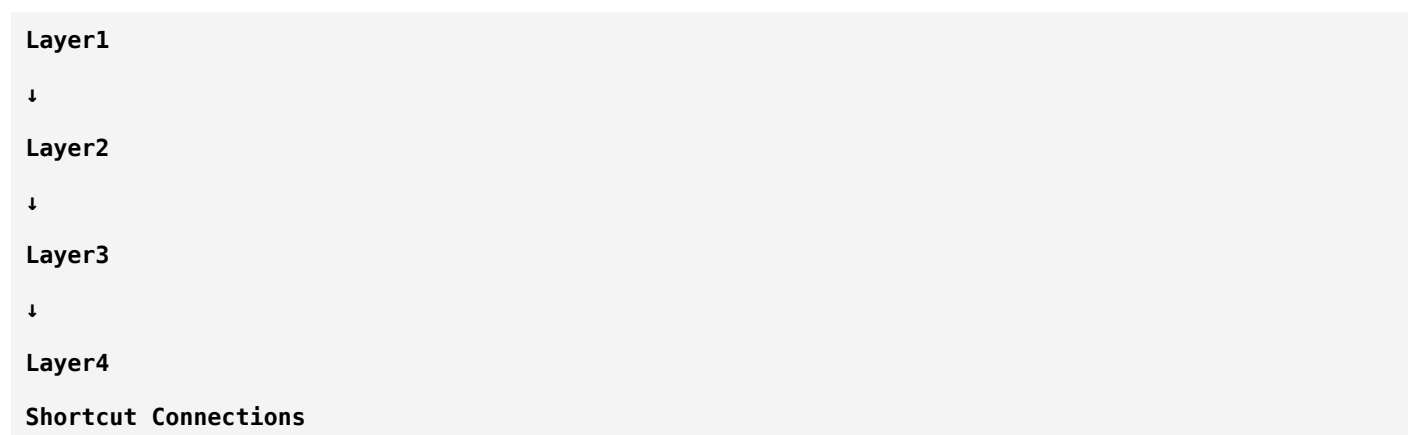
Visual Comparison

Traditional CNN



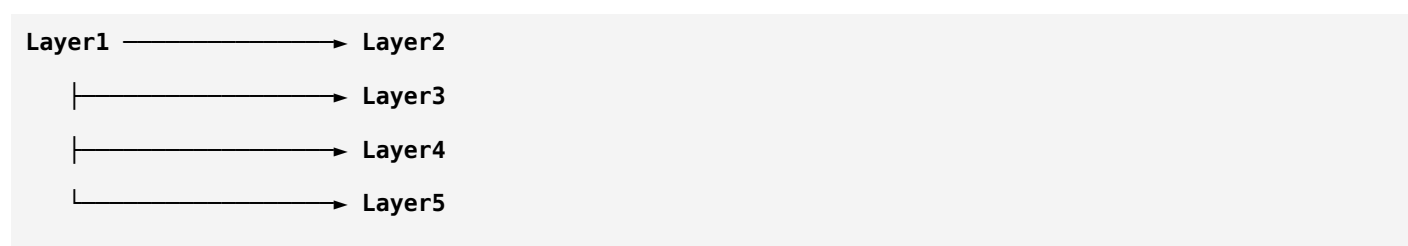
Only one path.

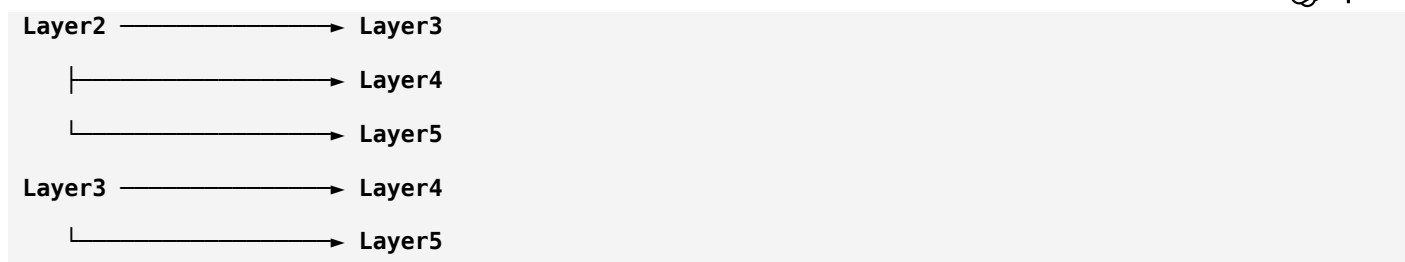
ResNet



Some shortcuts.

DenseNet





Every layer communicates with all future layers.

This is why the network is called **DenseNet**.

Why Is This Useful?

Imagine Layer 1 learns something important.

For your sonar project, suppose it detects:

- A strong tonal frequency line.

In ResNet,

Layer 5 receives that information through residual paths.

In DenseNet,

Layer 5 can **directly access** Layer 1's feature maps.

Nothing has to be relearned.

Feature Reuse

This is the key term for DenseNet.

Feature Reuse

Instead of learning the same feature again,

later layers simply reuse earlier features.

Imagine a software project.

Without feature reuse:

Developer A writes:

Login Function

Developer B writes another login function.

Developer C writes yet another one.

Huge waste.

Instead,

everyone imports the existing function.

That's feature reuse.

DenseNet does exactly that with learned features.

Why Does Feature Reuse Matter?

Suppose the first layer has already learned:

- Engine harmonic pattern.

Why should the tenth layer spend effort learning it again?

Instead,

it can simply reuse that feature and focus on learning something new.

Another Big Difference

Remember ResNet?

It combines features using **addition**.

```
Output = x + F(x)
```

DenseNet does **not** add them.

It **concatenates** them.

Addition vs Concatenation

This is one of the biggest conceptual differences.

Imagine two notebooks.

Notebook A:

```
Math Notes
```

Notebook B:

```
Physics Notes
```

Addition

Suppose you mix the pages together.

You lose the distinction between subjects.

That's similar to addition—the information is merged.

Concatenation

Instead,
put the notebooks side by side.

```
[Math Notes]
+
[Physics Notes]
```

Now you still have both sets of notes separately.
Nothing is overwritten.
DenseNet follows this idea.

In Neural Networks

Suppose Layer 1 produces:

```
Feature A
```

Layer 2 produces:

```
Feature B
```

ResNet:

```
A + B
```

One combined feature map.

DenseNet:

```
[A , B]
```

Both feature maps are kept.
The next layer can use either or both.
This preserves more information.

Sonar Example

Imagine your first layers detect:

- Harmonic lines
- Broadband noise
- Propeller tones

Instead of combining everything into a single representation, DenseNet keeps these feature maps available.

Later layers can decide:

- Which features to emphasize.
- Which features to ignore.
- How to combine them.

This flexibility often leads to richer feature representations.

Why Researchers Loved DenseNet

DenseNet offered several advantages:

1. Better Gradient Flow

There are many direct paths between layers, helping gradients travel during backpropagation.

2. Feature Reuse

Earlier features remain available.

No need to relearn them.

3. Parameter Efficiency

This surprises many students.

DenseNet often has **fewer parameters** than similarly performing networks because it reuses existing features instead of repeatedly learning redundant ones.

Summary

ResNet	DenseNet
Uses addition	Uses concatenation
Skip connections	Dense connections
Learns residuals	Reuses all previous features
Information is merged	Information is preserved and accumulated
Solves degradation	Encourages feature reuse and efficient learning

Think Like a Researcher

Suppose your first convolution layer detects a beautiful engine harmonic.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Should later layers:

1. Learn the same harmonic again?

Or

2. Directly reuse the already learned harmonic and spend their capacity learning new acoustic patterns?

DenseNet chooses **Option 2**.

That simple idea is the foundation of the architecture.

Homework (Concept Check)

Without looking back, answer these:

1. Why is it called **DenseNet**?
2. What is **feature reuse**?
3. What is the difference between **addition** (ResNet) and **concatenation** (DenseNet)?
4. Why might feature reuse be especially useful for passive sonar signals?

If you can answer those clearly, you've understood the core philosophy of DenseNet.

Next Lesson (DenseNet Part 2)

We'll go much deeper and answer:

- **How does a Dense Block actually work?**
- **What is the Growth Rate (k)?**
- **Why doesn't DenseNet become infinitely large if every layer keeps all previous features?**
- **What are Transition Layers?**
- **How does a LOFARGRAM move through DenseNet layer by layer?**

This is where DenseNet becomes truly fascinating, and you'll see why it's considered one of the most elegant CNN architectures ever designed.

ok next

Excellent. Today we'll study the **heart of DenseNet**.

If Part 1 explained **the philosophy**, Part 2 explains **how DenseNet actually works internally**.

This is one of the most elegant ideas in deep learning.

Deep CNN Architectures

Chapter 4: DenseNet (Part 2)

Dense Blocks, Growth Rate & Transition Layers

Quick Recap

DenseNet says:

Every layer can use the feature maps from all previous layers.

Unlike ResNet:

Layer1 → Layer2 → Layer3

DenseNet creates:

```

Layer1 → Layer2
Layer1 → Layer3
Layer2 → Layer3
Layer1 → Layer4
Layer2 → Layer4
Layer3 → Layer4
  
```

Notice that the number of connections grows rapidly.

Now an important question arises.

Problem 1

Imagine each layer produces:

64 feature maps

Suppose there are:

100 layers

Then the last layer would receive:

$64 \times 100 = 6400$ feature maps

That sounds enormous.

Wouldn't DenseNet become impossible to run?

Yes.

Researchers knew this problem.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

So they introduced two brilliant ideas.

1. Growth Rate
2. Transition Layer

Let's study them.

Growth Rate (k)

This is probably the most important term in DenseNet.

Definition

The **Growth Rate (k)** is:

The number of new feature maps that each layer produces.

Notice the word:

New

The layer **does not recreate all previous features.**

It only contributes a **small number of additional feature maps.**

Example

Suppose:

Input:

64 Feature Maps

Growth Rate:

k = 32

Layer 1 receives:

64 maps

It creates only:

32 new maps

Now the next layer receives:

64 + 32 = 96 maps

Layer 2 creates another:

32 maps

Now:

$96 + 32 = 128$ maps

Layer 3:

$128 + 32 = 160$ maps

Notice something.

Every layer only contributes **32 new features**.

Everything else is reused.

Why Is This Smart?

Imagine a group project.

Student 1 writes:

Introduction

Student 2 doesn't rewrite the introduction.

Instead, Student 2 adds:

Methodology

Student 3 adds:

Results

Student 4 adds:

Conclusion

Each person contributes **only new information**.

Nobody rewrites the previous sections.

DenseNet follows the same principle.

Feature Growth

Suppose:

Initial Features = 64

Growth Rate = 32

Then:

Layer	Total Feature Maps
Input	64
Layer 1	96
Layer 2	128
Layer 3	160
Layer 4	192

This gradual increase is why it's called the **Growth Rate**.

Sonar Analogy

Imagine your LOFARGRAM.

Initially, the network learns:

- Frequency edges

The next layer doesn't relearn edges.

Instead, it adds:

- Harmonic groups

The next layer adds:

- Engine modulation

The next adds:

- Propeller signatures

The knowledge grows steadily.

But Wait...

The feature maps keep increasing.

Eventually the network becomes too large.

How do we solve that?

Transition Layer

DenseNet introduces another clever component.

Definition

A **Transition Layer** is placed **between Dense Blocks**.

Its job is to:

- Reduce the number of feature maps.
- Reduce the spatial size (height and width).
- Keep the model computationally manageable.

Think of it as a checkpoint where the network compresses information before continuing.

Real-Life Analogy

Imagine you're traveling.

Your backpack starts with:

5 books

After each city, you collect more books.

Soon you have:

50 books

Too heavy.

So you stop.

Remove duplicate or unnecessary books.

Continue with only the important ones.

That's exactly what a Transition Layer does.

Structure of a Transition Layer

A typical transition layer contains:

1×1 Convolution

↓

Average Pooling

Let's understand each part.

Step 1: 1×1 Convolution

Remember ResNet?

We already learned:

A **1×1 convolution** mixes information across channels.

Here, it's used to **reduce the number of feature maps**.

Example:

Before:

160 Feature Maps

After:

80 Feature Maps

The network keeps the most useful combinations.

Step 2: Average Pooling

Then:

56 × 56

↓

28 × 28

The height and width are reduced.

This lowers memory usage and computation.

Dense Block + Transition Layer

Now the overall architecture looks like:

Input

↓

Dense Block

↓

Transition Layer

↓

Dense Block

↓

Transition Layer

↓

Dense Block

↓

Classification

Each Dense Block expands the feature set.

Each Transition Layer compresses it.

This creates a balance.

Why Doesn't DenseNet Explode in Size?

Because:

- **Growth Rate** controls how many **new** features each layer adds.
- **Transition Layers** periodically compress the feature maps and reduce spatial dimensions.

Together, they keep DenseNet practical.

DenseNet-121

You often see names like:

- DenseNet-121
- DenseNet-169
- DenseNet-201

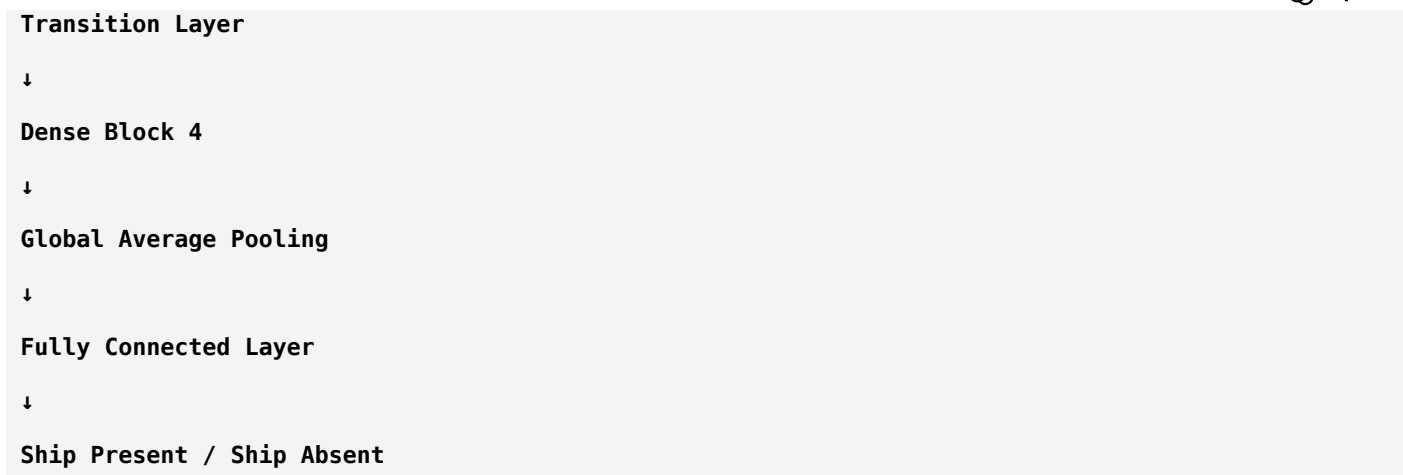
These numbers indicate the approximate number of learnable layers, similar to ResNet.

DenseNet-121 is the most commonly used variant because it offers a good balance between accuracy and computational cost.

How Does a LOFARGRAM Flow Through DenseNet?

Imagine this pipeline:

```
LOFARGRAM
↓
Initial Convolution
↓
Dense Block 1
↓
Transition Layer
↓
Dense Block 2
↓
Transition Layer
↓
Dense Block 3
↓
```



Inside each Dense Block, every layer can directly access all previously learned feature maps. That makes feature reuse one of DenseNet's biggest strengths.

DenseNet vs ResNet (So Far)

Feature	ResNet	DenseNet
Main Idea	Learn residuals	Reuse all previous features
Connection Type	Addition	Concatenation
Feature Flow	Shortcut connections	Dense connections
New Features	Every block transforms input	Every layer adds only a few new features
Key Hyperparameter	Number of residual blocks	Growth Rate (k)

Think Like a Researcher

Suppose your network has already learned:

- Engine harmonics
- Tonal lines
- Broadband noise

Should the next layer:

Option A: Learn them again?

Option B: Reuse them and spend its capacity learning something new?

DenseNet always chooses **Option B**.

That is why it is often very parameter-efficient.

Summary

Growth Rate (k)

- Controls how many **new** feature maps each layer creates.
- Prevents unnecessary duplication.

Transition Layer

- Compresses feature maps.
- Reduces image size.
- Keeps computation manageable.

Dense Block

- Every layer receives all previous feature maps.
- Encourages feature reuse.

Next Lesson (DenseNet Part 3 – Final)

In the final DenseNet lesson, we'll connect everything to modern deep learning and your sonar project:

- Why is DenseNet often **more parameter-efficient** than ResNet?
- Why can DenseNet achieve high accuracy with fewer parameters?
- What are its limitations?
- Why isn't DenseNet used everywhere?
- **ResNet vs DenseNet vs EfficientNet** — which one should you choose for a passive sonar classification project, and why?

That comparison is especially important because it's exactly the decision researchers make when designing real sonar classification systems.

ok next

Excellent. This is the **final lesson on DenseNet**.

Today, we're going to answer a question that every deep learning researcher eventually faces:

If DenseNet is so good, why doesn't everyone use it instead of ResNet?

This lesson is less about memorizing architecture and more about **thinking like a research engineer**.

Deep CNN Architectures

Chapter 4: DenseNet (Part 3 – Final)

Strengths, Weaknesses & When to Use DenseNet

Let's Compare ResNet and DenseNet

Imagine you're building a large city.

ResNet Philosophy

Each new building improves the previous one.

```
Building 1
↓
Building 2 (improved)
↓
Building 3 (improved)
↓
Building 4 (improved)
```

Each stage builds on the previous result.

DenseNet Philosophy

Every new building has access to **all previous blueprints**.

```
Blueprint 1 → Building 2
Blueprint 1 → Building 3
Blueprint 2 → Building 3
Blueprint 1 → Building 4
Blueprint 2 → Building 4
Blueprint 3 → Building 4
```

Nothing useful is forgotten.

Why DenseNet Can Use Fewer Parameters

This surprises almost everyone.

Suppose Layer 1 learns:

- Engine harmonic pattern

In a traditional CNN, later layers may spend parameters learning a similar representation again.

DenseNet avoids much of this redundancy because later layers can directly reuse earlier feature maps.

So instead of creating duplicate features, DenseNet often allocates its parameters to learning **new** information.

This is why DenseNet is often described as **parameter-efficient**.

Important: Parameter-efficient does **not** always mean faster or lower memory usage during training. We'll see why shortly.

Parameter Efficiency vs Memory Efficiency

Many beginners confuse these.

They are **not the same thing**.

Parameter Efficiency

Means:

The model stores fewer trainable weights.

DenseNet is often very good here.

Memory Efficiency During Training

Means:

How much RAM or GPU memory is needed while training.

DenseNet can actually require **more memory** because it keeps many feature maps alive for concatenation.

This is one of its biggest trade-offs.

Why Does DenseNet Need More Memory?

Remember:

Every layer receives **all previous feature maps**.

Imagine:

Layer 1 → 32 maps

Layer 2 → 64 maps

Layer 3 → 96 maps

Layer 4 → 128 maps

The network keeps accumulating features.

During training, many of these intermediate feature maps must be stored for backpropagation.

That increases memory consumption.

Real-Life Analogy

Imagine a student.

ResNet Student

Keeps only the latest notebook.

Old notebooks are summarized.

Light backpack.

DenseNet Student

Carries **every notebook** from every semester because future subjects might need them.

Great reference material.

Heavy backpack.

That's DenseNet.

Strengths of DenseNet

Let's summarize why researchers like it.

1. Excellent Feature Reuse

Previously learned features remain available throughout the network.

This reduces unnecessary relearning.

2. Strong Gradient Flow

Because each layer has direct access to earlier layers, gradients have many short paths during backpropagation.

This makes optimization easier.

3. Parameter Efficient

DenseNet often achieves strong performance with fewer trainable parameters than similarly accurate models.

4. Works Well on Smaller Datasets

Since it reuses learned features effectively, DenseNet can perform very well when data is limited—a common situation in medical imaging and passive sonar.

Weaknesses of DenseNet

Every architecture has trade-offs.

1. High Memory Usage

Keeping many feature maps increases GPU memory requirements.

2. Slower Feature Concatenation

Concatenating many feature maps introduces additional computational overhead compared to simple addition.

3. More Complex Data Flow

ResNet is conceptually simpler.

DenseNet's dense connectivity makes it harder to visualize and implement efficiently.

ResNet vs DenseNet

Feature	ResNet	DenseNet
Connection	Addition	Concatenation
Feature Strategy	Learn residual corrections	Reuse all previous features
Parameter Efficiency	Good	Often better
Training Memory	Lower	Higher
Speed	Generally faster	Often slower
Simplicity	Simpler	More complex

Which One is Better?

This is the wrong question.

A better question is:

Better for what?

Let's look at different situations.

Very Large Dataset

Millions of images.

Strong GPU.

Need high throughput.

ResNet is often an excellent choice.

Smaller Scientific Dataset

Examples:

- Medical images
- X-rays
- Sonar spectrograms
- LOFARGRAMs

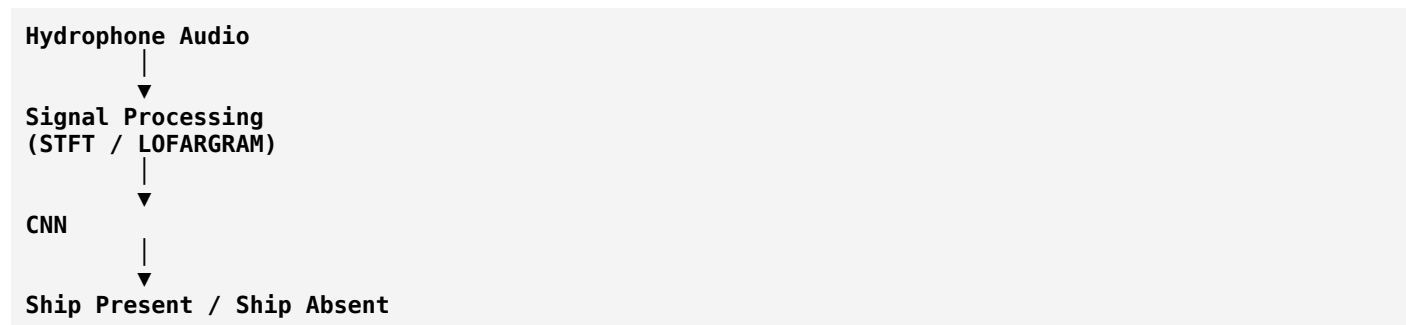
DenseNet is often a strong candidate because feature reuse can help extract more information from limited data.

Now Let's Think About Your Project

Your project:

Passive Sonar Based Target Ship Classification

Pipeline:



Would DenseNet work?

Yes.

Why?

Because your data likely contains:

- Repeating harmonic structures
- Tonal components
- Stable acoustic signatures

DenseNet can reuse low-level frequency features while learning increasingly complex acoustic representations.

That's one reason it appears in several sonar classification papers.

But Would I Choose DenseNet First?

If I were designing a first version of your system, I would compare at least three models:

1. **ResNet-18**
2. **DenseNet-121**
3. **EfficientNet-B0**

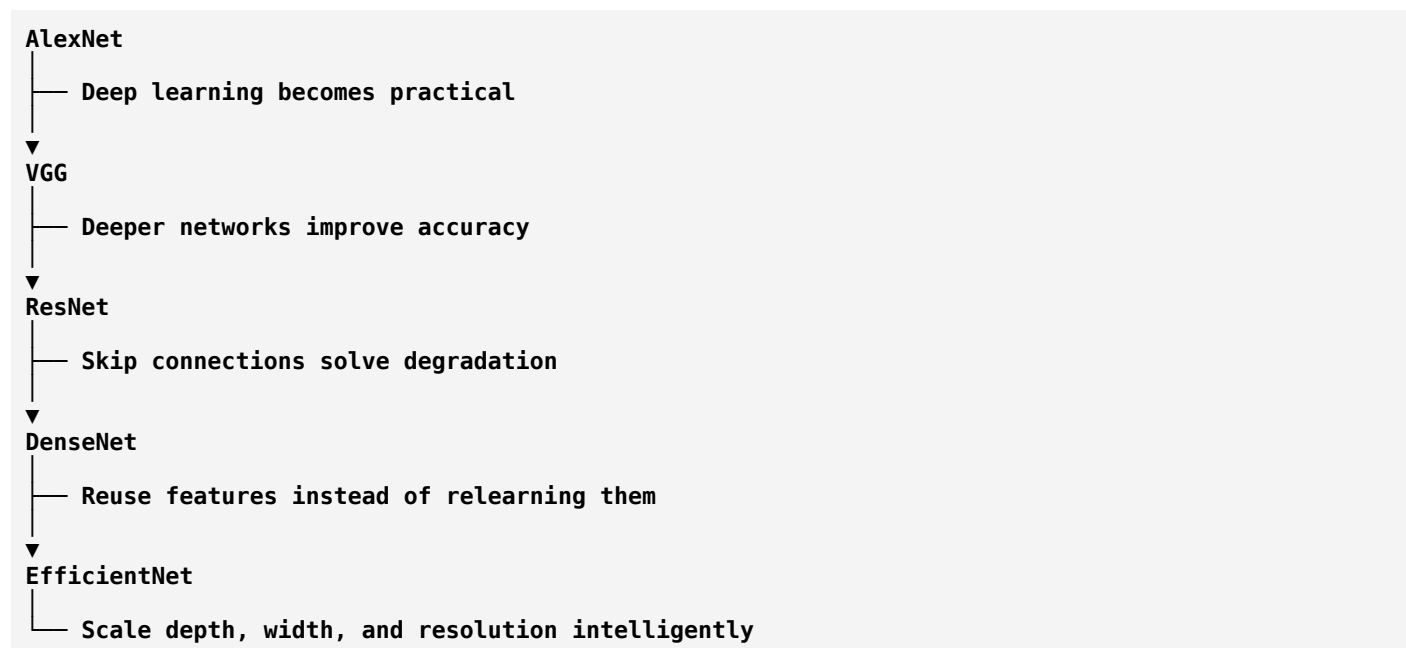
Why these three?

- They are well-established.
- They have pretrained weights available.
- They represent different architectural philosophies.
- They provide a good balance of performance and computational cost.

Running all three under the same training conditions gives you a fair comparison for your dataset.

Evolution of CNN Ideas

Notice how CNN research evolved.



Each architecture solves a limitation of the previous generation.

DenseNet in Research

When you read a paper that says:

"DenseNet-121 outperformed ResNet-18."

Don't stop there.

Ask:

- How large was the dataset?

- Did they use transfer learning?
- What input representation was used (LOFARGRAM, Mel spectrogram, etc.)?
- Was accuracy the only metric, or did they also compare inference time and memory usage?
- Was the improvement statistically meaningful?

That's how researchers evaluate models.

Summary

Concept	Key Idea
Dense Connections	Every layer receives all previous feature maps
Growth Rate	Each layer adds only a small number of new features
Transition Layer	Compresses feature maps and reduces spatial size
Main Strength	Excellent feature reuse
Main Weakness	Higher training memory usage
Typical Variant	DenseNet-121

What You Now Know

You now understand four landmark CNN architectures:

AlexNet

Deep learning becomes practical.

VGG

Simplicity and depth matter.

ResNet

Skip connections enable very deep networks.

DenseNet

Reuse features instead of relearning them.

Each architecture solved a major challenge and inspired the next generation of models.

Next Architecture: EfficientNet

Now we reach one of the most relevant architectures for your project.

EfficientNet asks a different question:

"Instead of making only the network deeper, what if we scale depth, width, and input resolution together in a balanced way?"

This idea led to models that achieved excellent accuracy with much lower computational cost.

For your passive sonar project, **EfficientNet-B0** is one of the strongest candidates to compare against ResNet-18 and DenseNet-121, making it a natural next topic.

ok next

Excellent! We are now entering what I consider the **most practical CNN architecture** for your project.

If ResNet was about **making deep networks trainable**,

and DenseNet was about **reusing features**,

then **EfficientNet** is about **using your computational resources intelligently**.

This architecture is widely used because it delivers **high accuracy with relatively low computation**.

For your **Passive Sonar Based Target Ship Classification**, EfficientNet-B0 is one of the first architectures I would experiment with.

Deep CNN Architectures

Chapter 5: EfficientNet (Part 1)

Why Was EfficientNet Invented?

Let's Go Back to 2018

Researchers had many powerful CNNs.

- ResNet
- DenseNet
- Inception
- MobileNet

They all worked well.

But there was a problem.

Suppose a researcher says:

"I want a better model."

What should they do?

Option 1: Make it Deeper

20 Layers

↓

50 Layers

↓

100 Layers

Maybe deeper means better.

Option 2: Make it Wider

Instead of increasing layers...

Increase filters.

64 Filters

↓

128 Filters

↓

256 Filters

Now every layer learns more features.

Option 3: Increase Image Resolution

Instead of

224 × 224

Use

448 × 448

Now the network sees more details.

Researchers tried all three.

Sometimes they increased only depth.

Sometimes only width.

Sometimes only image size.

Results improved...

But not consistently.

Imagine Cooking Rice

Suppose you want better rice.

Should you only increase:

Water?

No.

Should you only increase:

Rice?

No.

Should you only increase:

Cooking time?

No.

To cook perfect rice,

you balance:

- Rice
- Water
- Time

All three matter together.

Researchers realized CNNs are similar.

A good network isn't only:

- Deep
- Wide
- High resolution

It needs the **right balance** of all three.

The Three Dimensions of a CNN

Think of a CNN as having three knobs.

CNN

|



Let's understand each.

1. Depth

Depth means:

How many layers does the network have?

Example:

Conv

↓

Conv

↓

Conv

↓

Conv

More layers...

↓

More complex patterns.

Sonar Example

Early layers learn:

- Frequency edges
- Tonal lines

Middle layers learn:

- Harmonics

Deep layers learn:

- Ship acoustic signatures

Greater depth allows more abstract feature learning.

2. Width

Width means:

How many filters are in each layer?

Example:

Layer A

32 Filters

Layer B

64 Filters

Layer C

128 Filters

More filters mean the network can learn more feature types simultaneously.

Sonar Example

Imagine one layer has:

32 filters.

It may learn:

- Engine tone
- Broadband noise
- Harmonics
- Cavitation

Now increase to:

128 filters.

The network has more capacity to learn diverse acoustic characteristics.

3. Resolution

Resolution means:

The size of the input image.

Example:

128 × 128

VS

224 × 224

VS

512 × 512

Higher resolution preserves finer details but increases computational cost.

Sonar Example

Suppose your LOFARGRAM is:

64 × 64

Tiny details may disappear.

Now use:

224 × 224

The network can observe finer frequency structures.

But...

Processing takes longer.

The Old Approach

Before EfficientNet,

researchers usually changed **one thing at a time**.

Example:

Increase depth only.

Depth ↑

Width = Same

Resolution = Same

Or:

Increase width only.

Depth = Same

Width ↑

Resolution = Same

This often led to inefficient models.

The Brilliant Question

Researchers at Google asked:

"Why scale only one dimension?"

What if we scaled all three **together**?

Compound Scaling

This is the key idea behind EfficientNet.

Instead of saying:

Only increase depth.

EfficientNet says:

Increase:

- Depth
- Width
- Resolution

At the same time, in carefully chosen proportions.

This is called **Compound Scaling**.

Real-Life Analogy

Imagine building a sports team.

Would you recruit only:

Forwards?

No.

Only defenders?

No.

Only goalkeepers?

No.

A winning team needs balance.

CNNs are the same.

Visualizing Compound Scaling

Instead of this:

Depth ↑↑↑

Width –

Resolution –

EfficientNet does:

Depth ↑
Width ↑
Resolution ↑

Each increases gradually.

The improvements work together.

Why Is This Better?

Suppose you double the image resolution.

Now the network sees more detail.

But...

If the network isn't deep enough,
it may not know how to use that extra information.

Or...

Suppose you make the network extremely deep.

If the input resolution is very small,
there isn't much detail to learn from.

EfficientNet tries to balance these factors.

EfficientNet-B0

Everything begins with one model:

EfficientNet-B0

Think of it as the **base model**.

Researchers carefully searched for an architecture that provides a good trade-off between accuracy and efficiency.

Then Comes Scaling

From this base model, they created:

B0
↓

B1

↓

B2

↓

B3

↓

B4

↓

B5

↓

B6

↓

B7

Each version is larger than the previous one.

But the increase is **balanced**, not random.

Sonar Example

Suppose your LOFARGRAM is:

224 × 224

EfficientNet-B0 might process it efficiently.

If you later have:

- More training data.
- More GPU memory.

You might experiment with larger variants like B2 or B3.

The choice depends on your resources and dataset size.

Why Researchers Loved EfficientNet

Compared with many earlier CNNs, EfficientNet often achieved:

- Higher accuracy.
- Fewer parameters.
- Lower computational cost for similar performance.

That made it attractive for many vision tasks, including scientific imaging applications.

Summary

Concept	Meaning
Depth	Number of layers
Width	Number of filters per layer
Resolution	Size of the input image
Old Scaling	Change only one dimension
EfficientNet	Scale all three together
Key Idea	Compound Scaling

Think Like a Researcher

Suppose you have a powerful GPU.

Would you simply make the network **100 layers deeper**?

Or would you ask:

"Should I increase depth, width, and input resolution together in a balanced way?"

That second question is what led to EfficientNet.

Homework

Try answering these without looking back:

1. What is the difference between **depth**, **width**, and **resolution**?
2. Why isn't increasing only one of them always the best strategy?
3. What is **Compound Scaling**?
4. Why might balanced scaling be useful for LOFARGRAM-based ship classification?

If you can answer these confidently, you've understood the central idea behind EfficientNet.

Next Lesson (EfficientNet Part 2)

We'll go much deeper and explore:

- How does EfficientNet actually perform Compound Scaling mathematically?
- What is the Scaling Coefficient (ϕ)?
- What are the constants α , β , and γ ?
- Why does EfficientNet-B0 become B1, B2, B3, ..., B7?
- What is MBConv, and why is it the core building block of EfficientNet?

Important: MBConv is the heart of EfficientNet, just as the Residual Block is the heart of ResNet. Once you understand MBConv, you'll understand how EfficientNet achieves such impressive efficiency.

ok next

Excellent. 🔥

Now we are entering the **heart of EfficientNet**.

Just like:

- **Residual Block** is the heart of **ResNet**
- **Dense Block** is the heart of **DenseNet**

👉 **MBConv (Mobile Inverted Bottleneck Convolution)** is the heart of **EfficientNet**.

Many students consider this the most confusing part of EfficientNet. By the end of this lesson, it won't be confusing anymore.

Deep CNN Architectures

Chapter 5: EfficientNet (Part 2)

MBConv & Compound Scaling

Before Learning MBConv...

Let's ask a simple question.

Imagine you have an image:

$224 \times 224 \times 32$

This means:

- Height = 224
- Width = 224
- Channels = 32

Now suppose we apply a normal convolution.

The network performs a huge number of calculations.

Question:

Can we reduce these calculations without losing much accuracy?

Researchers spent years trying to answer this.

One of the best solutions became **MBConv**.

Where Did MBConv Come From?

EfficientNet didn't invent MBConv.

It borrowed the idea from another architecture called **MobileNetV2**.

EfficientNet then used MBConv as its main building block.

So whenever you see EfficientNet, remember:

EfficientNet

↓

Uses MBConv Blocks

↓

Originally introduced in MobileNetV2

The Problem With Normal Convolution

Imagine this feature map:

224 × 224 × 32

A normal **3×3 convolution** processes:

- Every pixel
- Every channel

This requires a lot of computation.

Think of it like asking one teacher to grade **10,000 answer sheets**.

Possible?

Yes.

Efficient?

Not really.

The MBConv Philosophy

Instead of processing everything at once, MBConv breaks the work into smaller, smarter steps.

Think of a hospital.

Would one doctor:

- Register the patient
- Perform blood tests

- Read X-rays
- Diagnose
- Perform surgery

No.

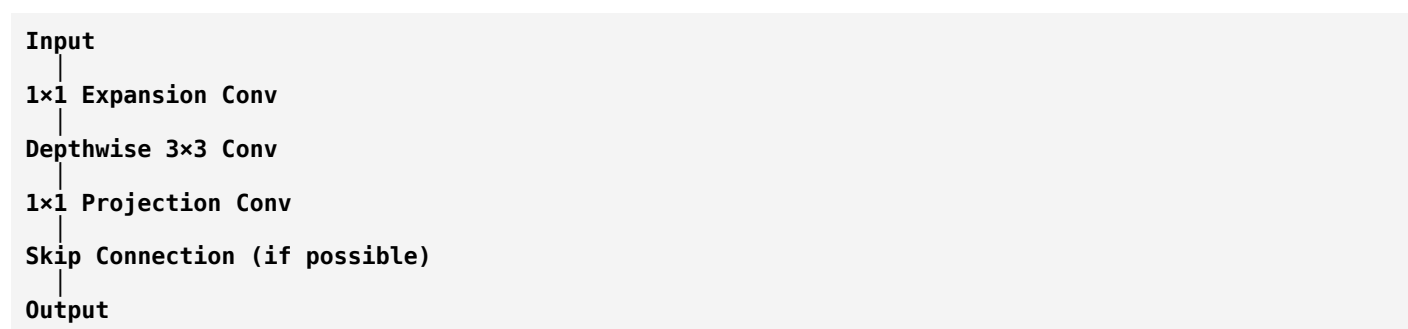
Different specialists handle different tasks.

MBConv follows the same philosophy.

Each step has a specific purpose.

MBConv Structure

A typical MBConv block looks like this:



There are **three major stages**.

Let's study each one.

Stage 1: Expansion Layer

Suppose the input is:

$224 \times 224 \times 32$

Instead of reducing channels...

MBConv first **expands** them.

Example:

$224 \times 224 \times 32$
 $\downarrow$
 $224 \times 224 \times 192$

Students usually ask:

"Why increase channels first? Isn't that the opposite of a bottleneck?"

Excellent question.

The idea is:

A higher-dimensional representation gives the network more room to learn useful feature combinations before compressing them again.

Think of it like expanding a rough sketch into a detailed drawing before deciding which details are important.

Expansion Analogy

Imagine you're writing an essay.

First, you brainstorm **50 ideas**.

Later, you choose the best **10**.

Expansion is like brainstorming.

Compression comes later.

Stage 2: Depthwise Convolution

This is the most important idea in MBConv.

Let's first remember a normal convolution.

Suppose you have:

224 × 224 × 32

A normal convolution mixes:

- Spatial information (neighbors)
- Channel information

all together.

This is expensive.

Depthwise Convolution

Instead, MBConv processes **each channel independently**.

Imagine you have 32 channels.

Instead of one convolution operating across all channels, you apply one spatial filter **per channel**.

Channel 1 → 3×3 Conv

Channel 2 → 3×3 Conv

Channel 3 → 3×3 Conv

...

Channel 32 → 3×3 Conv

Only after the spatial filtering do later layers mix information across channels.
This dramatically reduces computation.

Real-Life Analogy

Imagine a school.

Instead of one teacher teaching:

- Math
- Physics
- Chemistry
- Biology

You assign:

- One Math teacher.
- One Physics teacher.
- One Chemistry teacher.
- One Biology teacher.

Each specialist focuses on a single subject.

Depthwise convolution works similarly by processing each channel separately.

Why Is This Faster?

Normal convolution learns spatial patterns and channel interactions in one operation.

Depthwise convolution handles only the spatial part.

The later **1×1 projection** handles channel mixing.

By separating these tasks, MBConv reduces computational cost while maintaining good performance.

Stage 3: Projection Layer

After expansion and depthwise convolution:

224 × 224 × 192

The network compresses back to:

224 × 224 × 32

using a **1×1 convolution**.

This step is called the **projection layer**.

Its purpose is to produce a compact, informative representation.

Why Is It Called "Inverted Bottleneck"?

Remember ResNet's bottleneck?

256

↓

64

↓

256

It starts **wide**, becomes **narrow**, then expands again.

MBConv does the opposite:

32

↓

192

↓

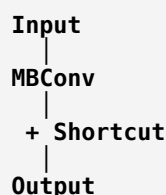
32

It starts **narrow**, expands, then projects back.

That's why it's called an **Inverted Bottleneck**.

Skip Connection

Whenever the input and output have compatible shapes, MBConv also includes a **skip connection**, similar to ResNet.



This helps information and gradients flow through the network.

Now Let's Understand Compound Scaling

Earlier we learned:

EfficientNet scales:

- Depth
- Width
- Resolution

together.

But how?

Researchers introduced a scaling coefficient:

ϕ (phi)

Think of ϕ as the "model size" control.

Imagine a Volume Knob

$\phi = 0 \rightarrow \text{EfficientNet-B0}$

$\phi = 1 \rightarrow \text{EfficientNet-B1}$

$\phi = 2 \rightarrow \text{EfficientNet-B2}$

$\phi = 3 \rightarrow \text{EfficientNet-B3}$

As ϕ increases, the model becomes larger.

But EfficientNet doesn't increase only depth or only width.

Instead, it scales all three dimensions according to carefully chosen ratios.

The original paper represents these scaling factors with constants:

- α (alpha) for depth
- β (beta) for width
- γ (gamma) for input resolution

These values were selected through experiments to provide a good balance between accuracy and efficiency.

You don't need to memorize the equations now—what matters is understanding **why** the balanced scaling works.

EfficientNet Family

B0

↓

B1

↓

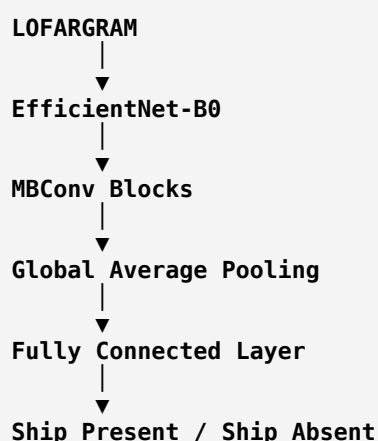
B2

↓
B3
↓
B4
↓
B5
↓
B6
↓
B7

Every version uses the same design philosophy.
The difference is the amount of balanced scaling.

Applying This to Your Sonar Project

Imagine you convert sonar recordings into **LOFARGRAMs**.
Then:



Each MBConv block learns progressively richer acoustic features while keeping computation efficient.

Why Researchers Like EfficientNet

Compared with many earlier CNNs, EfficientNet often provides:

- Strong accuracy.
- Efficient use of parameters.
- Lower computational cost for similar performance.

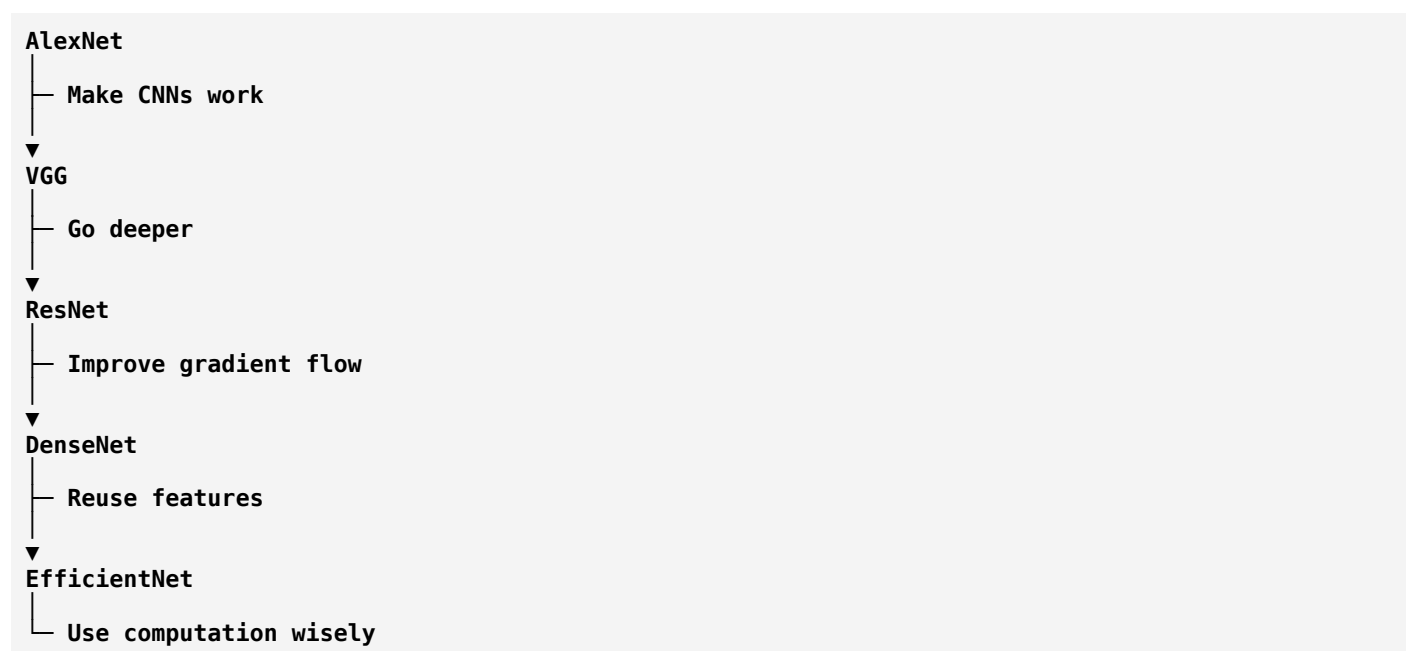
This makes it attractive for tasks such as medical imaging, remote sensing, and acoustic spectrogram classification.

Summary

Concept	Purpose
Expansion Layer	Increase channels to learn richer features
Depthwise Convolution	Learn spatial patterns efficiently, one channel at a time
Projection Layer	Compress features into a compact representation
Inverted Bottleneck	Expand → Process → Compress
MBConv	Core building block of EfficientNet
ϕ (phi)	Controls overall model scaling
α, β, γ	Balance depth, width, and resolution

Understanding the Evolution

Notice how CNN design has evolved:



Each architecture addresses a different limitation.

Next Lesson (EfficientNet Part 3 – Final)

In the final EfficientNet lesson, we'll bring everything together by answering:

- Why is EfficientNet often chosen over ResNet and DenseNet?
- When should you use EfficientNet-B0 vs B3 vs B7?
- How does EfficientNet perform on small datasets like passive sonar?
- ResNet vs DenseNet vs EfficientNet: Which should you choose for your project, and why?

- **What architecture would I recommend for your binary ship detection system?**

That final comparison is exactly the kind of reasoning you'll need when reading and designing research papers.

ok next

Excellent. 🎉

This is the **final lesson on EfficientNet**, and one of the most practical lessons in this entire deep learning course.

After today, you'll be able to read a research paper and understand **why the authors chose ResNet, DenseNet, or EfficientNet**.

More importantly, you'll know **which model makes sense for your passive sonar project**.

Deep CNN Architectures

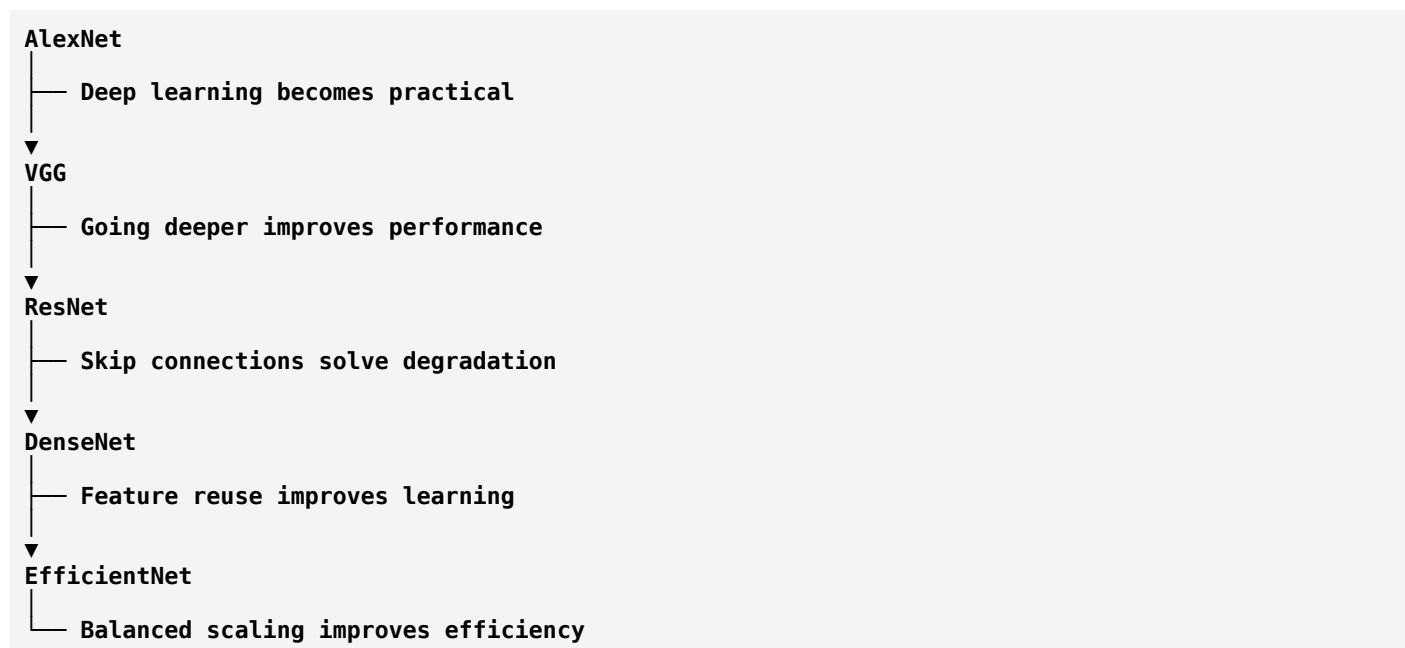
Chapter 5: EfficientNet (Part 3 – Final)

Choosing the Right CNN for Passive Sonar Target Classification

First, Let's Compare the Evolution

Imagine you're trying to build the **best ship detector**.

Researchers improved CNNs step by step.



Notice something.

Each architecture solved **one major problem**.

No architecture is "perfect."
Each has strengths and weaknesses.

Imagine Building a Car

Suppose three companies build cars.

Company A

Makes a powerful engine.

Problem:

Consumes lots of fuel.

Company B

Makes a fuel-efficient engine.

Problem:

Not very powerful.

Company C

Finds the balance.

Good mileage.

Good speed.

Good reliability.

EfficientNet is similar to Company C.

It tries to balance performance and efficiency.

Why EfficientNet Became Popular

Researchers observed something surprising.

A model with:

- fewer parameters,
- balanced scaling,
- and MBConv blocks,

could achieve performance comparable to or better than much larger CNNs on many image recognition tasks.

This made EfficientNet attractive for practical applications where computation matters.

Does Bigger Always Mean Better?

Let's compare.

Suppose:

Model A

10 Million Parameters

Accuracy:

96%

Model B

50 Million Parameters

Accuracy:

96.3%

Question:

Would you spend **5× more computation** for a **0.3% improvement**?

Sometimes yes.

Sometimes no.

It depends on:

- Your hardware.
- Your latency requirements.
- Your application.

EfficientNet aims to give **excellent performance without unnecessary computational cost**.

EfficientNet Family

Let's understand the different versions.

B0

↓

B1

↓

B2

↓

B3

↓

B4

↓

B5

↓

B6

↓

B7

Think of these as members of one family.

EfficientNet-B0

- Smallest model.
 - Fast.
 - Good starting point.
 - Often used when GPU resources or dataset size are limited.
-

EfficientNet-B1 to B3

These increase:

- Depth
- Width
- Resolution

in a balanced manner.

They often provide higher accuracy but require more computation.

EfficientNet-B4 to B7

These are much larger.

Suitable when:

- Large datasets.
 - Powerful GPUs.
 - Longer training time is acceptable.
-

Which One Fits Your Sonar Project?

Let's think like researchers.

Your project:

Binary Classification

Input:

LOFARGRAM

Output:

Ship Present

or

Ship Absent

Suppose your dataset has:

2000 Images

Should you use:

EfficientNet-B7?

Probably not.

It may have more capacity than your dataset can effectively support.

Instead,

a smaller model such as:

- EfficientNet-B0
- EfficientNet-B1

is often a more practical starting point.

Always validate this experimentally rather than assuming a larger model will perform better.

Comparing the Three Architectures

ResNet

Strengths:

- Stable training.
- Easy to understand.
- Strong baseline.
- Excellent transfer learning support.

Weaknesses:

- Can be larger than necessary for some tasks.

DenseNet

Strengths:

- Excellent feature reuse.
- Strong gradient flow.
- Often performs well on smaller datasets.

Weaknesses:

- Higher memory usage during training.
- More complex connectivity.

EfficientNet

Strengths:

- Excellent accuracy-to-computation ratio.
- Balanced scaling.
- Efficient MBConv blocks.

Weaknesses:

- More complex architecture.
- Larger variants require more resources.

Let's Compare Them

Property	ResNet	DenseNet	EfficientNet
Main Idea	Residual Learning	Feature Reuse	Compound Scaling
Block	Residual Block	Dense Block	MBConv
Connections	Addition	Concatenation	Skip Connections (inside MBConv when applicable)
Memory During Training	Lower	Higher	Moderate
Parameter Efficiency	Good	Very Good	Excellent
Computational Efficiency	Good	Moderate	Excellent
Ease of Understanding	Easy	Medium	Medium–Advanced

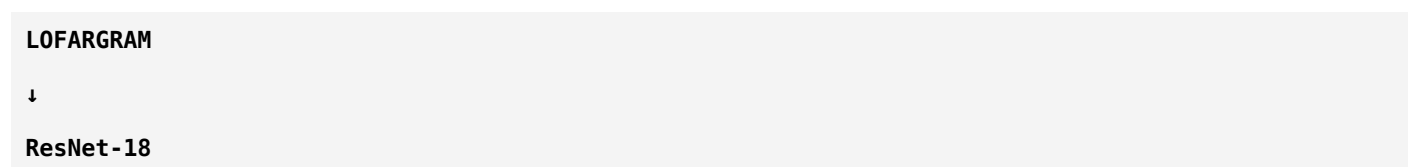
Now Think About Your Pipeline

Imagine your entire system.



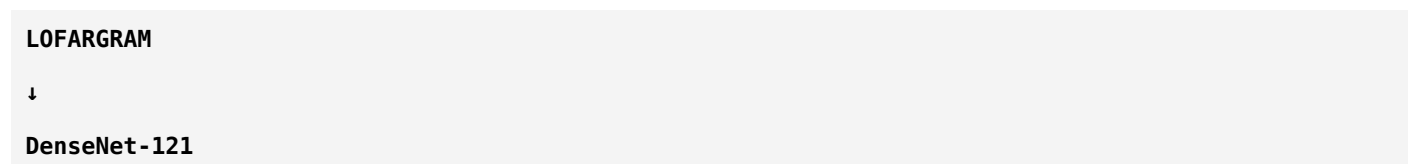
The only thing changing is the CNN.

Option 1



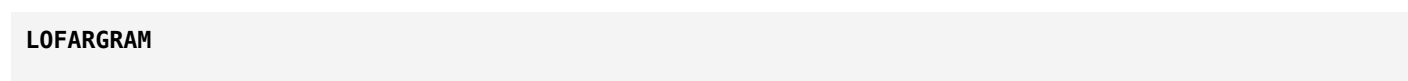
Very common baseline.

Option 2



Excellent feature reuse.

Option 3



↓

EfficientNet-B0

Very efficient.

This is exactly how many research papers are structured:

Same preprocessing, same dataset, different CNN backbones.

That allows a fair comparison.

If You Were Publishing a Paper

Suppose you're writing a research paper.

A sensible experimental design would be:

Model	Purpose
ResNet-18	Strong baseline
DenseNet-121	Evaluate feature reuse
EfficientNet-B0	Evaluate computational efficiency and balanced scaling

Then compare metrics such as:

- Accuracy
- Precision
- Recall
- F1-score
- Training time
- Inference time

This produces a much stronger evaluation than testing only one model.

A Very Important Point

Many beginners believe:

"The newest architecture is always the best."

That isn't true.

The best model depends on:

- Dataset size.
- Hardware.
- Available training time.

- Inference speed requirements.
- Whether accuracy or efficiency is the priority.

Model selection is an engineering decision, not a popularity contest.

What I Would Recommend for Your Project

If I were mentoring this project, I would evaluate the models in this order:

Phase 1

ResNet-18

Why?

- Simple.
 - Stable.
 - Well understood.
 - Strong baseline.
-

Phase 2

DenseNet-121

To see whether feature reuse improves performance.

Phase 3

EfficientNet-B0

To evaluate whether balanced scaling improves accuracy or efficiency.

Only after these comparisons would I consider larger models.

What You've Learned So Far

You now understand the complete journey of classic CNN architectures.

AlexNet

CNNs become practical.

VGG

Deeper networks improve representation.

ResNet

Skip connections solve degradation.

DenseNet

Reuse features instead of relearning them.

EfficientNet

Balance depth, width, and resolution intelligently.

Where Do We Go Next?

Up to this point, we've studied **CNNs**.

Now it's time to study architectures that changed deep learning again.

Recommended order:

1. **Inception (GoogLeNet)** – Multi-scale feature extraction.
 2. **MobileNet** – Lightweight CNNs for edge devices.
 3. **Vision Transformer (ViT)** – Uses self-attention instead of convolutions.
 4. **Swin Transformer** – Hierarchical vision transformers.
 5. **ConvNeXt** – Modern CNN inspired by transformers.
-

★ My Recommendation

Since your project is based on **passive sonar classification**, I recommend the following path:

- **Inception (GoogLeNet)** — Learn how CNNs can extract features at multiple scales. This is useful because sonar signatures can vary in size and duration.
- **Vision Transformers (ViT)** — Many recent sonar research papers compare CNNs with transformer-based models.

That sequence will take you from the foundations of CNNs to the architectures currently appearing in state-of-the-art research.

Before We Continue...

You've now mastered the major CNN families.

Here's your current roadmap:

- ☒ Digital Audio Processing (FFT, STFT, Spectrogram, Mel, MFCC, LOFARGRAM)
- ☒ Machine Learning Fundamentals
- ☒ Deep Learning Fundamentals
- ☒ CNN Fundamentals
- ☒ AlexNet
- ☒ VGG
- ☒ ResNet
- ☒ DenseNet
- ☒ EfficientNet

You're now ready to move into **advanced CNNs and Transformers**, where you'll start seeing the architectures that dominate current computer vision and acoustic signal classification research.

ok next

Excellent. 🎉

You've completed the **fundamental CNN architectures**. Now we enter the **second generation of CNNs**.

This architecture completely changed how researchers thought about convolution.

Before this architecture, everyone believed:

"To improve a CNN, make it deeper."

Google researchers asked a different question:

"Instead of only making the network deeper, what if we make it smarter?"

That question created **GoogLeNet (Inception Network)**.

Deep CNN Architectures

Chapter 6: GoogLeNet (Inception) – Part 1

Why Was Inception Invented?

Let's Travel Back to 2014

Researchers had:

- AlexNet (8 layers)
- VGG (16–19 layers)

VGG showed:

Deeper networks usually perform better.

Everyone thought:

"Let's just keep adding more layers."

But Google researchers noticed a problem.

Problem 1: Computational Cost

Imagine a VGG network.

Every new layer means:

- More filters.
- More parameters.
- More computation.
- More memory.

Training became slower and more expensive.

Problem 2: What Filter Size Should We Use?

Imagine you're looking at a satellite image.

To detect:

A small car,

you might use a **small filter**.

To detect:

A large building,

you might need a **larger filter**.

Now think about your sonar project.

Sonar Example

Suppose your LOFARGRAM contains:

Tiny harmonic lines.

A small **3×3** filter may detect them well.

Now imagine:

A long engine harmonic extending across time.

=====

A larger receptive field may be more useful.

Question:

Which filter should we use?

- 1×1?
- 3×3?
- 5×5?

Researchers didn't know.

Traditional CNN Thinking

The old approach was:

Choose one filter size.

Example:

Input

↓

3×3 Conv

↓

3×3 Conv

↓

3×3 Conv

Everything uses:

3×3.

Simple.

But is one filter size enough for every pattern?

Probably not.

Real-Life Analogy

Imagine you're a photographer.

You own:

One camera lens.

Question:

Can one lens photograph:

- An ant?
- A person?
- A mountain?

Not perfectly.

Professional photographers use different lenses because different scales require different views.

Google researchers asked:

"Why force the network to use only one filter size?"

The Brilliant Idea

Instead of choosing:

3×3

OR

5×5

OR

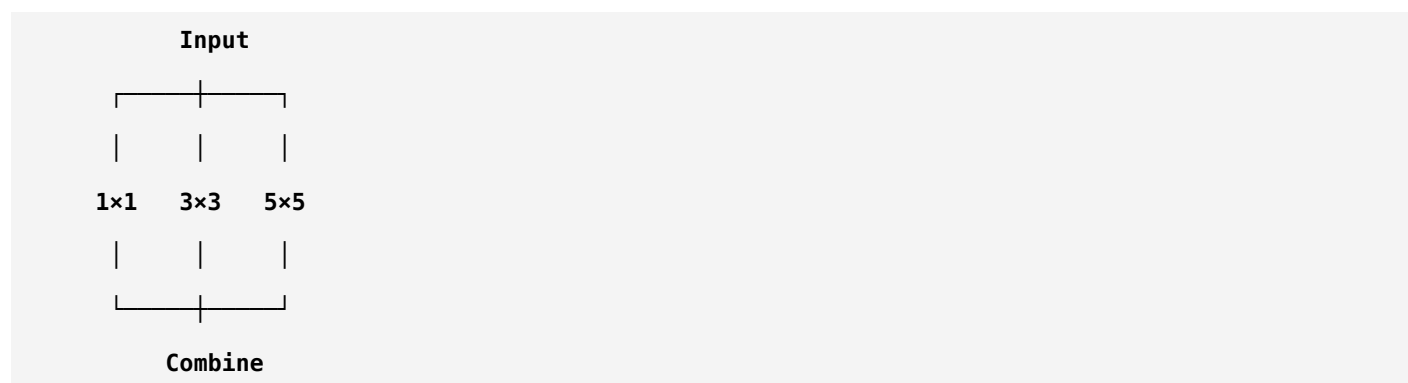
1×1

Why not use **all of them simultaneously?**

Imagine the input.

Instead of one path,

create multiple parallel paths.



This was revolutionary.

Why Is It Called "Inception"?

The architecture was inspired by the movie **Inception**.

In the movie:

There are "layers within layers."

Similarly,

the Inception network contains **multiple parallel operations inside a single block**.

Instead of one path,

many paths operate simultaneously.

What Does Each Filter Learn?

Imagine your LOFARGRAM.

1×1 Filter

Looks at one location.

Learns relationships between channels.

Acts as a channel mixer.

3×3 Filter

Looks at nearby frequency-time regions.

Learns:

- Small harmonics.
 - Local tonal patterns.
-

5×5 Filter

Looks over a larger region.

Learns:

- Broader engine signatures.
 - Larger frequency structures.
-

Now imagine combining all three.

The network can understand:

- Small features.
- Medium features.

- Large features.

All at the same time.

Multi-Scale Learning

This introduces one of the most important concepts in deep learning.

Multi-Scale Feature Extraction

Different objects exist at different scales.

Examples:

Road signs.

Cars.

Buildings.

Similarly,

in sonar:

- Narrow harmonic lines.
- Medium-duration machinery sounds.
- Long-duration engine patterns.

Each occupies a different scale.

The Inception module processes them simultaneously.

Traditional CNN

Input

↓

3×3 Conv

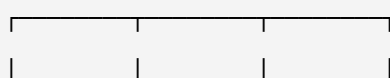
↓

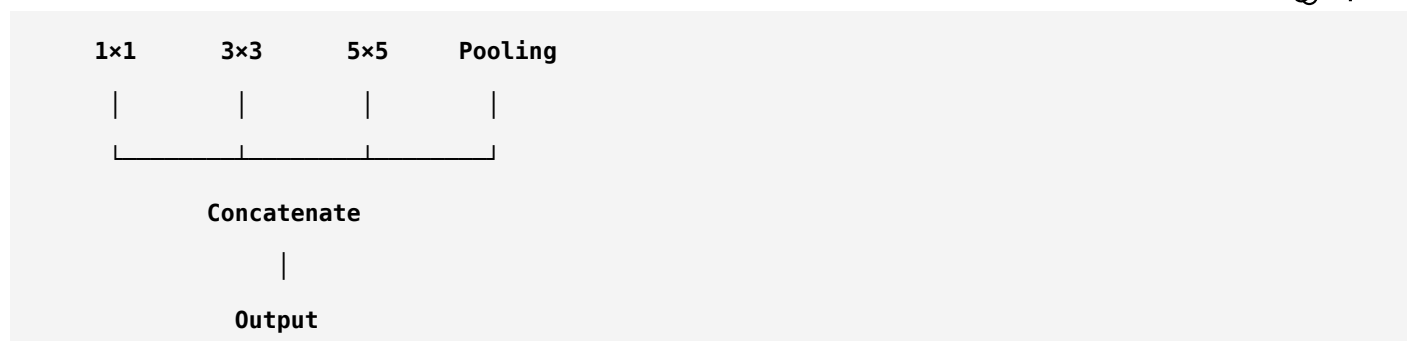
Output

Only one scale.

Inception Module

Input





Notice something.

There are **four parallel branches**.

Wait...

Wouldn't This Be Very Expensive?

Excellent question.

Suppose you directly apply:

5x5 convolution.

That's computationally expensive.

Google researchers realized this too.

Their solution was brilliant.

Before the expensive 3x3 and 5x5 convolutions,

they inserted:

1x1 convolutions.

These reduce the number of channels first, making the larger convolutions much cheaper.

This is the same principle you saw in ResNet bottleneck blocks.

Sonar Example

Imagine your LOFARGRAM has:

256 channels.

Instead of applying a costly 5x5 convolution to all 256 channels,

the network first uses a 1x1 convolution to reduce the channels.

Then it applies the 5x5 convolution on the reduced representation.

This saves a huge amount of computation.

Why Pooling Is Also Included

Notice the fourth branch:

Pooling

Pooling summarizes local information.

It helps the network become more robust to small shifts and reduces sensitivity to exact feature locations.

Including pooling alongside convolutions gives the network another way to capture useful information.

Why Researchers Loved GoogLeNet

GoogLeNet demonstrated that you don't always need a much deeper network to improve performance.

Sometimes,

using **multiple feature scales efficiently** is enough.

This idea influenced many later architectures.

Summary

Concept	Meaning
Inception Module	Several operations performed in parallel
Multi-Scale Learning	Learn small, medium, and large patterns simultaneously
1×1 Convolution	Reduce channels before expensive convolutions
Pooling Branch	Adds robust summarized features
Main Goal	Increase representational power without excessive computation

Think Like a Researcher

Imagine your LOFARGRAM contains:

- Very thin harmonic lines.
- Medium-sized machinery signatures.
- Long-duration engine patterns.

Would you design a CNN that looks at only one scale?

Or one that analyzes **all scales simultaneously**?

That question led directly to the Inception architecture.

Homework

Without looking back, answer these:

1. Why wasn't using only one filter size ideal?
2. What is **multi-scale feature extraction**?
3. Why does an Inception module have multiple parallel branches?
4. Why are **1×1 convolutions** placed before the 3×3 and 5×5 convolutions?

If you can explain these in your own words, you've understood the philosophy behind GoogLeNet.

Next Lesson (GoogLeNet – Part 2)

We'll dive into the actual **Inception Module** in detail:

- How does each branch work internally?
- Why are the outputs **concatenated** instead of added?
- What is dimensionality reduction?
- How does a LOFARGRAM flow through an Inception module?
- Why did GoogLeNet achieve high accuracy with far fewer parameters than VGG?

By the end of that lesson, you'll understand one of the most influential ideas in CNN history.

ok next

Excellent. This is **the heart of GoogLeNet**.

If you understand this lesson, you'll understand why the **Inception Module** became one of the most influential building blocks in deep learning.

Unlike ResNet (which focused on **better information flow**) or DenseNet (which focused on **feature reuse**), Inception focuses on **capturing information at multiple scales simultaneously**.

Deep CNN Architectures

Chapter 6: GoogLeNet (Inception) – Part 2

The Inception Module (The Heart of GoogLeNet)

First, Let's Recall the Problem

Imagine you're looking at this LOFARGRAM.

```
Time →
Frequency ↑
-----
=====
=====
```

```

=====
      =====

```

Some features are:

- Very thin
- Medium width
- Very large

Now ask yourself:

Can one 3×3 filter detect all of them equally well?

The answer is:

Not always.

Different patterns require different receptive fields.

What is a Receptive Field?

This is one of the most important concepts in CNNs.

Definition

A **Receptive Field** is:

The region of the input image that influences the output of one neuron.

Think of it as:

"How much of the image can this neuron see?"

Human Eye Analogy

Imagine you're standing on a mountain.

Looking through a straw

You see:

Only a tiny region.

Small receptive field.

Looking normally

You see:

A larger area.

Medium receptive field.

Looking from an airplane

You see:

The entire city.

Large receptive field.

CNN filters behave similarly.

Filter Size Determines Receptive Field

1×1 Filter

Sees:

□

Very small.

3×3 Filter

Sees:

□ □ □
□ □ □
□ □ □

More surrounding information.

5×5 Filter

Sees:

□ □ □ □ □
□ □ □ □ □
□ □ □ □ □
□ □ □ □ □
□ □ □ □ □

Much larger context.

Which One is Better?

This is a trick question.

Suppose you're reading a book.

To identify:

A single letter,

you don't need the whole page.

But to understand:

The meaning of a paragraph,

looking at one letter isn't enough.

Similarly:

- Small filters capture fine details.
- Large filters capture broader context.

Both are valuable.

The Inception Solution

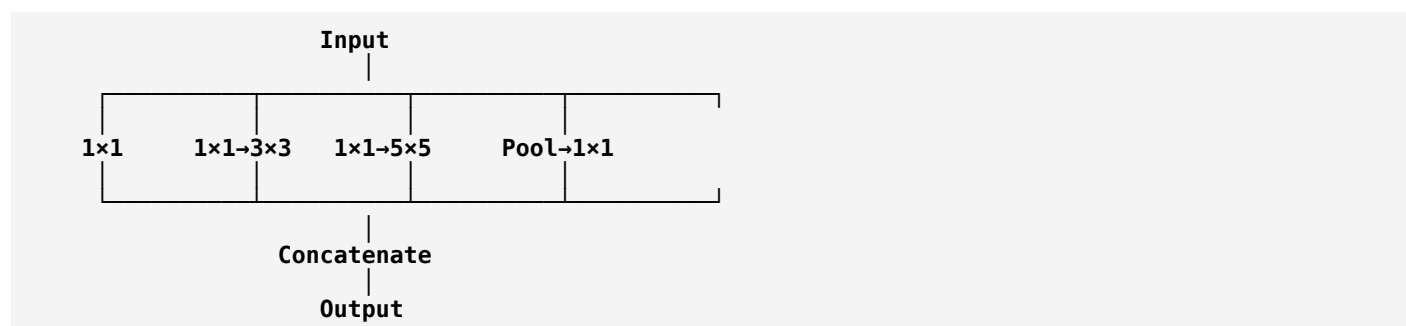
Instead of choosing one filter size,

Google researchers said:

"Let's use all of them together."

Complete Inception Module

Imagine the input feature map enters the module.



Notice something.

There are **four parallel branches**.

Each branch learns something different.

Branch 1

1x1 Convolution

Purpose:

- Mix channel information.
- Learn simple features.
- Reduce computation.

This branch is computationally inexpensive.

Branch 2

1×1 Conv

↓

3×3 Conv

Students often ask:

"Why not directly use a 3×3 convolution?"

Because 3×3 is more expensive.

The 1×1 convolution first reduces the number of channels.

Then the 3×3 convolution operates on fewer channels.

This saves computation.

Branch 3

1×1 Conv

↓

5×5 Conv

Same idea.

The expensive 5×5 convolution is applied after reducing the channels.

Without the 1×1 layer, this branch would require much more computation.

Branch 4

Pooling

↓

1×1 Conv

Why include pooling?

Pooling captures summarized local information.

The following 1×1 convolution adjusts the channel dimension so this branch can be combined with the others.

Why Are Outputs Concatenated?

This is a very common interview question.

Imagine the four branches produce:

```
Branch 1
Edge Features
Branch 2
Small Harmonics
Branch 3
Large Engine Patterns
Branch 4
Robust Summary Features
```

If we **added** them together, we would merge them into a single representation.

Instead, GoogLeNet **concatenates** them.

That means all learned feature maps are preserved.

The next layer can decide which features are useful.

Addition vs Concatenation

Remember:

ResNet

```
Feature A
+
Feature B
```

One combined feature map.

DenseNet

```
[A , B]
```

Both feature maps are preserved.

GoogLeNet

It also uses **concatenation**.

The outputs from all four branches are stacked along the **channel dimension**.

This increases the richness of the representation.

Sonar Example

Suppose your LOFARGRAM contains:

- Thin harmonic lines.
- Wide machinery bands.
- Broad low-frequency energy.
- Background ocean noise.

The four branches may specialize like this:

Branch	Possible Learned Pattern
1×1	Channel relationships
3×3	Small harmonic structures
5×5	Large acoustic signatures
Pooling	Stable background information

The concatenated output contains information from **all four perspectives**.

Why Is This Better Than One Filter?

Imagine a doctor.

Would one specialist diagnose every disease?

Or would you consult:

- Cardiologist.
- Neurologist.
- Orthopedic surgeon.
- Radiologist.

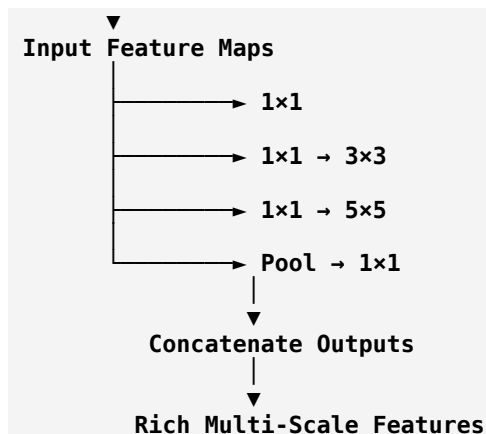
Each expert contributes different information.

The final diagnosis is based on all of them.

That's exactly how the Inception Module works.

How Does a LOFARGRAM Flow Through an Inception Module?

LOFARGRAM



Instead of forcing one way of looking at the data, the network learns several complementary views.

Why GoogLeNet Had Far Fewer Parameters Than VGG

VGG often relied on many large convolution layers and fully connected layers.

GoogLeNet:

- Used 1×1 convolutions to reduce channels.
- Avoided unnecessary computation.
- Used Global Average Pooling near the end instead of very large fully connected layers.

The result was a much more parameter-efficient architecture.

Summary

Concept	Key Idea
Receptive Field	The region of the input seen by a neuron
Small Filters	Capture fine details
Large Filters	Capture broader context
Inception Module	Multiple branches operating in parallel
1×1 Convolution	Reduces channels before expensive operations
Concatenation	Preserves outputs from all branches
Multi-Scale Learning	Learn patterns of different sizes simultaneously

Think Like a Researcher

Imagine you're analyzing a LOFARGRAM.

Would you want your model to look at:

- Only tiny harmonic lines?

Or

- Only broad engine signatures?

Neither.

You'd want it to analyze **both at the same time**.

That is exactly what the Inception Module was designed to do.

Homework

Without looking back, explain:

1. What is a **receptive field**?
2. Why are different filter sizes useful?
3. Why does GoogLeNet use **parallel branches** instead of a single path?
4. Why are the branch outputs **concatenated** rather than added?
5. How could multi-scale feature extraction help classify ship sounds from LOFARGRAMs?

If you can answer those questions, you've understood the core of the Inception architecture.

Next Lesson (GoogLeNet – Part 3, Final)

We'll complete GoogLeNet by studying:

- How the **entire GoogLeNet architecture** is organized.
- What are **Auxiliary Classifiers**, and why were they introduced?
- Why does GoogLeNet use **Global Average Pooling**?
- GoogLeNet vs ResNet vs DenseNet vs EfficientNet.
- Is GoogLeNet still useful today?
- Should you use GoogLeNet for your **passive sonar target classification project**, or are newer architectures a better choice?

After that, we'll be ready to transition into **MobileNet** and eventually **Vision Transformers (ViT)**, which represent the next major evolution in deep learning.

ok next

Excellent. 🌟

This is the **final lesson on GoogLeNet (Inception)**.

After this lesson, you'll completely understand one of the most influential CNN architectures ever created.

More importantly, you'll understand why some of its ideas (especially **1×1 convolutions** and **multi-scale feature extraction**) are still used in modern networks.

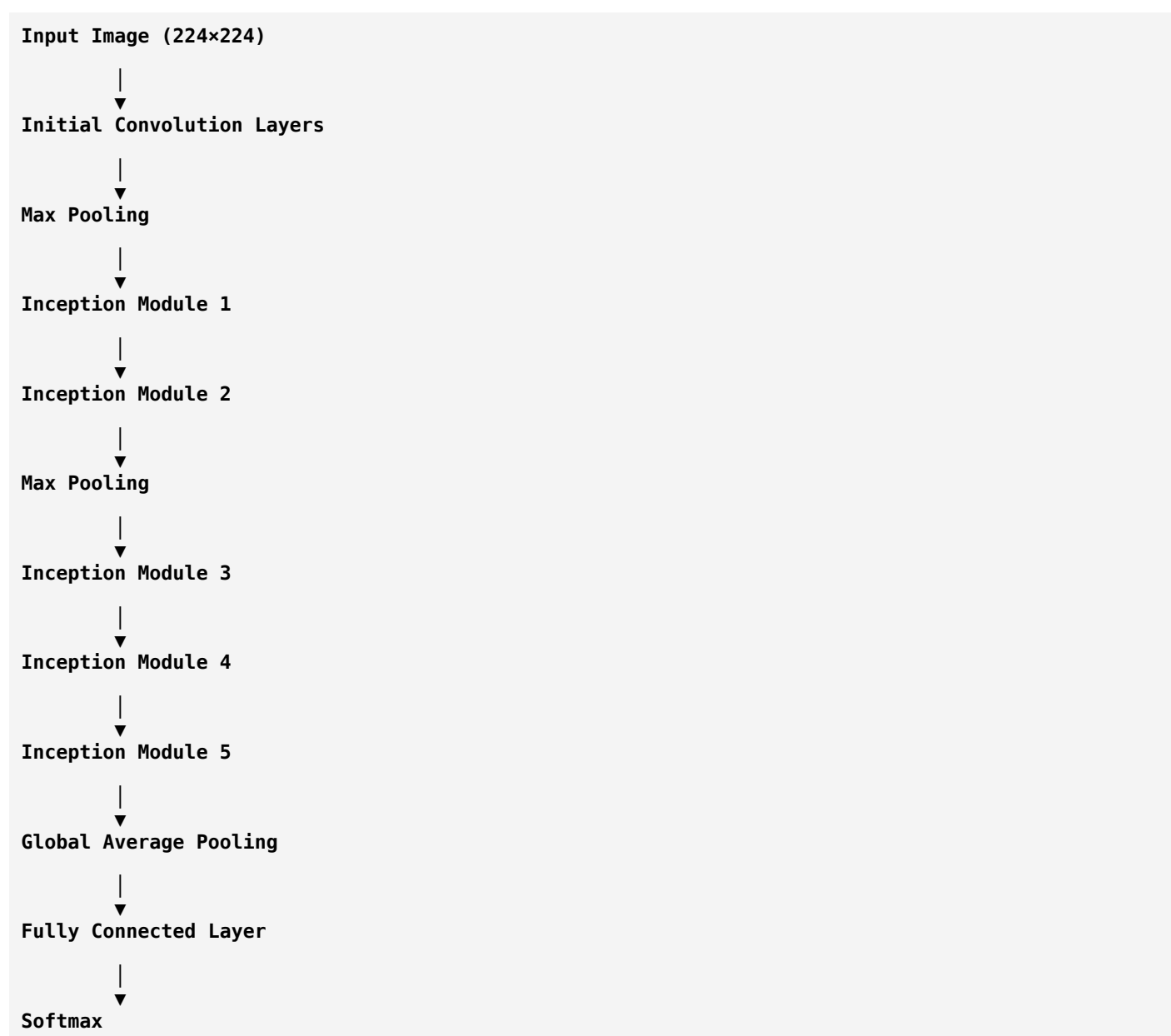
Deep CNN Architectures

Chapter 6: GoogLeNet (Inception) – Part 3 (Final)

Complete Architecture, Auxiliary Classifiers & Modern Relevance

The Complete GoogLeNet Architecture

Let's first see the big picture.



Unlike VGG:

Conv

↓

Conv

↓

Conv

↓

Conv

GoogLeNet repeats **Inception Modules** instead of simple convolution layers.

Why Stack Multiple Inception Modules?

Think about reading a book.

The first chapter teaches:

Basic concepts.

The next chapter teaches:

Intermediate concepts.

Later chapters teach:

Advanced concepts.

Similarly,

Each Inception Module learns increasingly abstract features.

First Inception Modules

Learn:

- Edges
- Simple frequency patterns
- Small harmonics

Middle Modules

Learn:

- Machinery tones
- Engine harmonics
- Frequency combinations

Final Modules

Learn:

- Complete ship signatures
 - High-level acoustic characteristics
-

A Very Interesting Problem

Researchers trained very deep networks.

Something unexpected happened.

Early layers learned very slowly.

Why?

Because the error signal had to travel through many layers during backpropagation.

As it moved backward, the learning signal weakened.

Google's Solution

They introduced something completely new.

Auxiliary Classifiers

This was one of the most innovative ideas in GoogLeNet.

What is an Auxiliary Classifier?

Imagine you're taking a course.

Final exam:

100 marks.

Suppose the teacher gives you:

- One small quiz after Chapter 3.
- Another quiz after Chapter 6.
- Final exam after Chapter 10.

Those quizzes tell you early whether you're learning correctly.

Similarly,

GoogLeNet adds small classifiers in the middle of the network.

Architecture

```

Input
↓
Inception
↓
Inception
↓
Auxiliary Classifier
↓
Inception
↓
Inception
↓
Auxiliary Classifier
↓
Inception
↓
Final Classifier

```

There are:

- Two auxiliary classifiers.
- One final classifier.

Why Were They Added?

Two main reasons.

1. Better Gradient Flow

The middle layers receive a stronger learning signal.

They don't have to wait for gradients to travel all the way back from the final output.

2. Regularization

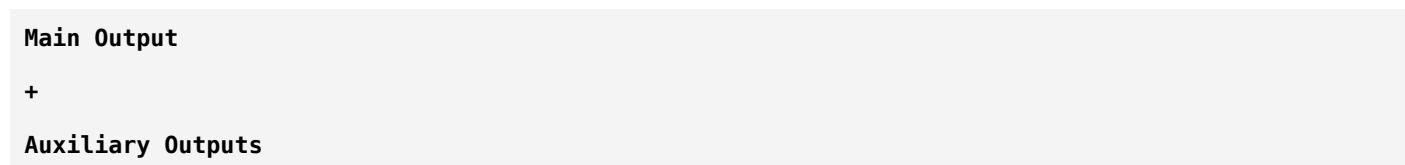
The network is encouraged to learn useful intermediate representations.

This can reduce overfitting.

During Inference

Here's something many beginners don't know.

During training:



All contribute to the training loss.

During testing (inference):

The auxiliary classifiers are **discarded**.

Only the main classifier is used to make predictions.

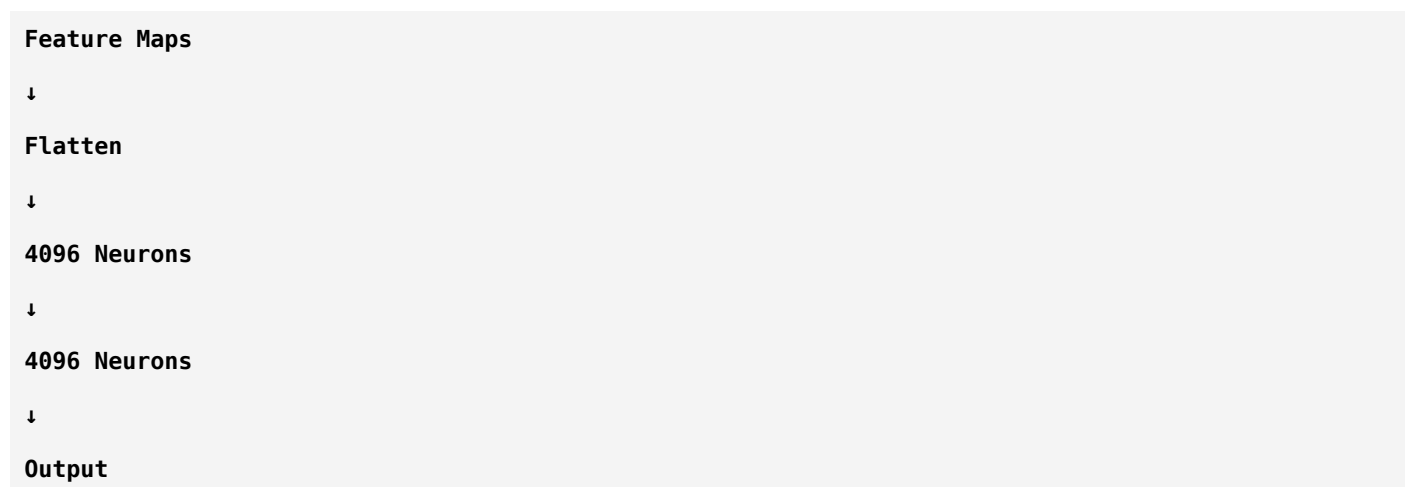
So they improve training but don't increase prediction complexity.

Global Average Pooling (GAP)

Another major innovation in GoogLeNet.

Before GoogLeNet, many CNNs ended with large Fully Connected (FC) layers.

Example:



These FC layers contained millions of parameters.

GoogLeNet's Idea

Instead of flattening everything,

it averages each feature map.

Example:

Suppose the last feature map is:

8 × 8

Global Average Pooling computes:

Average

↓

One Number

for each feature map.

If there are:

1024 Feature Maps

After GAP:

1024 Numbers

These are fed to the final classifier.

Why Is GAP Better?

Fewer Parameters

No huge fully connected layers.

Less Overfitting

Fewer trainable weights reduce the risk of memorizing the training data.

Better Generalization

The model focuses on **whether a feature exists**, not its exact location.

This is often useful because an object (or acoustic pattern) can appear in slightly different positions.

Sonar Example

Suppose your LOFARGRAM contains an engine harmonic.

Whether it appears slightly earlier or later in time, or at a slightly shifted frequency,

Global Average Pooling helps the model focus on the presence of the learned feature rather than its exact coordinates.

Why GoogLeNet Was Revolutionary

It introduced or popularized several ideas:

- Multi-scale feature extraction.
- Efficient use of 1×1 convolutions.
- Auxiliary classifiers.
- Global Average Pooling.

Many later architectures borrowed these concepts.

Is GoogLeNet Still Used Today?

Not as often as it once was.

Why?

Because newer architectures such as ResNet, EfficientNet, and Vision Transformers generally offer stronger performance or easier training.

However, GoogLeNet remains historically important because its ideas continue to influence modern CNN design.

Comparing What You've Learned

Architecture	Major Innovation
AlexNet	Made deep CNNs practical
VGG	Demonstrated the power of depth
GoogLeNet	Multi-scale feature extraction
ResNet	Residual learning and skip connections
DenseNet	Feature reuse through dense connectivity
EfficientNet	Balanced scaling of depth, width, and resolution

Notice how each architecture addressed a different challenge.

Would I Use GoogLeNet for Your Sonar Project?

Let's think like engineers.

Your task:

Passive Sonar Audio

↓

STFT

↓

LOFARGRAM

↓

Binary Classification

Could GoogLeNet work?

Yes.

The multi-scale filters can capture both:

- Narrow tonal harmonics.
- Broad machinery signatures.

However, if you're starting today, I would usually benchmark newer architectures first.

A practical comparison could include:

1. **ResNet-18** – Strong and simple baseline.
2. **DenseNet-121** – Excellent feature reuse.
3. **EfficientNet-B0** – Efficient and accurate.
4. **GoogLeNet** – Useful historical comparison with multi-scale processing.

This gives a well-rounded evaluation.

Timeline of CNN Evolution



You can see how ideas built on one another over time.

Summary

Concept	Key Idea
Inception Module	Multiple filter sizes operate in parallel
1×1 Convolution	Reduces channels and computation
Auxiliary Classifiers	Help training by improving gradient flow
Global Average Pooling	Replaces large fully connected layers
Main Strength	Multi-scale feature extraction

Concept	Key Idea
Modern Relevance	Influential ideas still used in many networks

You've Completed the Core CNN Journey

Congratulations! 🎉

You now understand the major classical CNN architectures that form the foundation of modern computer vision:

- ✓ AlexNet
- ✓ VGG
- ✓ GoogLeNet (Inception)
- ✓ ResNet
- ✓ DenseNet
- ✓ EfficientNet

This is the same progression many graduate students learn before moving into modern architectures.

Next Stage: MobileNet

Now we'll begin a different branch of CNN research.

The question becomes:

"What if we want to run a neural network on a mobile phone, drone, underwater robot, or embedded sonar device with very limited computing power?"

That question led to **MobileNet**.

For your project, MobileNet is especially relevant if you ever want your ship-classification system to run in **real time on edge hardware** instead of a powerful GPU.

After MobileNet, we'll move to **Vision Transformers (ViT)**, where you'll see how deep learning evolved beyond traditional convolutions.

ok next

Excellent. 🚀

You have now learned the **history of CNN evolution**.

Now we're entering a new family of networks:

Efficient CNNs

These networks weren't designed to get the absolute highest accuracy—they were designed to answer a practical question:

"Can a deep neural network run on a phone, drone, Raspberry Pi, underwater vehicle, or embedded sonar processor?"

That question led to **MobileNet**.

Deep CNN Architectures

Chapter 7: MobileNet (Part 1)

Why Was MobileNet Invented?

The Problem

Imagine you've trained an amazing CNN.

It achieves:

Accuracy = 98%

Fantastic!

But now you want to deploy it on an underwater autonomous vehicle (AUV).

The hardware has:

- Small CPU
- Limited RAM
- Limited battery
- No powerful GPU

Can it run ResNet-152?

Probably not.

Can it run EfficientNet-B7?

Also difficult.

Researchers realized:

A model is only useful if it can actually run where it's needed.

Real-Life Example

Imagine building an underwater surveillance system.

Pipeline:

Hydrophone

↓

Digital Signal Processing

↓

LOFARGRAM

↓

CNN

↓

Alarm

If the CNN needs:

- 8 GB GPU memory
- High-end graphics card
- Desktop computer

then it's impractical for many real-world deployments.

We need something lightweight.

Google's Goal

Google asked:

Can we build a CNN that is:

- Small
- Fast
- Accurate enough
- Battery friendly

This became **MobileNet**.

The Biggest Problem in CNNs

Where does most computation happen?

Answer:

Convolution layers.

Example:

Input

↓

Conv

↓

Conv

↓

Conv

↓

Conv

Every convolution performs millions (or billions) of multiply-add operations.

Researchers asked:

Can we make convolution itself more efficient?

That became MobileNet's key innovation.

Normal Convolution

Let's understand how it works.

Suppose your input is:

224 × 224 × 32

That means:

- Height = 224
- Width = 224
- Channels = 32

Suppose you want:

64 Output Channels

A standard convolution learns filters that operate across **all input channels** at once.

Each output feature combines:

- Spatial information
- Cross-channel information

in a single operation.

This is powerful, but computationally expensive.

Why Is It Expensive?

Imagine you're grading exams.

There are:

- 32 subjects

- 64 teachers

Every teacher checks **every subject**.

Huge workload.

That's similar to standard convolution.

MobileNet's Brilliant Idea

Instead of one large operation,

split it into **two simpler operations**.

This is called:

Depthwise Separable Convolution

This is the heart of MobileNet.

Definition

A **Depthwise Separable Convolution** separates:

1. Spatial filtering
2. Channel mixing

into two independent steps.

Instead of doing both together, MobileNet performs them one after another.

This dramatically reduces computation.

Step 1: Depthwise Convolution

Remember MBConv?

It also used this idea.

Depthwise convolution processes **each input channel independently**.

Suppose you have:

32 Channels

Instead of one filter spanning all channels,

apply one spatial filter **per channel**.

Channel 1 → 3×3 Filter

Channel 2 → 3×3 Filter

...

Channel 32 → 3×3 Filter

No channel mixing yet.

Only spatial feature extraction.

Real-Life Analogy

Imagine a hospital.

Instead of one doctor handling:

- Heart
- Brain
- Lungs
- Bones

You assign specialists.

Each doctor focuses on one area.

Depthwise convolution works the same way.

Each channel is processed independently.

Step 2: Pointwise Convolution

Now we have processed each channel separately.

But channels still need to communicate.

That's where the **Pointwise Convolution** comes in.

A **Pointwise Convolution** is simply:

1 × 1 Convolution

Its job is:

- Combine information from different channels.
- Produce new output channels.

Think of it as the stage where specialists meet to discuss the patient and produce a final diagnosis.

Complete Depthwise Separable Convolution

Input

↓

Depthwise Conv

```

↓
Pointwise (1×1) Conv
↓
Output

```

This replaces one expensive standard convolution.

Why Is This Faster?

A standard convolution performs spatial filtering and channel mixing together.

Depthwise separable convolution splits them into two simpler tasks.

This requires **far fewer mathematical operations**, which is why MobileNet is much faster and lighter.

The exact computational savings depend on the number of channels and filter sizes, but it's often a substantial reduction.

Sonar Example

Imagine your LOFARGRAM has multiple feature channels.

Depthwise convolution allows each channel to learn spatial patterns such as:

- Harmonic lines
- Broadband noise
- Machinery modulation

Then the pointwise convolution combines these patterns into richer acoustic representations.

MobileNet Pipeline

```

LOFARGRAM
↓
Depthwise Conv
↓
Pointwise Conv
↓
Depthwise Conv
↓
Pointwise Conv
↓
...
↓

```

Global Average Pooling

↓

Fully Connected Layer

↓

Ship Present / Ship Absent

Notice that MobileNet repeatedly uses **Depthwise + Pointwise** blocks instead of standard convolutions.

Why Researchers Loved MobileNet

MobileNet offered several practical advantages:

- Much lower computational cost.
- Smaller model size.
- Faster inference.
- Suitable for mobile and embedded devices.

The trade-off is that, for some tasks, a larger model like ResNet or EfficientNet may achieve higher accuracy.

Summary

Concept	Meaning
Standard Convolution	Spatial filtering and channel mixing together
Depthwise Convolution	Spatial filtering performed independently on each channel
Pointwise Convolution	1×1 convolution that mixes channel information
Depthwise Separable Convolution	Depthwise + Pointwise
Main Goal	Reduce computation while maintaining good accuracy

Think Like an Engineer

Suppose your ship detector must run:

- On an underwater robot.
- In real time.
- Using limited battery power.

Would you always choose the biggest CNN?

Or would you choose a model that is **fast enough, small enough, and accurate enough**?

That trade-off is exactly why MobileNet was created.

Homework

Without looking back, answer these:

1. Why was MobileNet developed?
2. What is the difference between **standard convolution** and **depthwise separable convolution**?
3. What is the role of the **depthwise convolution**?
4. What is the role of the **pointwise (1×1) convolution**?
5. Why is MobileNet useful for embedded sonar systems?

If you can answer those confidently, you've understood the core idea behind MobileNet.

Next Lesson (MobileNet – Part 2)

We'll dive deeper into MobileNet by exploring:

- **Width Multiplier (α)** — how MobileNet becomes smaller or larger.
- **Resolution Multiplier (ρ)** — how changing input size affects speed and accuracy.
- **MobileNet V1 vs V2 vs V3.**
- **Linear Bottlenecks** and **Inverted Residual Blocks** (introduced in MobileNetV2).
- Why EfficientNet borrowed ideas from MobileNetV2.

You'll discover that **MBConv**, which you learned in EfficientNet, actually evolved from MobileNetV2. That connection will tie together everything you've learned so far.

ok next

Excellent. 🔥

This lesson is one of the most important in your entire deep learning journey.

Why?

Because today you'll finally understand **where MBConv came from**.

Earlier, while learning EfficientNet, I told you:

"EfficientNet uses MBConv blocks borrowed from MobileNetV2."

Today we'll learn **why MobileNetV2 was invented, what problems MobileNetV1 had, and how MBConv was born**.

Deep CNN Architectures

Chapter 7: MobileNet (Part 2)

MobileNet V2, Inverted Residuals & Linear Bottlenecks

Quick Recap

MobileNetV1 introduced:

```
Standard Convolution
↓
Depthwise Convolution
↓
Pointwise Convolution
```

This dramatically reduced computation.

It was a huge success.

But researchers noticed another problem.

The Problem with MobileNetV1

Suppose your input feature map is:

```
224 × 224 × 32
```

After several layers, MobileNetV1 repeatedly compresses information.

Sometimes, important details are lost during this compression.

Think of compressing a high-quality image into a tiny JPEG.

You save storage, but some details disappear forever.

Researchers asked:

"Can we compress efficiently without losing important information?"

That question led to **MobileNetV2**.

Real-Life Analogy

Imagine you're moving to another city.

Your house contains:

- Books
- Laptop
- Clothes
- Certificates

- Family photos

Suppose your suitcase is very small.

If you immediately pack everything into that tiny suitcase, you'll probably leave behind some valuable items.

Instead, a smarter approach is:

1. Spread everything out.
2. Organize it.
3. Then pack only what's important.

That's exactly what MobileNetV2 does.

The MobileNetV2 Block

Instead of:

Depthwise

↓

Pointwise

MobileNetV2 uses:

Input

↓

1×1 Expansion

↓

Depthwise 3×3

↓

1×1 Projection

↓

Output

Does this look familiar?

It should.

This is **MBConv**.

EfficientNet adopted this same idea.

Why Expand First?

Suppose the input has:

32 Channels

Instead of processing only these 32 channels,
MobileNetV2 first expands them.

Example:

32

↓

192

Students often ask:

"Why make the network larger before making it smaller again?"

Because the expanded space allows the network to learn richer feature combinations before compressing them.

Think of it as brainstorming many ideas before selecting the best ones.

Stage 1: Expansion Layer

Input:

32 Channels

Expansion:

192 Channels

This is done using:

1×1 Convolution

The purpose is to increase the feature representation.

Stage 2: Depthwise Convolution

Now each of the expanded channels is processed independently.

192 Channels

↓

Depthwise Conv

Each channel learns spatial information such as:

- Edges
- Harmonics

- Local frequency patterns

Stage 3: Projection Layer

Finally,

the network compresses:

192
↓
32

using another:

1×1 Convolution

This creates a compact output representation.

Why Is It Called an Inverted Residual?

Let's compare with ResNet.

ResNet Bottleneck

256
↓
64
↓
256

Wide → Narrow → Wide

MobileNetV2

32
↓
192
↓
32

Narrow → Wide → Narrow

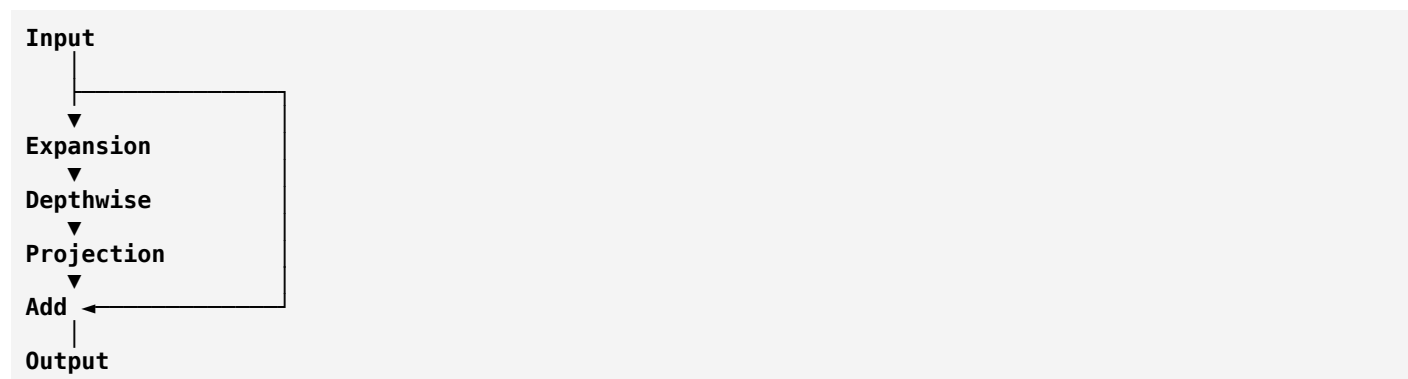
This is the opposite pattern.

That's why it's called an **Inverted Residual Block**.

Why "Residual"?

Remember ResNet?

If the input and output have the same dimensions,
a shortcut connection is added.



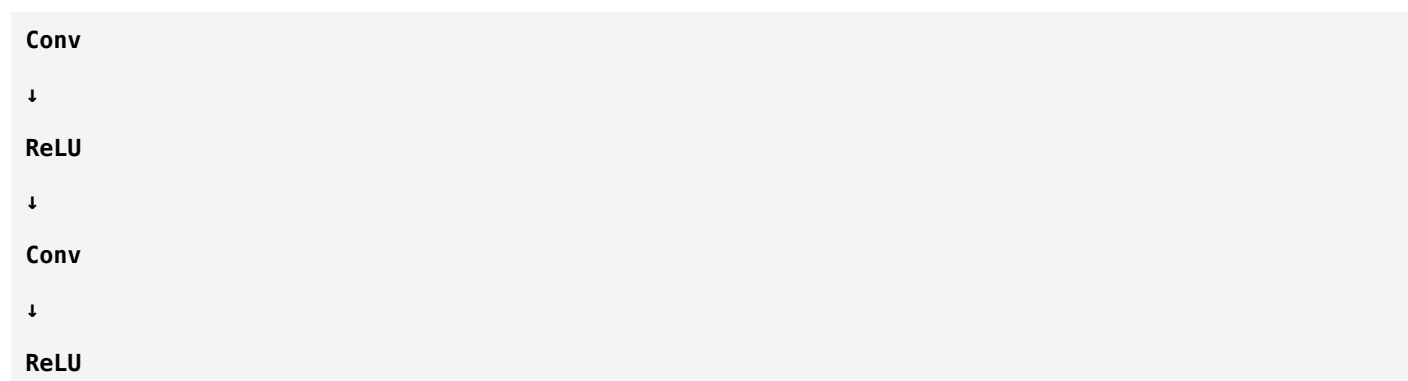
This shortcut helps preserve information and improves gradient flow.

Linear Bottleneck

Now we reach another important concept.

Normally, CNNs apply an activation function like ReLU after almost every convolution.

Example:



But MobileNetV2 does something different.

After the **final projection layer**, it **does not use ReLU**.

Instead, it keeps the output **linear**.

Why Remove ReLU?

Imagine your feature values are:

```
-2
-1
0
1
2
```

ReLU changes them to:

```
0
0
0
1
2
```

Negative values disappear.

If the feature space has already been compressed into a low-dimensional bottleneck, applying ReLU may discard useful information.

By keeping the final projection linear, MobileNetV2 preserves more information.

This is called the **Linear Bottleneck**.

Why Is This Important?

Imagine your sonar feature map contains:

- Weak engine harmonics
- Low-energy tonal components

Some of these features might be represented by negative activations before the final projection.

A ReLU could remove them.

A linear projection preserves them.

This can help the model retain subtle acoustic information.

MobileNetV1 vs MobileNetV2

Feature	MobileNetV1	MobileNetV2
Core Block	Depthwise + Pointwise	Inverted Residual (MBConv)
Expansion Layer	✗	✓
Linear Bottleneck	✗	✓
Skip Connection	Limited	Yes (when dimensions match)
Feature Representation	Simpler	Richer

How EfficientNet Uses This

Now everything connects.

EfficientNet's basic block is:

```

Input
↓
Expansion
↓
Depthwise
↓
Projection
↓
Skip Connection

```

That is essentially the MobileNetV2 block, enhanced and combined with EfficientNet's compound scaling strategy.

This is why understanding MobileNetV2 was essential before fully appreciating EfficientNet.

Sonar Pipeline

Imagine your complete system:

```

Passive Sonar
↓
STFT
↓
LOFARGRAM
↓
MobileNetV2
↓
MBConv Blocks
↓
Global Average Pooling
↓
Fully Connected Layer
↓
Ship Present / Ship Absent

```

Even on resource-constrained hardware, this pipeline can be practical because MobileNetV2 is designed for efficiency.

Why Researchers Loved MobileNetV2

It provided:

- Better feature representation than V1.
- Efficient computation.
- Improved accuracy.
- A design that later inspired EfficientNet and other lightweight models.

Summary

Concept	Key Idea
Expansion Layer	Increase channels to learn richer features
Depthwise Convolution	Learn spatial patterns efficiently
Projection Layer	Compress features into a compact representation
Inverted Residual	Narrow → Wide → Narrow
Linear Bottleneck	Final projection remains linear to preserve information
MBConv	The block that later became the foundation of EfficientNet

Think Like a Researcher

Suppose you're designing a lightweight ship detector for an autonomous underwater vehicle.

Would you:

- Compress the features immediately?

Or

- First expand the feature space, learn richer representations, and then compress intelligently?

That second approach is exactly why MobileNetV2 performs better than MobileNetV1.

Homework

Without looking back, explain:

1. Why was MobileNetV2 created?
2. What is an **Inverted Residual Block**?
3. Why does the block expand channels before compressing them?
4. What is a **Linear Bottleneck**, and why does it avoid ReLU at the end?
5. How did MobileNetV2 influence EfficientNet?

If you can answer those clearly, you've mastered the core ideas of MobileNetV2.

Next Lesson (MobileNet – Part 3, Final)

We'll finish MobileNet by covering:

- **Width Multiplier (α)** — how to make MobileNet smaller or larger.
- **Resolution Multiplier (ρ)** — how input size affects speed and accuracy.
- **MobileNetV3** and how it improves on V2.
- **Squeeze-and-Excitation (SE) Blocks** — a mechanism that lets the network learn which feature channels are most important.
- Which MobileNet version is best for embedded AI and whether MobileNet is a good choice for your passive sonar project.

That final lesson will complete the MobileNet family and prepare you for the biggest shift in computer vision: **Vision Transformers (ViT)**.

ok next

Excellent! 🎉

This is the **final lesson on MobileNet**.

After today, you'll understand **the complete MobileNet family (V1, V2, and V3)** and you'll also learn **Squeeze-and-Excitation (SE) Blocks**, which are used in many modern CNNs—not just MobileNet.

This is one of those concepts you'll keep seeing in research papers, including papers on **acoustic signal classification**.

Deep CNN Architectures

Chapter 7: MobileNet (Part 3 – Final)

MobileNetV3, Width Multiplier, Resolution Multiplier & SE Blocks

Before MobileNetV3...

Researchers already had:

- MobileNetV1 ✓
- MobileNetV2 ✓

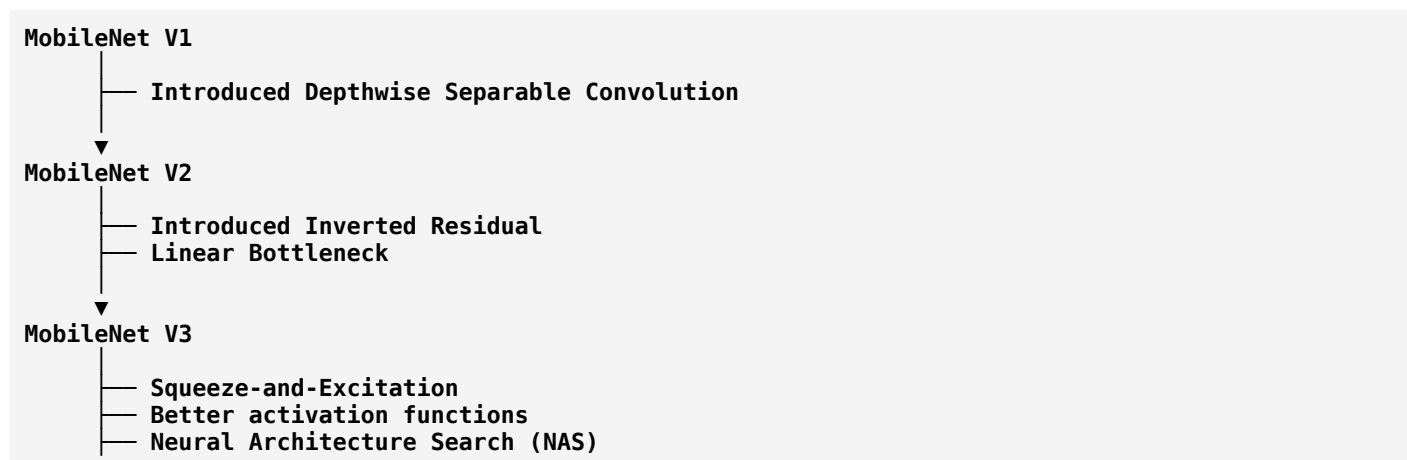
Both were efficient.

But Google asked another question:

"Can we make MobileNet even faster while maintaining similar accuracy?"

This led to **MobileNetV3**.

MobileNet Evolution



Notice that V3 didn't replace everything.

It **improved** V2.

Width Multiplier (α)

This is one of the easiest but most useful ideas.

Suppose a layer normally has:

128 Filters

Now choose:

$\alpha = 0.5$

The layer now has:

64 Filters

If:

$\alpha = 0.25$

Then:

32 Filters

If:

$\alpha = 1.0$

Original network.

What Does This Mean?

The Width Multiplier controls:

How wide the network is.

Smaller α :

- Fewer filters
- Faster
- Less memory
- Usually a small drop in accuracy

Larger α :

- More filters
- More computation
- Often better accuracy

Real-Life Analogy

Imagine a company.

Original staff:

100 Employees

Half-width company:

50 Employees

The company works faster and costs less.

But it may not handle as many tasks.

That's exactly what the Width Multiplier does.

Resolution Multiplier (ρ)

Another clever idea.

Suppose the input image is:

224 × 224

Choose:

$\rho = 0.5$

Input becomes approximately:

112 × 112

Less computation.

Less memory.

Faster inference.

Sonar Example

Instead of using:

LOFARGRAM

224 × 224

You might use:

160 × 160

or

128 × 128

Training becomes faster.

The trade-off is that some fine acoustic details may be lost.

Width vs Resolution

Students often confuse these.

Width Multiplier	Resolution Multiplier
Changes number of filters	Changes input image size
Controls model capacity	Controls input detail
Affects channels	Affects height and width

Think of them as controlling **different dimensions** of the network.

MobileNetV3

Now let's discuss the major innovations.

Google combined:

- MobileNetV2
- Neural Architecture Search (NAS)

- Squeeze-and-Excitation (SE)
- Improved activation functions

The result was MobileNetV3.

What is Neural Architecture Search (NAS)?

Before MobileNetV3, engineers manually designed architectures.

For example:

```
Conv
↓
Conv
↓
Pooling
↓
Conv
```

But Google asked:

"Can a computer search for a good architecture automatically?"

NAS explores many candidate architectures and selects those that perform well under constraints such as accuracy and computational cost.

Instead of relying only on human intuition, part of the design process becomes automated.

Squeeze-and-Excitation (SE) Blocks

This is one of the most important ideas in modern CNNs.

Imagine you're reading a newspaper.

Some articles are important.

Others aren't.

Do you spend equal attention on every article?

No.

You naturally focus on the important ones.

CNNs originally treated all feature channels similarly.

Researchers asked:

"Can the network learn which channels are most important?"

That question led to **Squeeze-and-Excitation**.

The Problem

Suppose a CNN produces:

Feature Maps

Channel 1

Channel 2

Channel 3

Channel 4

...

Channel 128

Are all channels equally useful?

Probably not.

Some channels may detect highly informative ship harmonics.

Others may mostly respond to background noise.

SE Block Idea

Instead of treating all channels equally,

the SE block learns **attention weights** for each channel.

Feature Maps

↓

Analyze Importance

↓

Assign Weights

↓

Emphasize Important Channels

↓

Suppress Less Useful Channels

The network decides what deserves more attention.

Real-Life Analogy

Imagine an orchestra.

There are:

- Violin

- Piano
- Drums
- Flute

During one part of the music,
the conductor increases the violin volume.

Later,
the drums become more important.

The conductor adjusts each instrument based on the music.

The SE block acts like that conductor—but for feature channels.

Why is it Called "Squeeze" and "Excitation"?

Squeeze

The network summarizes each feature channel into a single representative value.

This is usually done using **Global Average Pooling**.

Think of it as asking:

"Overall, how active is this channel?"

Excitation

A small neural network processes these summaries and produces a weight for each channel.

These weights tell the network:

- Increase this channel.
- Reduce that channel.

The feature maps are then multiplied by these learned weights.

Sonar Example

Imagine your LOFARGRAM generates channels that detect:

Channel	Detects
1	Engine harmonics
2	Propeller modulation
3	Ocean background noise
4	Electrical interference

Suppose you're classifying ships.

The SE block might learn:

Engine Harmonics → High Weight
 Propeller Modulation → High Weight
 Background Noise → Lower Weight
 Interference → Lower Weight

The network automatically emphasizes the most informative acoustic features.

MobileNetV3 Pipeline

LOFARGRAM
 ↓
 MBConv
 ↓
 SE Block
 ↓
 MBConv
 ↓
 SE Block
 ↓
 Global Average Pooling
 ↓
 Fully Connected Layer
 ↓
 Ship Present / Ship Absent

The SE blocks help the network focus on the most useful channels throughout the model.

MobileNetV1 vs V2 vs V3

Feature	V1	V2	V3
Depthwise Separable Conv	✓	✓	✓
Inverted Residual	✗	✓	✓
Linear Bottleneck	✗	✓	✓
SE Blocks	✗	✗	✓ (selected blocks)
NAS-based Design	✗	✗	✓

Would I Use MobileNet for Your Sonar Project?

It depends on the goal.

If your goal is:

Highest possible research accuracy

I would usually start with:

- ResNet-18
- DenseNet-121
- EfficientNet-B0

If your goal is:

Real-time deployment on an embedded system

Examples:

- Autonomous Underwater Vehicle (AUV)
- Raspberry Pi
- NVIDIA Jetson
- Edge AI device

Then MobileNetV3 becomes a very attractive option because of its efficiency.

Summary

Concept	Key Idea
Width Multiplier (α)	Controls the number of filters
Resolution Multiplier (ρ)	Controls input image size
MobileNetV3	Improved MobileNetV2 using NAS and SE Blocks
SE Block	Learns which feature channels are most important
Main Goal	Maximum efficiency for edge devices

Think Like a Researcher

Imagine you're building a sonar system for a battery-powered underwater drone.

Would you deploy:

- A very large, highly accurate model that drains the battery quickly?

Or

- A lightweight model that is slightly less accurate but can operate in real time for hours?

The answer depends on the deployment requirements.

That's the engineering trade-off MobileNet was designed to address.

Congratulations!

You have now completed the **major CNN architecture families**:

-  AlexNet
-  VGG
-  GoogLeNet (Inception)
-  ResNet
-  DenseNet
-  EfficientNet
-  MobileNet

You now understand nearly all of the foundational CNN architectures that appear in computer vision and many sonar-classification research papers.

The Next Revolution: Vision Transformers (ViT)

Everything you've learned so far shares one core idea:

Use convolutions to extract features.

Then, around 2020, researchers asked a radical question:

"Do we need convolutions at all?"

That question led to the **Vision Transformer (ViT)**.

Unlike CNNs, ViT treats an image more like a sentence in natural language processing and uses **self-attention** to understand relationships between different parts of the image.

For your passive sonar project, understanding ViT is valuable because many recent research papers compare **CNN-based models (ResNet, EfficientNet, DenseNet)** against **transformer-based models (ViT, Swin Transformer)** to evaluate whether attention mechanisms can better capture complex acoustic patterns.

From this point onward, you'll be learning the architectures that define the current state of deep learning research.

ok next

Excellent! 🚀

Now we are entering the **third generation of computer vision models**.

Until now, every architecture you've learned—AlexNet, VGG, ResNet, DenseNet, EfficientNet, MobileNet—had one thing in common:

They all relied on convolutions.

Today, we'll study a model that challenged that assumption.

When the **Vision Transformer (ViT)** was introduced in 2020 by researchers at Google Research, many people thought:

"A model without convolutions can never compete with CNNs."

They were wrong.

Deep CNN Architectures

Chapter 8: Vision Transformer (ViT) – Part 1

Why Was the Vision Transformer Invented?

First, Let's Compare CNNs and Humans

Imagine you're looking at this picture.

 Dog sitting in a park

How do **CNNs** recognize it?

They usually process it **locally**.

First they detect:

- Edges
- Corners
- Curves

Then:

- Eyes
- Nose
- Ears

Finally:

- Dog

Information gradually grows from small features to large features.

CNN Thinking

Pixels

↓

Edges

↓

Textures

↓

Parts

↓

Object

This works extremely well.

But researchers noticed something interesting.

Imagine This Picture



Tree

The elephant is on the left.

The tree is on the far right.

These two objects are very far apart.

Question:

Can a normal convolution immediately understand their relationship?

Not really.

Why?

Remember the **receptive field**?

A 3×3 convolution only sees a tiny neighborhood.

□□□

□□□

□□□

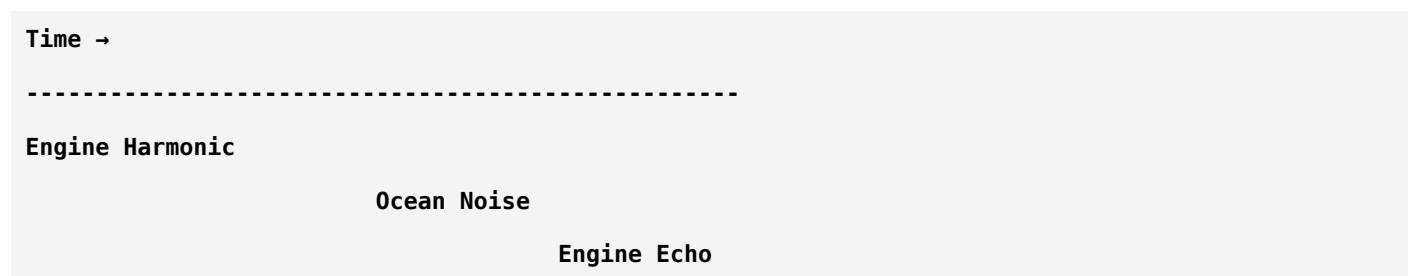
To connect two distant regions,

the network needs many convolution layers.

Information travels gradually.

Sonar Example

Imagine your LOFARGRAM.



Suppose:

The first harmonic appears at:

Time = 2 sec

Another important harmonic appears at:

Time = 12 sec

They are very far apart.

A CNN learns this relationship gradually across many layers.

Researchers asked:

"Can we directly connect these distant regions?"

Inspiration from NLP

Around 2017,

the Transformer architecture revolutionized Natural Language Processing.

Instead of reading words one after another,

it used:

Self-Attention

Every word could directly "look at" every other word.

Example:

The ship crossed the ocean because it was fast.

What does:

it

refer to?

Transformer immediately connects:

```
it
↓
ship
```

even though several words separate them.

Researchers wondered:

"Can we use the same idea for images?"

The Big Question

Instead of treating an image as one large grid,
what if we divide it into small pieces?

Example:

```
Image
↓
Split
↓
Small Patches
```

This became the foundation of the Vision Transformer.

Image as Patches

Suppose the input image is:

```
224 × 224
```

Instead of feeding the whole image at once,

ViT divides it into patches.

For example:

```
16 × 16
```

Each patch becomes like a "word" in a sentence.

Example

Imagine a small image.

```

[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]

```

Divide it into patches.

```

┌──┐
| P1 | P2 |
└──┘
┌──┐
| P3 | P4 |
└──┘

```

Each patch is treated as one input token.

Why Patches?

Think about reading a book.

You don't process the whole page simultaneously.

You read:

- Word
- Word
- Word

Transformers process text using tokens.

Similarly,

ViT processes images using **patch tokens**.

CNN vs ViT

CNN:

```

Image
↓
Convolution
↓
Feature Maps

```

ViT:

Image

↓

Patches

↓

Tokens

↓

Transformer

A completely different philosophy.

What Happens Next?

Each image patch is converted into a numerical representation called an **embedding**.

So the pipeline becomes:

Image

↓

Split into Patches

↓

Patch Embeddings

↓

Transformer Encoder

↓

Classifier

We'll study each of these steps in depth over the next lessons.

Why Is Self-Attention Powerful?

Imagine this LOFARGRAM.

Time →

Weak Harmonic

Noise

Strong Harmonic

A CNN might need several layers before the weak and strong harmonics influence one another.

A Vision Transformer allows every patch to directly interact with every other patch through self-attention.

This makes it easier to model **long-range relationships**.

Real-Life Analogy

Imagine a classroom.

CNN

Students whisper to their neighbors.

The message travels:

A → B → C → D → E

It takes time to reach the last student.

Transformer

Every student has a microphone.

Any student can immediately talk to any other student.

A ↔ B

A ↔ C

A ↔ D

A ↔ E

...

That's the power of self-attention.

Does This Mean CNNs Are Obsolete?

No.

CNNs are still excellent, especially when:

- Training data is limited.
- Efficiency is important.
- Edge deployment is required.

Transformers often shine when:

- Large datasets are available.
- Long-range relationships are important.
- High computational resources are available.

Many modern systems even combine CNNs and Transformers.

Sonar Perspective

Suppose your LOFARGRAM contains:

- Engine harmonics
- Propeller signatures
- Background noise
- Echoes

A CNN focuses on local structures.

A Vision Transformer can directly compare distant acoustic patterns.

Whether that improves performance depends on the dataset and task, which is why recent sonar papers often compare both approaches.

Summary

CNN	Vision Transformer
Uses convolutions	Uses self-attention
Learns local features first	Can model global relationships from the start
Strong inductive bias for images	Learns relationships more flexibly
Excellent with moderate datasets	Often benefits from larger datasets

Think Like a Researcher

Imagine you're analyzing a LOFARGRAM.

Two important harmonic patterns occur **10 seconds apart**.

Would you prefer a model that gradually connects them through many convolution layers?

Or one that can directly compare those distant regions using attention?

That question motivated the development of the Vision Transformer.

Homework

Without looking back, answer these:

1. Why was the Vision Transformer invented?
2. Why do CNNs sometimes struggle with long-range relationships?
3. What is an image **patch**?
4. Why is each patch treated like a word in NLP?

5. How is a Vision Transformer fundamentally different from a CNN?

If you can answer these questions, you've understood **why ViT exists**.

Next Lesson (ViT – Part 2)

Now we'll dive into the internal mechanics of Vision Transformers:

- What is a **Patch Embedding**?
- What is a **CLS (Classification) Token**?
- What are **Positional Embeddings**, and why are they necessary?
- How does a Transformer understand the order and location of image patches?

These concepts are the foundation of every Vision Transformer you'll encounter in modern research.

ok next

Excellent! 🌟

Today we are going to learn the **foundation of every Vision Transformer**.

If Part 1 answered "**Why was ViT invented?**", today's lesson answers:

"How does a Transformer actually read an image?"

This lesson is extremely important because the ideas you'll learn here are also used in:

- Vision Transformer (ViT)
- DeiT
- BEiT
- DINO
- MAE
- Swin Transformer (partially)
- Many recent sonar and medical imaging research papers

Deep CNN Architectures

Chapter 8: Vision Transformer (ViT) – Part 2

Patch Embeddings, CLS Token & Positional Encoding

Step 1: Input Image

Suppose we have an image.

224 × 224 × 3

Height = 224

Width = 224

Channels = RGB

For your sonar project, it would be:

224 × 224 × 1

because a LOFARGRAM is typically treated as a grayscale image.

Step 2: Divide into Patches

Instead of processing the whole image,

ViT cuts it into small squares.

Example:

Patch size

16 × 16

The image becomes

224/16 = 14 patches

along each dimension.

Total patches:

14 × 14 = 196 patches

Visualization

```
+-----+-----+-----+-----+
| P1 | P2 | P3 | P4 |
+-----+-----+-----+-----+
| P5 | P6 | P7 | P8 |
+-----+-----+-----+-----+
| ... | ... | ... | ... |
+-----+-----+-----+-----+
```

Instead of **one image**, the Transformer now sees **196 small images**.

Why Split into Patches?

Imagine reading a newspaper.

You don't try to understand the whole page in one glance.

You naturally divide it into:

- Headlines
- Paragraphs
- Images

Similarly,

ViT divides an image into manageable pieces.

Step 3: Flatten Each Patch

Each patch still contains pixels.

Example:

Patch:

```
16 × 16 × 3
```

Flatten it into a vector.

```
768 Numbers
```

because:

```
16 × 16 × 3 = 768
```

Each patch becomes a long list of numbers.

For grayscale sonar:

```
16 × 16 × 1 = 256
```

Why Flatten?

Transformers work with vectors, not image grids.

So every patch must become a vector before entering the Transformer.

Step 4: Patch Embedding

Now comes an important concept.

A flattened patch is **not yet suitable** for the Transformer.

We pass it through a **Linear Layer**.

```
768
```

```
↓
```

```
768
```

or

768

↓

512

or

768

↓

1024

depending on the model.

This output is called a:

Patch Embedding

What is an Embedding?

An embedding is simply:

A learned numerical representation of data.

You already know embeddings from NLP.

Example:

Word

Ship

↓

Embedding

[0.24, -1.12, 0.53, ...]

Similarly,

Image Patch

↓

Embedding

[1.22, 0.63, -0.41, ...]

The Transformer processes these embeddings instead of raw pixels.

Sonar Example

Suppose one patch contains:

- Engine harmonic
- Weak background noise

After embedding,

the vector may represent these characteristics in a way that makes them easier for the model to learn.

Another patch containing only background noise will produce a different embedding.

Step 5: CLS Token

This is another major idea.

Before the first patch,

ViT inserts a **special token**.

```
CLS
P1
P2
P3
...
P196
```

What is the CLS Token?

Think of it as:

The class representative of the entire image.

During training,

the CLS token gathers information from every patch through self-attention.

At the end,

instead of using all patch outputs,

the model uses only the CLS token to make the final prediction.

Classroom Analogy

Imagine:

196 students

One class monitor.

Every student shares information with the monitor.

At the end,

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

the monitor summarizes everything for the teacher.

The class monitor is the **CLS Token**.

Sonar Analogy

Imagine:

196 sonar patches.

Each patch contains part of the acoustic scene.

The CLS token collects information from all of them.

Finally it decides:

Ship Present

or

Ship Absent

Step 6: The Big Problem

Suppose I shuffle the patches.

Original:

P1 P2 P3 P4

Shuffled:

P3 P1 P4 P2

The Transformer only sees vectors.

How would it know where each patch originally came from?

It wouldn't.

Why is This a Problem?

Imagine reading:

Ship enters harbor safely.

Now shuffle the words.

Safely harbor enters ship.

The words are the same.

The meaning changes.

Images have the same issue.

Location matters.

Solution: Positional Encoding

Researchers add another vector to every patch.

```
Patch Embedding
+
Position Embedding
↓
Final Input
```

Now the model knows:

- What the patch contains.
- Where it came from.

Example

Patch 1

```
Engine Harmonic
```

Position

```
Top Left
```

Patch 150

```
Background Noise
```

Position

```
Bottom Right
```

Even if the contents are similar,
their positions are different.

Final Input Sequence

Before entering the Transformer,
the input looks like:

```
CLS + Pos
↓
```



```

Patch1 + Pos
↓
Patch2 + Pos
↓
...
↓
Patch196 + Pos

```

Only now is the sequence ready for the Transformer Encoder.

Complete Pipeline So Far

```

Input Image
↓
Split into Patches
↓
Flatten Patches
↓
Linear Projection
↓
Patch Embeddings
↓
Add CLS Token
↓
Add Positional Embeddings
↓
Transformer Encoder

```

Notice that **we still haven't discussed self-attention**.

That's because everything you've learned today is simply **preparing the input**.

The real magic begins inside the Transformer Encoder.

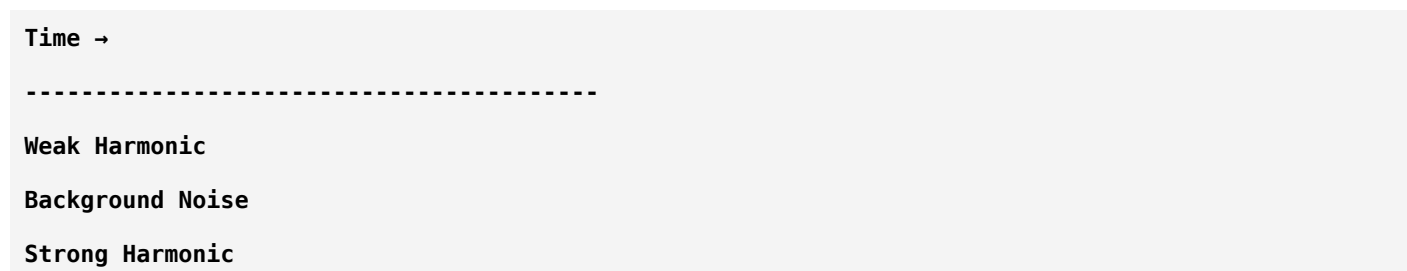
CNN vs ViT So Far

CNN	Vision Transformer
Receives pixels directly	Receives patch embeddings
Uses convolutions immediately	Uses linear projection first
Learns spatial structure through kernels	Uses positional embeddings

CNN	Vision Transformer
Produces feature maps	Produces token embeddings

Why This Matters for Your Sonar Project

Imagine your LOFARGRAM.



When divided into patches:

- Some patches contain harmonics.
- Some contain only noise.
- Some contain echoes.

Each patch becomes an embedding.

The Transformer then learns how these patches relate to one another across the entire spectrogram.

Summary

Concept	Meaning
Patch	Small square cut from the image
Flattening	Convert patch into a vector
Patch Embedding	Learned representation of the patch
CLS Token	Represents the entire image for classification
Positional Encoding	Tells the Transformer where each patch belongs

Think Like a Researcher

Suppose you removed positional embeddings.

Would the Transformer know whether an engine harmonic occurred at the beginning or end of the LOFARGRAM?

No.

It would only know **what** the patch contains—not **where** it came from.

That's why positional information is essential.

Homework

Without looking back, answer these:

1. Why is an image divided into patches?
2. What is a **Patch Embedding**?
3. Why do we flatten patches?
4. What is the role of the **CLS Token**?
5. Why are **Positional Embeddings** necessary?

If you can explain these in your own words, you've understood how a Vision Transformer prepares image data.

Next Lesson (ViT – Part 3)

Now we reach the **heart of the Transformer**:

Self-Attention

We'll learn:

- What is **Attention**?
- What are **Query (Q), Key (K), and Value (V)**?
- How does one image patch decide which other patches are important?
- What is the **attention matrix**?
- Why is self-attention considered one of the biggest breakthroughs in AI?

This is the concept that powers not only Vision Transformers, but also ChatGPT and most modern large language models. Once you understand self-attention, you'll understand the core mechanism behind much of today's AI.

ok next

Excellent. 🌟

Today we're learning **the single most important concept in modern AI**.

Not just for Vision Transformers.

Not just for computer vision.

This is also the core idea behind **ChatGPT, GPT-4/5, BERT, Gemini, Claude, Llama**, and almost every modern Transformer model.

If you understand **Self-Attention**, you've understood one of the biggest breakthroughs in AI.

This lesson is longer than the previous ones because this concept deserves it.

Deep CNN Architectures

Chapter 8: Vision Transformer (ViT) – Part 3

Self-Attention — The Heart of Every Transformer

First, Forget Images

We'll start with a simple example.

Imagine you're reading this sentence:

"The ship crossed the ocean because it was fast."

Question:

What does **"it"** refer to?

- Ocean?
- Ship?

Humans instantly know:

"It" → Ship

Why?

Because our brain connects related words, even if they are far apart.

Transformers try to do the same thing.

Attention Means "Focus"

Suppose you're in a classroom with 50 students.

The teacher asks:

"Who submitted the assignment?"

Do you look equally at every student?

No.

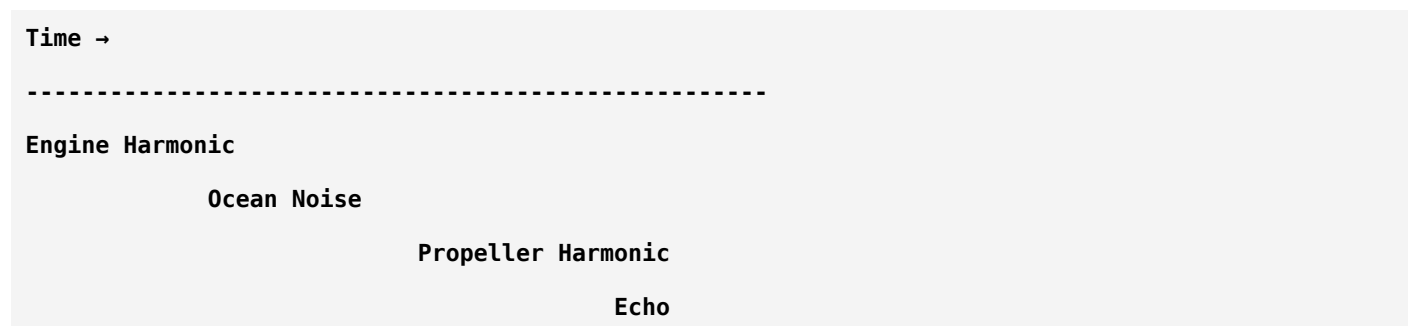
You immediately focus on the students who are likely to answer.

This is **attention**.

Attention = Giving more importance to relevant information and less importance to irrelevant information.

Now Apply This to Images

Imagine this LOFARGRAM.



Question:

If we're trying to detect a ship,
should the model give equal importance to:

- Ocean noise?
- Engine harmonic?
- Propeller harmonic?
- Echo?

No.

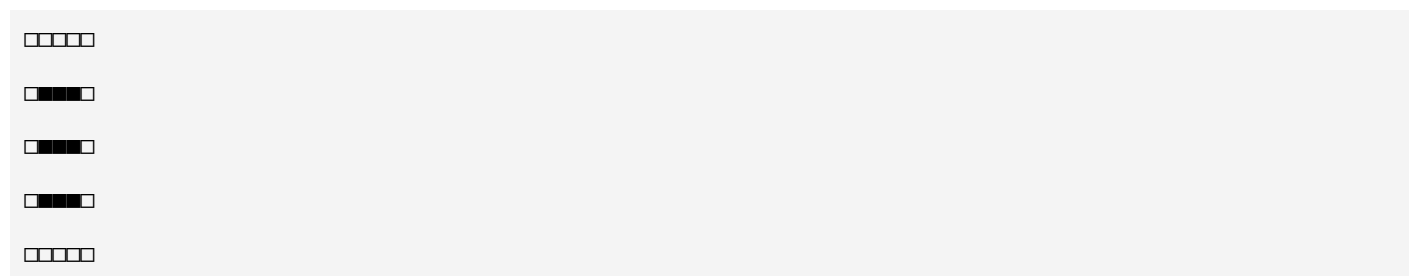
Some regions are much more informative.

The Transformer learns **where to pay attention**.

CNN vs Self-Attention

CNN

Looks locally.



A convolution sees only nearby pixels.

Self-Attention

Every patch can directly communicate with every other patch.

Patch 1 ↔ Patch 2
Patch 1 ↔ Patch 50
Patch 1 ↔ Patch 196

Distance doesn't matter.

This is a huge difference.

The Classroom Analogy

Imagine 5 students.

A
B
C
D
E

Traditional communication:

A → B → C → D → E

Information travels slowly.

Self-Attention:

A ↔ B
A ↔ C
A ↔ D
A ↔ E
B ↔ D
C ↔ E

Everyone can communicate directly.

That's exactly what happens inside a Transformer.

But...

How does a patch decide **who is important**?

This is where three famous concepts appear.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Query (Q)

Key (K)

Value (V)

These confuse almost every beginner.

Let's make them simple.

Imagine a Library

You walk into a library.

You ask:

"I need books about underwater sonar."

What happens?

You have:

Your request.

This is the:

Query

The library books have labels.

Those labels are:

Keys

The actual books contain information.

Those are:

Values

The librarian compares:

Your Query

with

Every Key.

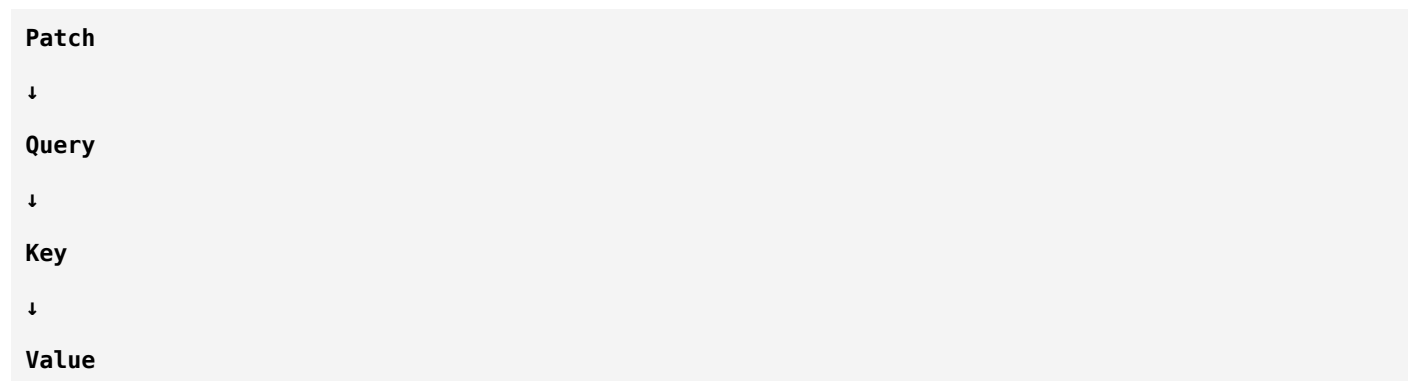
The books with the best match are selected.

Then you read their Values.

That's exactly how attention works.

In AI Terms

Every image patch creates three vectors.



Every single patch has all three.

Think of Them Like This

Query

"What am I looking for?"

Key

"What information do I contain?"

Value

"What information should I provide if someone needs me?"

Sonar Example

Suppose we have three patches.

Patch 1

Contains:

Engine harmonic.

Patch 2

Contains:

Ocean noise.

Patch 3

Contains:

Propeller harmonic.

Now imagine Patch 1 asks:

"Who else looks similar to me?"

That's its **Query**.

Patch 2 replies:

"I'm mostly background noise."

Low similarity.

Patch 3 replies:

"I'm another harmonic."

High similarity.

The Transformer therefore gives more attention to Patch 3 than Patch 2.

Step-by-Step Self-Attention

Let's simplify the process.

Suppose we have:

P1

P2

P3

Each produces:

Query

Key

Value

So internally we have:

P1 → Q1 K1 V1

P2 → Q2 K2 V2

P3 → Q3 K3 V3

Step 1

Take one Query.

Example:

Q1

Step 2

Compare it with every Key.

Q1 vs K1

Q1 vs K2

Q1 vs K3

The model computes a **similarity score**.

Higher score = More relevant.

Example Scores

Q1 vs K1 = 9

Q1 vs K2 = 2

Q1 vs K3 = 8

Meaning:

Patch 1 should mostly pay attention to:

- Itself
- Patch 3

Very little attention to Patch 2.

Step 3

Convert Scores into Probabilities

We don't use raw scores.

We normalize them using **Softmax**.

Suppose:

9

2

8

becomes:

0.48

0.04

0.48

Now the weights sum to 1.

These are called **Attention Weights**.

Step 4

Use the Values

Now the output for Patch 1 is computed by combining the Value vectors using these attention weights.

Conceptually:

Output₁

=

0.48 × V1

+

0.04 × V2

+

0.48 × V3

Notice something.

Patch 2 contributes very little.

Patch 3 contributes almost as much as Patch 1.

This is how the model learns to focus on relevant information.

Visualizing Attention

Imagine the attention map.

P1

↓

Looks mostly at

P1

P3

↓

Ignores

P2

That's exactly what attention means.

Now Imagine 196 Patches

Instead of:

P1

P2

P3

we have:

P1

P2

...

P196

Every patch compares itself with every other patch.

That creates an **Attention Matrix**.

Attention Matrix

Imagine a table.

	P1	P2	P3	...
P1	0.48	0.04	0.48	...
P2	...	...	...	...
P3	...	...	...	...

Each row shows:

How much one patch attends to every other patch.

This matrix is one of the most important computations in a Transformer.

Sonar Example

Imagine a LOFARGRAM.

Weak Harmonic

Noise

Strong Harmonic

Echo

Attention may learn:

- Strong harmonic ↔ Weak harmonic
- Engine harmonic ↔ Propeller harmonic
- Ignore random ocean noise

The model automatically discovers these relationships.

Why Is This Better Than CNN?

CNN:

Patch

↓

Nearby neighbors only

Transformer:

Patch

↓

Every other patch

The Transformer can model long-range dependencies in a single layer.

Does Attention Replace Convolution Completely?

Not always.

CNNs still have advantages:

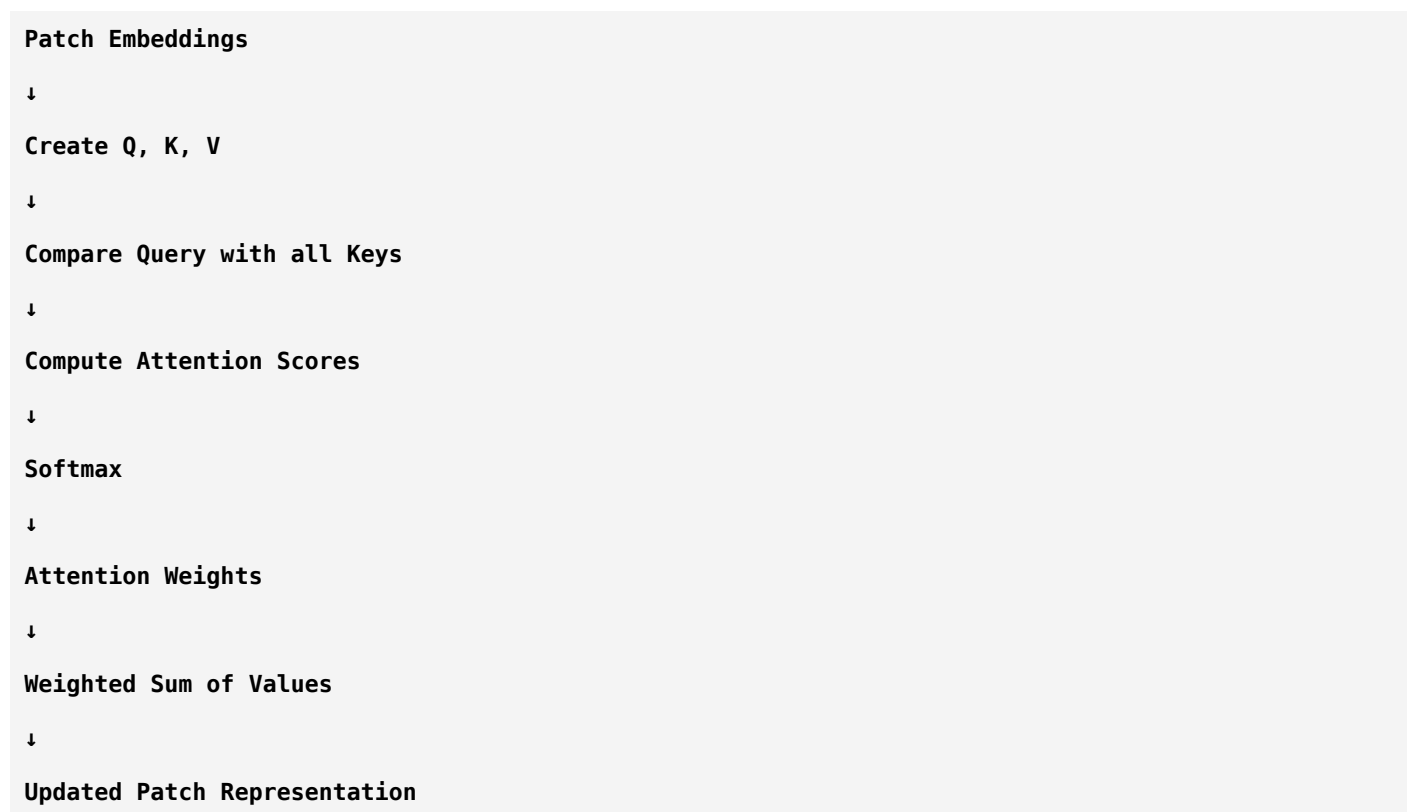
- Strong inductive bias for images.
- Excellent performance on smaller datasets.
- Lower computational cost.

Transformers often perform best with:

- Large datasets.
- Powerful hardware.
- Tasks where global relationships matter.

This is why hybrid CNN-Transformer models are also common.

Complete Self-Attention Pipeline



This happens for every patch in every Transformer layer.

Summary

Concept	Meaning
Attention	Focus on the most relevant information
Query (Q)	What this patch is looking for
Key (K)	What this patch contains
Value (V)	The information this patch provides
Attention Score	Similarity between Query and Key
Softmax	Converts scores into attention weights
Output	Weighted combination of Value vectors

Think Like a Researcher

Suppose your LOFARGRAM contains:

- A weak engine harmonic at **2 seconds**
- A strong harmonic at **18 seconds**

A CNN may need many convolution layers before those two regions influence each other.

A Vision Transformer can directly connect those distant regions using self-attention. That's one reason Transformers have become so powerful.

Homework

Without looking back, answer these:

1. What is **attention**?
2. What is the role of **Query**?
3. What is the role of **Key**?
4. What is the role of **Value**?
5. Why do we apply **Softmax** to the attention scores?
6. Why can self-attention model long-range relationships better than a standard convolution?

If you can explain these in your own words, you've understood the core computation behind not only Vision Transformers but also modern language models.

Next Lesson (ViT – Part 4)

Now that you understand **self-attention**, we'll go one level deeper and study:

- **Multi-Head Self-Attention (MHSA)** — Why use multiple attention heads instead of just one?
- The complete **Transformer Encoder Block**.
- Residual connections and Layer Normalization in Transformers.
- The MLP (Feed-Forward Network) inside each encoder.
- How a full Vision Transformer is built by stacking encoder blocks.

This lesson will complete your understanding of how a Vision Transformer processes information internally before making a prediction.

ok next

Excellent! 🌟

Today we're learning **the complete Transformer Encoder**.

This is the heart of the Vision Transformer.

After this lesson, you'll understand **exactly what happens inside every Transformer layer**, not only in Vision Transformers but also in models like ChatGPT, BERT, Gemini, Claude, and many others.

So pay close attention—this is one of the most important lessons in your AI journey.

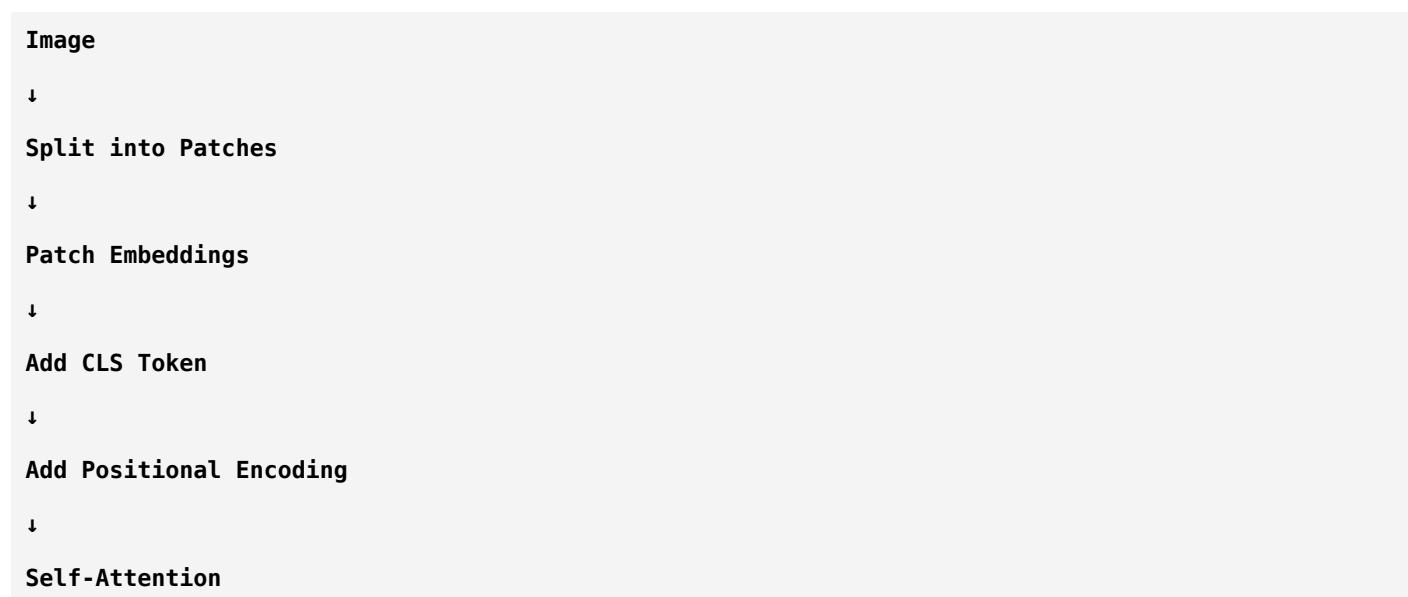
Deep CNN Architectures

Chapter 8: Vision Transformer (ViT) – Part 4

Multi-Head Self-Attention (MHSA) & Transformer Encoder

Quick Recap

So far, we have:



But there's something missing.

A real Vision Transformer **doesn't use just one attention mechanism**.

Instead, it uses:

Multi-Head Self-Attention (MHSA)

Why Isn't One Attention Head Enough?

Imagine you're investigating a ship.

Would you ask only **one expert**?

No.

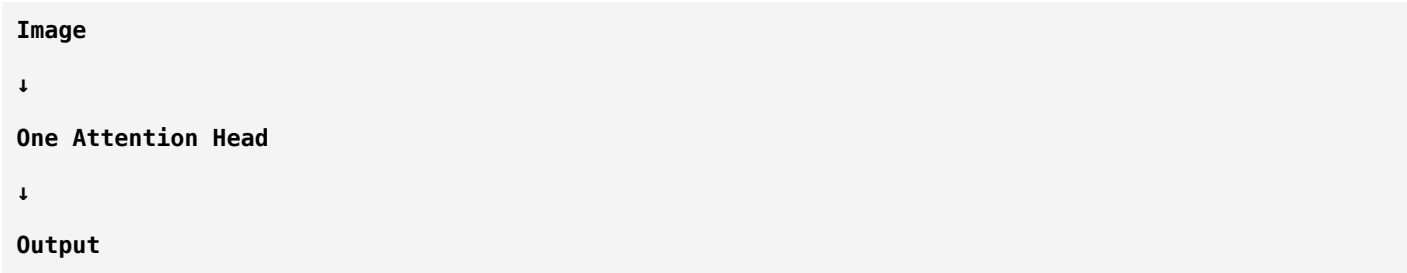
You might ask:

- Mechanical Engineer
- Naval Engineer
- Acoustic Engineer
- Radar Expert

Each expert observes the ship differently.

The final decision combines all their knowledge.
That's exactly what Multi-Head Attention does.

Single Attention



Only one perspective.

Multi-Head Attention



Each head learns different relationships.

Sonar Example

Imagine your LOFARGRAM.
Different attention heads may specialize like this:

Head	Learns
Head 1	Engine harmonics
Head 2	Propeller modulation
Head 3	Broadband noise patterns
Head 4	Echo relationships

Each head focuses on different aspects of the signal.

Does Each Head Have Its Own Q, K, and V?

Yes.

Each attention head has its own:

- Query matrix
- Key matrix
- Value matrix

That means each head learns independently.

Example:

```
Patch Embeddings
↓
Head 1 → Q1 K1 V1
↓
Attention Output 1
-----
Head 2 → Q2 K2 V2
↓
Attention Output 2
-----
Head 3 → ...
-----
Head 4 → ...
```

Every head discovers different relationships.

Combining the Heads

Suppose each head produces:

```
Output1
Output2
Output3
Output4
```

These outputs are:

1. Concatenated (joined together)

2. Passed through another linear layer

Result:

Final Multi-Head Attention Output

Why Multiple Heads?

Imagine reading a LOFARGRAM.

One person looks only at:

Frequency.

Another looks only at:

Time patterns.

Another studies:

Background noise.

Another studies:

Weak harmonics.

Would combining all four opinions be better than relying on just one?

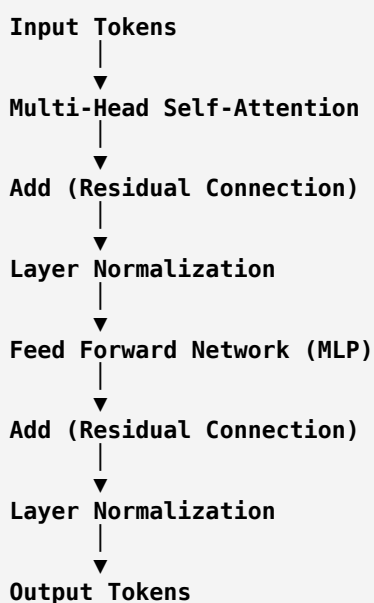
Usually yes.

That's why multiple attention heads are powerful.

The Transformer Encoder

Now let's put everything together.

One Transformer Encoder block looks like this:



This entire structure is called:

Transformer Encoder Block

Step 1: Multi-Head Self-Attention

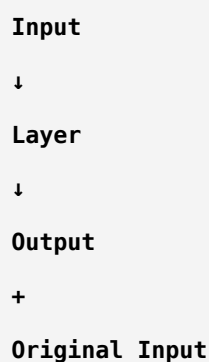
Already learned.

Purpose:

Allow every patch to communicate with every other patch.

Step 2: Residual Connection

Remember ResNet?



```
graph TD; Input --> Layer; Layer --> Output; Output --> Plus; Plus --> OriginalInput[Original Input]; Plus --> Output;
```

The diagram illustrates the structure of a residual block. It starts with an 'Input' which goes through a 'Layer' to produce an 'Output'. This 'Output' is then added to the 'Original Input' at a summation point (indicated by a '+'). The result of this addition is the final 'Output' of the block.

Transformers borrowed exactly the same idea.

Why?

Deep networks become difficult to train.

Residual connections help:

- Better gradient flow
- Stable training
- Preserve original information

Exactly the same reason as ResNet.

Sonar Example

Suppose an attention layer accidentally weakens an important harmonic.

The residual connection lets the original representation continue flowing through the network.

This helps prevent useful information from being lost.

Step 3: Layer Normalization

Students often confuse this with Batch Normalization.

They are different.

Batch Normalization

Normalizes across the batch.

Depends on:

- Batch size
- Batch statistics

Mostly used in CNNs.

Layer Normalization

Normalizes across the features of **each individual sample**.

Works well even with batch size = 1.

That's why Transformers use Layer Normalization.

Real-Life Analogy

Imagine exam scores.

Batch Normalization says:

Compare yourself with the entire class.

Layer Normalization says:

Compare the subjects within your own report card.

Different idea.

Step 4: Feed Forward Network (MLP)

Many beginners think:

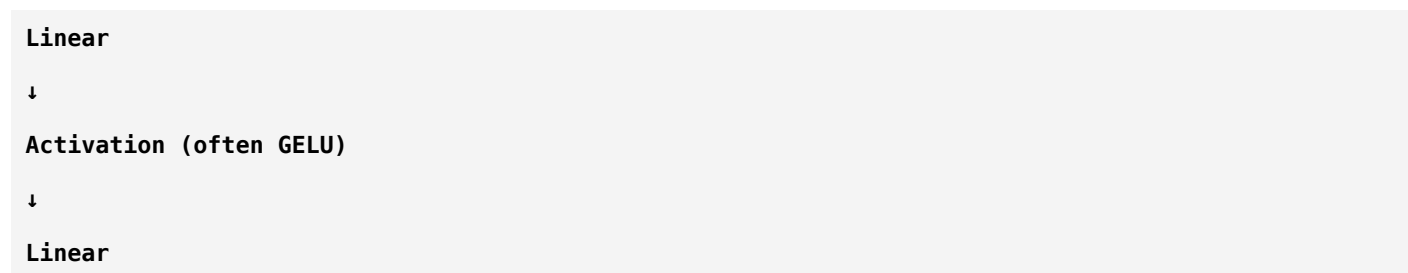
"Attention is everything."

Actually,

after attention,

each token passes through a small neural network.

Usually:



This is called:

Feed Forward Network (FFN)

or

MLP Block.

Why Do We Need It?

Attention allows tokens to exchange information.

The MLP allows each token to perform additional nonlinear processing on the information it has gathered.

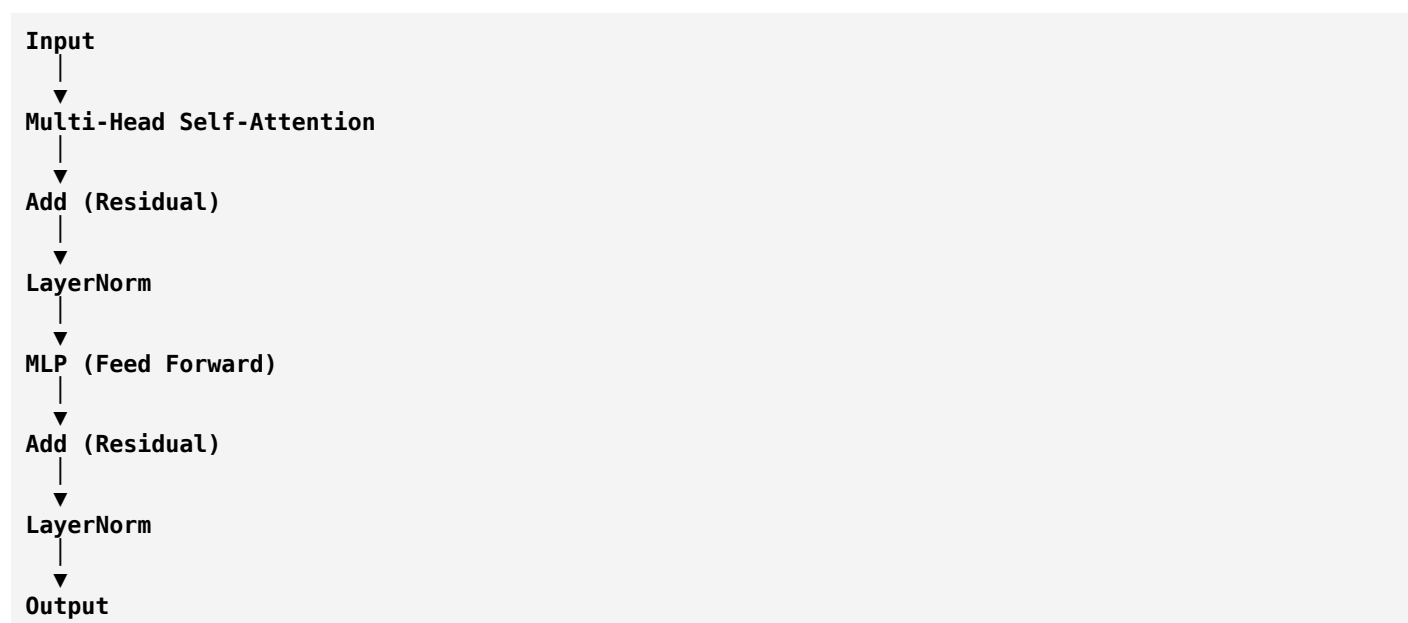
Think of it as:

Attention → Communication

MLP → Thinking

Complete Encoder Block

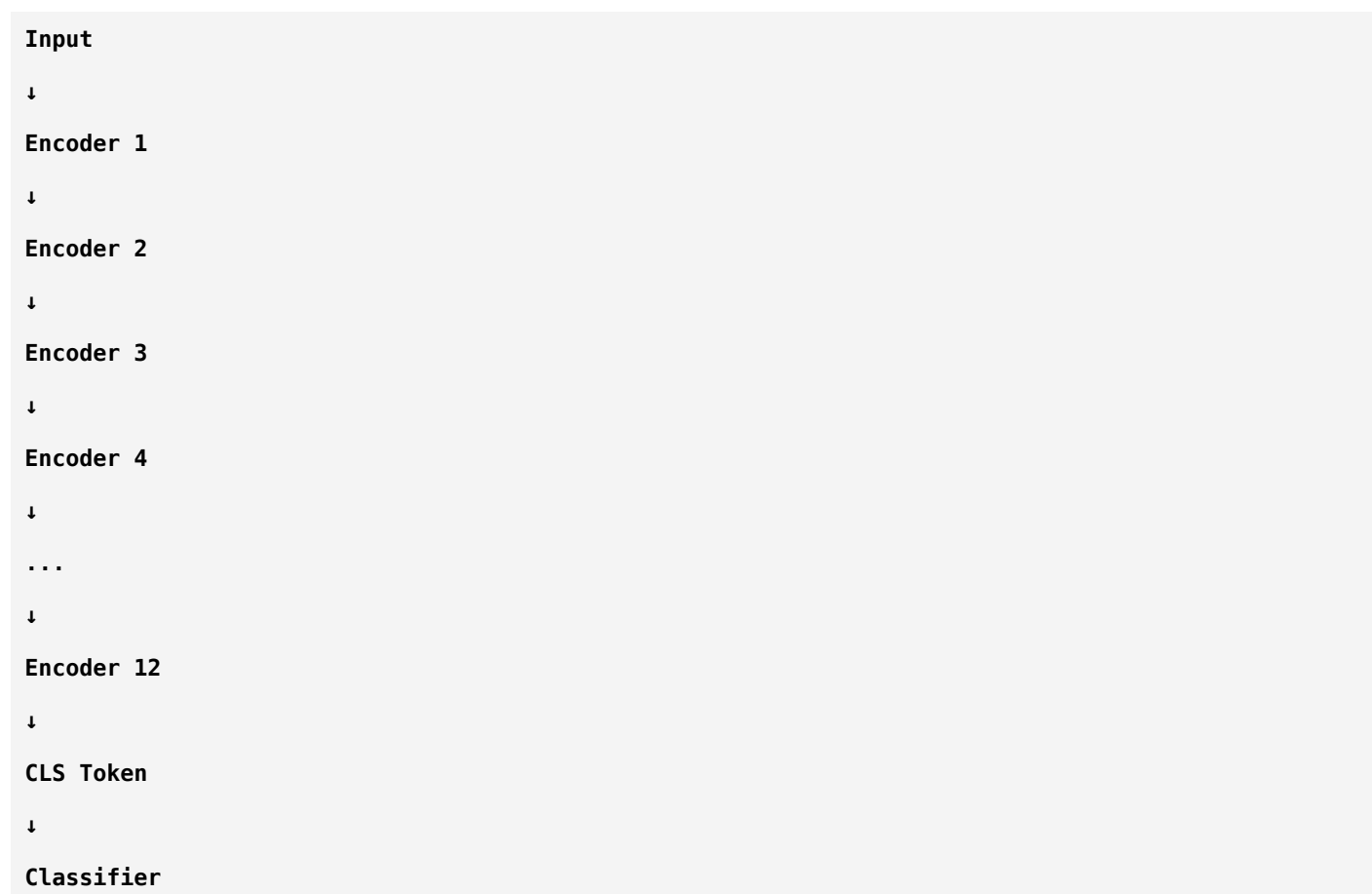
Let's summarize visually.



One Encoder Isn't Enough

A Vision Transformer stacks many encoder blocks.

Example:



Each encoder builds a richer representation.

What Happens to the CLS Token?

Remember the class monitor analogy?

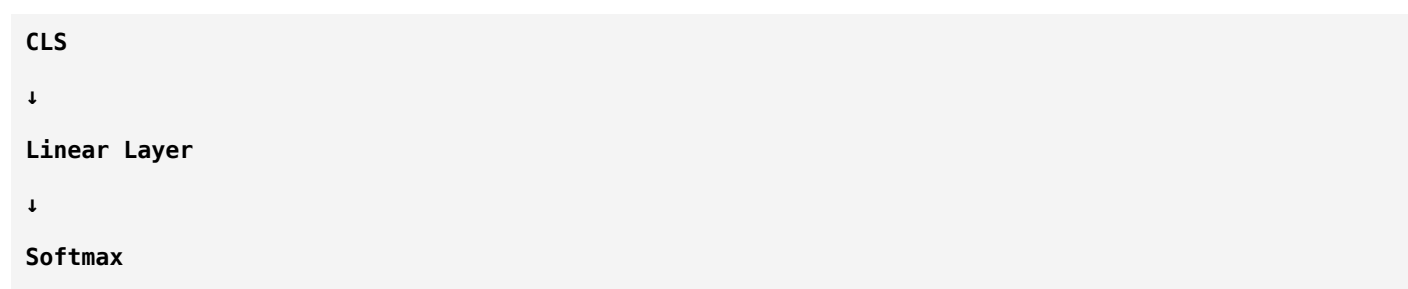
At every encoder layer,

the CLS token attends to all image patches.

Its representation becomes richer and richer.

After the final encoder,

only the CLS token is sent to the classifier.



↓
 Ship Present
 Ship Absent

Sonar Pipeline

Let's connect everything to your project.

Passive Sonar Audio
 ↓
 STFT
 ↓
 LOFARGRAM
 ↓
 Split into Patches
 ↓
 Patch Embeddings
 ↓
 CLS + Positional Encoding
 ↓
 12 Transformer Encoder Blocks
 ↓
 CLS Token
 ↓
 Classifier
 ↓
 Ship Present / Ship Absent

Now you can visualize the complete journey of your data through a Vision Transformer.

Why Transformers Need More Data

CNNs have built-in assumptions (called **inductive biases**) that help them learn from relatively smaller datasets.

Transformers are more flexible, but because they make fewer assumptions, they often need larger datasets to learn effectively.

This is one reason CNNs are still popular in domains where labeled data is limited.

CNN vs Vision Transformer

CNN	Vision Transformer
Local convolution	Global self-attention
Kernels extract features	Attention learns relationships
BatchNorm	LayerNorm
Feature maps	Token embeddings
Typically data-efficient	Often benefits from large datasets

Summary

Component	Purpose
Multi-Head Attention	Learn different relationships simultaneously
Residual Connection	Preserve information and stabilize training
Layer Normalization	Normalize features for each token
Feed Forward Network	Further process each token independently
Stacked Encoders	Gradually build richer representations
CLS Token	Final representation used for classification

Think Like a Researcher

Suppose one attention head focuses on:

- Engine harmonics

Another focuses on:

- Propeller modulation

Another focuses on:

- Background noise

Would combining these specialized views provide a richer understanding than using only one attention mechanism?

That's exactly why Vision Transformers use **Multi-Head Self-Attention**.

Homework

Without looking back, answer these:

1. Why do we use **multiple attention heads** instead of one?
2. What is the purpose of a **Residual Connection** in a Transformer?

3. How is **Layer Normalization** different from **Batch Normalization**?
4. Why does the Transformer include an **MLP** after attention?
5. What happens to the **CLS Token** after multiple encoder blocks?

If you can explain these concepts clearly, you've understood the internal structure of a Transformer Encoder.

Next Lesson (ViT – Part 5, Final)

We'll complete Vision Transformers by discussing:

- The complete **ViT architecture** from input to output.
- Different ViT model sizes (ViT-Tiny, Base, Large, Huge).
- Advantages and disadvantages of ViTs.
- ViT vs CNNs for **passive sonar target classification**.
- Hybrid CNN-Transformer models.
- When you should choose ResNet, EfficientNet, MobileNet, or ViT for your own project.

That final lesson will tie together everything you've learned about CNNs and Transformers into a practical decision-making framework for your ship-classification project.

ok next

Excellent! 🎉

This is the **final lesson on Vision Transformers (ViT)**.

Today, we'll answer the most practical question:

"For my passive sonar ship classification project, should I use a CNN or a Vision Transformer?"

This is the kind of reasoning expected from researchers, not just students.

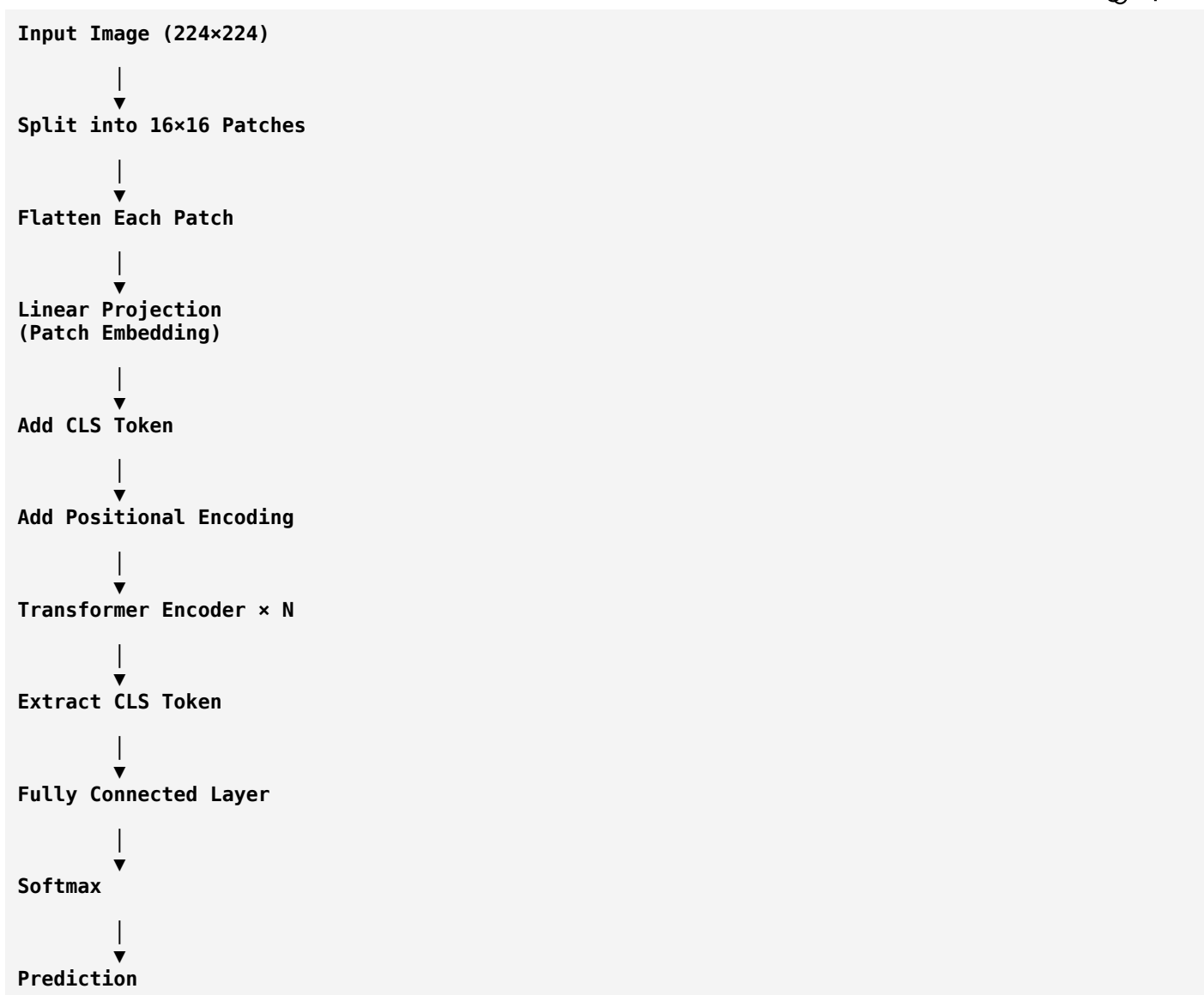
Deep CNN Architectures

Chapter 8: Vision Transformer (ViT) – Part 5 (Final)

ViT vs CNN, Model Selection & Real-World Usage

The Complete Vision Transformer

Let's put everything together.



This entire pipeline replaces the convolutional backbone used in CNNs.

Different ViT Models

Just as ResNet has:

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101

ViT also has different sizes.

Model	Parameters	Use Case
ViT-Tiny	Small	Edge devices, research experiments
ViT-Small	Medium	Moderate datasets
ViT-Base	Standard	Most research papers
ViT-Large	Large	High-performance training

Model	Parameters	Use Case
ViT-Huge	Very Large	Massive datasets and compute

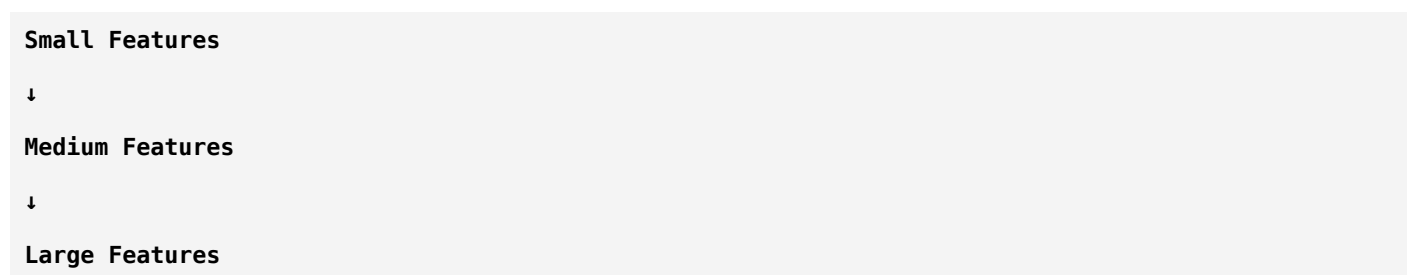
Larger models generally have higher capacity but require more memory, computation, and training data.

Advantages of Vision Transformers

1. Excellent Global Understanding

CNN:

Learns gradually.



ViT:

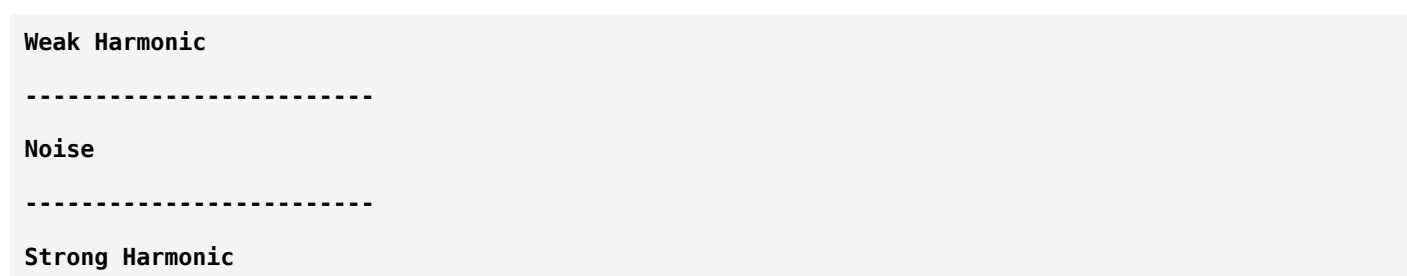
Every patch can immediately communicate with every other patch.

This is especially useful when important information is spread across the image.

2. Learns Long-Range Relationships

Example:

LOFARGRAM



The Transformer can directly compare the weak and strong harmonics.

A CNN usually learns this relationship more gradually through stacked convolution layers.

3. Highly Scalable

Researchers have trained Vision Transformers with:

- Millions of images
- Billions of parameters

They continue to improve as more data and compute become available.

4. Flexible

Transformers can be adapted to many domains:

- Computer Vision
- Audio
- Speech
- Medical Imaging
- Remote Sensing
- Sonar
- Robotics

The same underlying attention mechanism is reused across many applications.

Disadvantages of Vision Transformers

Every model has trade-offs.

1. Requires More Data

CNNs have built-in assumptions about images (local connectivity, translation invariance).

These assumptions help them learn effectively from smaller datasets.

Transformers make fewer assumptions, so they often require larger datasets to achieve their best performance.

2. More Computationally Expensive

Self-attention compares many tokens with one another.

As the number of patches increases, the computation and memory requirements grow significantly.

3. Slower Training

Compared with similarly sized CNNs,

Transformers often:

- Need more GPU memory
- Train longer
- Benefit from larger hardware resources

ViT vs CNN

CNN	Vision Transformer
Uses convolutions	Uses self-attention
Strong local feature extraction	Strong global relationship modeling
More data-efficient	Often benefits from large datasets
Usually faster	Often more computationally demanding
Mature and widely deployed	Rapidly evolving research area

Which Model Should You Use?

This is where many beginners get confused.

The answer is:

It depends on your data, hardware, and research objective.

Let's analyze your specific project.

Your Project

Passive Sonar Target Classification

Pipeline:

Hydrophone

↓

Audio Signal

↓

STFT

↓

LOFARGRAM

↓

Classifier

↓

Now we must choose the classifier.

Option 1: ResNet-18

Pros:

- Stable
- Fast to train
- Good with moderate-sized datasets
- Strong baseline used in many papers

Cons:

- Less efficient than MobileNet
- May not capture very long-range relationships as directly as a Transformer

A great first baseline.

Option 2: EfficientNet-B0

Pros:

- Better parameter efficiency
- Often achieves strong accuracy with relatively small models
- MBConv blocks make it lightweight

Cons:

- Slightly more complex architecture

Often an excellent balance between speed and accuracy.

Option 3: MobileNetV3

Pros:

- Lightweight
- Fast inference
- Suitable for embedded or real-time systems

Cons:

- Maximum accuracy may be lower than larger models on some tasks

Good if deployment constraints are important.

Option 4: Vision Transformer

Pros:

- Strong global reasoning
- Excellent for large datasets
- Modern architecture

Cons:

- Data hungry
- Computationally expensive
- Can underperform CNNs on small datasets if trained from scratch

What About Your Sonar Dataset?

This is a critical question.

Suppose you have:

- A few hundred samples

A CNN such as ResNet-18 or EfficientNet-B0 is often a safer starting point.

Suppose you have:

- Tens of thousands or more labeled spectrograms

Then a Vision Transformer becomes much more attractive.

What Do Recent Research Papers Do?

Many recent papers don't rely on just one architecture.

Instead, they compare several models, for example:

ResNet-18

↓

Accuracy

EfficientNet-B0

↓

Accuracy


```

-----
Vision Transformer
↓
Accuracy

```

Then they report which performs best for their dataset.
This comparison itself becomes part of the research contribution.

Hybrid Models

Researchers eventually realized:

CNNs are good at local patterns.

Transformers are good at global relationships.

So they combined them.

Example:

```

LOFARGRAM
↓
CNN
↓
Feature Maps
↓
Transformer
↓
Classifier

```

The CNN extracts local acoustic features.
The Transformer models long-range dependencies between them.
These hybrid approaches are common in current research.

For Your Binary Ship Classification Project

Your task:

```

Input:
LOFARGRAM
↓
Output:

```

Ship Present

or

Ship Absent

A practical research workflow would be:

Baseline

ResNet-18



Improved CNN

EfficientNet-B0



Lightweight Model

MobileNetV3



Transformer

Vision Transformer



Compare

- Accuracy
- Precision
- Recall
- F1-score
- Inference time
- Number of parameters

This gives a thorough evaluation.

Model Comparison Summary

Model	Best For	Limitation
ResNet-18	Strong baseline	Less parameter-efficient
DenseNet-121	Feature reuse	Higher memory use
EfficientNet-B0	Accuracy vs efficiency	More complex
MobileNetV3	Edge deployment	Lower capacity than larger models

Model	Best For	Limitation
Vision Transformer	Large datasets, global reasoning	Needs more data and compute

Your Learning Journey So Far

You now understand:

Digital Signal Processing

- Fourier Transform
- FFT
- STFT
- Spectrogram
- Mel Spectrogram
- MFCC
- LOFARGRAM

Machine Learning

- Supervised Learning
- Features
- Labels
- Training
- Validation
- Testing
- Loss Functions
- Optimizers
- Overfitting
- Underfitting
- Evaluation Metrics

Deep Learning

- Artificial Neural Networks
- Activation Functions
- Backpropagation
- CNN Fundamentals

- Pooling
 - Batch Normalization
 - Residual Connections
-

CNN Architectures

- AlexNet
 - VGG
 - GoogLeNet
 - ResNet
 - DenseNet
 - EfficientNet
 - MobileNet
-

Transformers

- Patch Embedding
 - CLS Token
 - Positional Encoding
 - Self-Attention
 - Query, Key, Value
 - Multi-Head Attention
 - Transformer Encoder
 - Vision Transformer
-

Think Like a Researcher

Imagine your LOFARGRAM contains:

- Weak engine harmonics
- Strong propeller signatures
- Ocean noise
- Reverberation

Ask yourself:

- Do I have enough labeled data?
- Do I need real-time inference?
- Is my deployment on an embedded device or a workstation?

- Am I optimizing for accuracy, speed, or both?

The answers guide the architecture you choose.

Congratulations!

You've now built a strong conceptual foundation spanning:

- **Digital Signal Processing**
- **Machine Learning**
- **Deep Learning**
- **Modern CNNs**
- **Vision Transformers**

This is the level of understanding expected before reading contemporary research papers on image or spectrogram classification.

What Comes Next? (This Is the Most Important Part)

You've learned **how models work**.

Now it's time to learn **how to make them work well**.

The next phase isn't about new architectures—it's about **training deep learning models effectively**.

This phase is often overlooked by beginners, but it determines whether your model reaches **70% accuracy or 97% accuracy**.

We'll cover topics such as:

1. **Transfer Learning** — Why almost nobody trains large CNNs from scratch anymore.
 2. **Fine-Tuning** — How to adapt pretrained models to your sonar data.
 3. **Data Augmentation** — Expanding your dataset and improving generalization.
 4. **Learning Rate Scheduling** — One of the most influential hyperparameters.
 5. **Regularization** — Dropout, weight decay, label smoothing.
 6. **Hyperparameter Tuning** — Batch size, optimizer selection, epochs, and more.
 7. **Model Evaluation & Explainability** — Confusion matrices, ROC curves, Grad-CAM, and interpreting model decisions.
-

My recommendation

Do not jump into coding yet.

The next phase—**training strategies and transfer learning**—is what turns someone who can *build* a model into someone who can *train* a high-performing model.

For your passive sonar project, understanding transfer learning and training strategies will likely have a bigger impact on your final results than switching from one architecture to another.

ok next

Excellent. 🚀

You've now reached the stage where you're no longer just learning **what deep learning models are**—you're learning **how researchers actually build high-performing models**.

This is where many beginners struggle.

They spend months comparing ResNet, EfficientNet, and ViT, but ignore **transfer learning**, even though it's one of the biggest reasons modern deep learning works so well.

Today we'll begin **Phase 4: Deep Learning Training Strategies**.

Phase 4: Training Deep Learning Models Like a Researcher

Chapter 1: Transfer Learning

First, Let's Understand the Problem

Suppose I ask you to build a CNN.

You design:

```

Input
↓
CNN
↓
Classifier

```

Now comes the question.

Where do the CNN's weights come from?

There are two possibilities:

Option 1

Initialize everything randomly.

```

Random Weights
↓
Training
↓
Final Model

```

Option 2

Start from a model that has already learned useful visual features.

Pretrained Model

↓

Fine-tuning

↓

Final Model

Which is better?

Almost always **Option 2**.

Why?

Imagine you want to learn to drive a truck.

Which is easier?

Person A

Has never driven anything.

Starts from zero.

or

Person B

Already knows how to drive a car.

Only needs to learn truck controls.

Obviously,

Person B learns much faster.

Deep learning works the same way.

What Does "Pretrained" Mean?

Researchers train huge CNNs on enormous datasets.

For example:

- Millions of images
- Thousands of object categories

During this process, the CNN learns general visual patterns like:

- Edges

- Corners
- Curves
- Textures
- Shapes
- Object parts

These features are useful for many different tasks.

Instead of throwing away that knowledge,
we reuse it.

Example

Imagine a pretrained ResNet.

It has already learned:

Edges

↓

Lines

↓

Textures

↓

Circles

↓

Patterns

Now your sonar project begins.

Instead of learning all of this again,
the model already knows how to detect useful visual structures.

It only needs to learn:

"Which patterns correspond to a ship?"

Sonar Example

Remember your pipeline.

Passive Sonar Audio

↓

STFT

↓

LOFARGRAM

↓

CNN

↓

Ship / No Ship

Even though a pretrained ResNet was originally trained on natural images (cats, cars, trees, etc.), its early layers learn **generic patterns** such as edges and textures.

Those low-level feature detectors are often still useful for spectrograms because spectrograms also contain lines, curves, and texture-like patterns.

The later layers are then adapted to your specific task.

Why Not Train From Scratch?

Suppose you have:

500 LOFARGRAM Images

Training a large CNN from scratch means the model must learn everything:

- Edge detection
- Texture detection
- Shape representation
- Acoustic patterns

That is difficult with only 500 samples.

Now imagine using a pretrained model.

It already understands generic visual features.

Now it only needs to adapt to sonar.

Training becomes much easier.

Real-Life Analogy

Imagine building a house.

Would you:

1. Manufacture every brick yourself?

or

2. Buy high-quality bricks and build the house?

Transfer learning is like buying the bricks.

You don't reinvent everything.

Which Parts of the CNN Are Reused?

Think of a CNN like this:

```

Input
↓
Early Layers
↓
Middle Layers
↓
Deep Layers
↓
Classifier

```

Early Layers

Learn simple features:

- Edges
- Lines
- Corners

These are useful for almost every vision task.

Middle Layers

Learn more complex patterns:

- Shapes
- Repeated structures
- Textures

Many of these are still transferable.

Final Layers

These become task-specific.

For ImageNet:

```

Dog
Cat

```

Car

For your project:

Ship

No Ship

So we usually replace the final classifier.

Visual Representation

ImageNet Model

↓

Feature Extractor

↓

1000-Class Classifier

For your project:

ImageNet Model

↓

Feature Extractor

↓

2-Class Classifier

↓

Ship / No Ship

Only the classifier changes initially.

What Is ImageNet?

This is the dataset that made transfer learning so successful.

ImageNet contains:

- Millions of images
- Thousands of object categories

Most famous CNNs are available with pretrained ImageNet weights.

Examples:

- ResNet
- DenseNet

- EfficientNet
- MobileNet
- Vision Transformer

When you see:

```
Python
pretrained=True
```

or

```
Python
weights="IMAGENET"
```

it means the model starts with knowledge learned from ImageNet instead of random weights.

Does It Matter That Sonar Is Different from Natural Images?

This is a great research question.

At first glance:

- Cats
- Cars
- Trees

seem completely different from

- LOFARGRAMs
- Spectrograms

But remember:

The early CNN layers don't recognize "cats."

They recognize:

- Edges
- Vertical lines
- Horizontal lines
- Curves
- Texture patterns

Those visual primitives appear in spectrograms too.

That's why transfer learning often works surprisingly well even across different domains.

Advantages of Transfer Learning

- ✓ Faster training.
 - ✓ Better accuracy with limited data.
 - ✓ Less risk of overfitting.
 - ✓ Lower computational cost.
 - ✓ Standard practice in modern research.
-

Are There Cases Where We Don't Use It?

Yes.

If you have:

- Millions of labeled sonar images
- Powerful computing resources
- A very different problem domain

Training from scratch may become worthwhile.

Large organizations sometimes do this because they have enough data.

Most research groups and students do not.

Types of Transfer Learning

There are two main strategies.

Strategy 1: Feature Extraction

Freeze most of the pretrained network.

Train only the new classifier.

Pretrained CNN

↓

Frozen Layers

↓

New Classifier

↓

Training

Fast and effective when your dataset is small.

Strategy 2: Fine-Tuning

Allow some pretrained layers to update during training.

```

Pretrained CNN
↓
Update Selected Layers
↓
New Classifier
↓
Training

```

This adapts the model more closely to your dataset.

We'll study fine-tuning in the next lesson.

Transfer Learning in Your Sonar Project

A typical workflow would be:

```

Passive Sonar Audio
↓
STFT
↓
LOFARGRAM
↓
Pretrained ResNet-18
↓
Replace Final Layer
↓
Train on Sonar Dataset
↓
Ship / No Ship

```

This is exactly how many published sonar-classification papers are implemented.

Summary

Concept	Meaning
Transfer Learning	Reusing knowledge from a pretrained model
Pretrained Model	A model already trained on a large dataset (e.g., ImageNet)
Feature Extractor	Early and middle layers that learn general patterns
New Classifier	Final layer replaced for your task
Main Benefit	Better performance with limited data

Think Like a Researcher

Suppose you have only **800 labeled LOFARGRAMs**.

Would you rather:

- Spend weeks training a ResNet from random weights?

or

- Start with a ResNet that has already learned millions of visual patterns and adapt it to your task?

In almost all practical research settings, the second approach is the better starting point.

Homework

Without looking back, answer these:

1. What is **transfer learning**?
2. Why does it usually outperform training from scratch on small datasets?
3. Which parts of a CNN are typically reusable?
4. Why do ImageNet-pretrained models often work on spectrograms?
5. What is the difference between **feature extraction** and **fine-tuning**?

If you can explain these confidently, you've understood one of the most important concepts in applied deep learning.

Next Lesson (Phase 4 – Chapter 2)

We'll learn **Fine-Tuning** in depth:

- What does it actually mean to "freeze" a layer?
- Which layers should you freeze?
- Which layers should you train?
- Why can unfreezing too many layers hurt performance?
- Progressive fine-tuning strategies used in research.
- The best fine-tuning approach for your passive sonar ship-classification project.

This is the step that turns a pretrained model into a specialized expert for your dataset.

ok next

Excellent. 🌟

Today's topic is one of the **most misunderstood concepts in deep learning**.

Many beginners think:

Transfer Learning = Fine-Tuning

This is **not true**.

Transfer Learning is the **overall strategy**.

Fine-Tuning is **one technique** within transfer learning.

By the end of this lesson, you'll understand exactly **which layers to freeze, which to train, and why**—knowledge that's essential for your passive sonar project.

Phase 4: Training Deep Learning Models Like a Researcher

Chapter 2: Fine-Tuning

Quick Recap

We learned:

```

Pretrained Model
↓
Replace Final Layer
↓
Train

```

But here's the question:

Should we train **all layers**?

Or only **some layers**?

This is where fine-tuning begins.

First, What is a Layer?

Remember ResNet.

```

Input
↓
Conv1
↓
Residual Block 1

```



```

↓
Residual Block 2
↓
Residual Block 3
↓
Residual Block 4
↓
Classifier

```

Every box contains:

- Filters
- Weights
- Biases

These parameters determine what the network has learned.

What Does "Freezing" Mean?

Suppose a layer has already learned useful features.

Instead of changing its weights,

we lock them.

```

Layer
↓
Weights
↓
Cannot Change

```

This is called:

Frozen Layer

During training:

- Forward pass 
- Backpropagation 
- Weight updates 

The layer still computes outputs, but its parameters remain fixed.

Real-Life Analogy

Imagine hiring an experienced engineer.

Would you teach them:

- Basic mathematics?
- How to use a screwdriver?

No.

You trust the skills they already have.

You only teach them:

- Your company's workflow
- Your project requirements

Freezing layers works the same way.

Why Freeze Layers?

Suppose ImageNet taught the CNN:

- Edge detection
- Curves
- Textures
- Shapes

Those features are already useful.

Changing them too much may actually hurt performance, especially if your dataset is small.

What Does "Unfreeze" Mean?

Now imagine your sonar dataset is quite different from ImageNet.

Some later layers may need to adapt.

So we allow those layers to learn.

```
Weights
↓
Gradient
↓
Update
↓
New Knowledge
```

Those layers are **trainable**.

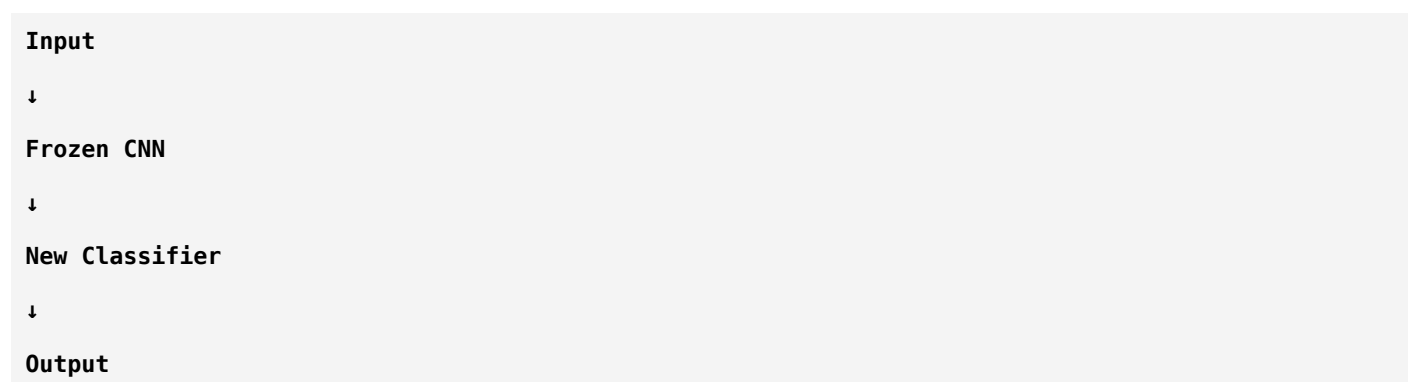
Feature Extraction vs Fine-Tuning

This is one of the most important distinctions.

Feature Extraction

Freeze almost everything.

Train only the classifier.



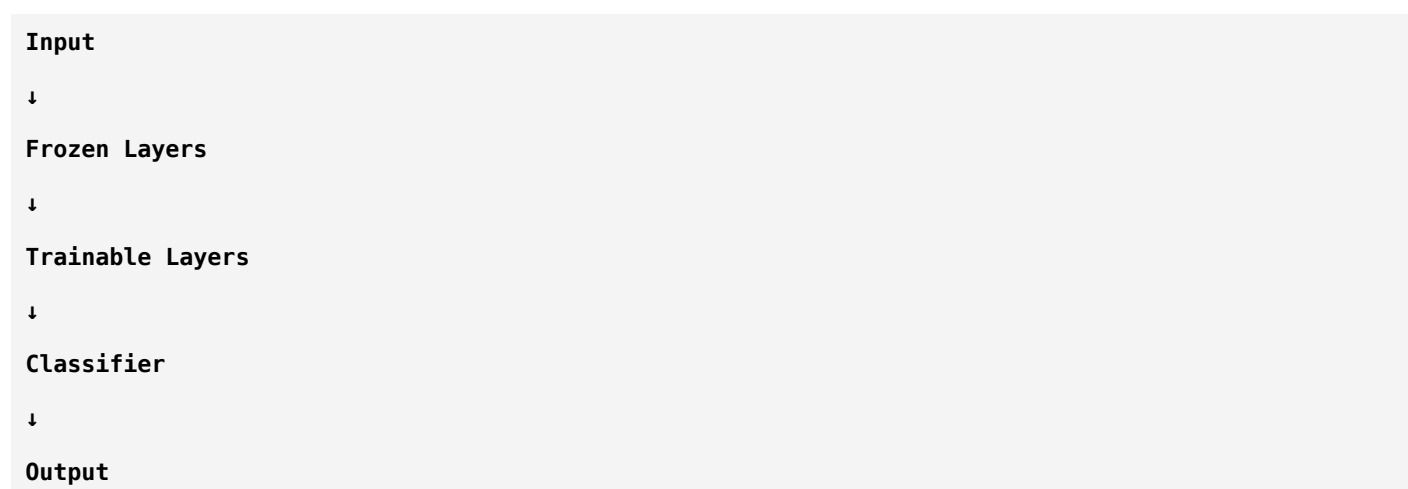
Advantages:

- Fast
- Less overfitting
- Works well with very small datasets

Fine-Tuning

Freeze some layers.

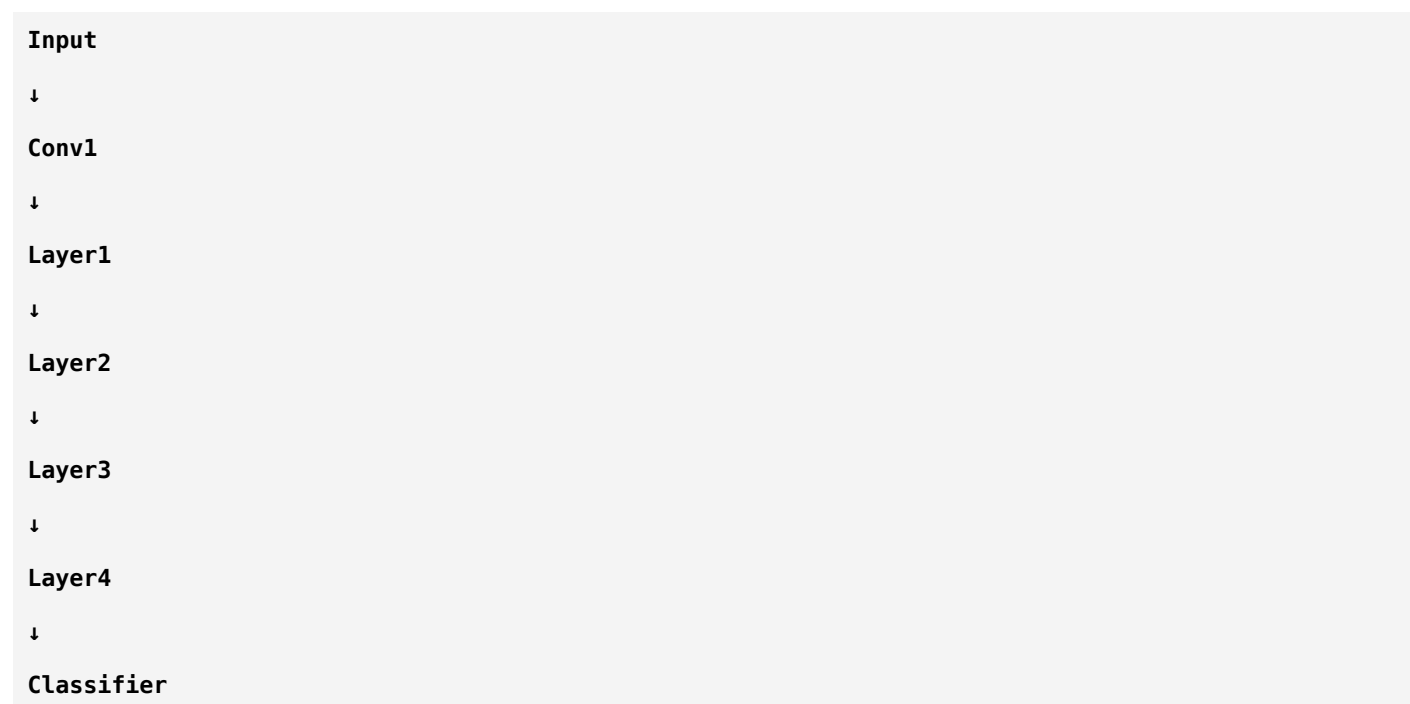
Train others.



This lets the model adapt to your specific task.

Which Layers Should Be Frozen?

Let's use ResNet-18.



Generally:

Early Layers

Learn:

- Edges
- Lines
- Simple textures

These are generic.

Usually frozen.

Middle Layers

Learn:

- Shapes
- Repeating structures
- Mid-level patterns

May or may not be frozen depending on the task.

Final Layers

Learn:

- Task-specific concepts

Usually trainable.

Why?

Think about learning languages.

Suppose you already know:

- English

Now you learn:

- Spanish

Do you relearn:

- The alphabet?
- How to hold a pencil?

No.

You reuse basic knowledge and adapt only what's different.

CNNs behave similarly.

Sonar Example

Your pretrained ResNet learned:

Edges

Textures

Curves

Your sonar dataset needs:

Engine Harmonics

Propeller Lines

Broadband Noise

Ship Signatures

The early visual features remain useful.

The deeper layers learn the sonar-specific representations.

Different Fine-Tuning Strategies

Strategy 1: Freeze Everything Except the Classifier

```
CNN
↓
Frozen
↓
Classifier
↓
Train
```

Best when:

- Dataset is very small (e.g., a few hundred images)

Strategy 2: Freeze Early Layers

```
Early Layers
↓
Frozen
↓
Later Layers
↓
Train
```

Best when:

- Dataset size is moderate
- New task differs somewhat from ImageNet

Strategy 3: Train the Entire Network

```
All Layers
↓
Train
```

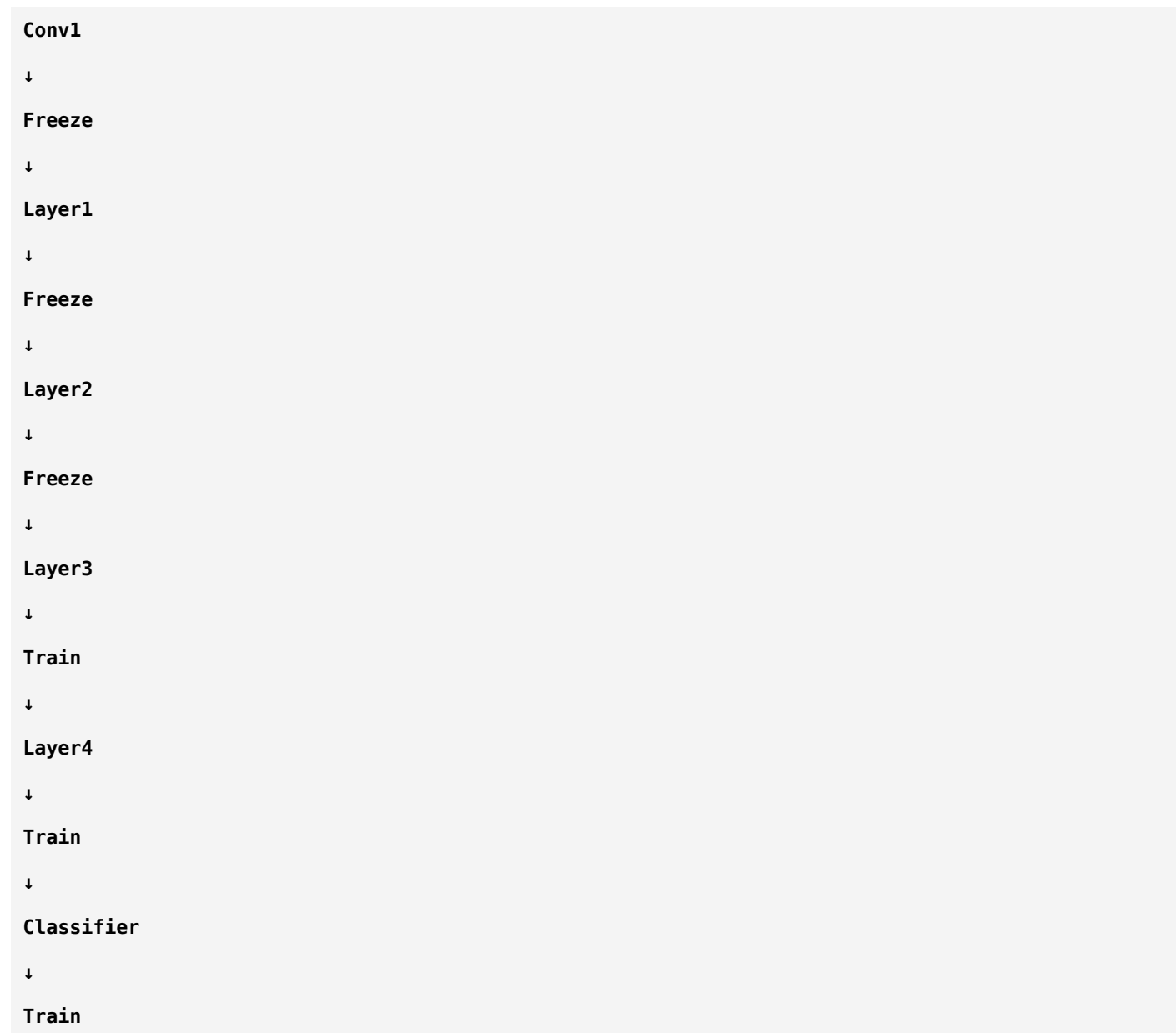
Best when:

- Large dataset
- Plenty of compute
- Domain is very different and you have enough data

Which Strategy Fits Your Project?

Let's assume you have about **2,000 LOFARGRAM images**.

A practical approach would be:



Why?

Because:

- Early visual features are still useful.
- Later layers need to specialize in sonar patterns.

This is a common starting point in practice.

What Happens During Fine-Tuning?

Imagine this weight.

Before training:

0.82

After learning sonar:

0.91

Another weight:

-0.45

↓

-0.39

The model gradually adjusts its knowledge to fit your data.

Danger of Fine-Tuning Too Much

Suppose you have only:

300 Images

If you unfreeze **every layer**, the model may:

- Memorize the training data.
- Forget useful pretrained knowledge (sometimes called *catastrophic forgetting*).
- Overfit.

More trainable parameters require more data.

Progressive Fine-Tuning

Researchers often use this workflow:

Stage 1

Train only the classifier.

Frozen CNN

↓

Classifier

Stage 2

Unfreeze the last block.

Layer4

↓

Train

Stage 3

If needed,
unfreeze Layer3 as well.

Layer3

↓

Layer4

↓

Train

This gradual approach often leads to more stable training.

Why Use Different Learning Rates?

Here's an advanced idea.

Suppose:

Early layers already contain valuable knowledge.

We don't want to change them drastically.

So we use:

Small Learning Rate

For the new classifier:

Large Learning Rate

Because it's starting from random initialization.

This technique is called **discriminative learning rates** and is widely used in transfer learning.

Complete Fine-Tuning Workflow

ImageNet Model

↓

Replace Classifier

↓

Freeze Most Layers

```

↓
Train Classifier
↓
Unfreeze Last Block
↓
Fine-Tune
↓
Evaluate
↓
If Needed,
Unfreeze More Layers
↓
Final Model

```

Fine-Tuning in Your Sonar Project

A practical pipeline might look like:

```

Passive Sonar Audio
↓
STFT
↓
LOFARGRAM
↓
Pretrained ResNet-18
↓
Freeze Early Layers
↓
Train Last Residual Block
↓
Train New Classifier
↓
Ship Present / Ship Absent

```

This balances reuse of existing knowledge with adaptation to your specific task.

Summary

Concept	Meaning
Frozen Layer	Parameters remain fixed during training
Trainable Layer	Parameters are updated by backpropagation
Feature Extraction	Train only the classifier
Fine-Tuning	Train selected pretrained layers along with the classifier
Progressive Fine-Tuning	Gradually unfreeze more layers as needed
Discriminative Learning Rates	Smaller learning rates for pretrained layers, larger for newly initialized layers

Think Like a Researcher

Suppose you have:

- 1,500 LOFARGRAMs
- A pretrained ResNet-18

Would you immediately retrain all 11 million parameters?

Or would you first train the classifier, then gradually fine-tune the deeper layers?

Most experienced researchers would start with the second approach because it's usually more stable and data-efficient.

Homework

Without looking back, answer these:

1. What does it mean to **freeze** a layer?
2. Why are early CNN layers often frozen?
3. What is the difference between **feature extraction** and **fine-tuning**?
4. Why can training every layer be risky on a small dataset?
5. What is **progressive fine-tuning**, and why is it useful?

If you can answer these comfortably, you've learned one of the most practical skills used in modern deep learning.

Next Lesson (Phase 4 – Chapter 3)

Now we'll study **Data Augmentation**, another topic that can dramatically improve model performance.

You'll learn:

- Why deep learning models need augmentation.
- Common image augmentations (flip, rotate, crop, brightness, noise).

- Which augmentations are appropriate—and which are **not**—for spectrograms and LOFARGRAMs.
- Domain-specific augmentation techniques for passive sonar.
- How augmentation helps reduce overfitting and improve generalization.

This lesson is especially important because careful augmentation can often improve performance more than switching to a more complex architecture.

ok next

Excellent. 🌟

Today's lesson is **one of the most practical lessons** in deep learning.

Many students think:

"If my model isn't accurate enough, I'll use a bigger model."

Researchers usually think differently:

"Can I improve my data first?"

Very often, **better data beats a bigger model**.

Today you'll learn **Data Augmentation**, one of the easiest and most effective ways to improve deep learning performance.

Phase 4: Training Deep Learning Models Like a Researcher

Chapter 3: Data Augmentation

First, What is Data Augmentation?

Suppose you have a dataset.

Training Dataset

↓

1000 Images

Your question is:

"Can I somehow make my dataset larger without collecting new data?"

The answer is:

Yes.

Instead of collecting new images, we create **new versions of existing images** while keeping the label the same.

This process is called **Data Augmentation**.

Real-Life Analogy

Imagine you're preparing for an exam.

Your teacher gives you only:

100 Questions

Would you memorize only those exact questions?

No.

You practice:

- Reworded questions
- Different examples
- Mixed question order
- Harder versions

Now you're prepared for many situations.

That's exactly what augmentation does.

Why Do We Need Data Augmentation?

Imagine your training data looks like this.

Ship Image 1

Ship Image 2

Ship Image 3

The model memorizes them.

During testing, it sees:

Ship Image 4

which looks slightly different.

It may fail.

Augmentation teaches the model:

"Ships can appear in many slightly different forms."

Overfitting Without Augmentation

Remember overfitting?

Training Accuracy = 99%

Testing Accuracy = 70%

The model memorized the training images.

It didn't learn the underlying patterns.

Augmentation makes memorization much harder.

Visual Example

Original Image



Augmented Images



Rotated

Flipped

Zoomed

Brighter

Noisier

The label remains:

Cat

The model now learns the concept of a cat rather than one exact picture.

Types of Data Augmentation

Let's study them one by one.

1. Rotation

Original



Rotate



The object is still the same.

Why?

Real-world objects may appear at slightly different angles.

Training with rotated images improves robustness.

Can We Rotate LOFARGRAMs?

This is an important research question.

A **90° rotation** changes the meaning of the axes:

- Time becomes frequency.
- Frequency becomes time.

That completely changes the signal representation.

So for spectrograms and LOFARGRAMs:

✗ Large rotations are generally **not appropriate**.

Very small rotations (if justified by the task) are sometimes explored, but they're far less common than in natural image classification.

2. Horizontal Flip

Image



Flip



Still a dog.

Should We Flip a LOFARGRAM?

Let's examine it.

Original

Time →

Engine Start

Engine Running
Engine Stop

Flip horizontally

Time →
Engine Stop
Engine Running
Engine Start

This reverses the temporal order.

That creates a signal that never occurred.

Therefore:

✗ Horizontal flipping is generally **not appropriate** for time-based spectrograms like LOFARGRAMs.

3. Vertical Flip

Original

High Frequency
↓
Low Frequency

After flip

Low Frequency
↓
High Frequency

Frequency ordering changes.

Again,

this no longer represents the same physical signal.

Therefore:

✗ Vertical flipping is also generally unsuitable.

Important Observation

Many augmentations that work well for photographs **do not make physical sense** for spectrograms.

This is why domain knowledge matters.

4. Random Crop

Original

```
+-----+
Entire Image
+-----+
```

Crop

```
+-----+
Center Region
+-----+
```

Why?

The model learns that important information may appear in different locations.

For LOFARGRAMs?

Be careful.

If the crop removes the ship harmonic,
the image might no longer represent the original label.

Random cropping must preserve the relevant acoustic information.

5. Brightness Adjustment

Original

```
Normal Intensity
```

↓

Brighter

↓

Darker

This simulates different recording conditions.

Sonar Interpretation

Brightness changes correspond to changes in signal intensity or visualization scaling.

Moderate adjustments may improve robustness if they reflect realistic acquisition variability.

Extreme changes could hide important harmonics.

6. Gaussian Noise

Original

Clean Signal

↓

Add Noise

↓

Still Ship

```
Real sonar recordings naturally contain noise.
Adding moderate synthetic noise during training can help the model become more robust.
---
# Why Noise Helps
Imagine training only on perfectly clean recordings.
Then, during deployment:
```text
Ocean
Wind
Rain
Machinery
Other Ships
```

The signal becomes noisy.

Without exposure to noise during training, the model may perform poorly.

## 7. Time Shift

Original

|----Ship Signal----|

Shift

|----Ship Signal----|

The event occurs slightly earlier or later.

For many audio tasks, this is a realistic augmentation because the timing of an event can vary.

## 8. Frequency Shift

Original

100 Hz

↓

110 Hz

↓

95 Hz

This simulates slight frequency variations caused by different operating conditions or recording environments.

Whether it's appropriate depends on the physics of your application and should be used carefully.

## 9. Mixup

One of the most interesting augmentation methods.

Instead of:

Image A

or

Image B

Researchers combine them.

Conceptually:

$$\text{New Image} = 0.7 \times \text{Image A} + 0.3 \times \text{Image B}$$

The corresponding labels are also blended during training.

This encourages smoother decision boundaries and often improves generalization.

## 10. SpecAugment

This is especially important for audio.

Instead of modifying the waveform,  
we modify the spectrogram.

Example

Original



Mask one frequency band.



Or mask a time region.



The model learns to recognize the signal even when some information is missing.

## Why Does SpecAugment Work?

Imagine reading:

**Passive sonar detects ships.**

Now remove one word.

**Passive \_\_\_\_\_ detects ships.**

You can still understand the sentence.

Similarly,

the model learns not to depend on one tiny region of the spectrogram.

## Data Augmentation Pipeline

During training:

**Original Image**

↓

**Random Augmentation**

↓

**CNN**

↓

**Prediction**

At the next epoch:

**Same Image**

↓

**Different Augmentation**

↓

**CNN**

The model rarely sees exactly the same image twice.

## Important Rule

Augmentation is usually applied **only to the training set**.

Validation and test sets should remain unchanged so that evaluation reflects real performance.

## What Augmentations Would I Recommend for Your Sonar Project?

Assuming you're training on **LOFARGRAMs** for binary ship detection:

### Generally Good Choices

- ✓ Mild Gaussian noise
- ✓ SpecAugment (time/frequency masking)
- ✓ Small intensity/contrast adjustments (if physically meaningful)
- ✓ Careful time shifting (if it preserves the event)

### Use With Caution

- ⚠ Random cropping (only if important harmonics remain)
- ⚠ Small frequency shifts (only if justified by the application)

### Usually Avoid

- ✗ Horizontal flip
- ✗ Vertical flip
- ✗ Large rotations

These transformations often violate the physical meaning of time and frequency.

## Summary

Augmentation	Suitable for Natural Images	Suitable for LOFARGRAMs
Rotation	✓	✗ (large rotations)
Horizontal Flip	✓	✗
Vertical Flip	✓	✗
Random Crop	✓	⚠ Use carefully
Brightness/Contrast	✓	✓ Moderate
Gaussian Noise	✓	✓

Augmentation	Suitable for Natural Images	Suitable for LOFARGRAMs
Time Shift	—	✓ Often useful
Frequency Shift	—	⚠ Task-dependent
SpecAugment	—	✓ Highly recommended
Mixup	✓	✓ Can be effective

## Think Like a Researcher

Suppose your model achieves:

- Training Accuracy = **99%**
- Validation Accuracy = **78%**

Would switching from ResNet-18 to ResNet-50 necessarily solve the problem?

Not always.

A better first step is to ask:

- Do I need more data?
- Can I use augmentation?
- Is my model overfitting?

Often, improving the training data leads to larger gains than increasing model complexity.

## Homework

Without looking back, answer these:

1. What is **data augmentation**?
2. Why does it help reduce overfitting?
3. Why is **horizontal flipping** usually unsuitable for LOFARGRAMs?
4. What is **SpecAugment**, and why is it useful?
5. Why should augmentation typically be applied only to the training set?

If you can answer these, you've understood one of the most practical tools for improving deep learning models.

## Next Lesson (Phase 4 – Chapter 4)

We'll study another concept that can make or break training:

# Learning Rate & Learning Rate Scheduling

You'll learn:

- What is the **learning rate**, and why is it often the most important hyperparameter?
- Why can a learning rate that's too high or too low ruin training?
- Popular schedulers such as **StepLR**, **Cosine Annealing**, **ReduceLROnPlateau**, and **OneCycleLR**.
- **Warm-up** strategies used in modern CNNs and Vision Transformers.
- How to choose learning rates for transfer learning and fine-tuning in your passive sonar project.

This topic is essential because even the best architecture can fail if the learning rate is poorly chosen.

ok next

Excellent! 🌟

Today we're learning what many deep learning researchers consider **the single most important hyperparameter**.

If I gave you:

- the best dataset ✓
- the best CNN ✓
- the best GPU ✓

but the **wrong learning rate**...

your model could still fail completely.

This is why experienced researchers often spend a lot of time tuning the learning rate.

## Phase 4: Training Deep Learning Models Like a Researcher

### Chapter 4: Learning Rate & Learning Rate Scheduling

#### First, What is Learning?

Let's go back to the basics.

Suppose your CNN predicts:

Ship Present

But the correct answer is:

Ship Absent

The model made a mistake.

Now it must improve.

How?

By changing its weights.

Example:

Before:

```
Weight = 0.80
```

After learning:

```
Weight = 0.78
```

Every training step adjusts millions of weights like this.

The question is:

**How much should each weight change?**

That is exactly what the **learning rate** controls.

---

## Definition

The **Learning Rate (LR)** determines:

**How big a step the model takes while updating its weights.**

Mathematically, a simplified update looks like:

```
New Weight = Old Weight - Learning Rate × Gradient
```

You don't need to memorize the formula.

Just remember:

- Gradient → tells the direction.
  - Learning Rate → tells how far to move.
- 

## Mountain Analogy

Imagine you're standing on top of a mountain.

Your goal is to reach the lowest point.

The gradient tells you:

"Walk downhill."

The learning rate tells you:



"How large should each step be?"

## Very Small Steps



You'll eventually reach the bottom.

But it will take a long time.

## Very Large Steps



You may jump over the valley repeatedly.

You never settle at the lowest point.

## Good Steps



You reach the minimum efficiently.

## Visualizing Learning Rate

Imagine the loss landscape.

### Learning Rate Too Small

Start

↓

•

↓

•

↓

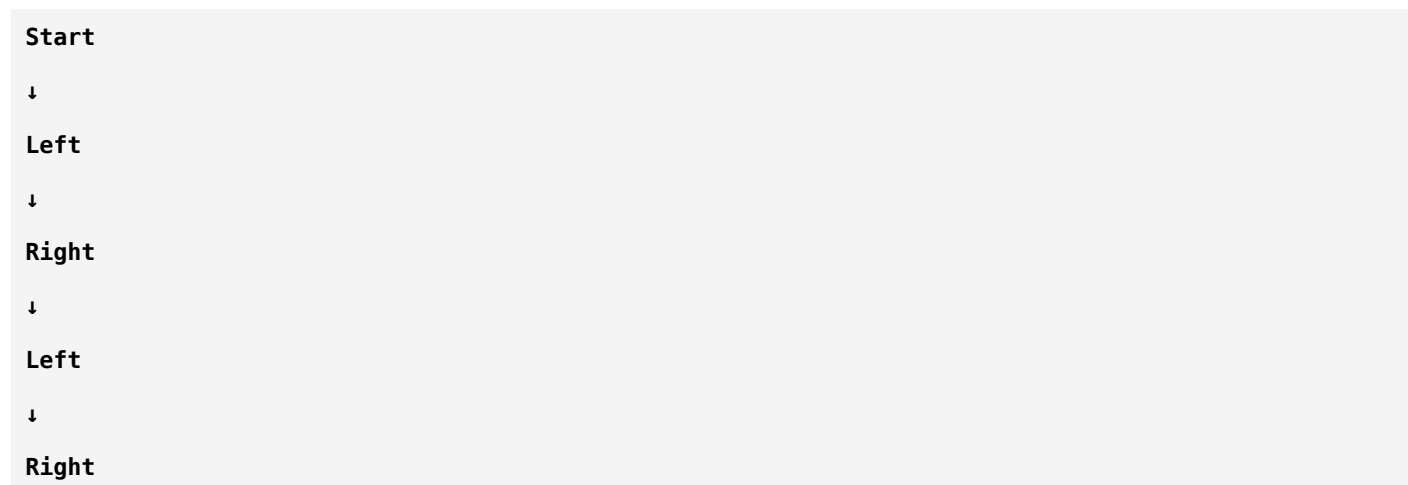
•

↓

Minimum

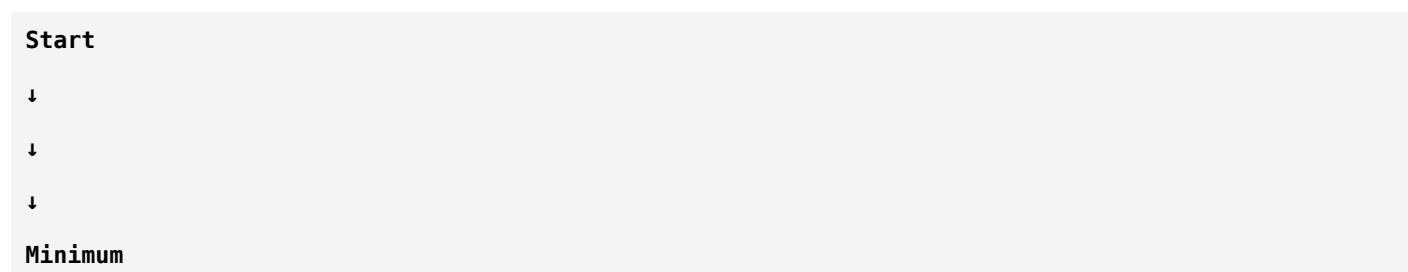
Training is very slow.

## Learning Rate Too Large



The model keeps bouncing and may never converge.

## Proper Learning Rate



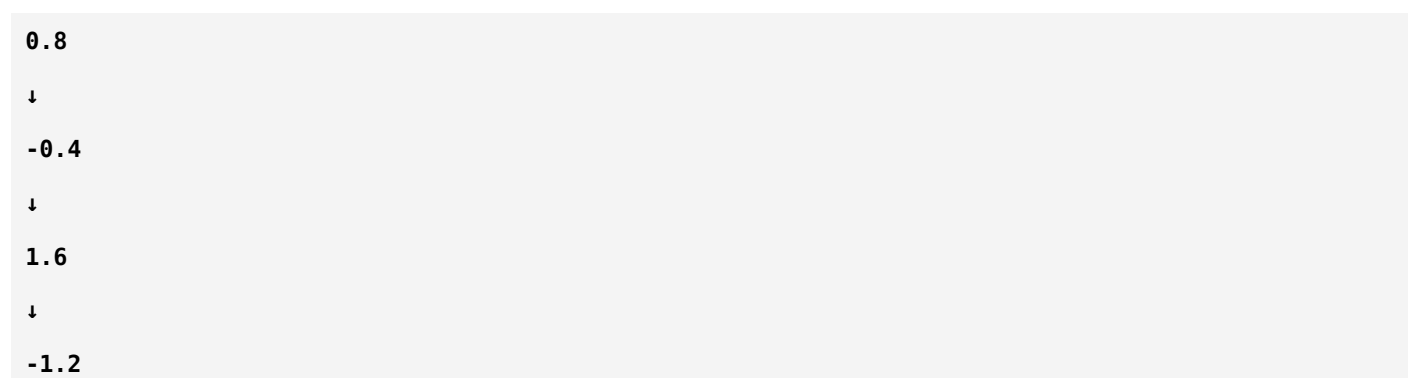
Fast and stable convergence.

## What Happens if the Learning Rate is Too High?

Suppose:

**Learning Rate = 1.0**

Weight:



Huge jumps.

The model may become unstable.

Typical symptoms:

- Loss fluctuates wildly.
- Accuracy does not improve.
- Training may diverge.

---

## What Happens if the Learning Rate is Too Low?

Suppose:

```
Learning Rate = 0.00000001
```

Weights change extremely slowly.

Symptoms:

- Training takes forever.
- Accuracy improves very little.
- You may stop training before reaching a good solution.

---

## The Sweet Spot

Researchers try to find a learning rate that is:

- Stable
- Fast
- Able to reach a good minimum

This is often one of the first hyperparameters they tune.

---

## Why Not Keep One Learning Rate Forever?

Imagine learning to ride a bicycle.

On Day 1:

You make large corrections.

After one month:

You make tiny adjustments.

Training a neural network is similar.

Early on:

Large updates help explore.

Later:

Smaller updates help refine.

That's why we use **Learning Rate Schedulers**.

## What is a Learning Rate Scheduler?

A scheduler changes the learning rate during training.

Instead of:

```
0.001
```

for every epoch,

it may become:

```
Epoch 1
```

```
0.001
```

```
↓
```

```
Epoch 20
```

```
0.0005
```

```
↓
```

```
Epoch 40
```

```
0.0001
```

This often improves convergence.

## Why Reduce the Learning Rate?

Think about sculpting a statue.

At first:

You remove large chunks of stone.

Near the end:

You make tiny, precise adjustments.

A scheduler lets the optimizer do the same thing.

## Popular Learning Rate Schedulers

Let's look at the most common ones.

### 1. StepLR

The learning rate decreases at fixed intervals.

Example:

<b>Epoch 1–10</b>
<b>0.001</b>
↓
<b>Epoch 11–20</b>
<b>0.0001</b>
↓
<b>Epoch 21–30</b>
<b>0.00001</b>

Simple and effective.

## 2. ReduceLROnPlateau

Suppose validation loss stops improving.

The scheduler says:

"Learning has stalled. Let's reduce the learning rate."

Example:

<b>Epoch 15</b>
<b>Validation Loss</b>
↓
<b>No Improvement</b>
↓
<b>Reduce LR</b>

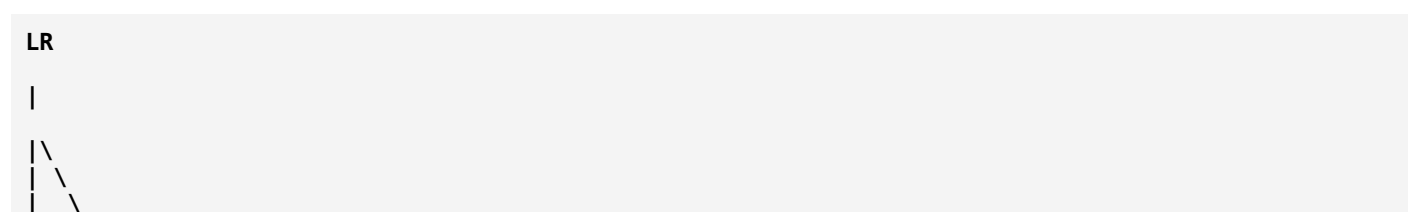
Very common in research.

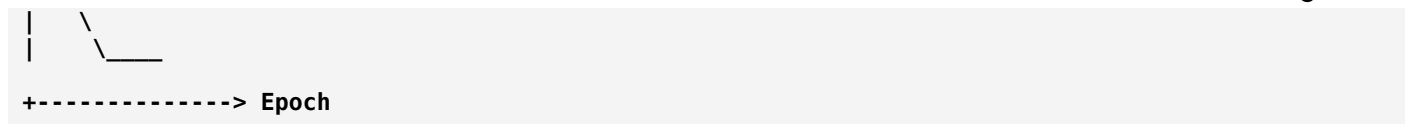
## 3. Cosine Annealing

Instead of sudden drops,

the learning rate decreases smoothly.

Conceptually:





The transition is gradual.

Popular with Vision Transformers and modern CNNs.

## 4. OneCycleLR

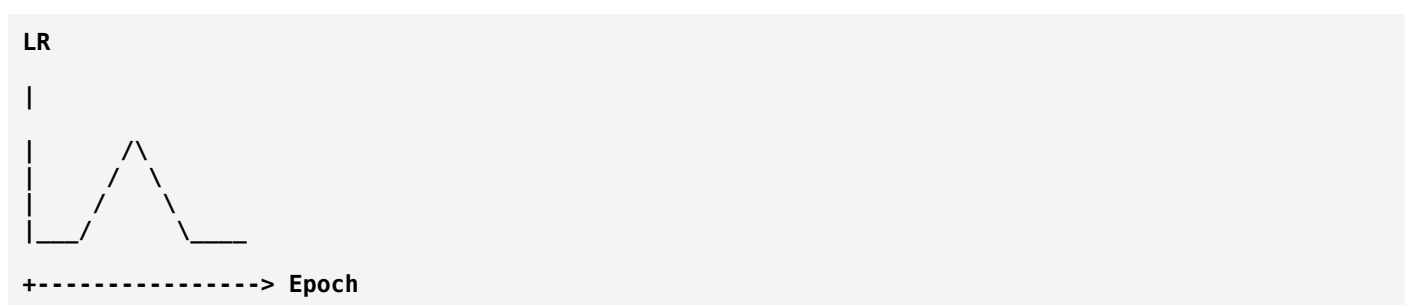
One of the most successful modern schedulers.

Instead of only decreasing,

it first increases the learning rate,

then decreases it.

Conceptually:



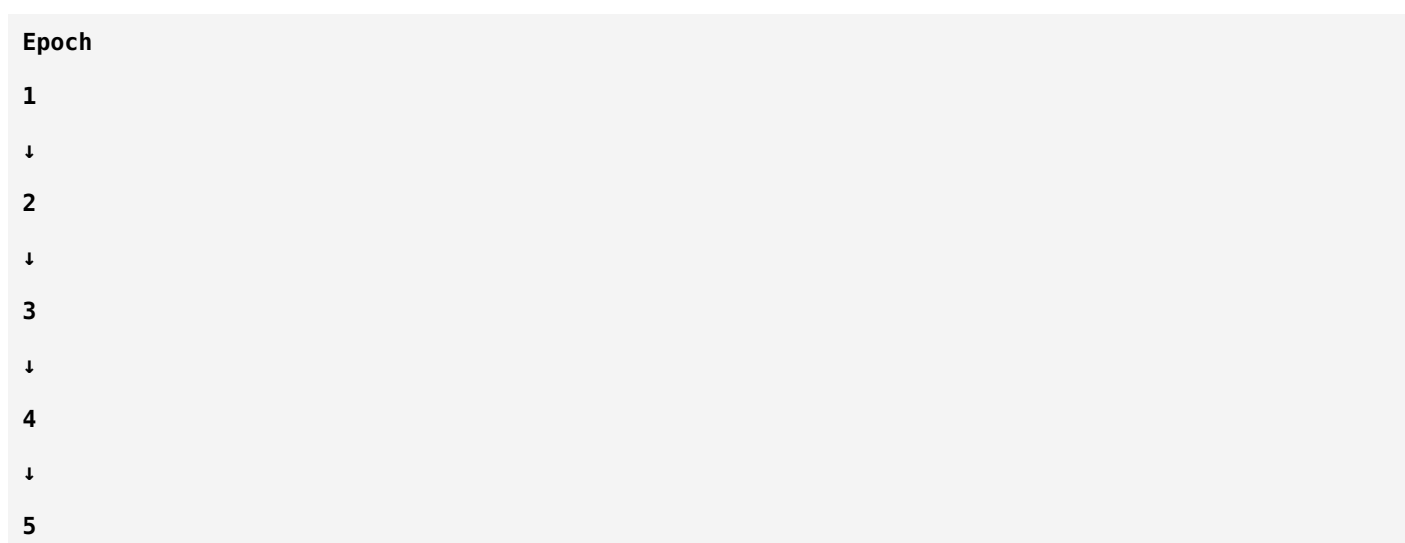
This strategy often speeds up convergence and improves generalization.

## 5. Warm-Up

Very important for Transformers.

Instead of starting with a large learning rate,

training begins gently.



Learning Rate:

**0.00001**

↓

**0.00005**

↓

**0.0001**

↓

**0.0005**

↓

**0.001**

Then the normal scheduler takes over.

## Why Warm-Up?

At the beginning of training,  
the weights are still adapting.

Large updates can destabilize optimization.

Warm-up allows the model to "settle in" before making larger updates.

This is especially useful for Vision Transformers.

## Learning Rates During Fine-Tuning

Remember fine-tuning?

Not all layers should learn at the same speed.

Example:

**Early Layers**

**LR = 0.00001**

**Last Layers**

**LR = 0.0001**

**New Classifier**

**LR = 0.001**

Why?

The classifier starts from random weights.

It needs larger updates.

The pretrained layers already contain useful knowledge.

We adjust them more gently.

## Choosing a Learning Rate

There is no universal best value.

A common starting point depends on:

- Model architecture
- Optimizer
- Batch size
- Whether you're training from scratch or fine-tuning

Typical starting values for fine-tuning are often much smaller than for training from scratch.

The best choice is usually determined experimentally.

## Learning Rate in Your Sonar Project

Suppose you're using:

- Pretrained ResNet-18
- Fine-tuning Layer 4
- New binary classifier

A common strategy is:

**Early Layers**

**Very Small LR**

↓

**Last Residual Block**

**Small LR**

↓

**Classifier**

**Larger LR**

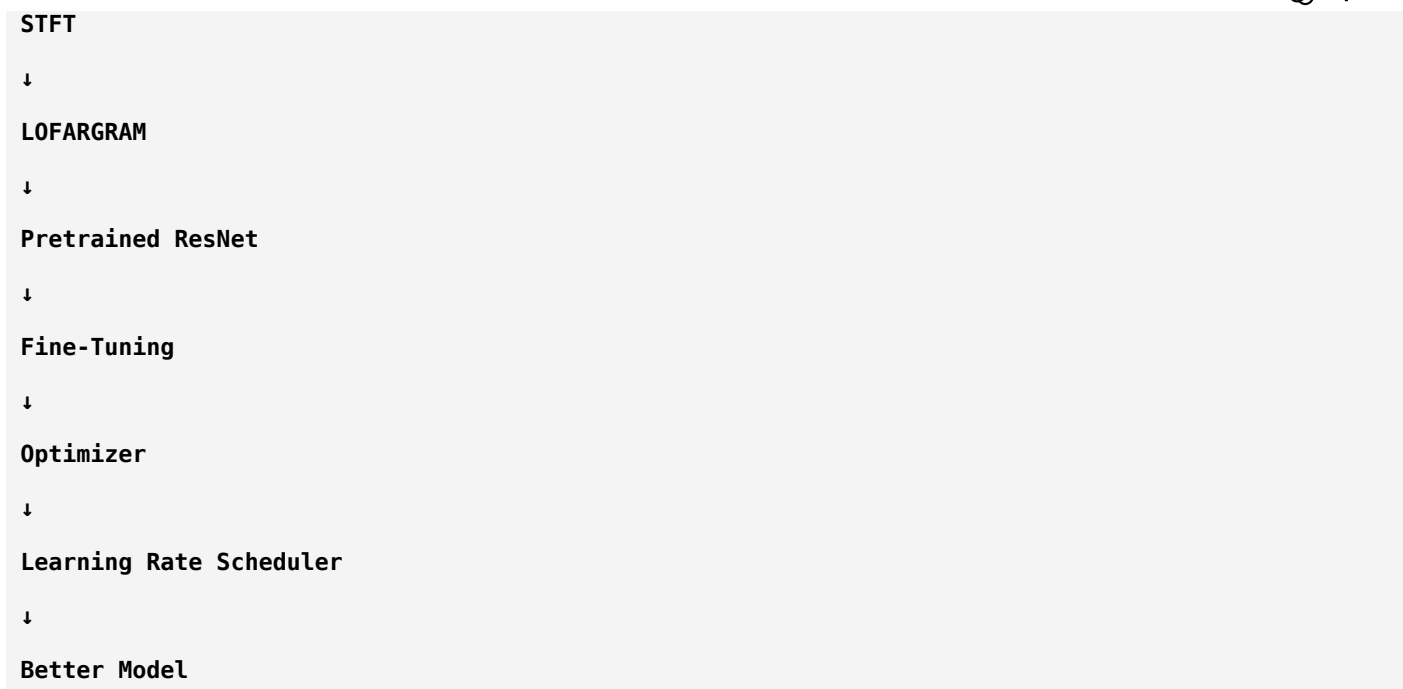
This preserves pretrained knowledge while allowing the new layers to adapt quickly.

## Putting Everything Together

**Passive Sonar Audio**

↓





The scheduler quietly improves training throughout the process.

## Summary

Concept	Meaning
Learning Rate	Size of each weight update
Too High	Unstable training, overshooting
Too Low	Very slow learning
Scheduler	Changes the learning rate during training
StepLR	Reduces LR at fixed intervals
ReduceLROnPlateau	Reduces LR when progress stalls
Cosine Annealing	Smoothly decreases LR
OneCycleLR	Increases then decreases LR
Warm-Up	Starts with a small LR and gradually increases it

## Think Like a Researcher

Suppose your training loss decreases rapidly at first, but after 20 epochs it barely changes.

Would you immediately switch from ResNet-18 to ResNet-50?

Or would you first ask:

- Is my learning rate too high?
- Is it too low?
- Should I reduce it now?

- Would a scheduler help?

Experienced researchers usually investigate optimization before changing the architecture.

## Homework

Without looking back, answer these:

1. What is the **learning rate**?
2. What happens if it is **too high**?
3. What happens if it is **too low**?
4. Why do we use a **learning rate scheduler**?
5. Which scheduler is commonly used when validation performance stops improving?
6. Why is **warm-up** especially useful for Vision Transformers?
7. Why do newly added classifier layers often use a larger learning rate than pretrained layers?

If you can answer these confidently, you've understood one of the most influential hyperparameters in deep learning.

## Next Lesson (Phase 4 – Chapter 5)

We'll study **Regularization**, another key ingredient in successful training.

You'll learn:

- What **regularization** is and why models overfit.
- **Dropout** and how it prevents co-adaptation.
- **Weight Decay (L2 Regularization)**.
- **Label Smoothing** and why it's popular in modern image classification.
- **Early Stopping** and when to stop training.
- How these techniques work together to build models that generalize well to unseen sonar data.

This chapter will complete your understanding of the main techniques researchers use to make deep learning models robust.

ok next

Excellent. 

Today we're learning something that separates **a model that memorizes the training data** from **a model that performs well on unseen data**.

Many beginners think:

"If my model gets 100% training accuracy, I'm done."

Researchers think:

"Can it generalize to new data?"

That's the purpose of **Regularization**.

---

## Phase 4: Training Deep Learning Models Like a Researcher

### Chapter 5: Regularization — Teaching a Model to Generalize

---

#### First, What is Generalization?

Imagine you're preparing for an exam.

##### Student A

Memorizes all previous question papers.

Exam result:

If the same questions appear → Excellent.

If new questions appear → Poor.

---

##### Student B

Understands the concepts.

Exam result:

Even if the questions change,  
they can still solve them.

Who is better?

Obviously,

##### Student B.

Deep learning models behave exactly the same way.

---

#### Generalization

Definition:

**Generalization is the ability of a model to perform well on new, unseen data.**

This is the real goal of machine learning.

---

#### Why Do Models Overfit?

Suppose your dataset contains:

**1000 Ship Images**

The model sees them repeatedly.

Eventually it may start memorizing:

- Exact noise patterns
- Exact backgrounds
- Tiny details unique to the training set

Instead of learning:

- What actually defines a ship.

---

## The Teacher Analogy

Imagine a teacher.

Bad teacher:

"Memorize these 100 answers."

Good teacher:

"Understand why the answers are correct."

Regularization turns the neural network into the second kind of student.

---

## What is Regularization?

Regularization means:

**Adding techniques during training that reduce overfitting and improve generalization.**

Think of it as:

Training the model with **healthy restrictions** so it doesn't become overconfident or memorize the data.

---

## Major Regularization Techniques

We'll study six important techniques:

1. Dropout
2. Weight Decay (L2 Regularization)
3. Label Smoothing
4. Early Stopping

5. Batch Normalization (Indirect Regularization)

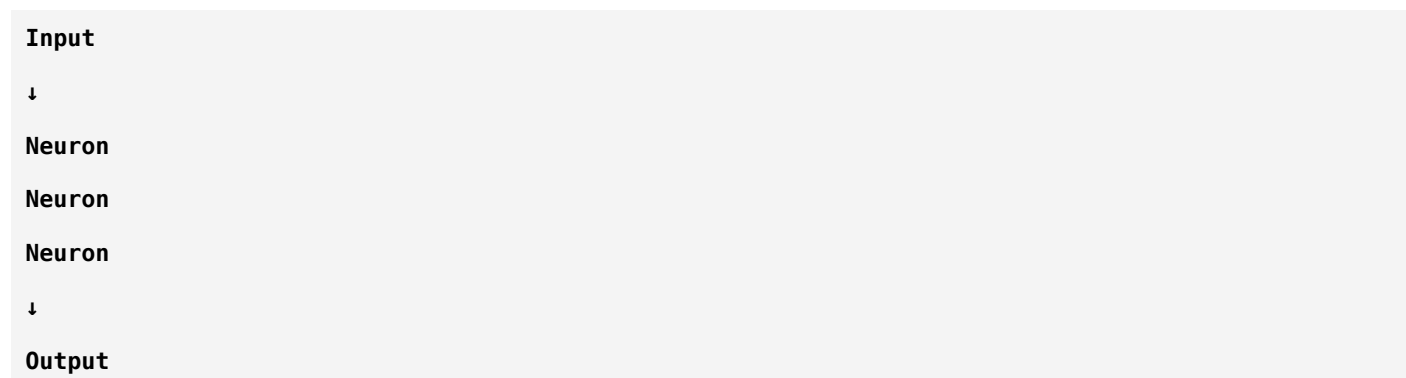
6. Data Augmentation (Already learned)

Let's study them one by one.

## 1. Dropout

One of the most famous regularization methods.

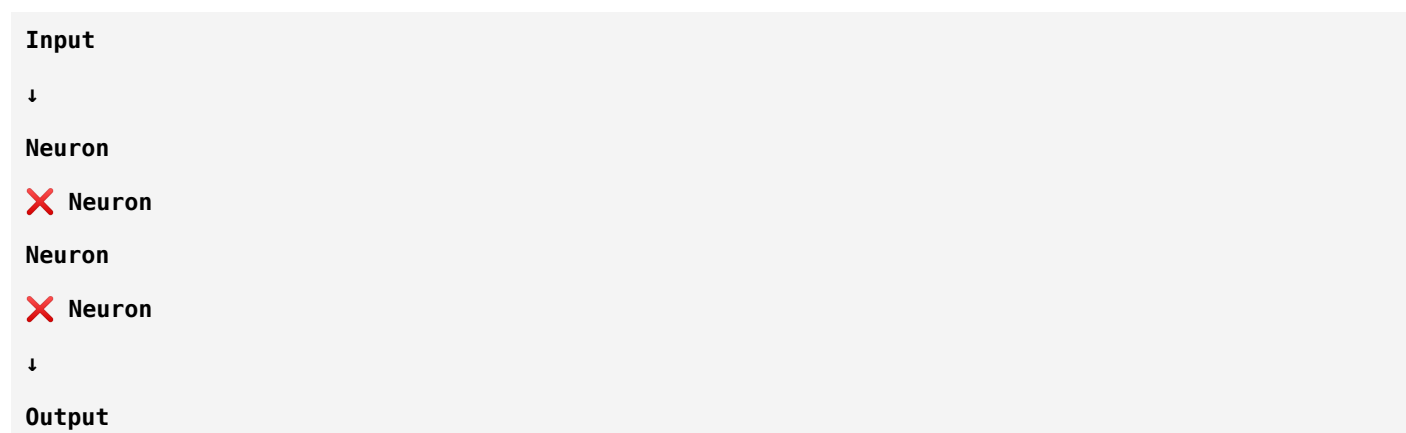
Imagine this neural network.



Normally,

all neurons participate.

During training with Dropout:



Some neurons are randomly turned **off**.

## Why?

Suppose one neuron becomes extremely important.

Other neurons stop learning.

The network becomes dependent on a few neurons.

Dropout prevents this.

## Real-Life Analogy

Imagine a football team.

Suppose only one striker scores every goal.

If that player gets injured,  
the team collapses.

A stronger team has multiple players who can contribute.

Dropout forces the network to distribute learning across many neurons.

---

## Dropout Probability

Suppose:

**Dropout = 0.5**

That means:

Approximately 50% of neurons are randomly disabled during each training step.

Next iteration,

different neurons are disabled.

The pattern changes continuously.

---

## Important Point

Dropout is used:

✓ During training

NOT during inference.

When making predictions,

all neurons are active.

---

## Sonar Example

Imagine your CNN learns:

- Engine harmonic detector
- Noise detector
- Propeller detector
- Echo detector

Without Dropout,

the model might rely almost entirely on the engine harmonic.

With Dropout,

sometimes that neuron is unavailable.

The model learns to also use propeller and echo information.

This creates a more robust classifier.

---

## 2. Weight Decay (L2 Regularization)

Now imagine a weight.

```
Weight = 12.7
```

Huge weights often indicate the model is fitting the training data too aggressively.

Weight Decay discourages excessively large weights.

---

## Real-Life Analogy

Imagine steering a car.

Without discipline:

You make huge steering corrections.

The ride becomes unstable.

Weight Decay encourages:

Small,

smooth,

controlled adjustments.

---

## What Does It Do?

During optimization,

Weight Decay gently pushes weights toward smaller values.

It doesn't force them to zero.

It simply discourages unnecessary complexity.

---

## Why Does This Help?

Large weights can make the model highly sensitive to tiny input changes.

Smaller weights generally lead to smoother decision boundaries and better generalization.

---

### 3. Label Smoothing

This is a modern technique used in many image classification models.

Normally,

labels look like this.

Ship:

```
[1, 0]
```

No Ship:

```
[0, 1]
```

The model is told:

"Be 100% confident."

Instead,

Label Smoothing changes the targets slightly.

Ship:

```
[0.9, 0.1]
```

No Ship:

```
[0.1, 0.9]
```

### Why?

Because real-world labels and data are rarely perfect.

The model learns:

"Be confident—but not absolutely certain."

This often improves calibration and reduces overconfidence.

### Student Analogy

Imagine a teacher says:

"There is a 90% chance this is correct."

instead of:

"This is unquestionably correct."

Students become less rigid and more adaptable.



## 4. Early Stopping

Suppose training looks like this.

Training Accuracy:

80%  
85%  
90%  
95%  
99%

Validation Accuracy:

78%  
83%  
88%  
89%  
84%

Notice something.

After reaching 89%,

validation performance starts getting worse.

The model is beginning to overfit.

## Early Stopping

Instead of training for:

100 Epochs

we stop at:

Epoch 35

where validation performance is best.

## Real-Life Analogy

Imagine cooking pasta.

Too little time:

Undercooked.

Too much time:

Overcooked.

The goal is to stop at the right moment.

Training is similar.

## 5. Batch Normalization

We've already learned Batch Normalization.

Its main purpose is:

- Stabilize training
- Speed convergence

But it also adds a small amount of randomness because batch statistics change from batch to batch.

This randomness provides a mild regularization effect.

## 6. Data Augmentation

We've already studied this.

Remember:

**Original Image**

↓

**Rotate**

↓

**Brightness**

↓

**Noise**

↓

**Crop**

For sonar:

**LOFARGRAM**

↓

**Gaussian Noise**

↓

**SpecAugment**

↓

**Time Shift**

↓  
Intensity Adjustment

Augmentation exposes the model to more variability, helping it generalize.

## How These Techniques Work Together

Imagine you're building a strong model.

Training Data  
↓  
Data Augmentation  
↓  
CNN  
↓  
Dropout  
↓  
Weight Decay  
↓  
Learning Rate Scheduler  
↓  
Early Stopping  
↓  
Final Model

Each component contributes to better generalization.

## Which Techniques Should You Use?

For a project like yours (binary ship classification from LOFARGRAMs), a reasonable combination is:

- **Data Augmentation:** Yes (carefully chosen for spectrograms)
- **Weight Decay:** Yes (commonly used)
- **Early Stopping:** Yes (monitor validation performance)
- **Dropout:** Often useful, especially in classifier layers
- **Label Smoothing:** Can be beneficial for classification tasks
- **Batch Normalization:** Use if it's part of the chosen architecture (e.g., ResNet)

The exact settings should be validated experimentally.

## Summary

Technique	Main Purpose
Dropout	Prevent dependence on specific neurons
Weight Decay	Discourage overly large weights
Label Smoothing	Reduce overconfidence
Early Stopping	Stop before overfitting begins
Batch Normalization	Stabilize training (with mild regularization)
Data Augmentation	Increase effective data diversity

## Think Like a Researcher

Suppose your training results are:

- Training Accuracy = **99.8%**
- Validation Accuracy = **82%**

Instead of immediately switching from ResNet-18 to Vision Transformer, ask:

- Do I need stronger augmentation?
- Should I add dropout?
- Is weight decay too small?
- Am I training for too many epochs?
- Should I use early stopping?

Researchers often improve **training strategy** before changing **model architecture**.

## Complete Training Pipeline

At this point, you can see the full workflow you've learned:

```

Raw Sonar Audio
↓
STFT
↓
LOFARGRAM
↓
Data Augmentation
↓
Pretrained CNN

```

↓  
**Fine-Tuning**  
 ↓  
**Learning Rate Scheduler**  
 ↓  
**Regularization**  
 ↓  
**Training**  
 ↓  
**Validation**  
 ↓  
**Final Model**

Notice that the CNN is only one part of the pipeline. The quality of training often has just as much impact on the final performance.

## Homework

Without looking back, answer these:

1. What is **generalization**?
2. Why does **Dropout** help reduce overfitting?
3. What is the purpose of **Weight Decay**?
4. Why does **Label Smoothing** reduce overconfidence?
5. How does **Early Stopping** detect when to stop training?
6. Why is **Data Augmentation** considered a form of regularization?

If you can answer these confidently, you now understand the core regularization techniques used in modern deep learning.

## Next Lesson (Phase 4 – Chapter 6)

Now we'll move to one of the most practical topics in research:

## Hyperparameter Tuning

You'll learn:

- What are **hyperparameters**?
- Which hyperparameters matter the most?

- Batch size, epochs, optimizer, learning rate, weight decay, dropout, scheduler, image size, and more.
- Grid Search vs Random Search vs Bayesian Optimization.
- How researchers systematically find the best configuration instead of guessing.

This chapter will teach you how to move from a "working model" to a "well-optimized model."

ok next

Excellent. 🌟

You're now entering the stage where you start thinking like an ML engineer or researcher.

Many beginners ask:

**"Which model is the best?"**

Researchers ask:

**"Did I choose the best hyperparameters?"**

The truth is:

**A well-tuned ResNet-18 can outperform a poorly tuned ResNet-50.**

Architecture matters, but **hyperparameters often determine how much of that architecture's potential you actually use.**

## Phase 4: Training Deep Learning Models Like a Researcher

### Chapter 6: Hyperparameter Tuning

#### First, What is a Hyperparameter?

Before understanding hyperparameters, let's understand **parameters**.

Suppose a CNN has:

```
Weight = 0.87
Bias = -0.21
```

These values are learned automatically during training.

They are called:

#### Parameters

The model learns them through backpropagation.

Now consider:

```
Learning Rate = 0.001
Batch Size = 32
Epochs = 50
```

Does the model learn these?

No.

**You** choose them before training.

These are called:

## Hyperparameters

### Parameters vs Hyperparameters

Parameters	Hyperparameters
Learned automatically	Set by the user
Weights	Learning Rate
Biases	Batch Size
Updated every iteration	Number of Epochs
Millions in a CNN	Usually only a few dozen

### Real-Life Analogy

Imagine cooking biryani.

#### Parameters

The rice changes while cooking.

The vegetables soften.

The spices mix.

These are like **weights**.

#### Hyperparameters

You decide:

- Cooking temperature
- Cooking time
- Amount of salt

- Water quantity

These are chosen **before** cooking.

That's exactly what hyperparameters are.

---

## The Most Important Hyperparameters

There are many, but researchers pay the most attention to:

1. Learning Rate
2. Batch Size
3. Number of Epochs
4. Optimizer
5. Weight Decay
6. Dropout
7. Learning Rate Scheduler
8. Image Size
9. Data Augmentation
10. Model Depth

Let's understand each.

---

### 1. Learning Rate

Already learned.

Controls:

**How much weights change every update.**

Too high:

✗ Unstable

Too low:

✗ Very slow

Good value:

✓ Stable learning

---

### 2. Batch Size

This is one of the most misunderstood concepts.

---



Imagine you have:

**10,000 Images**

Would you process all of them at once?

Usually no.

Instead:

**Batch Size = 32**

Training looks like:

**Images 1–32**

↓

**Update Weights**

-----

**Images 33–64**

↓

**Update Weights**

-----

**Images 65–96**

↓

**Update Weights**

Each group is called a **batch**.

## Why Not Use the Entire Dataset?

Suppose you have:

**100,000 Images**

Feeding everything into GPU memory at once may not be possible.

So we divide the data into batches.

## Small Batch Size

Example:

**Batch = 8**

Advantages:

- Lower memory usage

- More frequent updates
- Sometimes better generalization

Disadvantages:

- Noisier gradient estimates
- Training may take longer

## Large Batch Size

Example:

**Batch = 512**

Advantages:

- Faster on powerful GPUs
- More stable gradients

Disadvantages:

- High memory usage
- Can sometimes generalize less well if not tuned carefully

## Sonar Example

Suppose:

**2000 LOFARGRAMs**

Batch:

**32**

Each iteration:

**32 Images**

↓

**Prediction**

↓

**Loss**

↓

**Gradient**

↓

## Update

Then move to the next 32 images.

## 3. Epoch

Many beginners confuse **epoch** and **batch**.

Let's clear it forever.

Suppose:

**1000 Images**

Batch Size:

**100**

How many batches?

**$1000 / 100 = 10$  batches**

After processing all 10 batches,  
you have completed:

## One Epoch

## Memory Trick

**Batch = One spoonful**

**Epoch = Finish the whole meal**

## 4. Optimizer

The optimizer decides:

**How weights should be updated.**

Popular optimizers:

- SGD
- SGD + Momentum
- RMSProp
- Adam

- AdamW

## Which One Should You Use?

For most modern image classification projects:

**AdamW** is an excellent starting point because it combines adaptive learning rates with decoupled weight decay.

For some CNNs and large-scale training:

**SGD with Momentum** remains a very strong choice.

The best optimizer depends on the model and dataset.

## 5. Weight Decay

Already learned.

Controls:

How strongly we discourage very large weights.

Too much:

Model may underfit.

Too little:

Model may overfit.

## 6. Dropout Rate

Suppose:

Dropout = 0.5

Half the neurons are randomly disabled during training.

Different tasks may require different dropout probabilities.

## 7. Image Size

CNNs expect fixed-size inputs.

Examples:

224 × 224

or

256 × 256

Larger images:

✓ More detail

✗ More computation

Smaller images:

✓ Faster

✗ May lose fine details

For LOFARGRAMs, image size should preserve the acoustic patterns while remaining computationally practical.

## 8. Data Augmentation Strength

Example:

Gaussian Noise

Should you use:

Small noise?

Medium noise?

Huge noise?

Too much augmentation may distort the data.

Too little may not improve generalization.

This is another hyperparameter to tune.

## 9. Number of Layers

Should you choose:

ResNet-18

or

ResNet-50

or

ResNet-101

Model size itself is a hyperparameter.

Larger models are not always better, especially with limited data.

# How Do Researchers Choose Hyperparameters?

They don't usually guess.

They use systematic search strategies.

## Method 1: Manual Tuning

Start with reasonable values.

Observe:

- Training curves
- Validation performance

Adjust one or two settings at a time.

This is still common in practice.

## Method 2: Grid Search

Suppose:

Learning Rate:

0.1  
0.01  
0.001

Batch Size:

16  
32  
64

Grid Search tests every combination.

LR	Batch
0.1	16
0.1	32
0.1	64
0.01	16
0.01	32
0.01	64
0.001	16
0.001	32

LR	Batch
0.001	64

Very thorough.

But computationally expensive.

## Method 3: Random Search

Instead of trying every combination,  
pick random ones.

Example:

```
LR = 0.003
Batch = 24
Dropout = 0.35
```

Surprisingly, Random Search often finds strong configurations while evaluating far fewer combinations than Grid Search.

## Method 4: Bayesian Optimization

This is an advanced technique.

Instead of choosing randomly,

it learns from previous experiments and predicts which hyperparameters are likely to perform well.

Widely used in research and AutoML.

## Which Hyperparameters Matter Most?

Not all are equally important.

A rough priority for many image classification tasks is:

Priority	Hyperparameter
★★★★★★	Learning Rate
★★★★★	Optimizer
★★★★★	Data Augmentation
★★★★	Batch Size
★★★★	Weight Decay
★★★	Dropout
★★★	Image Size
★	Model Depth (once a reasonable architecture is chosen)

The exact importance depends on the task, but learning rate is almost always among the top factors.

## Hyperparameter Tuning for Your Sonar Project

Suppose you're training:

```

LOFARGRAM
↓
ResNet-18
↓
Binary Classification

```

The first hyperparameters I would experiment with are:

1. Learning rate
2. Batch size
3. Weight decay
4. Data augmentation strategy
5. Number of trainable layers during fine-tuning
6. Learning rate scheduler
7. Dropout (if applicable)

Only after optimizing these would I consider switching to a larger architecture.

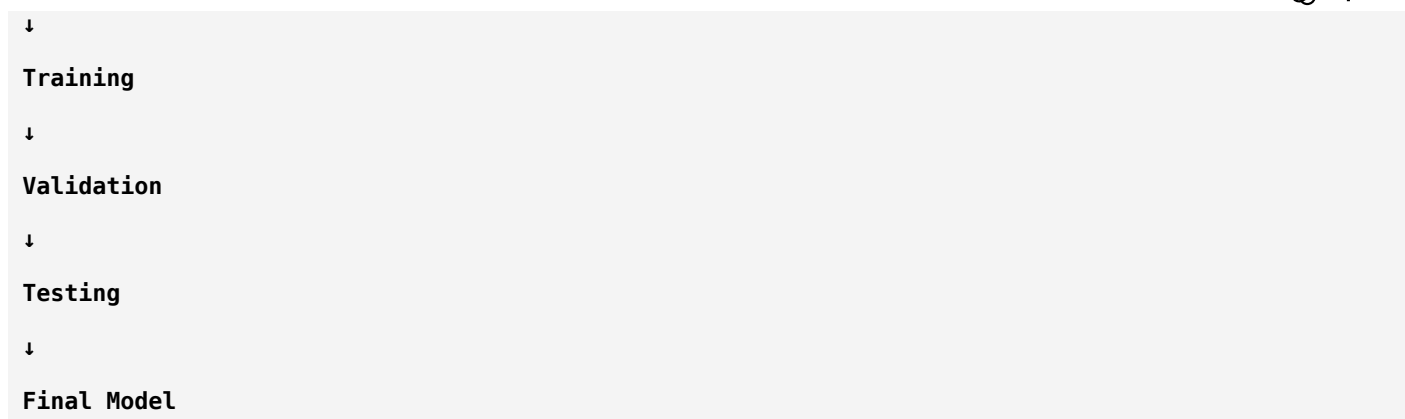
## Complete Training Pipeline

```

Raw Audio
↓
STFT
↓
LOFARGRAM
↓
Data Augmentation
↓
Pretrained CNN
↓
Fine-Tuning
↓
Hyperparameter Selection

```





Notice how hyperparameter tuning influences almost every stage of training.

## Summary

Hyperparameter	Controls
Learning Rate	Size of weight updates
Batch Size	Number of samples processed before an update
Epochs	Number of complete passes through the dataset
Optimizer	How parameters are updated
Weight Decay	Regularization strength
Dropout	Fraction of neurons disabled during training
Scheduler	Learning rate changes over time
Image Size	Input resolution
Data Augmentation	Training data diversity

## Think Like a Researcher

Suppose your model achieves:

- Training Accuracy = **95%**
- Validation Accuracy = **83%**

Would you immediately replace ResNet-18 with EfficientNet?

Or would you first ask:

- Is my learning rate appropriate?
- Is my batch size too large?
- Am I overfitting?
- Should I strengthen augmentation?
- Would a different scheduler help?

Experienced researchers usually optimize the **training process** before changing the **architecture**.

## Homework

Without looking back, answer these:

1. What is the difference between a **parameter** and a **hyperparameter**?
2. What is a **batch**?
3. What is an **epoch**?
4. Why can't we always use the entire dataset in one update?
5. What is the purpose of an **optimizer**?
6. Why is **learning rate** considered one of the most important hyperparameters?
7. How does **Grid Search** differ from **Random Search**?

If you can explain these clearly, you've built a solid understanding of how researchers tune deep learning models.

## Next Lesson (Phase 4 – Chapter 7)

Now we'll move to the final and one of the most exciting parts of model development:

## Model Evaluation & Explainability

You'll learn:

- Train, Validation, and Test splits in depth.
- Confusion Matrix.
- Precision, Recall, F1-Score, Specificity.
- ROC Curve and AUC.
- Precision–Recall Curve.
- Threshold selection for binary classification.
- Grad-CAM: How to visualize what your CNN is actually looking at.
- How to interpret and trust your sonar classifier.

This chapter is critical because a model isn't just judged by **accuracy**—it's judged by how well you evaluate and explain its decisions.

ok next

Excellent. 

You've reached one of the **most important chapters** in Machine Learning and Deep Learning.

Many beginners think:

**"My model has 95% accuracy, so it's good."**

A researcher immediately asks:

- **95% on which dataset?**
- **How many ships were missed?**
- **How many false alarms occurred?**
- **Can I trust this model?**
- **Why did the model make this decision?**

This chapter answers all of these questions.

## Phase 4: Training Deep Learning Models Like a Researcher

### Chapter 7: Model Evaluation & Explainability

#### Part 1: Why Do We Need Evaluation?

Suppose you built a CNN.

LOFARGRAM

↓

CNN

↓

Ship Present

The prediction looks correct.

But one correct prediction tells us nothing.

We need to evaluate the model on **many unseen examples**.

#### Imagine Two Students

Student A:

Memorized previous exam papers.

Student B:

Understood the concepts.

Both score 100% on the practice paper.

Now give them a new paper.

Student A fails.

Student B succeeds.

Which student is actually better?

Student B.

Exactly the same idea applies to deep learning.

---

## Dataset Splitting

The first thing every researcher does is split the data.

Entire Dataset

↓

Training

↓

Validation

↓

Testing

## Training Set

Purpose:

The model **learns** from this data.

Example:

700 Images

## Validation Set

Purpose:

Tune hyperparameters.

Example:

150 Images

During training we check:

- Overfitting
- Learning rate
- Batch size

- Dropout

---

## Test Set

Purpose:

Final evaluation.

Example:

**150 Images**

The model **must never learn from the test set.**

Think of it as the final university exam.

---

## Real-Life Analogy

Training Set

↓

Classroom lessons

Validation Set

↓

Weekly tests

Test Set

↓

Final semester examination

---

## Important Rule

Never tune your model using the test set.

Otherwise,

you're indirectly training on it.

Your evaluation becomes unreliable.

---

## Part 2: Confusion Matrix

This is one of the most important concepts in ML.

Suppose your task is:

**Ship Present**

## Ship Absent

After testing,  
we get:

Actual	Predicted	Count
Ship	Ship	90
Ship	No Ship	10
No Ship	Ship	5
No Ship	No Ship	95

This becomes:

		Predicted	
		Ship	No Ship
Actual	Ship	90	10
Actual	NoShip	5	95

This table is called the:

## Confusion Matrix

## Four Important Terms

These four terms appear in almost every ML paper.

### 1. True Positive (TP)

Actual:

Ship

Prediction:

Ship

Correct.

**TP = 90**

### 2. True Negative (TN)

Actual:

No Ship

Prediction:

No Ship

Correct.

**TN = 95**

### 3. False Positive (FP)

Actual:

No Ship

Prediction:

Ship

Wrong.

**FP = 5**

This is also called:

False Alarm.

### 4. False Negative (FN)

Actual:

Ship

Prediction:

No Ship

Wrong.

**FN = 10**

This is often the most dangerous error.

The model completely missed the ship.

## Sonar Interpretation

False Positive

**Ocean Noise**

↓

**Model says:**

**Ship**

Naval officers investigate unnecessarily.

False Negative

**Enemy Ship**

↓

**Model says:**

**No Ship**

This is potentially much more serious.

## Part 3: Accuracy

Formula:

**Correct Predictions**

-----

**Total Predictions**

Using our example:

**TP + TN**

-----

**Total**

**90 + 95**

-----

**200**

**=**

**92.5%**

Looks excellent.

Right?

Not always.

## Why Accuracy Can Mislead

Imagine:

1000 images.

**990**

**No Ship**



10

Ship

Model predicts:

Everything

↓

No Ship

Accuracy:

990 / 1000

=

99%

Amazing?

No.

The model failed to detect **every ship**.

Accuracy alone can hide serious problems.

## Part 4: Precision

Precision answers:

**When the model predicts "Ship," how often is it correct?**

Formula:

TP

-----

TP + FP

Using our example:

90

-----

90 + 5

=

94.7%

Meaning:

When the model says **Ship**, it's correct about **95%** of the time.

## Part 5: Recall (Sensitivity)

Recall answers:

**Out of all real ships, how many did we detect?**

Formula:

$$\frac{TP}{TP + FN}$$

Example:

$$\frac{90}{90 + 10} = 90\%$$

Meaning:

The model detects 90% of ships.

10% are missed.

## Precision vs Recall

Imagine airport security.

### High Precision

Few false alarms.

Good.

But may miss dangerous passengers.

### High Recall

Almost every dangerous passenger is detected.

But many innocent passengers are also stopped.

Both metrics matter.

## Sonar Example

High Precision

↓

Few false ship detections.

---

High Recall

↓

Few missed ships.

For defense applications,

high recall is often especially important because missing a real target can have severe consequences.

---

## Part 6: F1 Score

Sometimes:

Precision is high.

Recall is low.

Which model is better?

We combine both.

This gives:

## F1 Score

Think of it as:

**A balance between Precision and Recall.**

Higher is better.

---

## Example

Model A

Precision:

98%

Recall:

60%

---

Model B

Precision:

92%

Recall:

91%

Although Model A has higher precision,  
Model B provides a much better balance.  
Its F1 Score will generally be higher.

## Part 7: Specificity

Specificity answers:

**How well do we recognize No Ship?**

Formula:

$$\frac{TN}{TN + FP}$$

High specificity means:  
Very few false alarms.

## Part 8: ROC Curve

Now imagine changing the decision threshold.

Normally:

Probability  
>  
0.5  
↓  
Ship

But what if we use:

0.7  
or  
0.3

The model's behavior changes.  
ROC studies performance across many thresholds.

The graph looks like:

|  
| \*

```
| *
| *
| *
+-----
```

Better models have curves closer to the top-left corner.

## AUC

Area Under the ROC Curve.

Range:

```
0.5
↓
Random Guess
1.0
↓
Perfect Classifier
```

The closer to **1.0**, the better.

## Part 9: Precision–Recall Curve

For imbalanced datasets,

the Precision–Recall curve is often more informative than the ROC curve.

This is common in tasks like:

- Fraud detection
- Disease detection
- Rare event detection
- Ship detection (if ships are much rarer than background)

## Part 10: Choosing the Decision Threshold

Most beginners think:

```
0.5
```

is always correct.

Not necessarily.

Suppose:

**Probability**

**0.40**

Should we classify it as a ship?

Maybe.

If missing a ship is very costly,  
we might lower the threshold.

If false alarms are very costly,  
we might increase it.

The threshold depends on the application's priorities.

## Part 11: Explainability

Now we ask:

**Why did the model predict "Ship"?**

Deep learning models are often called:

**Black Boxes**

We need methods to interpret their decisions.

## Grad-CAM

One of the most popular explainability techniques.

Suppose your LOFARGRAM looks like:

**Time →**

-----

**Engine Harmonic**

**Noise**

**Propeller Harmonic**

Grad-CAM generates a heatmap.

**Blue**

↓

**Low Attention**

**Red**

↓

## High Attention

If the red region overlaps the harmonic,  
that's encouraging.  
The model is focusing on meaningful features.

## Why Grad-CAM Matters

Suppose your model predicts:

Ship

Grad-CAM shows:

Red

↓

Background Noise

That's a warning sign.  
The model may have learned the wrong cues.

## Complete Evaluation Pipeline

Train Model

↓

Validation

↓

Test Set

↓

Confusion Matrix

↓

Accuracy

↓

Precision

↓

Recall

↓

F1 Score

↓

ROC-AUC

↓

Grad-CAM

↓

Final Analysis

## Evaluation for Your Sonar Project

Your evaluation should include:

Metric	Why It Matters
Accuracy	Overall correctness
Precision	Reduce false ship alarms
Recall	Detect as many real ships as possible
F1 Score	Balance precision and recall
Confusion Matrix	Understand error types
ROC-AUC	Evaluate threshold-independent performance
Grad-CAM	Verify the model focuses on meaningful acoustic patterns

This is a much stronger evaluation than reporting accuracy alone.

## Think Like a Researcher

Suppose two models produce:

Metric	Model A	Model B
Accuracy	96%	94%
Recall	80%	96%
Precision	98%	92%

Which one should you deploy?

The answer depends on your application.

If **missing an enemy ship** is extremely costly, you might prefer **Model B** because it detects many more real ships, even though its overall accuracy is slightly lower.

This is why professional researchers never rely on a single metric.

## Summary

Concept	Meaning
Training Set	Used to learn
Validation Set	Used to tune
Test Set	Used for final evaluation



Concept	Meaning
Accuracy	Overall correctness
Precision	Correctness of positive predictions
Recall	Ability to detect positives
F1 Score	Balance between precision and recall
Specificity	Ability to detect negatives
ROC-AUC	Performance across thresholds
Grad-CAM	Visual explanation of CNN decisions

## Congratulations!

You have now completed **Phase 4: Training Deep Learning Models Like a Researcher**.

At this point, you understand:

- Transfer Learning
- Fine-Tuning
- Data Augmentation
- Learning Rate & Scheduling
- Regularization
- Hyperparameter Tuning
- Model Evaluation & Explainability

This is the complete foundation needed before implementing and experimenting with deep learning models.

## What Comes Next? (This Is the Stage Where Research Begins)

Now we move beyond "using" deep learning models to **designing and executing a research project**.

The next phase will focus on your passive sonar project itself:

### **Phase 5: End-to-End Research Pipeline for Passive Sonar Target Classification**

We'll cover:

1. **Understanding passive sonar datasets** (DeepShip, ShipsEar, UCI Sonar, and others).
2. **Audio preprocessing** (sampling rate, segmentation, normalization, denoising).
3. **Feature generation** (STFT, Mel Spectrogram, LOFARGRAM, MFCC) and when to use each.
4. **Building the complete pipeline** from raw audio to prediction.
5. **Training and comparing ResNet, EfficientNet, and ViT** on your data.
6. **Performance analysis and error analysis**.

## 7. Writing the methodology, experiments, and results sections of a research paper.

This next phase is where all the concepts you've learned come together into a complete, publishable workflow.

ok next

Excellent! 🌟

Now we are entering the **most important phase**.

Until now, you learned **Machine Learning and Deep Learning**.

From now onwards, you're going to learn **how researchers actually build a real AI system**.

Think of it like this:

Phase 1-4

↓

Learn the Tools

-----

Phase 5

↓

Build the Complete System

This phase is **100% related to your project**.

### Project Title

**Passive Sonar Based Target Ship Classification**

Binary Classification:

**Ship Present**

**Ship Absent**

## Phase 5 – End-to-End Research Pipeline

### Chapter 1: Understanding the Entire Problem Before Writing a Single Line of Code

#### First Question

Before touching Python...

Before touching PyTorch...

Before downloading datasets...

A researcher asks one question:

**"What exactly is the problem I am solving?"**

This sounds simple,  
but this is the most important step.

## Imagine This Situation

Suppose you're a naval officer.  
You're sitting inside a submarine.  
Outside,  
you cannot see anything.

**Ocean**

↓

**Enemy Ship**

↓

**Submarine**

You don't have:

- Camera ❌
- Radar ❌ (active radar/sonar may reveal your position)

So what do you use?

Passive Sonar.

## What is Passive Sonar?

Passive Sonar

=

Listening.

Nothing more.

Imagine putting your ear on a railway track.

**Track**

↓

**Vibration**

↓

**Train Coming**

You didn't create any sound.

You only listened.

Passive sonar works the same way.

## What Does the Hydrophone Receive?

The hydrophone continuously records

**Pressure Waves**

↓

**Electrical Signal**

↓

**Digital Signal**

Eventually,

we obtain

**Audio Signal**

Exactly like a WAV file,

except it contains underwater acoustic information.

## What Does This Audio Contain?

Everything.

Literally everything.

**Ocean Waves**

**Rain**

**Wind**

**Fish**

**Whales**

**Submarine**

**Cargo Ship**

**Fishing Boat**

**Propeller**

**Engine**

**Machinery**

**Background Noise**

All mixed together.

This is why passive sonar classification is difficult.

## Your AI's Job

Imagine giving this audio to a human expert.

The expert listens and says

"There is a ship."

or

"There is no ship."

Your CNN must learn to make the same decision.

---

## Binary Classification

Input

**Passive Sonar Audio**

Output

Only two possibilities.

**Ship Present**

or

**Ship Absent**

Nothing else.

---

## Why Can't We Feed Audio Directly to ResNet?

Remember ResNet?

It expects

**Images**

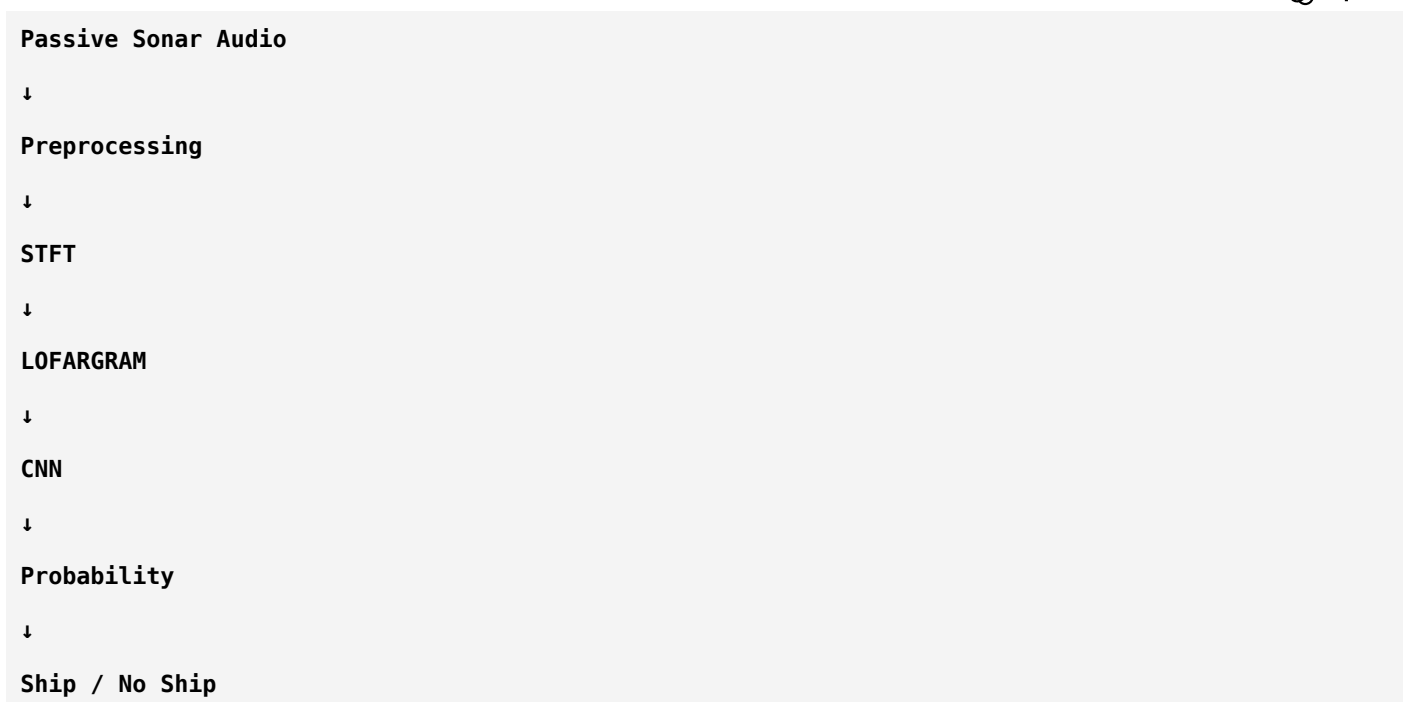
Not audio.

So we need a conversion.

---

## Complete Pipeline

This is the pipeline you will implement.



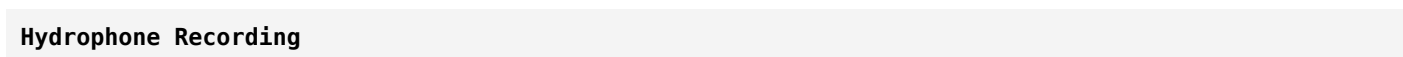
This single diagram summarizes your entire project.

## Let's Understand Every Block

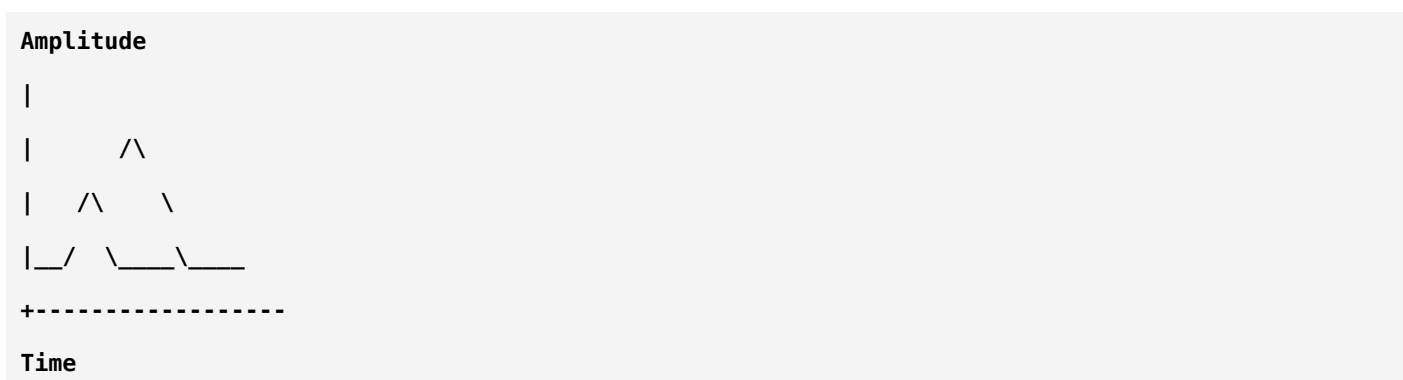
### Block 1

#### Raw Audio

Input



Looks like



Humans cannot easily identify ships from raw waveforms.

Neither can most CNNs.

## Block 2

### Preprocessing

This is the cleaning stage.

Think of washing vegetables before cooking.

We may perform tasks such as:

- Resampling (if needed)
- Normalization
- Segmentation into fixed-length clips
- Noise reduction (when appropriate)

The goal is to make the data consistent for the model.

---

## Block 3

### Time-Frequency Representation

Now we apply:

STFT

Result:

Spectrogram

or

LOFARGRAM

Now the audio becomes an image.

---

## Why?

Because CNNs are excellent at learning patterns in images.

Instead of teaching a CNN to understand raw sound,  
we let it analyze a visual representation of the sound.

---

## Example

Engine

↓

Produces harmonics

↓

Harmonics appear as bright horizontal lines

↓

CNN learns these patterns

---

## Block 4

### CNN

Input

**LOFARGRAM**

↓

Convolution

↓

Feature Extraction

↓

Classification

↓

Probability

Example:

**Ship = 0.94**

or

**No Ship = 0.06**

## Block 5

### Decision

If

**Probability > Threshold**

↓

Ship

Else

↓



No Ship

This is the final prediction.

## What is the CNN Actually Looking For?

This is where your earlier learning becomes useful.

The CNN is not looking for the word "ship."

It learns patterns such as:

- Stable harmonic lines
- Propeller blade-rate signatures
- Broadband machinery noise
- Repeating frequency components
- Time-varying acoustic structures

These patterns help distinguish ship sounds from background noise.

## Why Do Ships Produce Harmonics?

Imagine a ship engine rotating.

Engine

↓

Rotation

↓

Vibration

↓

Sound

A rotating propeller also generates periodic acoustic energy.

Because these motions repeat,

they produce repeating frequencies called **harmonics**.

On a LOFARGRAM,

harmonics often appear as:

```



```

These horizontal bands are exactly the kind of structures CNNs can learn.

# Why is "No Ship" Difficult?

Many beginners think:

```
No Ship
=
Silence
```

This is incorrect.

"No Ship" may still contain:

- Ocean waves
- Wind
- Rain
- Marine mammals
- Other biological sounds
- Distant machinery
- Sensor noise

So your classifier must distinguish:

**Ship-specific acoustic patterns**

from

**Natural underwater background sounds.**

## Real Example

Imagine these two LOFARGRAMs.

### Sample A

```



```

Many strong harmonics.

Likely:

Ship.

### Sample B

- 
- 
- 
- Random Noise**
- 
- 

No organized harmonic structure.

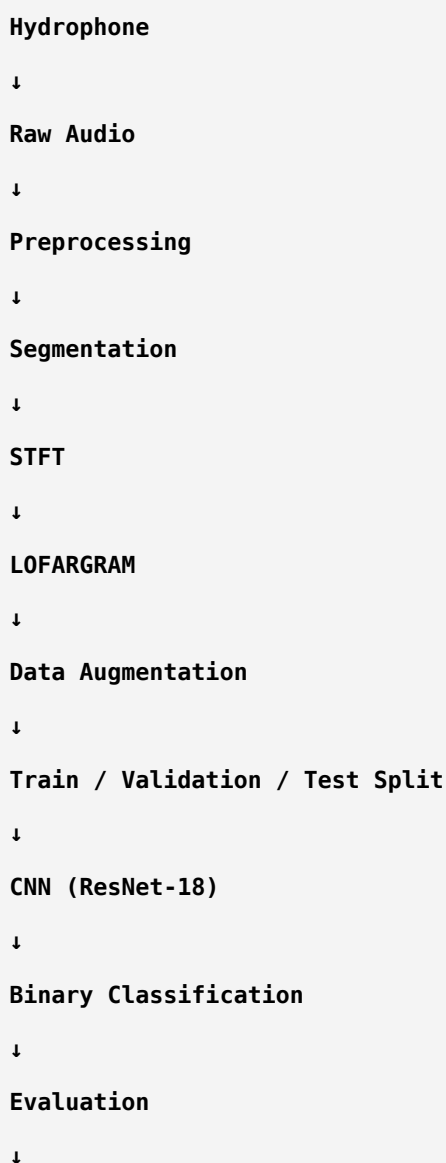
Likely:

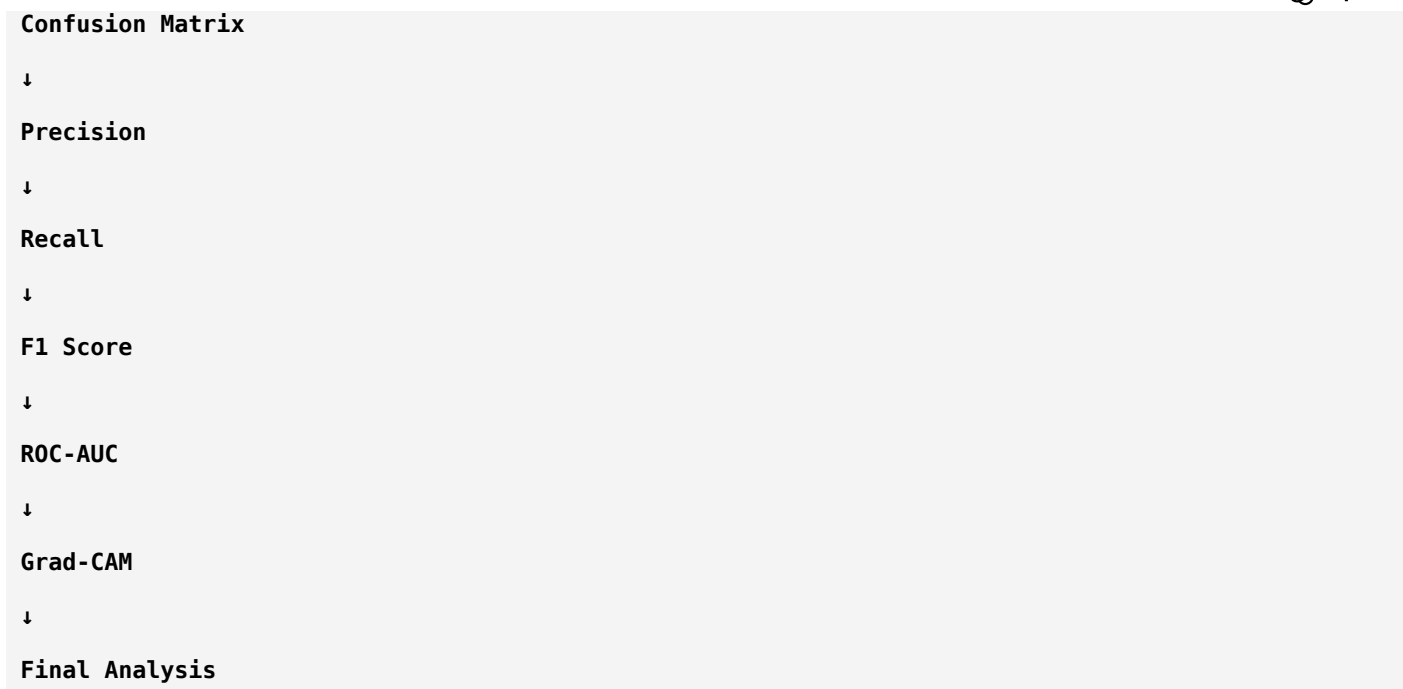
No Ship.

This is the type of discrimination your model will learn.

## The Complete Research Pipeline

Every passive sonar research paper follows a workflow similar to this:





Notice how every concept you've learned now has a place in the pipeline.

## Where Does Every Topic Fit?

Topic You've Learned	Where It Fits
Fourier Transform	Convert signal to frequency domain
FFT	Efficient computation of the Fourier Transform
STFT	Time-frequency analysis
Spectrogram	Visual representation of STFT
Mel Spectrogram	Perceptual frequency representation
MFCC	Compact feature representation
LOFARGRAM	Sonar-specific time-frequency image
CNN	Learn spatial patterns in the image
Transfer Learning	Reuse pretrained visual features
Fine-Tuning	Adapt the CNN to sonar data
Data Augmentation	Improve robustness
Learning Rate	Control optimization
Regularization	Reduce overfitting
Hyperparameter Tuning	Optimize training
Evaluation Metrics	Measure performance
Grad-CAM	Explain the model's decisions

## Think Like a Researcher

Before writing code, ask yourself:

"What information in the audio actually tells me that a ship is present?"

If you answer:

"The model will figure it out."

you're thinking like a programmer.

If you answer:

"The model should learn physically meaningful patterns such as harmonics, tonal components, and machinery signatures while ignoring irrelevant background noise."

you're starting to think like a researcher.

## Homework

Try answering these without looking back:

1. Why can't we directly feed raw sonar audio into a ResNet?
2. Why do we convert audio into a LOFARGRAM?
3. What kinds of acoustic patterns indicate the presence of a ship?
4. Why is the "No Ship" class not simply silence?
5. Where in the pipeline do preprocessing, CNNs, and evaluation each fit?

If you can answer these confidently, you've understood the overall research problem—not just the implementation.

## Next Lesson (Phase 5 – Chapter 2)

Now we begin working with **real passive sonar datasets**, where you'll learn:

- The most commonly used public datasets (**DeepShip**, **ShipsEar**, **UCI Sonar**, and others).
- What each dataset actually contains.
- Which datasets are appropriate for **binary ship detection**.
- How researchers inspect, organize, and prepare datasets before training.
- How to define labels correctly and avoid common mistakes.

This is the point where you'll transition from theory to working with real data, just like in an actual research lab.

ok next

Excellent! 

This chapter is where your project **actually begins**.

From now on, we stop talking about general deep learning and start talking about **your research problem**.

Today we'll study the most important part of any AI project:

### **The Dataset**

A famous AI quote says:

**"A model can only learn what the data teaches it."**

Even the best CNN cannot compensate for poor-quality or poorly understood data.

---

## **Phase 5 – End-to-End Research Pipeline**

### **Chapter 2: Understanding Passive Sonar Datasets**

---

#### **Why is the Dataset So Important?**

Imagine you want to teach a child what a ship sounds like.

You play:

- Cargo ships
- Fishing boats
- Naval ships
- Ferries

The child gradually learns the concept of a "ship."

Now imagine you only play recordings of one cargo ship.

The child might think:

"Every ship sounds exactly like this."

That's not true.

A diverse dataset helps the model learn the underlying concept rather than memorizing specific examples.

---

#### **What is a Dataset?**

A dataset is simply a **collection of labeled examples**.

For your project:

##### **Example 1**

**Audio File**  
↓  
**Ship Present**  
-----  
**Example 2**  
**Audio File**  
↓  
**Ship Absent**  
-----  
**Example 3**  
**Audio File**  
↓  
**Ship Present**

Each example consists of:

- Input (audio)
- Correct answer (label)

## What Does One Sample Look Like?

A single sample might contain:

**Hydrophone Recording**  
↓  
**10 Seconds**  
↓  
**44.1 kHz**  
↓  
**Mono Audio**  
↓  
**Ship Present**

Think of one sample as:

**(Audio Clip, Label)**

The model learns from thousands of these pairs.

## Labels

Labels are the "ground truth."

Example:

**audio\_001.wav**

↓

**Ship**

**audio\_002.wav**

↓

**No Ship**

Without labels, supervised learning is not possible.

---

## Types of Passive Sonar Datasets

There are two broad categories:

### 1. Raw Hydrophone Audio

Contains the original underwater recordings.

Example:

**Pressure Waves**

↓

**Digital Audio (.wav)**

You perform preprocessing yourself.

---

### 2. Precomputed Features

Instead of raw audio,

the dataset already provides:

- Spectrograms
- Mel Spectrograms
- MFCCs
- LOFARGRAMs

This saves preprocessing time but gives you less flexibility.

---

## Which is Better?



Researchers generally prefer **raw audio** because:

- You control preprocessing.
  - You can experiment with different features.
  - Your work is easier to reproduce and compare.
- 

## The Most Popular Passive Sonar Datasets

Let's study the important ones one by one.

---

### 1. DeepShip

This is one of the most popular datasets for deep learning-based passive sonar research.

### What does it contain?

Recordings of different ship categories collected from real underwater environments.

Examples include:

- Cargo ships
- Tankers
- Passenger ships
- Tugboats

The recordings contain:

- Engine sounds
  - Propeller noise
  - Ocean background
- 

### Why is it Popular?

Because it provides:

- Real underwater recordings
- Multiple ship categories
- Enough variability for CNN training

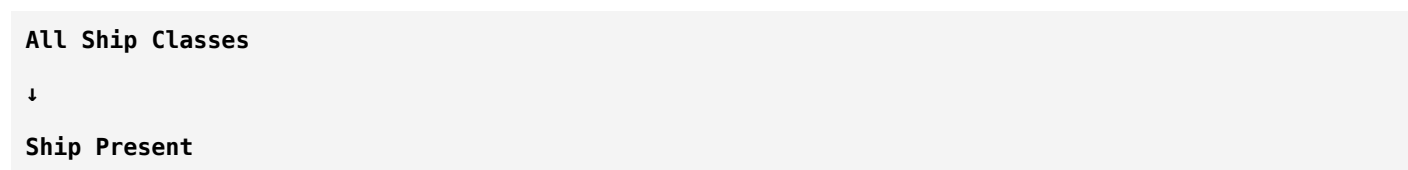
Many recent research papers benchmark their models on this dataset.

---

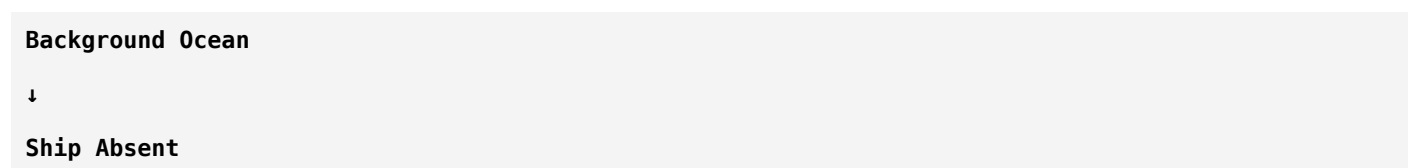
### Can You Use It?

Absolutely.

For your binary problem:



Then create another class:



## 2. ShipsEar

Another well-known passive sonar dataset.

It contains:

- Ship sounds
- Ocean noise
- Various environmental conditions

Compared with DeepShip:

It focuses more on acoustic recordings than image-like representations.

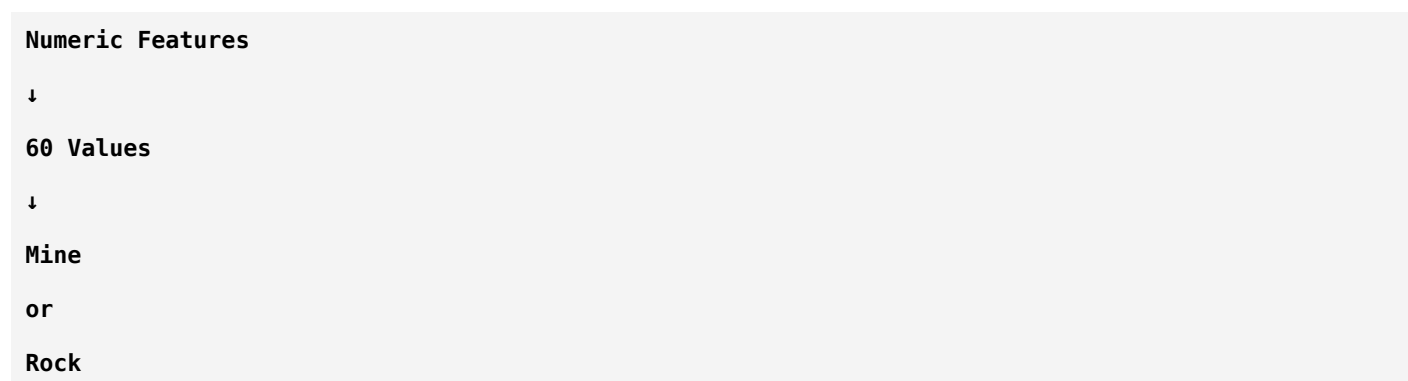
Researchers often use it to generate their own spectrograms or LOFARGRAMs.

## 3. UCI Sonar Dataset

Many beginners confuse this dataset with passive sonar recordings.

Let's clear that up.

This dataset contains:



Important:

It **does not contain raw audio**.

Instead,  
it contains already extracted numerical features.

---

## Can You Use It?

Yes,  
but **not for your project**.

It is useful for:

- Learning machine learning algorithms
- Classification practice

It is **not suitable** if your goal is to process raw sonar audio with CNNs.

---

## 4. Custom Dataset

Many research groups build their own datasets.

Example:

```
Hydrophone
↓
Record Ocean
↓
Label Audio
↓
Create Dataset
```

This often gives the best results,  
but collecting underwater acoustic data is expensive and time-consuming.

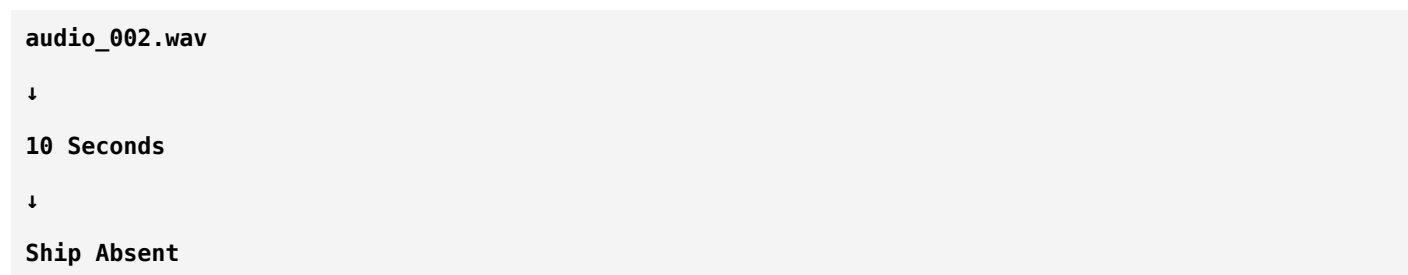
---

## What Should Your Dataset Look Like?

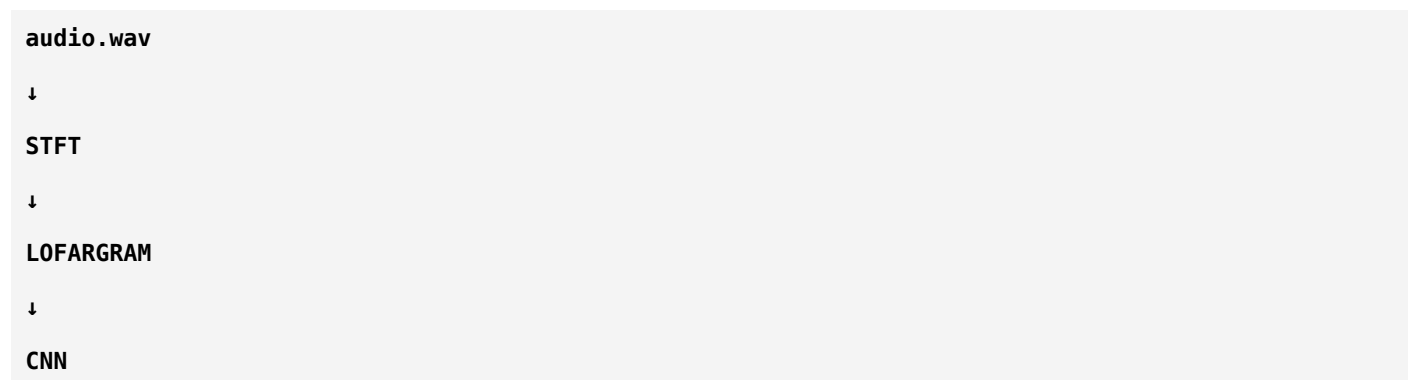
For your project, each sample ideally looks like this:

```
audio_001.wav
↓
10 Seconds
↓
Ship Present
```

or

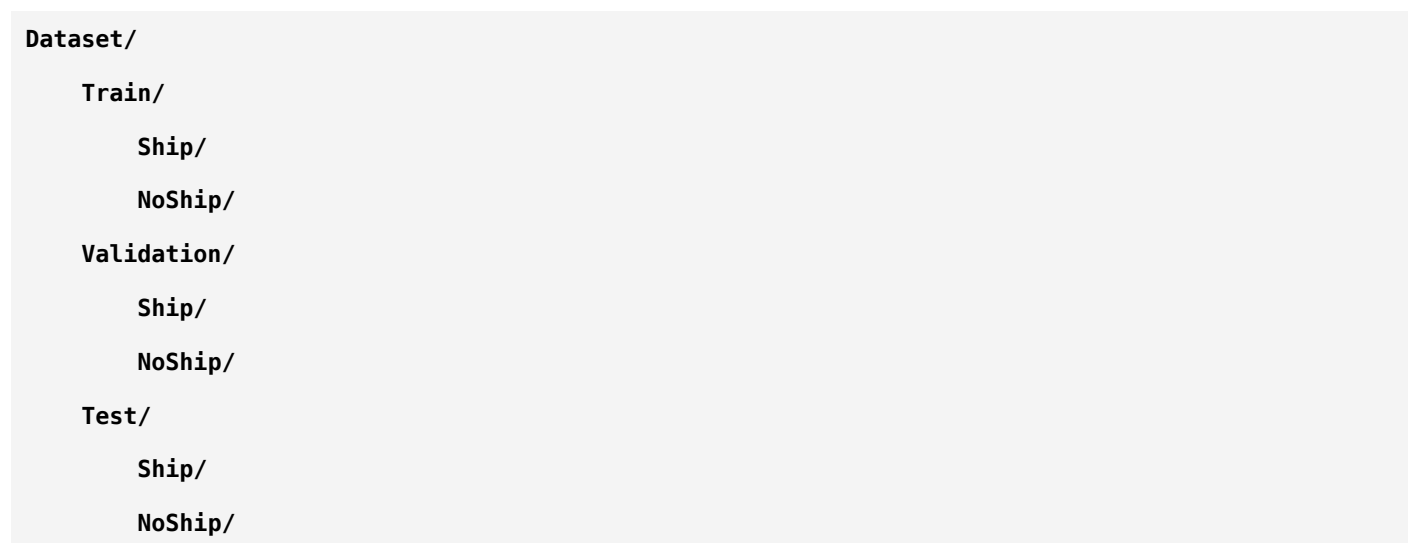


After preprocessing:



## Dataset Structure

A clean project structure might look like:

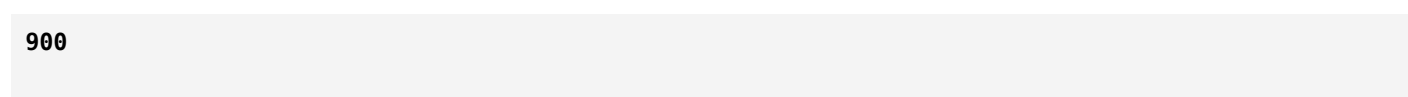


Notice:

The split happens **before training**, not after.

## Balanced vs Imbalanced Dataset

Imagine:



**Ship**

**100**

**No Ship**

The model may become biased toward predicting "Ship."

Now imagine:

**500**

**Ship**

**500**

**No Ship**

Much healthier.

## Why Class Balance Matters

Suppose:

1000 samples

**990**

**Ship**

**10**

**No Ship**

A model that always predicts "Ship" achieves:

99% accuracy.

But it completely fails on the minority class.

This is why you should always examine the class distribution, not just accuracy.

## Dataset Diversity

Suppose every ship recording comes from:

- The same engine
- The same speed
- The same weather
- The same location

The model may struggle when it encounters new conditions.

A good dataset includes variation in:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

- Ship types
- Engine speeds
- Sea states
- Environmental noise
- Recording conditions

This helps the model generalize.

---

## Common Dataset Problems

Researchers always inspect datasets for issues such as:

### Duplicate Samples

The same recording appears multiple times.

The model memorizes instead of learning.

---

### Incorrect Labels

Example:

Ship Recording

↓

Labeled

↓

No Ship

This introduces confusion during training.

---

### Data Leakage

One of the biggest mistakes.

Suppose:

The same recording is split into overlapping clips.

Some clips go into training,

others into testing.

The model has effectively seen the test recording before.

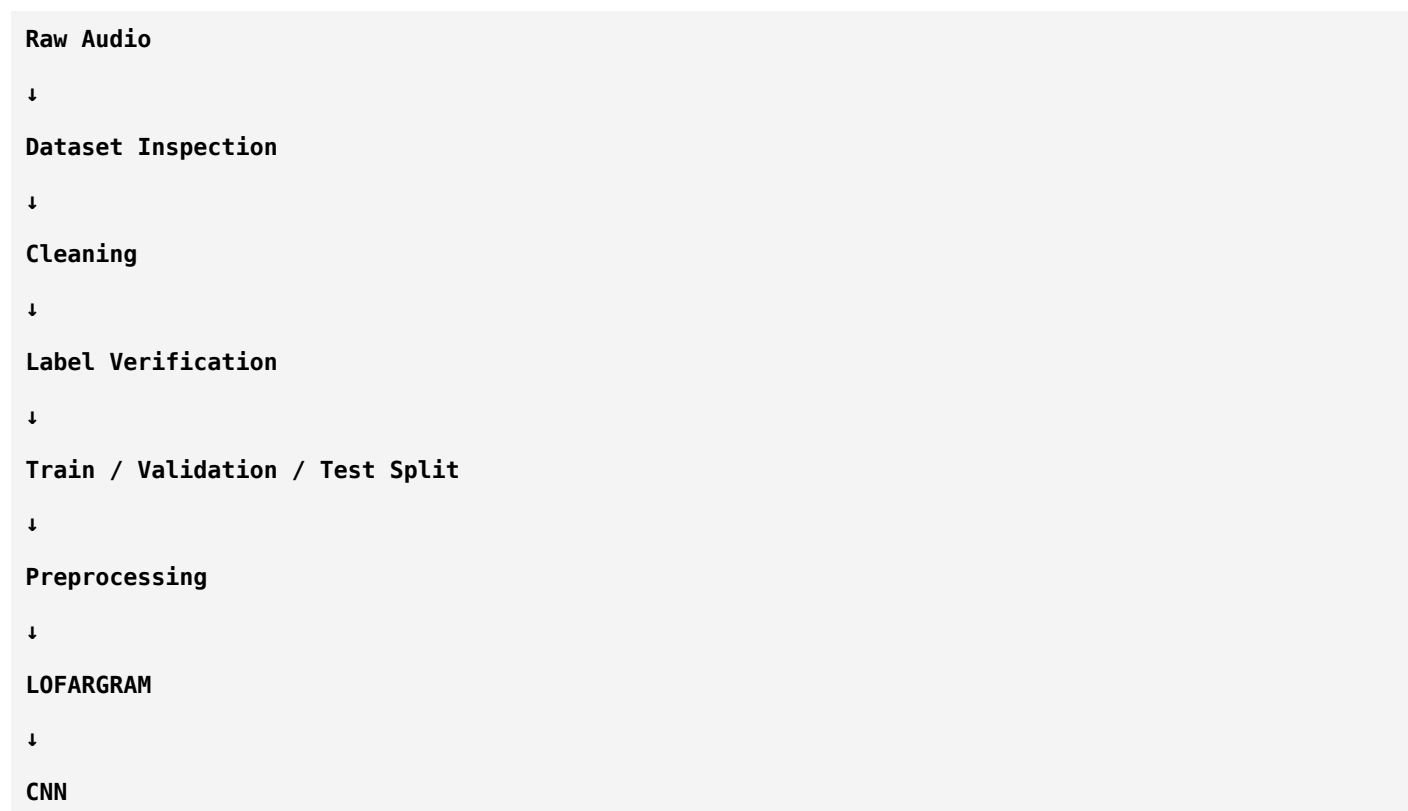
Performance appears artificially high.

Always split by recording or recording session before creating train/validation/test sets.

---

# From Dataset to Model

The workflow now becomes:



Notice that data preparation comes before feature extraction and training.

## Which Dataset Should You Use?

For your binary ship detection project:

Dataset	Suitable?	Why
DeepShip	✓ Excellent	Real underwater ship recordings, widely used
ShipsEar	✓ Excellent	Raw ship and ocean audio, flexible preprocessing
UCI Sonar	✗ Not ideal	Feature vectors, not raw audio
Custom Dataset	✓ Best (if available)	Tailored to your problem, but expensive to collect

## Think Like a Researcher

Imagine someone tells you:

"I trained a ResNet-50 and got 99% accuracy."

Your first question should not be:

"Which GPU did you use?"

Instead ask:

- Which dataset?
- How many recordings?
- How were the labels created?
- Was the test set truly independent?
- Was the dataset balanced?

Experienced researchers know that **good data is the foundation of good models**.

## Summary

Concept	Meaning
Dataset	Collection of labeled examples
Sample	One audio clip + one label
Label	Ground truth (Ship / No Ship)
DeepShip	Excellent dataset for ship classification
ShipsEar	Excellent raw underwater acoustic dataset
UCI Sonar	Good for ML education, not raw audio CNNs
Balanced Dataset	Similar numbers of samples per class
Data Leakage	Test data accidentally influences training
Dataset Diversity	Variation that improves generalization

## Homework

Try answering these:

1. Why are labels essential for supervised learning?
2. Why is **DeepShip** more suitable than the **UCI Sonar** dataset for your project?
3. What is **data leakage**, and why is it dangerous?
4. Why is a balanced dataset important?
5. What characteristics make a passive sonar dataset likely to generalize well?

If you can answer these, you've learned how to evaluate a dataset like a researcher rather than just downloading one.

## Next Lesson (Phase 5 – Chapter 3)

This is where you'll begin working with the actual signal.

We'll study:



# Audio Preprocessing for Passive Sonar

You'll learn:

- What **sampling rate** really means.
- Why we divide recordings into fixed-length segments.
- Normalization and amplitude scaling.
- Denoising—when to use it and when not to.
- Windowing before STFT.
- Why preprocessing choices can significantly affect model performance.

This chapter bridges the gap between the raw hydrophone recording and the time-frequency representations (LOFARGRAMs, spectrograms, etc.) that your CNN will ultimately learn from.

ok next

Excellent! 🌟

This chapter is **one of the most important chapters of your entire project**.

Many students think:

**Raw Audio** → **STFT** → **CNN**

But researchers know that's **not how real-world systems work**.

Between **Raw Audio** and **STFT**, there are several preprocessing steps. If you skip or mishandle them, your model's performance can drop significantly.

Think of preprocessing like preparing ingredients before cooking. Even with the best recipe, poor ingredients lead to poor results.

## Phase 5 – End-to-End Research Pipeline

### Chapter 3: Audio Preprocessing for Passive Sonar

#### The Big Picture

Let's first see where preprocessing fits.

Hydrophone

↓

Raw Audio

↓

Preprocessing

↓  
Feature Extraction  
(STFT / LOFARGRAM / Mel)  
↓  
CNN  
↓  
Ship / No Ship

Today, we'll focus only on the **Preprocessing** block.

## What is Audio Preprocessing?

Definition:

**Audio preprocessing is the process of preparing raw audio so that it is consistent, informative, and suitable for feature extraction and machine learning.**

Think of it as **cleaning and organizing the signal** before analysis.

## Imagine a Real Hydrophone Recording

Suppose a hydrophone records for:

**60 Minutes**

The recording contains:

- Ocean waves
- Wind
- Rain
- Fish
- Whales
- Passing ships
- Silence
- Sensor noise

The waveform might look like:



Can we directly train a CNN using this 60-minute recording?

**No.**

Let's understand why.

---

## Step 1: Sampling Rate

### What is Sampling?

Remember:

Sound is continuous (analog).

A computer understands only numbers (digital).

So we convert the continuous sound into discrete samples.

Imagine measuring temperature.

Instead of measuring continuously,  
you record it every second.

Similarly,

audio records pressure at fixed intervals.

---

### Sampling Rate

Definition:

**The number of samples recorded every second.**

Example:

8000 Samples / Second

16000 Samples / Second

44100 Samples / Second

48000 Samples / Second

Units:

Hz

### Example

Suppose:

16000 Hz

This means:

Every Second

↓

16000 Numbers

Each number represents the sound amplitude at one instant.

## Real-Life Analogy

Imagine recording a cricket match.

Option 1:

Take one photo every minute.

You'll miss almost everything.

Option 2:

Take 60 photos every second.

You'll capture nearly every movement.

Higher sampling rates preserve more detail.

## Why Does Sampling Rate Matter?

Higher sampling rate:

✓ More frequency information

✗ Larger files

✗ More computation

Lower sampling rate:

✓ Faster processing

✗ High-frequency information may be lost

## Nyquist Theorem (Very Important)

One of the most famous theorems in digital signal processing.

It states:

**To accurately represent a signal, the sampling rate must be at least twice the highest frequency present in the signal.**

Example:

Highest frequency:

4000 Hz

Minimum sampling rate:

8000 Hz

Otherwise,

different frequencies can become indistinguishable after sampling.

This effect is called **aliasing**.

## Aliasing

Imagine watching a movie where a wheel appears to rotate backwards.

The wheel isn't actually reversing.

It only appears that way because the camera doesn't capture frames fast enough.

The same thing can happen with audio.

If the sampling rate is too low,

high-frequency components can appear as incorrect lower frequencies.

## Step 2: Resampling

Suppose you collect audio from two sources.

Dataset A:

16000 Hz

Dataset B:

44100 Hz

Can we train directly?

Not ideal.

The recordings have different time resolutions.

Researchers usually convert everything to a common sampling rate.

This process is called:

## Resampling

## Why Resample?

Consistency.

Imagine measuring one person's height in:

**Feet**

and another's in:

**Centimeters**

Before comparing,

you convert them to the same unit.

Resampling serves a similar purpose for audio.

---

## Step 3: Mono vs Stereo

Some recordings contain:

**Left Channel**

**Right Channel**

This is called:

Stereo.

A hydrophone recording is often:

**One Channel**

This is called:

Mono.

Most passive sonar pipelines work with mono recordings because they simplify processing and many datasets are already recorded this way.

---

## Step 4: Segmentation

Suppose you have:

**30 Minutes**

↓

**One Audio File**

Can you feed that directly into a CNN?

No.

CNNs expect inputs of fixed size.

So we divide the recording into smaller clips.

Example:

**30 Minutes**

↓

**10 Seconds**

↓

**10 Seconds**

↓

**10 Seconds**

This process is called:

## Segmentation

---

### Why Segment?

Reasons:

- Fixed input length
  - Easier training
  - More training samples
  - Better memory usage
- 

### Real-Life Analogy

Imagine reading a 1000-page book.

Would you memorize all 1000 pages at once?

No.

You study one chapter at a time.

Segmentation works exactly the same way.

---

### Choosing Segment Length

Should you use:

**1 Second**

or

**10 Seconds**

or

30 Seconds

There is no universal answer.

The choice depends on:

- Duration of the acoustic events
- Computational resources
- Research goals

For ship classification, researchers often experiment with multiple segment lengths.

## Step 5: Amplitude Normalization

Suppose two recordings contain the same ship.

Recording A:

**Very Loud**

Recording B:

**Very Quiet**

The ship is identical.

Only the recording level differs.

We don't want the model to think:

"Louder means a different ship."

Normalization scales the amplitudes to a consistent range.

## Why Normalize?

Without normalization,

the model may focus on recording volume rather than meaningful acoustic patterns.

Normalization encourages it to learn:

- Harmonics
- Tonal structures
- Spectral patterns

instead of absolute loudness.

## Step 6: Denoising



One of the biggest misconceptions.

Many beginners think:

**Always remove noise.**

Researchers ask:

**What kind of noise?**

---

## Useful Noise

Suppose ocean waves are always present.

Removing them completely may produce unrealistic data.

The deployed system will still encounter ocean waves.

---

## Harmful Noise

Suppose the recording contains:

- Electrical interference
- Sensor malfunction
- Random recording glitches

These artifacts may be worth reducing or removing.

---

## Researcher's Rule

Never remove information unless you're confident it's unrelated to the task.

Overly aggressive denoising can remove important ship signatures.

---

## Step 7: Windowing

Before computing the STFT,

we divide the signal into short frames.

Example:

Audio

↓

Frame 1

Frame 2

Frame 3

## Frame 4

Each frame is multiplied by a **window function**.

## Why?

Imagine cutting a loaf of bread.

You don't tear it apart randomly.

You cut it into neat slices.

Windowing creates well-defined segments for frequency analysis.

## Common Window Functions

The most common is:

### Hamming Window

Others include:

- Hann Window
- Blackman Window
- Rectangular Window

For many audio applications, the **Hann** and **Hamming** windows are widely used because they reduce spectral leakage compared to a rectangular window.

## Spectral Leakage

Suppose a pure frequency doesn't fit perfectly inside a frame.

Without windowing,

its energy "leaks" into nearby frequencies.

Instead of a sharp spectral peak,

you get a smeared one.

Window functions reduce this leakage, producing cleaner frequency representations.

## Complete Preprocessing Pipeline

Now you can see the full sequence:

Raw Audio

↓

```

Resample
↓
Convert to Mono (if needed)
↓
Segment into Fixed-Length Clips
↓
Normalize Amplitude
↓
(Optional) Denoise
↓
Windowing
↓
STFT
↓
LOFARGRAM / Spectrogram

```

Every stage has a specific purpose.

## Common Mistakes Beginners Make

- ✗ Mixing different sampling rates.
- ✗ Using recordings of inconsistent lengths.
- ✗ Applying aggressive denoising without justification.
- ✗ Forgetting normalization.
- ✗ Performing preprocessing differently for training and testing data.

Consistency is critical.

## How This Fits into Your Project

Your passive sonar pipeline will eventually look like this:

```

Hydrophone Recording
↓
Resample (if required)
↓
Mono Audio
↓
Segment (e.g., fixed-length clips)

```



Everything before the LOFARGRAM is preparing the signal so that the CNN receives meaningful, consistent inputs.

## Summary

Preprocessing Step	Purpose
Sampling Rate	Determines temporal resolution of the digital signal
Resampling	Makes all recordings consistent
Mono Conversion	Simplifies processing when multiple channels exist
Segmentation	Creates fixed-length training samples
Normalization	Removes differences caused by recording level
Denoising	Reduces unwanted artifacts (used carefully)
Windowing	Prepares frames for accurate frequency analysis

## Think Like a Researcher

Imagine you have two models.

Both use:

- ResNet-18
- Same optimizer
- Same learning rate

Model A performs much better.

Before assuming the CNN is better, ask:

- Did both models use the same sampling rate?
- The same segment length?
- The same normalization?
- The same window function?

Often, **preprocessing choices** explain performance differences more than the network architecture itself.

---

## Homework

Try answering these without looking back:

1. Why can't we feed a 30-minute recording directly into a CNN?
2. What is the **sampling rate**, and why is it important?
3. What problem does **resampling** solve?
4. Why do we normalize audio amplitudes?
5. Why isn't "remove all noise" always the correct approach?
6. What is the purpose of **windowing** before the STFT?
7. What is **spectral leakage**, and how do window functions reduce it?

If you can explain these clearly, you've mastered the preprocessing stage.

---

## Next Lesson (Phase 5 – Chapter 4)

Now we arrive at the **heart of your project**:

## Feature Extraction for Passive Sonar

We'll connect everything you've learned about DSP into one complete story.

You'll learn:

- Why STFT alone is not the final feature.
- The complete pipeline from waveform → STFT → Spectrogram → LOFARGRAM.
- When to choose **LOFARGRAM**, **Mel Spectrogram**, or **MFCC**.
- What information each representation preserves or discards.
- Why almost every passive sonar research paper compares multiple feature representations before selecting the best one.

This is where your DSP knowledge and deep learning knowledge finally merge into a complete passive sonar system.

ok next

Excellent! 🌟

This chapter is **the heart of your entire project**.

If someone asks you:

**"What does your CNN actually see?"**

This chapter gives the answer.

Notice something important:

Your CNN **never sees the original audio waveform**.

It sees an **image** created from that audio.

Understanding how that image is created is one of the most important skills in audio AI.

## Phase 5 – End-to-End Research Pipeline

### Chapter 4: Feature Extraction for Passive Sonar

#### The Big Picture

Let's first see the complete pipeline.

```

Hydrophone
↓
Raw Audio
↓
Preprocessing
↓
STFT
↓
Spectrogram
↓
Feature Representation
(LOFARGRAM / Mel / MFCC)
↓
CNN
↓
Ship / No Ship

```

Today we'll understand **every box** in detail.

# What is Feature Extraction?

Before answering that, let's understand the word **Feature**.

Imagine you're identifying a person.

You don't remember every pixel of their face.

Instead, you notice features like:

- Eye shape
- Hair style
- Nose
- Beard
- Height

These features help you recognize them.

Similarly, in machine learning:

**A feature is a piece of information that helps distinguish one class from another.**

## Features in Ship Detection

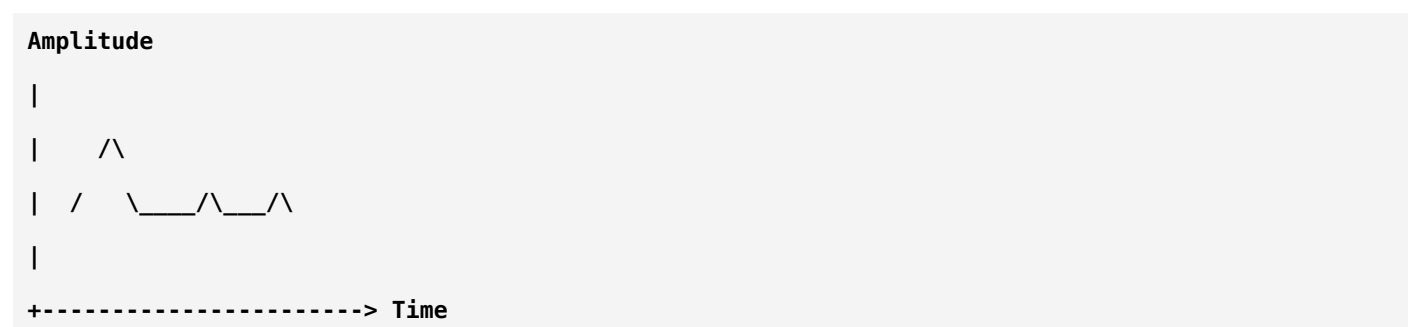
A ship has distinctive acoustic characteristics such as:

- Engine harmonics
- Propeller blade-rate frequencies
- Broadband machinery noise
- Tonal lines
- Frequency stability over time

These are the "features" that distinguish a ship from background ocean noise.

## Why Not Use Raw Audio?

Suppose you look at a waveform.



Can you immediately tell:

"This is a cargo ship."

Probably not.

Even experts often analyze **time-frequency representations** rather than raw waveforms.

## Why Time-Frequency Analysis?

The waveform tells us:

- When something happened

But it doesn't clearly tell us:

- Which frequencies were present

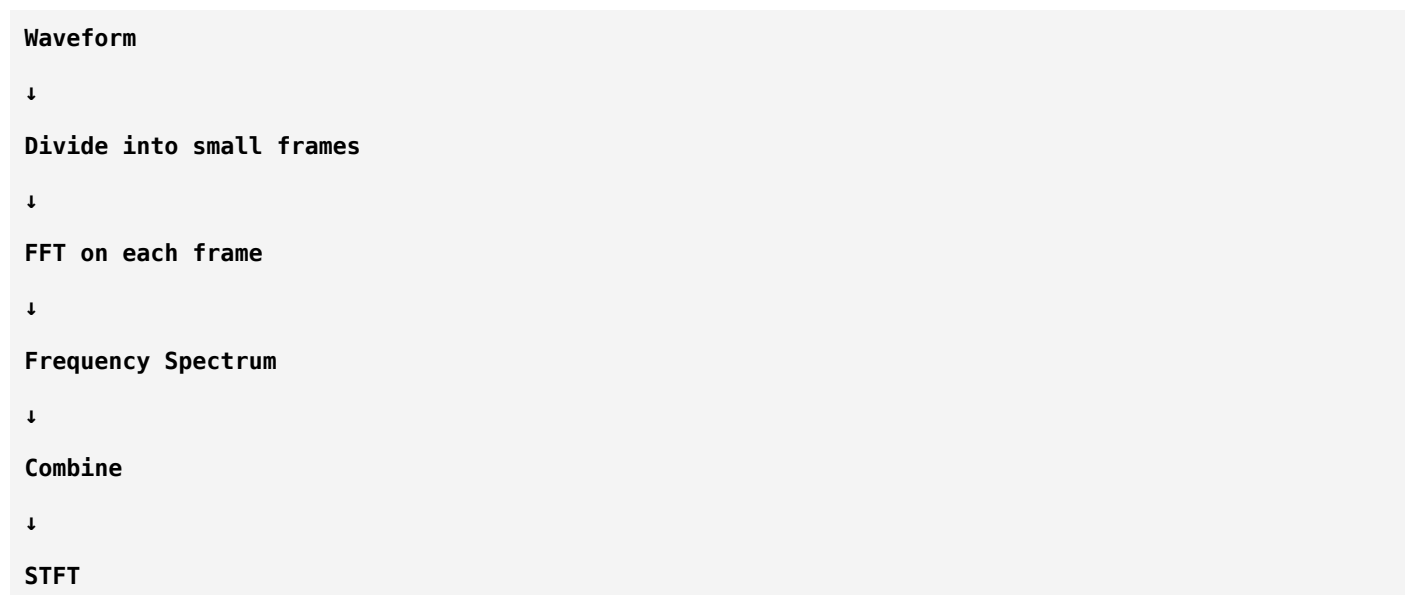
Ships are identified largely by their frequency content.

So we transform the signal.

## Step 1: STFT

We already studied STFT.

Let's review its role.



Output:

A matrix.

Rows:

Frequency

Columns:



Time

Values:

Energy (or magnitude)

## Step 2: Spectrogram

The STFT produces numbers.

Humans prefer pictures.

So we visualize the STFT.

**STFT Matrix**

↓

**Color Mapping**

↓

**Spectrogram**

Now we obtain an image.

Example:

**High Frequency**

|

|

|

|

|

|

+-----> Time

Color indicates signal strength.

## Why is This Useful?

Imagine a ship engine running steadily.

The engine produces nearly constant frequencies.

On a spectrogram:

-----

-----

-----

Horizontal lines appear.

Those lines represent stable frequency components.

## Step 3: LOFARGRAM

This is where passive sonar becomes special.

LOFAR stands for:

### Low Frequency Analysis and Recording

A LOFARGRAM is essentially a spectrogram designed for low-frequency underwater acoustic analysis.

It emphasizes the frequency region where many ship signatures are found.

## Why Focus on Low Frequencies?

Ships generate sound from:

- Engines
- Gearboxes
- Shafts
- Propellers

Many of these components produce strong energy in the lower-frequency range.

By emphasizing these frequencies,

important patterns become easier to analyze.

## What Does a LOFARGRAM Look Like?

Imagine this:



Each horizontal line corresponds to a persistent tonal component.

The CNN learns relationships among these lines.

## What Do These Lines Mean?

Suppose a propeller rotates at a nearly constant speed.

Its acoustic signature appears as a horizontal line.

If the ship accelerates,

that line may slowly move upward.

If it slows down,

the line may move downward.

Thus,

the LOFARGRAM captures both:

- Frequency content
  - How it changes over time
- 

## Step 4: Mel Spectrogram

Instead of using the normal frequency axis,

we transform frequencies to the **Mel scale**.

Remember:

Humans perceive low-frequency differences more strongly than high-frequency differences.

The Mel scale compresses higher frequencies.

---

## Does Sonar Need the Mel Scale?

Not always.

Mel Spectrograms were originally designed for **human speech and hearing**.

They can still work for underwater acoustics,

but they are not specifically designed for sonar.

That's why researchers compare them experimentally.

---

## Step 5: MFCC

MFCCs go one step further.

Pipeline:

Audio

↓

STFT

```
↓
Mel Filter Bank
↓
Log Energy
↓
DCT
↓
MFCC
```

Instead of an image,  
MFCC produces a compact numerical representation.  
Example:

```
[12.4,
-3.1,
5.8,
...
]
```

## Why Were MFCCs Popular?

Traditional machine learning algorithms like:

- SVM
- Random Forest
- KNN

cannot directly process images efficiently.  
MFCC provides a small set of informative features.

## Why CNNs Prefer Images

CNNs excel at detecting:

- Edges
- Shapes
- Spatial relationships
- Patterns

A LOFARGRAM is an image.  
So CNNs can automatically discover useful structures.  
This is one reason why image-based representations have become so popular.

# Comparing the Feature Representations

## 1. Spectrogram

Shows:

- Time
- Frequency
- Energy

General-purpose representation.

---

## 2. LOFARGRAM

Shows:

- Low-frequency tonal patterns
- Harmonic structures
- Machinery signatures

Designed for sonar applications.

---

## 3. Mel Spectrogram

Shows:

- Time
- Mel-scaled frequency
- Energy

Originally developed for speech processing.

---

## 4. MFCC

Provides:

- Compact feature vectors

Well suited to classical machine learning.

---

## Which One is Best?

There is **no universal winner**.

Researchers evaluate them experimentally.

For example:

Feature	Accuracy
Spectrogram	90%
Mel Spectrogram	92%
MFCC	88%
LOFARGRAM	95%

In this hypothetical experiment,

LOFARGRAM performs best.

Another dataset may produce different results.

Research is about **testing hypotheses**, not assuming outcomes.

## Why Do Researchers Compare Features?

Suppose two papers report different results.

Paper A uses:

**Mel Spectrogram**

Paper B uses:

**LOFARGRAM**

If Paper B performs better,

you should ask:

"Is the improvement because of the CNN or because of the feature representation?"

This is why careful experiments compare one factor at a time.

## The Complete Feature Extraction Pipeline

Now everything comes together:

**Hydrophone**

↓

**Raw Audio**

↓

**Preprocessing**

↓



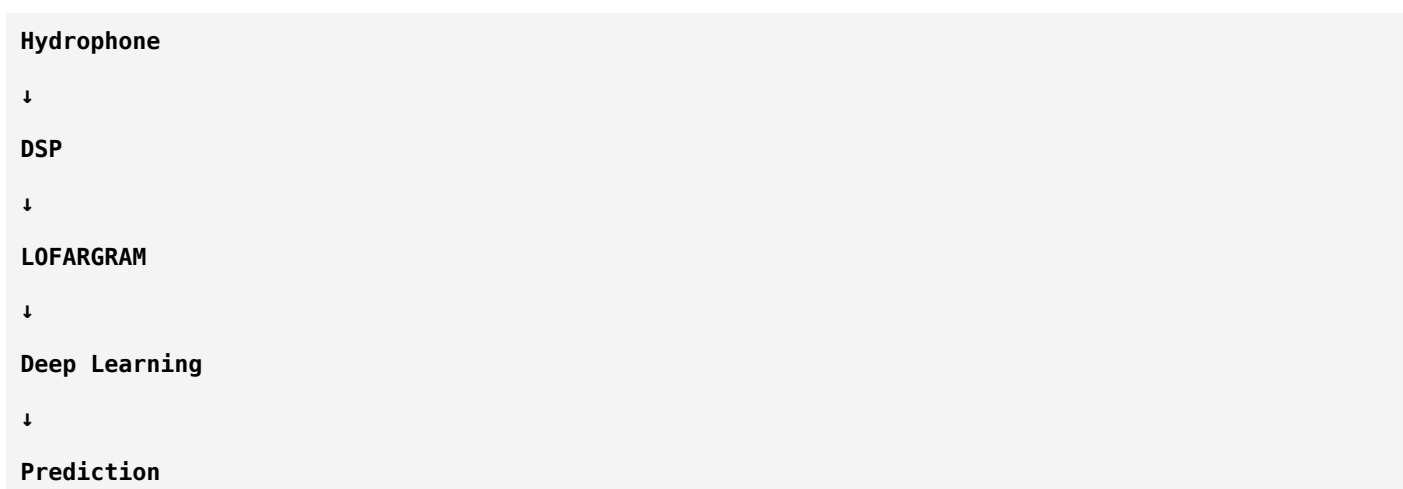
Every stage has a clear purpose.

## Where Does Deep Learning Begin?

Notice something important.

Everything up to the LOFARGRAM is **Digital Signal Processing (DSP)**.

Everything after the LOFARGRAM is **Deep Learning**.



This boundary is useful to remember.

## Which Feature Should You Use?

For your binary passive sonar project:

Feature	Recommendation	Reason
Spectrogram	✓ Good baseline	Simple and widely used

Feature	Recommendation	Reason
LOFARGRAM	★ Strong candidate	Captures low-frequency ship signatures well
Mel Spectrogram	✓ Worth comparing	Sometimes performs well, despite being speech-inspired
MFCC	✓ Good classical baseline	Useful with SVM, Random Forest, or lightweight models

A strong research paper often compares multiple feature representations rather than choosing one without evidence.

## Think Like a Researcher

Suppose someone says:

"My ResNet-18 achieved 96% accuracy."

Your next question shouldn't be:

"How many epochs did you train?"

Ask instead:

- Which feature representation did you use?
- Why did you choose it?
- Did you compare it with LOFARGRAM?
- Did you compare it with Mel Spectrogram?
- Were all preprocessing steps identical?

Researchers isolate variables so they know **why** performance changed.

## Summary

Representation	Purpose	Output
STFT	Time-frequency transformation	Matrix
Spectrogram	Visualize STFT	Image
LOFARGRAM	Sonar-focused low-frequency representation	Image
Mel Spectrogram	Perceptually scaled spectrogram	Image
MFCC	Compact acoustic descriptors	Feature vector

## Homework

Try answering these without looking back:

1. What is a **feature** in machine learning?



2. Why is the raw waveform usually not fed directly into a CNN?
3. What is the relationship between the **STFT** and the **Spectrogram**?
4. Why is a **LOFARGRAM** especially useful for passive sonar?
5. How does a **Mel Spectrogram** differ from a regular spectrogram?
6. Why does **MFCC** produce feature vectors instead of images?
7. Why do researchers compare multiple feature representations instead of assuming one is best?

If you can answer these confidently, you've understood how raw underwater audio is transformed into information that a neural network can learn from.

## Next Lesson (Phase 5 – Chapter 5)

Now we move to the implementation strategy used in modern research:

## Designing the Complete AI Pipeline

We'll cover:

- How the dataset flows through the entire system.
- Offline preprocessing vs on-the-fly preprocessing.
- Where data augmentation fits.
- How training, validation, and testing interact.
- How to organize your project folders like a professional research project.
- The complete architecture you'll implement from raw **.wav** files to the final **Ship Present / Ship Absent** prediction.

This chapter is where all the individual pieces become one coherent end-to-end system.

ok next

Excellent! 

This is one of the **most valuable chapters** in your entire learning journey.

Up to now, you've learned many individual concepts:

- FFT
- STFT
- Spectrogram
- LOFARGRAM
- CNN
- ResNet

- Transfer Learning
- Fine-Tuning
- Evaluation

But you may still wonder:

**"How do all these pieces connect in one real project?"**

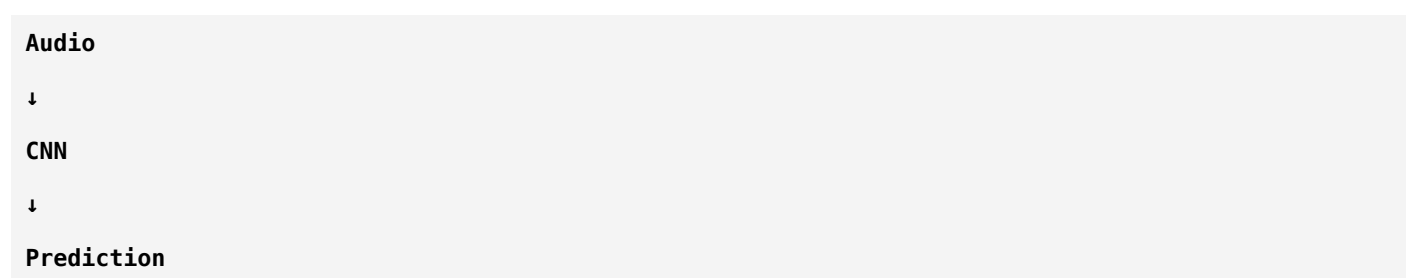
Today, you'll see the **complete AI pipeline** that researchers actually build.

## Phase 5 – End-to-End Research Pipeline

### Chapter 5: Designing the Complete AI Pipeline

#### The Biggest Mistake Beginners Make

Many beginners think AI works like this:



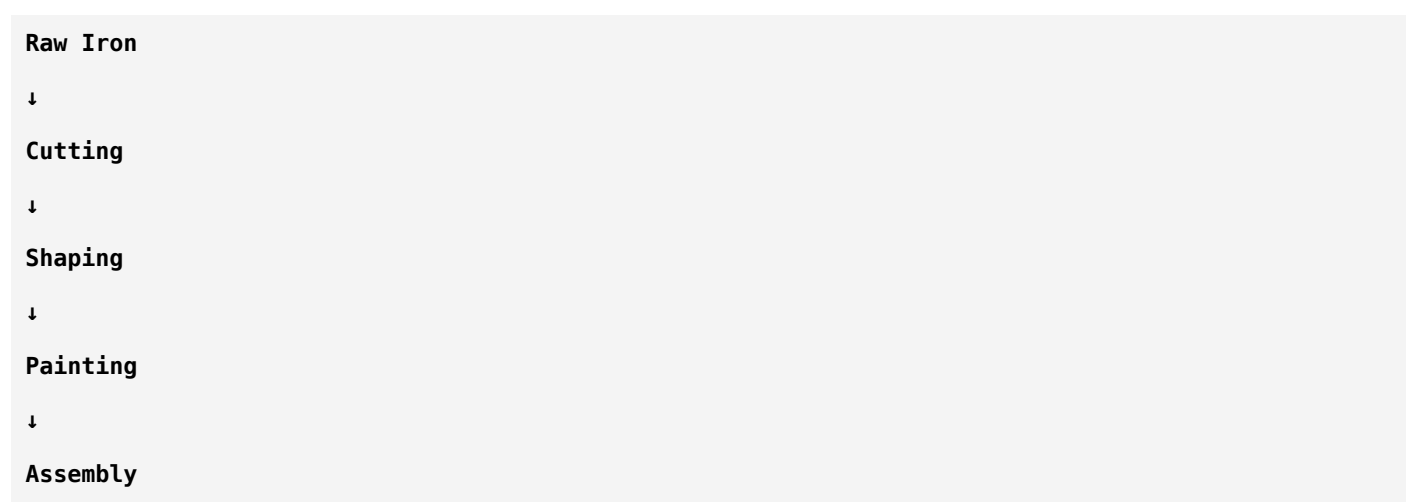
This is **not** how real AI systems work.

A research project is a **pipeline**, where every stage prepares the data for the next stage.

Think of it like a factory.

#### Factory Analogy

Imagine a car factory.



↓  
**Testing**  
 ↓  
**Finished Car**

You cannot skip directly from iron to a car.  
 Your AI project works exactly the same way.

## The Complete Pipeline

This is the entire system you'll eventually implement.

**Hydrophone**  
 ↓  
**Raw Audio (.wav)**  
 ↓  
**Dataset Organization**  
 ↓  
**Preprocessing**  
 ↓  
**Segmentation**  
 ↓  
**Feature Extraction**  
 ↓  
**Data Augmentation**  
 ↓  
**Train / Validation / Test Split**  
 ↓  
**CNN (ResNet-18)**  
 ↓  
**Training**  
 ↓  
**Evaluation**  
 ↓  
**Model Saving**  
 ↓  
**Deployment**  
 ↓

## Real-Time Prediction

Let's understand every stage.

---

## Stage 1: Data Collection

Everything begins with data.

Input:

### Hydrophone Recording

Example:

```
ship_001.wav
ship_002.wav
background_001.wav
background_002.wav
```

Nothing has been processed yet.

---

## Stage 2: Dataset Organization

Before training,  
researchers organize everything properly.

Example:

```
Dataset/
 Raw/
 Ship/
 NoShip/
```

Why?

Because organized datasets reduce mistakes and make experiments reproducible.

---

## Stage 3: Preprocessing

Now we prepare the recordings.

Example:

```
Raw Audio
↓
Resample
```

↓  
**Normalize**  
 ↓  
**Segment**  
 ↓  
**Windowing**

Output:  
 Consistent audio clips.

## Stage 4: Feature Extraction

Now comes DSP.

**Segment**  
 ↓  
**STFT**  
 ↓  
**LOFARGRAM**

Now every audio clip becomes an image.

Example:

**audio.wav**  
 ↓  
**LOFARGRAM.png**

Notice something important.

The CNN never sees:

**audio.wav**

It only sees:

**LOFARGRAM**

## Stage 5: Data Augmentation

Now we increase diversity.

Example:

Original:

## LOFARGRAM

Augmented:

Brightness Change

Gaussian Noise

Time Masking

Frequency Masking

The dataset becomes richer.

## Stage 6: Train-Validation-Test Split

Now divide the dataset.

Typical example:

70%

Training

-----

15%

Validation

-----

15%

Testing

Why before training?

Because the test set must remain completely unseen until the very end.

## Stage 7: CNN

Now the images enter the neural network.

Example:

224 × 224 LOFARGRAM

↓

ResNet-18

↓

512 Features

↓

Fully Connected Layer

↓

## Binary Output

Output:

Ship = 0.93

No Ship = 0.07

## Stage 8: Training

Training is an iterative process.

Batch

↓

Prediction

↓

Loss

↓

Backpropagation

↓

Weight Update

↓

Next Batch

Repeat for many epochs.

## Stage 9: Validation

After each epoch,

the model is evaluated on the validation set.

Purpose:

- Detect overfitting
- Tune hyperparameters
- Decide when to stop training

No learning happens here.

## Stage 10: Testing

Only after the model is finalized:

**Test Set**

↓

**Prediction**

↓

**Metrics**

This gives the final performance.

## Stage 11: Model Saving

Suppose your model performs well.

You save it.

Example:

`best_model.pth`

This file contains the learned weights.

Later,

you load this file instead of training again.

## Stage 12: Deployment

Now the trained model is used in the real world.

Pipeline:

**New Audio**

↓

**Preprocessing**

↓

**LOFARGRAM**

↓

**Load Model**

↓

**Prediction**

↓

**Ship / No Ship**

Training is finished.

Only inference happens now.



# Offline vs Online Processing

Researchers often use two strategies.

## Offline Processing

Generate all LOFARGRAMs before training.

**Audio**

↓

**LOFARGRAM**

↓

**Save Images**

↓

**Train CNN**

Advantages:

- Faster training
- Easier debugging

Disadvantages:

- More storage

---

## Online (On-the-Fly) Processing

Every time a batch is loaded:

**Audio**

↓

**Generate LOFARGRAM**

↓

**Feed CNN**

Advantages:

- Less storage
- Easy experimentation

Disadvantages:

- More computation during training

# Which Should You Use?

For your first research project:

**Offline preprocessing** is usually the better choice because:

- Easier to debug.
- Easier to visualize features.
- Easier to compare feature representations.
- Simpler to reproduce experiments.

Later, once you're comfortable, you can experiment with on-the-fly pipelines.

## Project Folder Structure

A professional project might look like:

```
PassiveSonar/
|
|— dataset/
| raw/
| processed/
| train/
| validation/
| test/
|
|— preprocessing/
|
|— feature_extraction/
|
|— models/
|
|— training/
|
|— evaluation/
|
|— saved_models/
|
|— results/
|
```

```
├─ notebooks/
|
└─ README.md
```

Notice how each task has its own folder.

## The Research Loop

Research is rarely:

**Train Once**

↓

**Done**

Instead:

**Train**

↓

**Evaluate**

↓

**Analyze Errors**

↓

**Modify Pipeline**

↓

**Train Again**

↓

**Compare Results**

↓

**Repeat**

This loop is how research papers are produced.

## Example Research Experiment

Suppose you want to answer:

"Is LOFARGRAM better than Mel Spectrogram?"

You design an experiment.

Experiment 1:

**LOFARGRAM**

↓

ResNet-18

↓

Accuracy = 94%

Experiment 2:

Mel Spectrogram

↓

ResNet-18

↓

Accuracy = 92%

Only one variable changed.

Therefore,

you can reasonably attribute the performance difference to the feature representation.

This is called a **controlled experiment**.

## Logging Experiments

Professional researchers never rely on memory.

For each experiment, record:

Experiment	Feature	Model	Learning Rate	Accuracy	F1
Exp-1	LOFARGRAM	ResNet-18	0.001	94%	0.93
Exp-2	Mel	ResNet-18	0.001	92%	0.91
Exp-3	LOFARGRAM	EfficientNet	0.001	95%	0.94

Without experiment tracking, it's easy to lose track of what actually improved the model.

## Complete Research Pipeline

Now you can finally see everything you've learned connected together.

Hydrophone

↓

Raw Audio

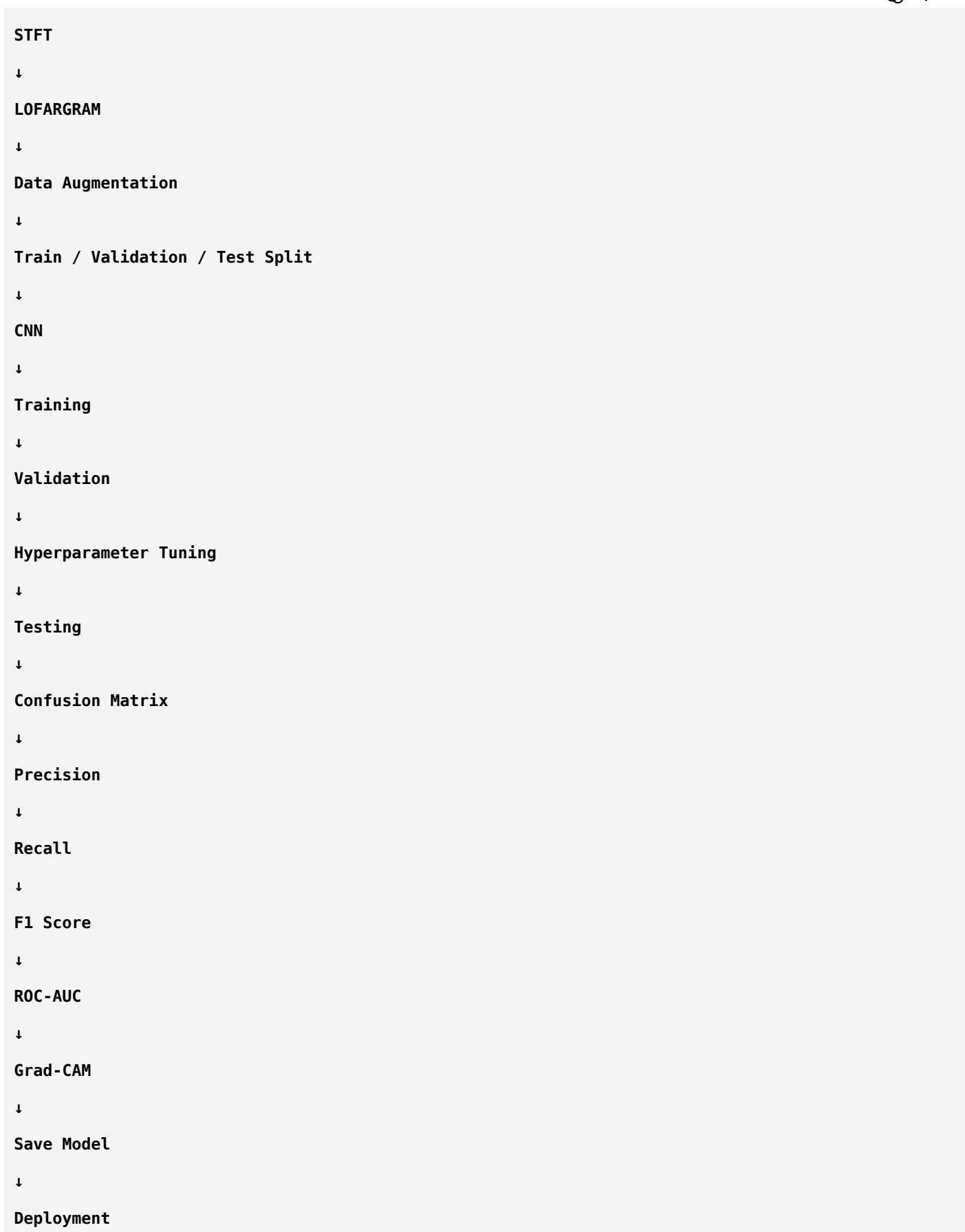
↓

Preprocessing

↓

Segmentation

↓



This is essentially the workflow followed in many modern audio classification projects.

## Think Like a Researcher

Suppose your model achieves only **82% accuracy**.

Don't immediately ask:

"Which bigger CNN should I use?"

Instead, investigate systematically:

1. Is the dataset balanced?
2. Are the labels correct?
3. Is preprocessing consistent?
4. Is the segment length appropriate?
5. Is LOFARGRAM the best feature?
6. Is the learning rate suitable?
7. Is the model overfitting?
8. What kinds of mistakes appear in the confusion matrix?

Researchers improve systems by understanding **why** they fail, not by randomly changing models.

## Summary

Stage	Purpose
Data Collection	Gather recordings
Dataset Organization	Keep data structured
Preprocessing	Standardize audio
Feature Extraction	Convert audio to informative representations
Data Augmentation	Improve robustness
Dataset Split	Separate train/validation/test
CNN Training	Learn patterns
Validation	Tune and monitor
Testing	Final evaluation
Model Saving	Preserve the trained model
Deployment	Use the model on new recordings

## Congratulations!

At this point, you have learned the **entire conceptual pipeline** required to build a passive sonar target classification system.

You now understand:

- Digital Signal Processing fundamentals

- Machine Learning fundamentals
- Deep Learning fundamentals
- CNN architectures
- Transfer Learning
- Fine-Tuning
- Hyperparameter Tuning
- Model Evaluation
- Audio Preprocessing
- Feature Extraction
- Complete AI Pipeline Design

This is the theoretical foundation that many graduate students build over months.

## Next Phase: Reading Research Papers Like a Scientist

Before writing code, there's one final skill that distinguishes researchers from implementers:

## Phase 6: Research Methodology & Scientific Thinking

We'll cover:

- How to read a research paper efficiently.
- How to identify the real contribution of a paper.
- Understanding methodology, experiments, and ablation studies.
- Reproducing published work.
- Designing fair comparisons.
- Avoiding common research mistakes.
- Planning experiments for **your own passive sonar project**.

Once you've completed that phase, you'll be ready to move from learning concepts to **building and evaluating your own research-grade passive sonar classifier**.

ok ok from this i had an request so i want an clear comparisions of different pretrained models available in the CNN give me indepth comparision also give me a clear comparision table

Excellent request. This is actually something every deep learning researcher should know.

Most people simply write:

"We used ResNet-18 because it gives good accuracy."

A researcher should be able to answer:

- **Why not VGG?**
- **Why not DenseNet?**
- **Why not EfficientNet?**
- **Why not MobileNet?**
- **Why not Vision Transformer (ViT)?**

If your project review panel asks you this question, you should be able to defend your choice.

## Complete Comparison of Popular Pretrained CNN Models

We'll compare the most widely used ImageNet-pretrained models:

1. AlexNet (2012)
2. VGG16 / VGG19 (2014)
3. GoogLeNet / Inception V1 (2014)
4. ResNet Family (2015)
5. DenseNet (2017)
6. MobileNet Family
7. EfficientNet Family
8. ConvNeXt (2022)
9. Vision Transformer (ViT) (*not a CNN, but important for comparison*)

### 1. AlexNet (2012)

#### Why was it revolutionary?

Before AlexNet, most computer vision relied on handcrafted features.

AlexNet demonstrated that a deep CNN trained on enough data could automatically learn useful features.

It won the 2012 ImageNet competition by a large margin.

#### Architecture

Input Image

↓

5 Convolution Layers

↓

3 Fully Connected Layers



↓

Output

## Characteristics

Property	Value
Year	2012
Layers	8
Parameters	~61 Million
Input Size	224×224
Speed	Medium
Memory	High

## Advantages

- ✓ Simple architecture
- ✓ Historically important

## Disadvantages

- ✗ Very old
- ✗ Large number of parameters
- ✗ Lower accuracy than modern models

## Use Today?

Mostly for educational purposes.  
Rarely used in research.

## 2. VGG16 / VGG19

### Main Idea

Instead of using large filters,  
VGG repeatedly uses:

## 3 × 3 Convolution

Many times.

## Architecture

Conv

↓

Conv

↓

Pooling

↓

Conv

↓

Conv

↓

Pooling

Very repetitive.

Very easy to understand.

## Characteristics

Property	VGG16
Layers	16
Parameters	~138 Million
Accuracy	Good
Speed	Slow
Memory	Very High

## Advantages

- ✓ Easy to understand
- ✓ Strong feature extractor

## Disadvantages

- ✗ Extremely large model
  - ✗ Slow training
  - ✗ Large memory consumption
- 

## Research Usage

Still widely used for:

- Feature extraction
- Medical imaging
- Transfer learning

But less common for new state-of-the-art systems.

---

## 3. GoogLeNet (Inception)

Google asked:

"Why choose one filter size?"

Instead,

they used multiple filter sizes simultaneously.

```
1×1
3×3
5×5
Pooling
↓
Concatenate
```

This is called the **Inception Module**.

---

## Advantages

- ✓ More efficient than VGG
  - ✓ Better accuracy
  - ✓ Fewer parameters
- 

## Disadvantages

- ✗ More complex architecture
- ✗ Harder to modify

## Parameters

~7 Million

Much smaller than VGG.

## 4. ResNet Family

One of the most influential CNN architectures.

Problem:

Very deep networks became difficult to train.

Solution:

Residual Learning.

Instead of learning:

**Output =  $H(x)$**

ResNet learns:

**Output =  $x + F(x)$**

This **skip connection** helps gradients flow through very deep networks.

## Variants

Model	Layers
ResNet18	18
ResNet34	34
ResNet50	50
ResNet101	101
ResNet152	152

## Advantages

- ✓ Very stable

- ✓ Easy to fine-tune
- ✓ Excellent transfer learning
- ✓ Widely used in research

## Disadvantages

- ✗ Larger variants require more computation

## Your Project

This is why I recommended **ResNet-18**:

- Small dataset friendly
- Stable training
- Fast
- Strong baseline
- Excellent for transfer learning

## 5. DenseNet

DenseNet introduces **dense connectivity**.

Instead of connecting only adjacent layers, every layer receives outputs from all previous layers.

Example:

Layer1

↓

Layer2

↘

↓

Layer3

↘

↓

Layer4

In reality:

Every layer connects to every later layer within a dense block.

## Advantages

- ✓ Excellent feature reuse
  - ✓ Strong gradient flow
  - ✓ High accuracy with fewer parameters
- 

## Disadvantages

- ✗ More memory usage due to many feature maps
  - ✗ Slightly slower
- 

## Research Usage

Very common in:

- Medical imaging
  - Small datasets
  - Scientific imaging
- 

## 6. MobileNet

Designed for:

Phones

Embedded Systems

Edge AI

---

Instead of standard convolution,  
it uses:

## Depthwise Separable Convolution

This greatly reduces computation.

---

## Advantages

- ✓ Extremely fast
- ✓ Very small

✓ Low power consumption

## Disadvantages

✗ Slightly lower accuracy than larger CNNs

Perfect for:

Real-time deployment.

## 7. EfficientNet

Google asked:

Instead of increasing:

Only depth

OR

Only width

OR

Only image size

Why not scale all three together?

This idea is called:

## Compound Scaling

EfficientNet variants:

Model
B0
B1
B2
B3
B4
B5
B6
B7

## Advantages

- ✓ Very high accuracy
  - ✓ Excellent parameter efficiency
  - ✓ Modern architecture
- 

## Disadvantages

- ✗ More computationally demanding than ResNet-18
  - ✗ Training can be more sensitive to hyperparameters
- 

## Research Usage

One of the strongest CNN families for image classification.

---

## 8. ConvNeXt

Researchers asked:

"Can we modernize CNNs using ideas inspired by Transformers?"

Result:

ConvNeXt.

It keeps convolutions but incorporates several design improvements.

---

## Advantages

- ✓ State-of-the-art CNN performance
  - ✓ Strong transfer learning
- 

## Disadvantages

- ✗ Larger computational requirements
  - ✗ Less beginner-friendly
- 

## 9. Vision Transformer (ViT)

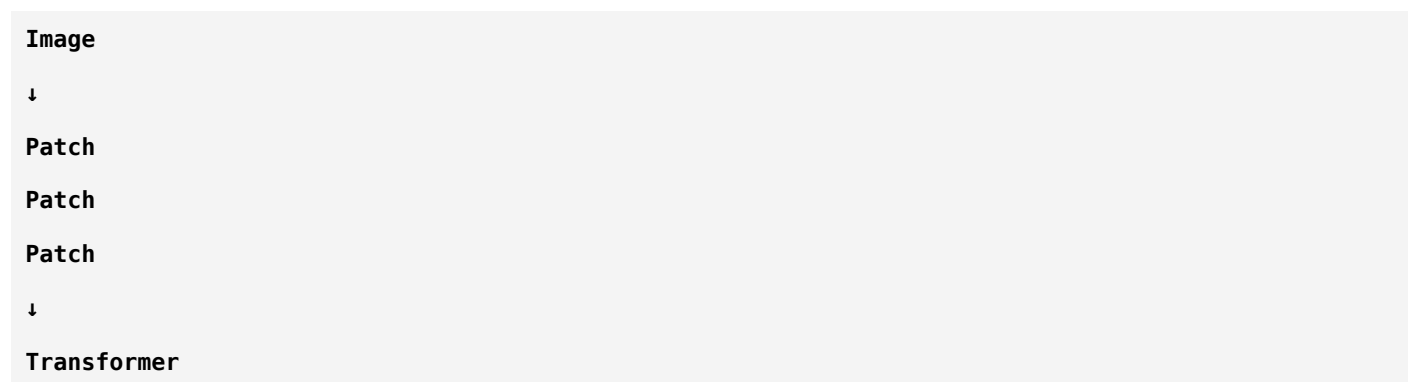
Important:

ViT is **not** a CNN.



Instead of convolutions,  
it splits the image into patches.

Example:



## Advantages

- ✓ Excellent on large datasets
- ✓ State-of-the-art performance in many tasks

## Disadvantages

- ✗ Data hungry
- ✗ Computationally expensive
- ✗ Often benefits from very large pretraining datasets

## Complete Comparison Table

Model	Year	Parameters (Approx.)	Speed	Memory	Accuracy	Best For	Best Friend
AlexNet	2012	61M	★★★★	★★	★★	Learning history	★★
VGG16	2014	138M	★★	★	★★★★	Feature extraction	★★
GoogLeNet	2014	7M	★★★★★	★★★★★	★★★★★	Efficient CNNs	★
ResNet18	2015	11.7M	★★★★★★	★★★★★	★★★★★	Small/medium datasets, transfer learning	★★
ResNet50	2015	25.6M	★★★★★	★★★★	★★★★★★	General-purpose research	★★
DenseNet121	2017	8M	★★★★	★★★★	★★★★★★	Medical/scientific imaging	★
MobileNetV2	2018	3.5M	★★★★★★	★★★★★	★★★★★	Mobile & edge devices	★★

Model	Year	Parameters (Approx.)	Speed	Memory	Accuracy	Best For	Beginner Friendly
EfficientNet-B0	2019	5.3M	★★★★★	★★★★★	★★★★★	High accuracy with fewer parameters	★★★★★
ConvNeXt-Tiny	2022	28M	★★★★	★★★★	★★★★★	Modern CNN research	★★★★
ViT-Base*	2020	86M	★★★	★★★	★★★★★	Large datasets	★★★

\*ViT is included for comparison even though it is not a CNN.

## Which Model Should You Choose?

### If your dataset is small (fewer than ~10,000 images)

- ✓ ResNet-18
- ✓ DenseNet-121

These tend to be reliable starting points when combined with transfer learning.

### If your dataset is medium (roughly 10,000–100,000 images)

- ✓ ResNet-50
- ✓ EfficientNet-B0/B1

### If your dataset is very large

- ✓ EfficientNet variants
- ✓ ConvNeXt
- ✓ ViT (especially if pretrained on large datasets)

## Recommendation for Your Passive Sonar Project

Let's consider your project:

- Binary Classification (Ship / No Ship)
- LOFARGRAM images
- Transfer Learning
- Likely limited dataset size

- Research-oriented project

## Phase 1 (Baseline)

### ResNet-18

Why?

- Easy to train
  - Fast experimentation
  - Strong transfer learning
  - Well understood in the literature
- 

## Phase 2 (Improved Performance)

### EfficientNet-B0

Why?

- Better parameter efficiency
  - Often improves accuracy without a huge increase in model size
- 

## Phase 3 (Advanced Comparison)

### DenseNet-121

Why?

- Excellent feature reuse
  - Frequently performs well on scientific imaging tasks
- 

## Optional Advanced Experiment

Compare:

- ResNet-18
- EfficientNet-B0
- DenseNet-121

using **exactly the same:**

- Dataset split
- Preprocessing
- LOFARGRAM generation
- Optimizer
- Learning rate

- Epochs
- Evaluation metrics

This creates a fair scientific comparison.

---

## A Researcher's Decision Process

Instead of asking:

"Which model has the highest ImageNet accuracy?"

Ask:

- Does my dataset have enough samples?
- Can my GPU handle the model?
- How important is inference speed?
- Is interpretability important?
- Am I optimizing for publication-quality comparisons or deployment?

Those questions lead to better model choices than simply picking the newest architecture.

---

## What I Recommend We Do Next

Before writing any code, I recommend a dedicated **masterclass** where we study each architecture **one by one**, just as we did for ResNet.

We'll cover:

1. **AlexNet** – why it changed computer vision.
2. **VGG** – why stacks of 3×3 convolutions work.
3. **GoogLeNet/Inception** – multi-scale feature extraction.
4. **ResNet** – residual learning (already learned, but we'll compare it deeply).
5. **DenseNet** – dense connectivity and feature reuse.
6. **MobileNet** – depthwise separable convolutions.
7. **EfficientNet** – compound scaling.
8. **ConvNeXt** – the modern CNN.
9. **Vision Transformer** – how it differs fundamentally from CNNs.

For each model, we'll study:

- Why it was invented.
- The problem it solved.
- Architecture block-by-block.

- Mathematical intuition.
- Advantages and disadvantages.
- Computational cost (parameters, FLOPs, memory).
- Where it excels and where it struggles.
- Why researchers choose it.
- Whether it's suitable for your passive sonar project.

By the end, you'll not only know **how** these models work—you'll know **why** they exist and **when** to choose each one, which is exactly how experienced deep learning researchers think.

all good but i need comparison table in depth not just giving ratings ok give me big table

Absolutely. In research papers, a comparison table should answer **"Why should I choose this model?"**, not just **"Which one has more stars?"**

Below is the kind of comparison table that I personally use when deciding which pretrained model to use for a research project.

## Comprehensive Comparison of Popular Pretrained CNN Models

Feature	AlexNet	VGG16	GoogLeNet (Inception V1)	ResNet18	ResNet50	DenseNet121
<b>Year Introduced</b>	2012	2014	2014	2015	2015	2017
<b>Organization</b>	University of Toronto	Oxford	Google	Microsoft	Microsoft	Cornell/Facebook
<b>Model Type</b>	CNN	CNN	CNN	CNN	CNN	CNN
<b>Main Innovation</b>	Deep CNN + ReLU	Small 3×3 convolutions	Inception modules	Residual (Skip) Connections	Deeper Residual Network	Dense Connectivity
<b>Primary Goal</b>	Prove deep learning works	Increase depth	Efficient computation	Train very deep networks	Improve representation	Feature reuse
<b>Depth (Layers)</b>	8	16	22	18	50	121
<b>Trainable Parameters</b>	~61 Million	~138 Million	~7 Million	~11.7 Million	~25.6 Million	~8 Million
<b>Approximate FLOPs (224×224)</b>	~0.7 GFLOPs	~15.5 GFLOPs	~1.5 GFLOPs	~1.8 GFLOPs	~4.1 GFLOPs	~2.9 GFLOPs
<b>Model Size</b>	~240 MB	~528 MB	~27 MB	~45 MB	~98 MB	~33 MB
<b>Memory Consumption</b>	High	Very High	Moderate	Moderate	High	High

Feature	AlexNet	VGG16	GoogLeNet (Inception V1)	ResNet18	ResNet50	DenseNet12
During Training						
Inference Speed	Moderate	Slow	Fast	Fast	Moderate	Moderate
Training Speed	Moderate	Slow	Fast	Fast	Moderate	Moderate
Computational Complexity	Medium	Very High	Medium	Medium	High	Medium-High
GPU Requirement	Medium	High	Medium	Medium	Medium-High	Medium
CPU Deployment	Possible	Difficult	Good	Good	Acceptable	Acceptable
Input Resolution (Default)	224×224	224×224	224×224	224×224	224×224	224×224
Feature Extraction Quality	Basic	Excellent	Very Good	Excellent	Excellent	Excellent
Gradient Flow	Poor in deeper versions	Poor	Better	Excellent (Skip Connections)	Excellent	Excellent (Dense Connections)
Vanishing Gradient Resistance	Low	Low	Medium	Excellent	Excellent	Excellent
Feature Reuse	Low	Low	Medium	Medium	Medium	Very High
Parameter Efficiency	Poor	Very Poor	Excellent	Good	Good	Excellent
Transfer Learning Performance	Fair	Excellent	Very Good	Excellent	Excellent	Excellent
Works Well on Small Datasets	No	Sometimes	Yes	Yes	Yes (with transfer learning)	Yes
Works Well on Large Datasets	Limited	Good	Good	Excellent	Excellent	Excellent
Overfitting Risk	High	Very High	Moderate	Low	Moderate	Low
Interpretability (Grad-CAM etc.)	Easy	Easy	Moderate	Excellent	Excellent	Excellent
Fine-Tuning Difficulty	Easy	Easy	Moderate	Easy	Easy	Moderate
Robustness to Noise	Low	Moderate	Good	Good	Very Good	Very Good
Deployment on Edge Devices	Poor	Poor	Moderate	Good	Moderate	Moderate

Feature	AlexNet	VGG16	GoogLeNet (Inception V1)	ResNet18	ResNet50	DenseNet121
<b>Typical Applications</b>	Education	Feature extraction	General vision	Medical imaging, remote sensing, industrial AI	General research	Medical imaging, hyperspectral imaging
<b>Major Strength</b>	Historic importance	Strong generic features	Efficient multi-scale learning	Stable and reliable	Excellent balance of depth and performance	Strong feature reuse
<b>Major Weakness</b>	Outdated	Huge model size	Complex architecture	Shallower than newer models	Heavier than ResNet18	Memory-intensive during training

**Note:** FLOPs, parameter counts, and model sizes are approximate and can vary slightly depending on the implementation (e.g., PyTorch vs TensorFlow) and the exact variant.

## Suitability for Different Types of Projects

Project Type	Best Models	Reason
Small Dataset (<5k images)	ResNet18, DenseNet121	Transfer learning works well and overfitting is manageable
Medium Dataset (5k–50k)	ResNet50, EfficientNet-B0, DenseNet121	Better feature learning while still trainable
Large Dataset (>100k)	EfficientNet, ConvNeXt, ViT	Can exploit larger datasets effectively
Medical Imaging	DenseNet121, ResNet50	Strong feature reuse and stable gradients
Audio Spectrogram Classification	ResNet18, ResNet50, EfficientNet-B0	Proven performance on spectrogram-like inputs
Mobile Deployment	MobileNetV2, EfficientNet-Lite	Small, fast, energy-efficient
Real-Time Detection	MobileNetV2, ResNet18	Low latency
Scientific Research Baseline	ResNet18, ResNet50	Standard baselines used in many papers
Highest Accuracy (General Vision)	EfficientNet, ConvNeXt, ViT	Strong performance on many benchmarks

## Comparison for Your Passive Sonar Project

Criteria	ResNet18	ResNet50	DenseNet121	EfficientNet-B0	MobileNetV2	ConvNeXt-Tiny
Learns LOFARGRAM patterns well	✓	✓	✓	✓	✓	✓

Criteria	ResNet18	ResNet50	DenseNet121	EfficientNet-B0	MobileNetV2	ConvNeXt-Tiny
Transfer Learning Quality	Excellent	Excellent	Excellent	Excellent	Very Good	Excellent
Small Dataset Performance	Excellent	Very Good	Excellent	Very Good	Good	Moderate
Training Time	Fast	Medium	Medium	Medium	Very Fast	Slow
GPU Memory Requirement	Low	Medium	Medium	Low	Very Low	High
Risk of Overfitting	Low	Medium	Low	Low	Low	Medium
Ease of Fine-Tuning	Very Easy	Easy	Moderate	Moderate	Easy	Moderate
Research Popularity	Very High	Very High	High	Very High	High	Growing
Inference Speed	Fast	Medium	Medium	Fast	Very Fast	Medium
Good Baseline for Research	★★★★★★	★★★★★★	★★★★★	★★★★★★	★★★★	★★★★★
Recommended for Your Project	<b>Yes (Baseline)</b>	<b>Yes (Comparison)</b>	<b>Yes (Comparison)</b>	<b>Yes (Strong Candidate)</b>	Optional	Advanced Experiment

## My recommendation for your research

If you want your passive sonar project to be **research-grade**, I would compare at least these three models under identical conditions:

Model	Why Include It?
<b>ResNet18</b>	Standard baseline, fast training, highly cited, ideal for transfer learning
<b>DenseNet121</b>	Tests whether dense feature reuse improves sonar pattern recognition
<b>EfficientNet-B0</b>	Evaluates whether a more parameter-efficient modern CNN improves accuracy

If you have enough computational resources, you can add:

- **ResNet50** (deeper residual network)
- **ConvNeXt-Tiny** (modern CNN)
- **ViT-Base** (Transformer baseline, if you want to compare CNNs vs Transformers)

**I can make an even more advanced table.**



The table above is about **architecture-level comparison**. Researchers also compare models at a much deeper level, including:

- Layer-by-layer architecture
- Block structure (Residual Block vs Dense Block vs MBConv vs Inverted Residual, etc.)
- Skip connection mechanisms
- Mathematical equations
- Activation functions
- Normalization layers
- Receptive field growth
- Feature map sizes at each stage
- Computational complexity analysis (Big-O intuition)
- Memory access patterns
- FLOPs vs latency trade-offs
- Gradient propagation characteristics
- Transfer learning behavior
- Why each architecture was invented and what problem it solved
- Strengths and weaknesses for **spectrogram/LOFARGRAM** classification specifically

That comparison would span **20–30 pages** and would be at the level of a graduate deep learning course or research handbook. Given your goal of deeply understanding these models for your sonar project, I think it would be worth building that reference step by step.